

Agentic AI for Chemical Information Extraction from Scientific Publications

Kasper Huysentruyt

Student number: 02109082

Supervisors: Prof. dr. ir. Kevin Van Geem, Prof. dr. ir. Christian Stevens

Counsellor: Dr. ir. Maarten Dobbelaere

Master's dissertation submitted in order to obtain the degree of Master of Science in Chemical Engineering

Academic year 2025-2026

Acknowledgments

I would like to express my sincere gratitude to my promoters, Prof. dr. ir. Kevin Van Geem and Prof. dr. ir. Christian Stevens, for the opportunity to undertake this thesis at the Laboratory for Chemical Technology, and for the trust and intellectual freedom they extended to me in exploring the intersection of vision-language models and chemical reaction extraction. Their willingness to support a topic at the boundary of two disciplines has been instrumental in shaping the direction of this work.

I am particularly indebted to my supervisor, Dr. ir. Maarten Dobbelaere, whose guidance has been central to every stage of this project. I am grateful for his sustained technical mentorship, for the development of Rxn-INSIGHT, which underpins the post-processing stage of this work, and for the considerable time he invested in our many discussions on the design and evaluation of the pipelines. His thoughtful feedback on scientific writing has likewise been of great value to me, and I have learned a great deal from working under his supervision.

I also wish to extend my sincere thanks to Dr. ir. Istvan Lengyel for our regular sessions, which extended well beyond the immediate scope of this thesis and from which I have benefited both scientifically and personally.

Finally, I am grateful to my family and friends for their unwavering support throughout an intense and demanding year, and to the broader open-source community, whose contributions made this thesis possible.

Kasper Huysentruyt

Ghent, May 2026

Declaration concerning the accessibility of the master thesis

Undersigned,

Kasper Marian Roland Huysentruyt

Graduated from Ghent University, academic year 2022-2023 and is author of the master thesis with title:

Agentic AI for Chemical Information Extraction from Scientific Publications

The author(s) gives (give) permission to make this master dissertation available for consultation and to copy parts of this master dissertation for personal use. In the case of any other use, the copyright terms have to be respected, in particular with regard to the obligation to state expressly the source when quoting results from this master dissertation.

May 2026

Kasper Marian Roland Huysentruyt

Preamble

In this thesis, artificial intelligence (AI) tools were used extensively across the research, the implementation, and the writing of the dissertation. AI assistance contributed to the design and coding of the agentic pipelines, to the execution and aggregation of benchmark runs across the evaluated models, to the systematic analysis of per-image failure modes, and to the drafting, editing, and structural refinement of the prose in every chapter. Given that the thesis itself studies agentic AI systems for chemical information extraction, this dual role of AI as both research subject and working tool was an intentional element of the project. The author retained responsibility for the research direction, the chemistry-domain reasoning, the interpretation of empirical results, and the verification of every AI-generated artefact, whether code, prose, or analytical output, before it was incorporated into the thesis.

A personal motivation underlies this choice. The author believes that students who decline to engage with AI and agentic tools will increasingly fall behind those who learn to use them well, and that the appropriate response is therefore not avoidance but disciplined investigation: figuring out how to use these tools in a way that still preserves and deepens the learning that a master's thesis is meant to produce, while achieving substantially more than would be possible without them. The methodology adopted throughout this dissertation reflects that conviction. AI was treated as an amplifier of the author's own reasoning rather than a replacement for it, with every generated artefact read, challenged, and verified before being accepted, so that the use of AI accompanied learning instead of bypassing it.

Agentic AI for Chemical Information Extraction from Scientific Publications

Kasper Marian Roland Huysentruyt

Master's dissertation submitted in order to obtain the academic degree of Master of
Science in
Chemical Engineering

Academic Year 2025-2026

Promotors: prof. dr. ir. Kevin M. Van Geem, prof. dr. ir. Christian V. Stevens
Counsellor: dr. ir. Maarten Dobbelaere

GHENT UNIVERSITY

Faculty of Engineering and Architecture

Department of Materials, Textiles and Chemical Engineering

Abstract

The chemical literature grows faster than it can be read. The CAS REGISTRY now exceeds 279 million substances, yet almost all of this chemistry stays trapped in figures and PDFs that machines cannot parse. The structured reaction records that retrosynthesis, yield-prediction and condition-prediction models depend on are still built by hand, a process that is slow, costly and prone to transcription error. Recent vision-language models, placed inside tool-using agentic loops, could automate this work, but whether extraction quality is set by the model itself or by the orchestration around it has not been clear. Here we build and benchmark four extraction pipelines on sixteen reaction figures drawn from five chemistry journals, ranging from a large multi-agent system to a single model equipped with two chemistry tools. We find that the model, not the framework, decides the outcome. Holding prompt, tools and output schema fixed, swapping only the underlying model produced a 0.61-point spread in product-level accuracy (intersection-over-union), far wider than any change caused by altering the orchestration. The simplest single-agent design surpassed every multi-step system, with Claude Opus 4.7 giving the best overall extraction quality and Gemini 3 Flash Preview matching it at roughly one-tenth the cost. These results suggest that progress in automated reaction extraction will come mainly from stronger vision encoders rather than from more complex agent architectures.

Keywords: Chemical reaction extraction, vision-language models, agentic AI, multi-agent systems, structured data extraction, cheminformatics, LLM tool use, benchmark evaluation.

Agentic AI for Chemical Information Extraction from Scientific Publications

Kasper Marian Roland Huysentruyt

Promotors: prof. dr. ir. Kevin M. Van Geem, prof. dr. ir. Christian V. Stevens
Counsellor: dr. ir. Maarten Dobbelaere

Abstract: The chemical literature grows faster than it can be read. The CAS REGISTRY now exceeds 279 million substances, yet almost all of this chemistry stays trapped in figures and PDFs that machines cannot parse. The structured reaction records that retrosynthesis, yield-prediction and condition-prediction models depend on are still built by hand, a process that is slow, costly and prone to transcription error. Recent vision-language models, placed inside tool-using agentic loops, could automate this work, but whether extraction quality is set by the model itself or by the orchestration around it has not been clear. Here we build and benchmark four extraction pipelines on sixteen reaction figures drawn from five chemistry journals, ranging from a large multi-agent system to a single model equipped with two chemistry tools. We find that the model, not the framework, decides the outcome. Holding prompt, tools and output schema fixed, swapping only the underlying model produced a 0.61-point spread in product-level accuracy (intersection-over-union), far wider than any change caused by altering the orchestration. The simplest single-agent design surpassed every multi-step system, with Claude Opus 4.7 giving the best overall extraction quality and Gemini 3 Flash Preview matching it at roughly one-tenth the cost. These results suggest that progress in automated reaction extraction will come mainly from stronger vision encoders rather than from more complex agent architectures.

Keywords: Chemical reaction extraction, vision-language models, agentic AI, multi-agent systems, structured data extraction, cheminformatics, LLM tool use, benchmark evaluation.

I. INTRODUCTION

The chemical literature is expanding at an unprecedented rate. The CAS REGISTRY exceeded 279 million unique substances in early 2025, up from 105 million a decade earlier [1], and global publication output doubles roughly every seventeen years [2]. Yet almost all of this chemistry remains locked inside PDFs whose text and figures are not intrinsically machine readable [3]. Trusted reaction databases such as Reaxys and SciFinder are still curated by hand, which is slow, expensive, and prone to transcription error [4]. As a result, only a small fraction of the published reaction record is available in a structured form that downstream models can consume (Figure 1), and a substantial share of the underlying primary literature additionally sits behind paywalls that further constrain reuse. The goal of this thesis is to evaluate whether modern agentic AI, in which a vision-language model (VLM) is wrapped in a tool-using decision loop, can close the gap between the published reaction record and the structured tables that retrosynthesis, condition prediction, yield regression models and reaction-class analysis tools such as Rxn-INSIGHT [5] require [6, 7].

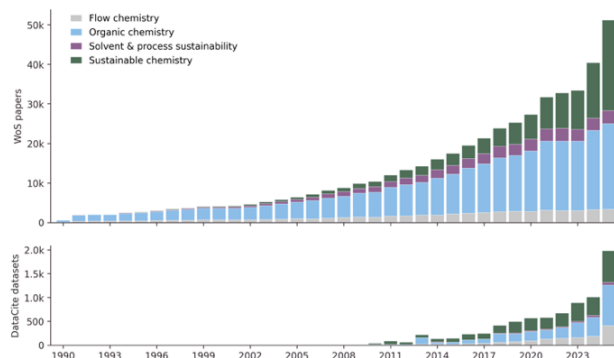


Figure 1: Number of research papers (top) versus structured datasets (bottom) deposited per year for four chemistry sub-fields. Even after two decades of growth in open data infrastructure, structured records lag publications by several orders of magnitude

Earlier generations of reaction-extraction pipelines relied on dictionary lookups, hand-written grammars and small machine-learning taggers trained on manually annotated corpora; tools such as ChemicalTagger, ChemDataExtractor and OSCAR exemplify this generation [4, 8, 9]. They achieve high accuracy on narrowly scoped inputs but scale poorly across the heterogeneous formats of contemporary chemistry papers. The advent of large language models (LLMs) and VLMs has shifted the landscape: a single multimodal model can now read a reaction figure end to end without task-specific fine tuning. Agentic frameworks extend this further by giving the model a decision loop, structured output schemas, and external tools (RDKit, optical chemical structure recognition (OCSR), and name-to-structure resolvers), so that symbolic chemistry primitives remain available alongside the model's generative reasoning [7]. The recently published ChemEAGLE pipeline [10] is representative of this shift and reports state-of-the-art numbers on its own benchmark, but its design space (number of agents, choice of backbone model, division of labour between deterministic tools and the VLM) has barely been mapped.

This thesis takes that mapping as its central research question: on a shared benchmark of sixteen chemistry figures, how much of the extraction quality is decided by the multimodal model itself, and how much by the orchestration that surrounds it? To answer this, four extraction pipelines were built and evaluated under identical conditions: (i) the published ChemEAGLE pipeline reinstalled locally as the reference baseline; (ii) an original multi-agent framework that integrates a deterministic R-group decomposition layer absent from ChemEAGLE; (iii) ChemEAGLE Hybrid, a variant designed independently; and (iv) a single-agent VLM pipeline instantiated behind nine multimodal models (Claude Opus 4.6 and 4.7, GPT-5.4 and 5.5, three Gemini variants, xAI Grok 4.3,

and the open-weights Qwen3.6-27B). Every system writes to the same Pydantic schema, is scored against the same ground truth, and is benchmarked on quality, cost, wall time and per-system failure modes.

II. DATA AND METHODS

A. Benchmark

All systems are evaluated on a curated 16-image subset of the GT3 ground-truth corpus published alongside ChemEAGLE [10]. The images cover reaction-scope tables, condition-survey panels and picture-style schemes, drawn from five journals. Each figure has been cropped from its journal page so no surrounding running text is available, which isolates the visual extraction task from out-of-figure context. The ground-truth schema is a flat list of reactions, each carrying reactant SMILES, product SMILES, and a conditions list whose role field is drawn from a closed enumeration covering reagent, solvent, catalyst, temperature, time, yield, atmosphere, loading and pressure. R-group placeholders are encoded as atom-mapped wildcards ([1*], [2*]). Sixteen figures are too few for fine-grained ranking, but sufficient for the qualitative cross-system comparison this thesis pursues, and small enough to keep the inference cost of a nine-model sweep within budget.

B. Evaluation Methodology

Every system is scored against the same ground truth with the same evaluation harness. SMILES are canonicalised through RDKit before comparison; chirality is stripped because some ground-truth files use abbreviated stereo notation. Per-image reactant and product intersection over union (IoU) are micro-averaged Jaccard indices on the corresponding SMILES sets. Reaction-level matches are then computed under two

bucketing, SMILES-based name unification) and roughly doubles the strict scores, the gap interpreted as formatting noise rather than chemistry error.

C. ChemEAGLE Reference Baseline

ChemEAGLE was reinstalled locally as the reference baseline. It coordinates a Planner with six specialist agents (template parser, molecular recognition, R-group substitution, condition interpretation, text extraction, data structuring) under two observer modules, and integrates Microsoft Florence-2 for figure detection, OCSR via MolNexTR [11], and OPSIN/PubChem/CIR for name-to-structure resolution [12-14]. Despite the published 0.78 soft-F1 on its own benchmark, the local rerun reached only 0.29 soft-F1 and crashed on four of the sixteen images inside the MolNexTR canonicalization step. The reproduction gap motivated two parallel research lines: an alternative multi-agent orchestration over the same chemistry toolchain, and a single-agent baseline that strips orchestration away entirely.

D. Original Multi-Agent Framework

The original multi-agent framework wires seven agents into a Selector Group Chat [15] team in which five specialist agents emit plain-text findings cards on a shared image, the Data Structure agent assembles a single Pydantic-valid JSON, and a Verifier critiques and re-triggers up to two retry cycles (Figure 2). The development progressed through ten numbered phases. Three changes carried most of the headline gains: (a) a Pydantic schema with a retry loop lifted schema validity from below 0.50 to 1.00; (b) replacing the VLM’s ad-hoc molecule reading with a per-bounding-box MolDetect [16] and MolNexTR routine (Phase 8) doubled product IoU; and (c) writing a deterministic RDKit [17] RGroupDecompose-and-molzip chain (Phase 9)

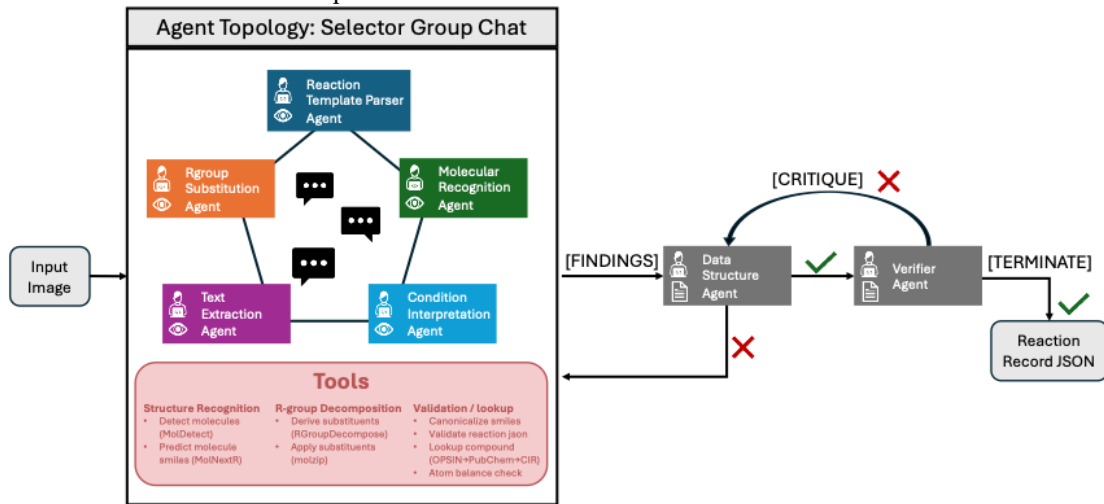


Figure 2: Topology of the original multi-agent framework. Five specialist agents (top row) emit findings cards on the shared image; the Data Structure agent is the only JSON producer; the Verifier critiques and re-prompts up to two retries before forced termination.

criteria: a soft match requires the reactant and product sets to coincide after canonicalization; a hard match additionally requires the condition set to match. A bipartite (Hopcroft–Karp) matcher aligns predicted and ground-truth reactions to neutralise ordering, and precision, recall and F1 are reported under both criteria together with a partial-match F1 based on a Jaccard-0.5 threshold and a constitution-F1 with stereochemistry stripped. Conditions are scored separately under strict and lenient matchers; the lenient matcher absorbs cosmetic differences (whitespace, parenthesisation, role

raised reactant IoU on the canonical R-group case from 0 to 1.0. The combined posture, constrain the output, delegate perception to specialist deep-learning models, and reserve symbolic manipulation for deterministic code, is the design principle that the original multi-agent framework operationalises.

E. Single-Agent Pipeline

The single-agent pipeline strips the orchestration to its simplest form: one multimodal model with two tools (an

RDKit-backed SMILES validator and a Pydantic-validated extraction terminator) and one system prompt that mirrors Anthropic’s prompt-engineering recommendations for Opus 4.7 [18]. The agent loop is implemented once per provider and shares the same schema, the same tools and the same prompt across nine models: Claude Opus 4.6 and 4.7 via the Anthropic SDK [18, 19]; GPT-5.4 and 5.5 via Microsoft Azure AI Foundry [20, 21]; Gemini 3.1 Pro Preview, Gemini 3 Flash Preview and Gemini 3.5 Flash via the Google Generative AI SDK [22-24]; xAI Grok 4.3 via the xAI API [25]; and the open-weights Qwen3.6-27B served through OpenRouter [26]. The five provider adapters differ only in their wire-format translation layer, which means any quality difference across models is attributable to the model itself rather than to changes in agent design. Substrate-scope figures (Figure 3) are particularly demanding because most variants must be reconstructed by structural reasoning from the products rather than read directly off the page; they form the discriminating case in the cross-system comparison.

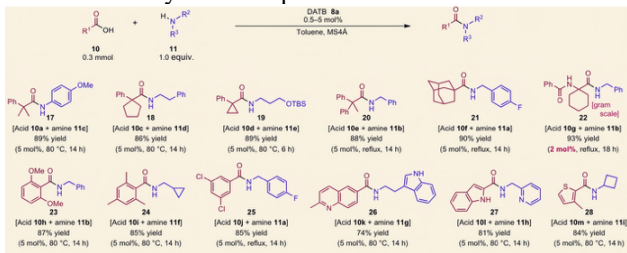


Figure 3: Substrate-scope figure (NC_2017, page 4). Only the generic template (top) and concrete products (bottom) are drawn; variant reactants must be reconstructed by the deterministic RDKit R-group chain (Phase 9).

III. RESULTS AND DISCUSSION

A. Phase Progression of the Multi-Agent Framework

Inside the multi-agent framework, the phase-by-phase progression on the canonical R-group case (Figure 4) is the cleanest illustration of where extraction quality is decided. Product IoU stabilises at 1.0 once MolNexTR is invoked once per detected bounding box (Phase 8); reactant IoU on the same image only reaches 1.0 after the deterministic RDKit chain replaces VLM reasoning (Phase 9). Both steps come from delegating sub-tasks to specialised models or deterministic code rather than to the VLM.

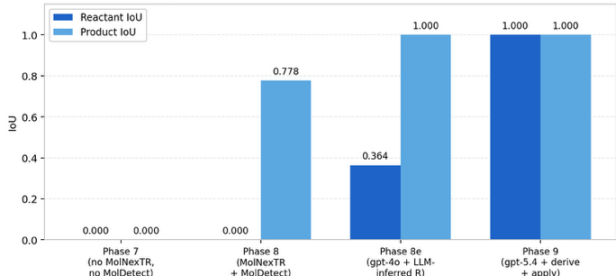


Figure 4: Phase progression on a reaction image with 8 reactions. Product IoU stabilises at 1.0 with per-bbox MolNexTR (Phase 8); reactant IoU reaches 1.0 only after the deterministic RDKit chain (Phase 9).

B. Cross-System Comparison

On the full sixteen-image benchmark, all twelve evaluated systems land in a heatmap that puts the strong single-agent variants clearly above every multi-step baseline on the structural metrics (Figure 5). Opus 4.7 achieves the highest

Partial F1 of 0.85, surpassing all other pipelines. However, Gemini 3 Flash Preview attains product IoU of 0.79 and Claude Opus 4.7 attains 0.76, both about 0.21-0.24 above the strongest multi-step pipeline (the original multi-agent framework at 0.55). Within the single-agent family, however, the spread is dramatic: GPT-5.4 collapses to product IoU of 0.18, the lowest in the table. This is due to consistent vision-encoder misreads of specific atom-count and connectivity motifs. Identical agent infrastructure, holding prompt, tools and schema constant, therefore produces a 0.61-point absolute spread on product IoU when only the multimodal model is varied. This spread is substantially larger than any orchestration-driven gap observed in the comparison, and it is what motivates the central claim of this thesis: on this benchmark, the multimodal model itself is the dominant determinant of extraction quality.

	Product IoU	Reactant IoU	Soft F1	Partial F1 (≥ 0.5)	Cond. recall (len=)
multi_agent	55%	39%	22%	37%	77%
ChemEagle	39%	45%	29%	39%	55%
ChemEagle_Hybrid	45%	32%	18%	20%	32%
Opus 4.7 *	76%	68%	60%	85%	77%
GPT-5.4 *	18%	24%	9%	27%	71%
Gemini 3.1 Pro Preview *	57%	59%	40%	83%	85%
Gemini 3 Flash Preview *	79%	57%	46%	79%	81%
Grok 4.3 *	64%	53%	42%	78%	74%
Opus 4.6 *	55%	55%	41%	64%	78%
GPT-5.5 *	62%	56%	43%	72%	76%
Qwen3.6-27B *	40%	54%	40%	66%	80%
Gemini 3.5 Flash *	62%	49%	38%	80%	82%

Figure 5: Per-system heatmap of the 12 evaluated extraction systems on the 16-image benchmark (colour scaled per column, rows marked with * are single-agent runs). Strong single-agent variants (Opus 4.7, GPT-5.5, Gemini family, Grok 4.3) outperform every multi-step baseline on the structural metrics.

C. Cost-Quality Trade-off

The cost-quality picture (Figure 6) makes the practical case. Claude Opus 4.7 leads on partial-F1 at 0.85 but at \$0.22 per image; Gemini 3 Flash Preview reaches partial-F1 0.79 at \$0.018 per image, the cheapest competitive option in the comparison; Gemini 3.5 Flash sits between them at \$0.055 per image with a partial-F1 of 0.80; and the open-weights Qwen3.6-27B served via OpenRouter sits near the fitted line at \$0.085 per image with a partial-F1 of 0.66. The original multi-agent framework is clearly off the Pareto front: the highest cost in the table for a partial-F1 below 0.40. Wall time and token spend tell the same story (Figure 7 and Figure 8): Opus 4.7 is simultaneously the fastest single-agent variant (41s per image) and the highest scoring, with no monotonic trend linking tokens spent to quality (R^2 of the log-linear fit is essentially zero, $R^2 = 0.00$). The choice of model and pipeline shape dominates raw resource budget on this task.

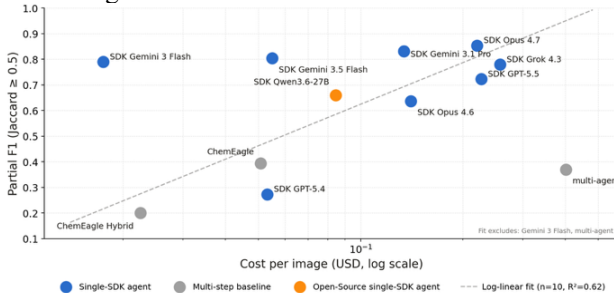


Figure 6: Cost vs. partial-F1 (log x). Claude Opus 4.7 leads on partial-F1, but at a high cost compared to Gemini 3 Flash. Gemini 3 Flash Preview and the multi-agent framework are excluded as outliers from the trend ($R^2=0.62$).

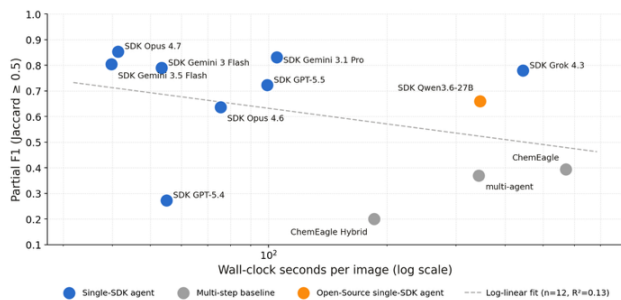


Figure 7: Wall-clock seconds vs. partial-F1 (log x). Opus 4.7 leads on both quality and speed (41 s/image); the trend is slightly negative ($R^2 = 0.13$).

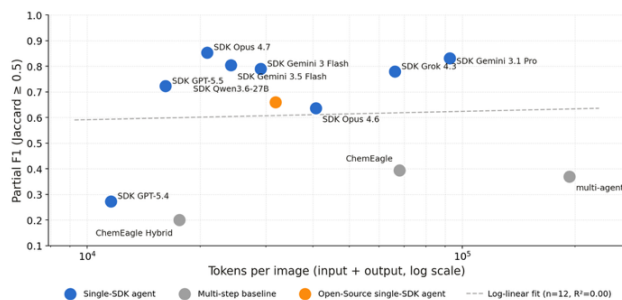


Figure 8: Tokens per image vs. partial-F1 (log x). Multi-agent consumes the most tokens per image. The log-linear fit is essentially flat ($R^2 = 0.00$); token spend is not predictive of quality.

D. Failure Modes

Per-image inspection of the low-scoring cases attributes most multi-agent failures to two structural weaknesses. First, MolDetect was trained on chemistry-figure crops and treats dense text fragments as molecule boxes when fed page-cropped scans; MolNexTR then fabricates RDKit-valid but chemically wrong SMILES, and the verifier loop never fires. Second, on dense substrate-scope grids the agent sometimes deposits intermediate R-group fragments into the output JSON without re-invoking the apply substituents step, a non-deterministic adherence failure that no prompt tightening resolved. On the single-agent side, the four-image Opus 4.6 versus Opus 4.7 case study traces the headline gap to perception events at the input stage (an isopropyl group read as methyl, a CH_2 linker read as a carbonyl, a benzothiophene confused with a thiophene). When vision fails, Opus 4.6 falls back to chemically plausible priors from training data rather than to low-confidence output; Opus 4.7 with its 3.75-megapixel image cap reads the page more literally. The implication is that improvements in the vision encoder, not in the orchestration around it, are what minimises the remaining quality gap on this benchmark.

IV. CONCLUSION AND FUTURE WORK

A. Conclusion

The multimodal model itself is the dominant variable in agentic chemical reaction extraction: identical agent infrastructure produces a 0.61-point spread in product IoU when only the model is varied, much larger than the spread produced by varying the orchestration at fixed model. The framework matters when the model is weak; the model matters more than the framework when the model is strong. Among the evaluated systems, Claude Opus 4.7 achieves the highest quality at the highest cost, Gemini 3 Flash Preview is the

cheapest competitive option, and Gemini 3.5 Flash sits between them with full schema coverage at a quarter of Opus 4.7's per-image cost. Every multi-step pipeline, including the original multi-agent framework, ChemEAGLE and ChemEAGLE Hybrid, is outperformed in partial-F1 by the best single-agent configurations at comparable or lower cost. The multi-agent value proposition is therefore no longer highest partial-F1 but agent-level auditability and guaranteed SMILES validity through an enforced RDKit round-trip, which matter for downstream regulatory documentation and integration with structural verifiers.

B. Future Work

Three follow-on directions are flagged. First, the benchmark itself should be extended with multi-annotator ground truth and inter-annotator agreement scores, both to break the dependence on the sample and to support fine-grained model ranking. Second, the proprietary models that currently dominate the cost-quality Pareto are accessed through provider APIs and have a finite useful lifetime; distilling the Opus 4.7 single-agent behaviour into a smaller open-weights student (Qwen3.6-27B as the natural first target) would make the pipeline durable against vintage shifts and enable on-premises deployment for groups whose data cannot leave the lab. Third, the cost-quality Pareto front motivates hybrid routing in which the agent decides per image which model to query and cascades from a fast model to a more expensive one only when the fast model's confidence is low. Beyond the benchmark, the cost numbers reported here (\$0.02 to \$0.22 per image, under one minute per image with Gemini 3 Flash Preview) are low enough that the extractor can live inside interactive chemist workflows and inside the literature-conditioning step of the emerging autonomous-lab platforms that closed-loop chemistry.

V. REFERENCING

- (CAS), C.A.S. *Annual Report 2015*. 2016. 29 oct 2025].
- Larsen, P. and M. Von Ins, *The rate of growth in scientific publication and the decline in coverage provided by Science Citation Index*. *Scientometrics*, 2010. **84**(3): p. 575–603.
- Rajan, K., et al., *DECIMER-Segmentation: Automated extraction of chemical structure depictions from scientific literature*. *Journal of Cheminformatics*, 2021. **13**(1): p. 20.
- Hawizy, L., et al., *ChemicalTagger: A tool for semantic text-mining in chemistry*. *Journal of Cheminformatics*, 2011. **3**(1): p. 17.
- Dobbelaere, M.R., et al., *Rxn-INSIGHT: fast chemical reaction analysis using bond-electron matrices*. *Journal of Cheminformatics*, 2024. **16**(1): p. 37.
- Schilling-Wilhelmi, M., et al., *From text to insight: large language models for chemical data extraction*. *Chemical Society Reviews*, 2025. **54**(3): p. 1125–1150.
- Schwaller, P., et al., *Machine intelligence for chemical reaction space*. *WIREs Computational Molecular Science*, 2022. **12**(5): p. e1604.
- Mavračić, J., et al., *ChemDataExtractor 2.0: Autopopulated Ontologies for Materials Science*. *Journal of Chemical Information and Modeling*, 2021. **61**(9): p. 4280–4289.
- Jessop, D.M., et al., *OSCAR4: a flexible architecture for chemical text-mining*. *Journal of Cheminformatics*, 2011. **3**(1): p. 41.
- Chen, Y. and C.Y.F. Group, *ChemEagle: A Multi-Agent Pipeline for Structured Reaction Extraction from Chemistry Figures*. *arXiv preprint*, 2025. **2507.20230**.
- Chen, Y., et al., *MolNexTR: A generalized deep learning model for molecular image recognition*. *Research Square*, 2024.
- Lowe, D.M., et al., *Chemical Name to Structure: OPSIN, an Open Source Solution*. *Journal of Chemical Information and Modeling*, 2011. **51**(3): p. 739–753.
- Kim, S., et al., *PubChem substance and compound databases*. *Nucleic acids research*, 2016. **44**(D1): p. D1202–D1213.
- NCI/CADD Chemical Identifier Resolver.
- Selector Group Chat — AutoGen.

16. Qian, Y., et al., *RxnScribe: A Sequence Generation Model for Reaction Diagram Parsing*. Journal of Chemical Information and Modeling, 2023. **63**(13): p. 4030–4041.
17. Landrum, G., *RDKit: Open-source cheminformatics*. 2024, RDKit.
18. *Introducing Claude Opus 4.7*.
19. *Claude Opus 4.6*.
20. *GPT-5.4 Model | OpenAI API*.
21. *GPT-5.5 Model | OpenAI API*.
22. *Gemini 3.1 Pro Preview | Gemini API*. Google AI for Developers.
23. *Gemini 3 Flash | Gemini Enterprise Agent Platform*. Google Cloud Documentation.
24. *Gemini 3.5 Flash*. Google DeepMind.
25. *Grok 4.3 | xAI Docs*.
26. *Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model*. Alibaba Cloud Community.

Table of Contents

NOMENCLATURE	IV
CHAPTER 1. INTRODUCTION	1
1.1 UNSTRUCTURED CHEMICAL DATA	2
1.2 REACTION DATABASE APPLICATIONS	4
1.3 THE POTENTIAL OF AGENTIC AI	6
1.4 OUTLINE	7
1.5 REFERENCES	8
CHAPTER 2. AUTOMATED EXTRACTION OF CHEMICAL REACTION PROCEDURES	11
2.1 INTRODUCTION	13
2.2 REPORTING OF CHEMICAL REACTION DATA	14
2.2.1 <i>Scientific Literature</i>	14
2.2.2 <i>Patents</i>	18
2.3 MACHINE-READABLE REACTION DATA FORMATS	19
2.3.1 <i>Strings</i>	19
2.3.2 <i>Other Reaction Data Exchange Formats</i>	23
2.4 THE EVOLUTION OF CHEMICAL REACTION EXTRACTION TOOLS	28
2.4.1 <i>The Roots</i>	28
2.4.2 <i>Chronological Overview of Chemical Data Extraction</i>	29
2.5 GENERAL PIPELINE FOR EXTRACTING CHEMICAL DATA	31
2.5.1 <i>Preprocessing</i>	32
2.5.2 <i>Tokenisation</i>	33
2.5.3 <i>Feature Processing</i>	34
2.5.4 <i>Chemical Named Entity Recognition</i>	35
2.5.5 <i>Parsing</i>	36
2.5.6 <i>Normalization Step and Relation Extraction</i>	37
2.6 OVERVIEW CHEMICAL REACTION EXTRACTION TECHNIQUES	39
2.6.1 <i>Dictionary-Based Systems</i>	39
2.6.2 <i>Rule-Based Systems</i>	40
2.6.3 <i>Machine-Learning Systems</i>	43
2.6.4 <i>Optical Chemical Structure Recognition (OCSR)</i>	53
2.6.5 <i>Hybrid and Multi-Modal Systems</i>	58

2.7	LLMs AND VLMS AS NEW TOOLS FOR CHEMICAL REACTION DATA EXTRACTION	59
2.7.1	<i>Conceptual Positioning Relative to Classical ML and Hybrid Systems</i>	60
2.7.2	<i>LLM/VLM-based Extraction Methodology</i>	61
2.7.3	<i>Strengths and Limitations</i>	69
2.8	EVALUATION CRITERIA AND METRICS	70
2.9	CONCLUSION AND CURRENT CHALLENGES	72
2.10	REFERENCES	74
CHAPTER 3.	DATA AND METHODS	86
3.1	INTRODUCTION	87
3.2	DATA	87
3.3	SOFTWARE	89
3.3.1	<i>ChemEAGLE</i>	89
3.3.2	<i>Multi-Agent Framework</i>	95
3.3.3	<i>ChemEAGLE Hybrid</i>	103
3.3.4	<i>Single-Agent Pipeline</i>	104
3.3.5	<i>LeMath-synth</i>	113
3.3.6	<i>Rxn-INSIGHT</i>	116
3.4	CLOSED VERSUS OPEN-SOURCE MODELS	119
3.4.1	<i>Closed-Source Models</i>	120
3.4.2	<i>Open-Source Models</i>	123
3.4.3	<i>Hardware</i>	125
3.5	COST CALCULATIONS	125
3.6	BENCHMARK EVALUATION METHODOLOGY	126
3.6.1	<i>Evaluation Methodology</i>	126
3.7	REFERENCES	130
CHAPTER 4.	RESULTS AND DISCUSSION	133
4.1	INTRODUCTION	134
4.2	CHEMEAGLE	135
4.2.1	<i>Reproduction Outcome</i>	135
4.2.2	<i>VLM Evaluation in ChemEAGLE</i>	136
4.3	MULTI-AGENT FRAMEWORK	137
4.3.1	<i>Phase Progression</i>	137
4.3.2	<i>Multi-Agent versus ChemEAGLE on the Sixteen-Image Benchmark</i>	139
4.3.3	<i>Multi-Agent with Claude Opus 4.7</i>	140
4.3.4	<i>Failure-mode Case Studies</i>	142
4.3.5	<i>Multi-Agent Limitations</i>	144
4.4	SINGLE AGENT	144

4.4.1	<i>Tool-Use Behaviour across Providers</i>	145
4.4.2	<i>Redesigned Prompt</i>	146
4.4.3	<i>Opus 4.6 versus Opus 4.7 Case Studies</i>	146
4.4.4	<i>Input-Image Resolution Ablation</i>	154
4.5	CROSS-SYSTEM EVALUATION	157
4.6	LE MATH-SYNTH	167
4.6.1	<i>Test Case</i>	167
4.7	DISCUSSION	168
4.7.1	<i>Model Choice as the Dominant Determinant of Extraction Quality</i>	169
4.7.2	<i>Comparative Performance and Residual Advantages of the Multi-Agent Pipeline</i>	170
4.8	REFERENCES	171
CHAPTER 5.	CONCLUSION AND FUTURE WORK	172
5.1	INTRODUCTION	173
5.2	LIMITATIONS	173
5.3	CONCLUSIONS	174
5.4	FUTURE WORK	177
5.4.1	<i>Benchmark and Evaluation</i>	177
5.4.2	<i>Reactant-Side Improvements</i>	178
5.4.3	<i>Architecture Experiments</i>	178
5.4.4	<i>Knowledge Distillation</i>	179
5.5	REFERENCES	180
CHAPTER 6.	SOCIETAL REFLECTION	182
6.1	INTRODUCTION	183
6.2	SUSTAINABILITY AND THE SUSTAINABLE DEVELOPMENT GOALS	183
6.3	ETHICAL IMPLICATIONS	184
6.4	SCIENTIFIC AND EPISTEMIC IMPACT	185

Nomenclature

Abbreviation	Definition
ACS	American Chemical Society
AI	Artificial Intelligence
API	Application Programming Interface
BBOX	Bounding Box
BE	Bond-Electron matrix representation
BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory
CAS	Chemical Abstracts Service
CDXML	ChemDraw Extensible Markup Language
CIR	Chemical Identifier Resolver (NCI/CADD)
CML	Chemical Markup Language
CNER	Chemical Named Entity Recognition
CNN	Convolutional Neural Network
CoT	Chain-of-Thought
CRF	Conditional Random Field
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
DCM	Dichloromethane
DL	Deep Learning
ECFP	Extended-Connectivity Fingerprint
F1	F1 score (harmonic mean of precision and recall)
FAIR	Findable, Accessible, Interoperable, Reusable (data principles)
GED	Graph Edit Distance
GPT	Generative Pre-trained Transformer

GPU	Graphics Processing Unit
GT	Ground Truth
HPC	High-Performance Computing
HRMS	High-Resolution Mass Spectrometry
HTTP	HyperText Transfer Protocol
InChI	International Chemical Identifier
IoU	Intersection over Union
IUPAC	International Union of Pure and Applied Chemistry
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
LSTM	Long Short-Term Memory
MACCS	Molecular ACCess System (chemical fingerprint set)
MCP	Model Context Protocol
MCS	Maximum Common Substructure
MEMM	Maximum Entropy Markov Model
ML	Machine Learning
MLLM	Multimodal Large Language Model
MolDetect	Pix2Seq-style molecule bounding-box detector
MolNexTR	Swin-Transformer image-to-SMILES model
NER	Named Entity Recognition
NLP	Natural Language Processing
OCR	Optical Character Recognition
OCSR	Optical Chemical Structure Recognition
OPSIN	Open Parser for Systematic IUPAC Nomenclature
ORD	Open Reaction Database
PDF	Portable Document Format
PEFT	Parameter-Efficient Fine-Tuning
POS	Part-of-Speech (tagging)

RAG	Retrieval-Augmented Generation
RDKit	Open-source cheminformatics toolkit
RInChI	Reaction International Chemical Identifier
RNN	Recurrent Neural Network
Rxn-INSIGHT	Reaction-analysis toolkit using bond-electron matrices
SDK	Software Development Kit
SFT	Supervised Fine-Tuning
SMARTS	SMILES Arbitrary Target Specification
SMILES	Simplified Molecular-Input Line-Entry System
SMIRKS	SMILES reaction transformation language (extends SMARTS)
SVM	Support Vector Machine
USD	United States Dollar
USPTO	United States Patent and Trademark Office
VDU	Visual Document Understanding
VLM	Vision-Language Model
XML	Extensible Markup Language

Chapter 1.

Introduction

1.1	UNSTRUCTURED CHEMICAL DATA	2
1.2	REACTION DATABASE APPLICATIONS	4
1.3	THE POTENTIAL OF AGENTIC AI	6
1.4	OUTLINE	7
1.5	REFERENCES	8

1.1 Unstructured Chemical Data

The Internet has greatly influenced the chemical publication environment, [1] not only when considering the access and distribution of published literature, but also during the actual review and writing process. The scientific literature continues to expand at a high rate [2], with cross-database analysis reporting an average annual growth rate of 4.10%, suggesting that global publications double approximately every 17 years. Chemistry contributes significantly to this surge: the CAS REGISTRY® (Chemical Abstracts Service) exceeded 279 million unique substances in early 2025, up from just over 105 million a decade earlier, with CAS scientists adding 13.5 million structures in 2015 alone [3].

Most of this chemical information remains encapsulated in PDFs whose text and figures are not intrinsically machine-readable, creating a critical bottleneck for large-scale data utilisation [4]. Recently there have been many advances in research data management, nevertheless only a minuscule fraction of all available research data is available in a usable structured form (see Fig. 1). Thus, there is a large untapped potential in leveraging this unstructured text data [5].

Furthermore, most of the trusted data sets that exist are created by trained human curators who manually read and extract chemical information (e.g. substances, reactions, properties) from journal articles and other sources [6]. Companies like Elsevier and CAS [7] sell these services and human-curated scoped collections of scientific data, respectively SciFinder [8] and Reaxys [9], to speed up chemical research. Apart from the considerable labour cost and delay after the original publication, human action is also a considerable source of error and data corruption [10].

Compounding these issues, much of the underlying primary literature sits behind paywalls, which not only restricts access for researchers at institutions without costly subscriptions but also limits the broader scientific community's ability to verify, reproduce, and build upon prior work, contrary to the cumulative nature on which scientific progress depends. Open-access preprint servers such as arXiv [11], and its chemistry-focused counterpart ChemRxiv [12], partially mitigate this barrier by allowing authors to share their work freely before or alongside formal publication, providing the

open-source and machine-learning communities with a steadily growing corpus of accessible scientific text and figures to build upon.

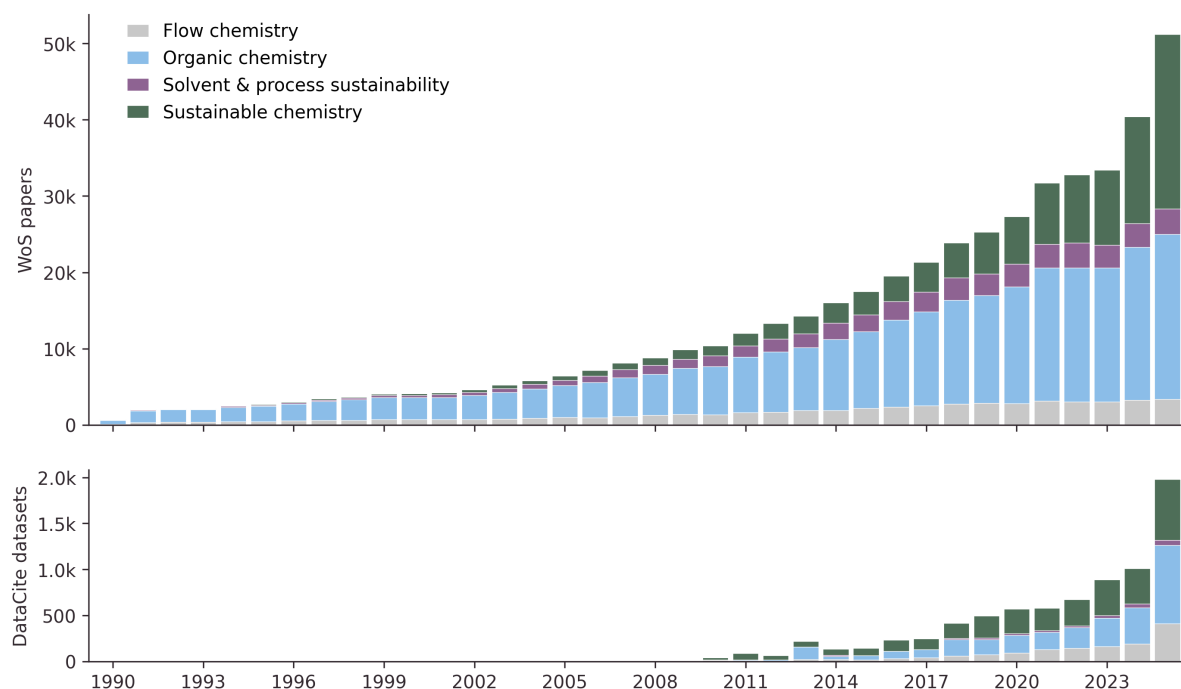


Figure 1-1: Number of research papers vs. datasets deposited in data repositories in materials science and chemistry per year. The top graph shows the number of publications from 1996 to 2023. The number of records was obtained from the search queries for Flow chemistry (“Flow chemistry”, “continuous flow synthesis”, “microfluidic reactor chemistry” and “microreactor synthesis”), Organic synthesis (“organic synthesis and “synthetic methodology”), Solvent & process sustainability (“aqueous organic synthesis”, “bio-based solvent” and “solvent-free synthesis”) and Sustainable chemistry (“environmentally benign synthesis”, “green chemistry” and “sustainable synthesis”) in the Web of Science Core Collection on February 2026. The graph below shows the number of datasets in chemistry and materials science deposited in the DataCite repositories from 1996 to 2023. The number of records was obtained from similar queries by restricting the document type to “dataset”. Note the different y-axis scale between the top and bottom graphs. While this figure highlights the large difference in the availability of structured datasets compared to papers, we note that a one-to-one comparison of these numbers is not always fair. This is because sometimes multiple papers can be used to create a single dataset, as in the case of curated databases, or vice versa, where multiple structured datasets can result from a single paper's work [5].

As a result of these challenges of manual curation and the promise of leveraging data “hidden” in scientific articles (interconnectivity), researchers became motivated to develop dedicated extraction pipelines over the past two decades [13, 14]. These pipelines often relied on dictionary and rule-based approaches [6, 15-17], or smaller semantic machine learning (ML) models trained on manually annotated corpora [18-20]. Those approaches, however, face challenges with the diversity of topics and reporting formats in chemistry and materials research as they are hand-tuned for very specific

use cases. All things together, automating the extraction for one particular case might not be too difficult, but the sheer scale of possible variations makes the overall problem intractable [5].

1.2 Reaction Database Applications

Reaction databases are useful for a variety of applications. Figure 1-2, adapted from Schwaller *et al.* [21], organises these applications into seven canonical tasks that span most of contemporary chemical reaction informatics.

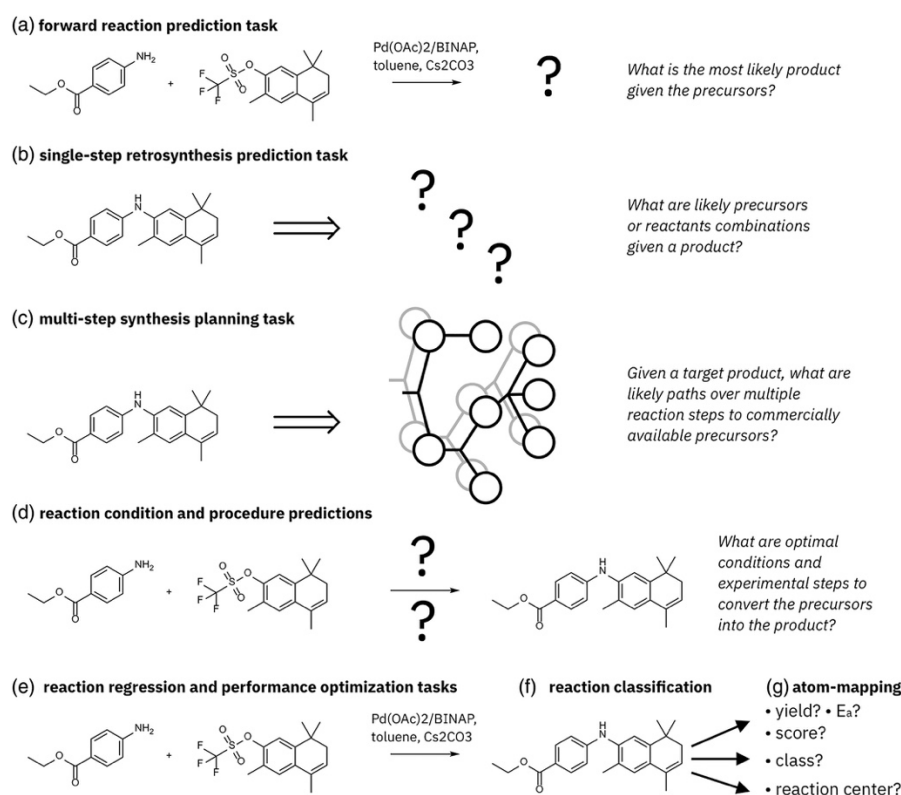


Figure 1-2: Overview of the principal machine intelligence tasks defined over chemical reaction space. (a) Forward reaction prediction infers the most likely product given a set of precursors. (b) Single-step retrosynthesis suggests likely precursor combinations that would yield a target product. (c) Multi-step synthesis planning chains retrosynthetic steps until commercially available building blocks are reached. (d) Reaction condition and procedure prediction proposes the catalysts, solvents, temperatures and experimental operations required to realise a transformation. (e) Reaction regression and performance optimisation tasks predict scalar outcomes such as yield, activity, selectivity or activation energy. (f) Reaction classification assigns a transformation to a named class or template. (g) Atom-mapping links atoms in the reactants to their counterparts in the products and is the prerequisite for reaction-centre and template extraction. Adapted from Schwaller *et al.* [21].

Forward reaction prediction (panel a) asks what product is most likely given a set of precursors [21]. Reliable forward prediction allows chemists to triage hypothetical transformations *in silico* before committing them to bench experiments [22].

Single-step retrosynthesis (panel b) inverts that task and proposes plausible precursors for a given target product. Template-based, graph-edit-based and sequence-based formulations have all been studied; their performance is largely set by the diversity, balance and atom-mapping quality of the reaction dataset on which the model is trained [21]. However, the use cases of single-step retrosynthesis methods are limited as most synthesis pathways require many reaction steps to produce the desired product [23, 24]. Multi-step synthesis planning (panel c) chains single-step retrosynthetic predictions with a search algorithm, typically Monte Carlo tree search, until a route to commercially available building blocks is found. Open-source platforms expose this pipeline to synthetic chemists. Their suggestions are only as good as the database from which the underlying single-step model has been trained.[21-23]

Reaction condition and procedure prediction (panel d) addresses how to run a reaction. Condition models propose catalysts, solvents and temperature given the product and a reaction fingerprint, while procedure-level models go further and predict the discrete operations a chemist or a robotic platform would execute, such as the order of addition, filtration, work-up and dropwise addition. Reaction performance prediction (panel e) treats the yield, activity, selectivity (chemo-, regio-, stereo- and enantioselectivity) or activation energy of a reaction as a regression target conditioned on the structural and condition features of the entry. High-throughput experimentation (HTE) campaigns produce particularly clean training data and have enabled successful models on Buchwald-Hartwig, Suzuki-Miyaura and other cross-couplings [21]. Literature-mined sets remain biased toward high-yield outcomes (positive publication bias [25]), which limits extrapolation to unsuccessful or moderate-yield conditions.[21, 24]

Reaction classification (panel f) groups each entry into a named class, for example Buchwald-Hartwig amination, or into one of thousands of reaction templates. Rule-based tools operate on atom-mapped reactions and recognise more than 1800 classes; neural classifiers learn class boundaries directly from reaction SMILES and additionally produce a data-driven fingerprint that

can be visualised as an interactive reaction atlas [21]. Reliable classification is what turns a flat list of reactions into a query able taxonomy. Atom-mapping (panel g) identifies which atom in a reactant becomes which atom in the product. It is the prerequisite for extracting reaction centres, bond changes and templates and therefore underpins most of the tasks above.

Across these seven applications the demands on the underlying database can be reduced to a small set of extractable elements. Every record needs the structures of the reactants and the products, preferably stored as canonical SMILES so that atom-mapping, template extraction and classification can be run automatically. The reagents (catalysts, ligands and additives that take part in the reaction without contributing atoms to the product) are required as soon as forward, retrosynthetic, procedure or performance models move beyond the simplest pair of precursors. Quantitative conditions, namely temperature, time, atmosphere, pressure and catalyst loading, are required for procedure and performance prediction. The reported yield closes the loop for performance regression, and an explicit reaction-class or template label, if available, accelerates classification and is a useful filter on the training corpus. In summary, a record that captures reactants, reagents, products, conditions (reagent, solvent, catalyst, temperature, time, yield, atmosphere, loading, pressure) and an optional reaction class is sufficient to support every application illustrated in Figure 1-2.

1.3 The Potential of Agentic AI

Agents AI couples vision language models (VLM) models and large language models (LLM) with an explicit decision loop and a set of external tools. An agent decides which tool to call, inspects the result, and revises its reasoning before producing a final structured output [25]. The underlying models have become sufficiently capable at reading and structuring chemistry that they can be applied to new corpora without task-specific fine-tuning [5]. Agent can leverage the symbolic strengths older extraction methods as specific tools, such as dictionary lookup, IUPAC name parsing, and rule-based pattern matching, and combine it with the flexibility of an LLM without rebuilding any of those components. This offers a scalable alternative to the dictionary and grammar-based pipelines that dominated the previous generation of extraction tools [26].

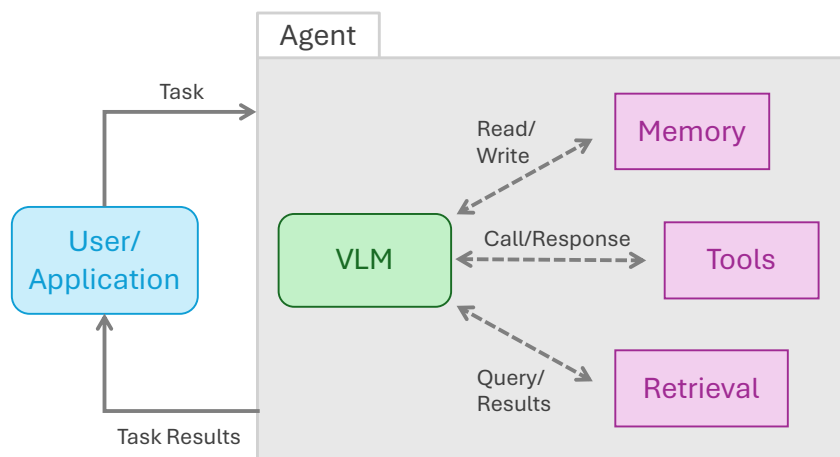


Figure 1-3: Schematic of a vision-language model (VLM) agent. The agent receives a task from a user or external application and returns task results upon completion. Internally, the VLM acts as the reasoning core, coordinating three auxiliary modules: a memory component for persisting and recalling state across steps, a set of tools invoked through structured call/response interactions, and a retrieval component that supplies external information on demand. This decomposition separates the model's generative reasoning from deterministic operations and external knowledge access.

Looking forward, agentic capabilities are extending from data extraction into autonomous scientific discovery. Recently, multi-agent systems have been developed that generate, critique and refine research hypotheses following the scientific method [27-29]. These systems show that multi-agent architectures can already carry the full hypothesis-experiment-analysis loop in biomedicine and in scientific software engineering. The analogous frontier in chemistry remains gated by an upstream bottleneck: the extraction of structured reaction records from unstructured journal articles.

1.4 Outline

Agentic AI is a promising approach for extracting structured chemical information from scientific papers. Its viability, however, depends on several practical aspects (hallucination, cost, performance and accessibility) that still need careful evaluation. To probe these aspects, four extraction pipelines were developed and scored against a shared sixteen-image benchmark, with the recently published ChemEAGLE system [31] serving as the comparison baseline.

Chapter 2 reviews the foundations on which the present work builds, from classical and machine-learning-based chemical reaction extraction tools to the recent generation of large language and vision-language models and the building blocks of agentic pipelines (tool use, structured output schemas and multi-agent orchestration).

Chapter 3 documents the benchmark and the four extraction pipelines developed in this work. The first is the published ChemEAGLE pipeline [30], reinstalled locally as the reference baseline. The second is an original multi-agent framework that redesigns the orchestration layer while retaining the chemistry deep-learning toolchain. The third is ChemEAGLE Hybrid, a variant of the upstream system. The last is an original single-agent vision-language model, instantiated behind several multimodal models and evaluated under identical conditions. The chapter also describes the data set, schema, tool layers, model choices, post-processing and evaluation pipeline with aggregate reaction-level matching metrics, per-system failure modes and a cost-quality analysis.

In Chapter 4 particular attention is given to how performance scales across the multimodal models behind the single-agent pipeline and how the original multi-agent framework compares with the ChemEAGLE Hybrid variant and the published baseline.

Chapter 5 discusses what the cross-system numbers say about the model-versus-framework trade-off, what the present evaluation cannot say given its size and scope, and which directions are most likely to extend the agentic-extraction paradigm beyond the current benchmark.

1.5 References

1. McEwen, L., *Taking a Long View: Traverses of 21st Century Chemical Information Stewardship*, in *The Future of the History of Chemical Information*. 2014, American Chemical Society. p. 1–18.
2. Larsen, P. and M. Von Ins, *The rate of growth in scientific publication and the decline in coverage provided by Science Citation Index*. *Scientometrics*, 2010. **84**(3): p. 575–603.
3. (CAS), C.A.S. *Annual Report 2015*. 2016 [29 oct 2025].
4. Rajan, K., et al., *DECIMER-Segmentation: Automated extraction of chemical structure depictions from scientific literature*. *Journal of Cheminformatics*, 2021. **13**(1): p. 20.

5. Schilling-Wilhelmi, M., et al., *From text to insight: large language models for chemical data extraction*. Chemical Society Reviews, 2025. **54**(3): p. 1125–1150.
6. Hawizy, L., et al., *ChemicalTagger: A tool for semantic text-mining in chemistry*. Journal of Cheminformatics, 2011. **3**(1): p. 17.
7. CAS. CAS data. Available from: <https://www.cas.org/cas-data>
<https://www.cas.org/cas-data#:~:text=The>.
8. Gabrielson, S.W., SciFinder. J. Med. Libr. Assoc., 2018. **106**(4).
9. Reaxys. 29 oct 2025]; Available from: <https://www.elsevier.com/products/reaxys>.
10. Jessop, D.M., et al., *OSCAR4: a flexible architecture for chemical text-mining*. Journal of Cheminformatics, 2011. **3**(1): p. 41.
11. arXiv.org e-Print archive. arXiv.
12. chemrxiv. Available from: <https://chemrxiv.org/>.
13. Hahn-Powell, G., M.A. Valenzuela-Escárcega, and M. Surdeanu, Swanson linking revisited: Accelerating literature-based discovery across domains using a conceptual influence graph, in *Proceedings of ACL 2017, System Demonstrations*. 2017. p. 103–108.
14. Krallinger, M., et al., *Information Retrieval and Text Mining Technologies for Chemistry*. Chemical Reviews, 2017. **117**(12): p. 7673–7761.
15. Mavračić, J., et al., *ChemDataExtractor 2.0: Autopopulated Ontologies for Materials Science*. Journal of Chemical Information and Modeling, 2021. **61**(9): p. 4280–4289.
16. Swain, M. and J. Cole, *ChemDataExtractor: A Toolkit for Automated Extraction of Chemical Information from the Scientific Literature*. Journal of Chemical Information and Modeling, 2016. **56**(10): p. 1894–1904.
17. Eltyeb, S. and N. Salim, *Chemical named entities recognition: a review on approaches and applications*. Journal of Cheminformatics, 2014. **6**(1): p. 17.
18. Guo, J., et al., *Automated Chemical Reaction Extraction from Scientific Literature*. Journal of Chemical Information and Modeling, 2022. **62**(9): p. 2035–2045.
19. Rocktäschel, T., M. Weidlich, and U. Leser, *ChemSpot: a hybrid system for chemical named entity recognition*. Bioinformatics, 2012. **28**(12): p. 1633–40.
20. Klinger, R., et al., *Detection of IUPAC and IUPAC-like chemical names*. Bioinformatics, 2008. **24**(13): p. i268–i276.
21. Schwaller, P., et al., *Machine intelligence for chemical reaction space*. WIREs Computational Molecular Science, 2022. **12**(5): p. e1604.
22. Dobbelaere, M.R., et al., *Rxn-INSIGHT: fast chemical reaction analysis using bond-electron matrices*. Journal of Cheminformatics, 2024. **16**(1): p. 37.
23. Tan, Z., Q. Yang, and S. Luo, *AI molecular catalysis: where are we now?* Organic Chemistry Frontiers, 2025. **12**(8): p. 2759–2776.

24. Mlinarić, A., M. Horvat, and V. Šupak Smolčić, *Dealing with the positive publication bias: Why you should really publish your negative results*. *Biochem Med (Zagreb)*, 2017. 27(3): p. 030201.
25. Shen, Y., et al., *Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face*. *Advances in Neural Information Processing Systems*, 2023. 36: p. 38154–38180.
26. Runcie, N.T., C.M. Deane, and F. Imrie, *Assessing the Chemical Intelligence of Large Language Models*. arXiv preprint arXiv:2505.07735, 2025.
27. Aygün, E., et al., *An AI system to help scientists write expert-level empirical software*. *Nature*, 2026.
28. Gottweis, J., et al., *Accelerating scientific discovery with Co-Scientist*. *Nature*, 2026.
29. Ghareeb, A.E., et al., *A multi-agent system for automating scientific discovery*. *Nature*, 2026.
30. Chen, Y. and C.Y.F. Group, *ChemEagle: A Multi-Agent Pipeline for Structured Reaction Extraction from Chemistry Figures*. arXiv preprint, 2025. 2507.20230.

Chapter 2.

Automated Extraction of Chemical Reaction Procedures

2.1	INTRODUCTION	13
2.2	REPORTING OF CHEMICAL REACTION DATA	14
2.2.1	SCIENTIFIC LITERATURE	14
2.2.2	PATENTS	18
2.3	MACHINE-READABLE REACTION DATA FORMATS	19
2.3.1	STRINGS	19
2.3.2	OTHER REACTION DATA EXCHANGE FORMATS	23
2.4	THE EVOLUTION OF CHEMICAL REACTION EXTRACTION TOOLS	28
2.4.1	THE ROOTS	28
2.4.2	CHRONOLOGICAL OVERVIEW OF CHEMICAL DATA EXTRACTION	29
2.5	GENERAL PIPELINE FOR EXTRACTING CHEMICAL DATA	31
2.5.1	PREPROCESSING	32
2.5.2	TOKENISATION	33
2.5.3	FEATURE PROCESSING	34
2.5.4	CHEMICAL NAMED ENTITY RECOGNITION	35
2.5.5	PARSING	36
2.5.6	NORMALIZATION STEP AND RELATION EXTRACTION	37
2.6	OVERVIEW CHEMICAL REACTION EXTRACTION TECHNIQUES	39
2.6.1	DICTIONARY-BASED SYSTEMS	39
2.6.2	RULE-BASED SYSTEMS	40
2.6.3	MACHINE-LEARNING SYSTEMS	43
2.6.4	OPTICAL CHEMICAL STRUCTURE RECOGNITION (OCSR)	53
2.6.5	HYBRID AND MULTI-MODAL SYSTEMS	58

2.7	LLMS AND VLMS AS NEW TOOLS FOR CHEMICAL REACTION DATA EXTRACTION	59
2.7.1	CONCEPTUAL POSITIONING RELATIVE TO CLASSICAL ML AND HYBRID SYSTEMS	60
2.7.2	LLM/VLM-BASED EXTRACTION METHODOLOGY	61
2.7.3	STRENGTHS AND LIMITATIONS	69
2.8	EVALUATION CRITERIA AND METRICS	70
2.9	CONCLUSION AND CURRENT CHALLENGES	72
2.10	REFERENCES	74

2.1 Introduction

Automated chemical information extraction tools have emerged as a necessary response to the limitations of manually curated databases. The volume of chemical reactions reported annually in scientific articles and patents far exceeds what can be captured through manual curation, leading to an increasing disconnect between published knowledge and structured chemical databases. Automated extraction methods therefore aim to narrow this gap by transforming unstructured literature into machine-readable representations, enabling large-scale data-driven analysis and reuse.

Building on this motivation, the objective of this chapter is to provide a systematic overview of methods for automated chemical reaction extraction from the literature, with a particular focus on how these methods have evolved over time. The chapter examines both classical and modern approaches, highlighting their respective strengths and limitations, and explains why recent advances such as hybrid systems, multimodal pipelines, and VLMs/LLMs, represent a significant conceptual shift. Establishing this methodological landscape is essential for understanding the design choices, evaluation criteria, and trade-offs underlying contemporary reaction extraction systems.

To achieve this aim, the chapter first introduces the fundamental data representations and formats used to encode chemical reactions, as these define the structure that extraction systems must extract and produce. It then reviews the historical progression of extraction techniques, from rule-based and dictionary-driven methods to machine-learning-based and hybrid approaches, before extending this discussion to multimodal systems that integrate text and visual information. Finally, the chapter examines the role of LLMs, VLMs, and agentic frameworks, culminating in a discussion of postprocessing, validation, and evaluation strategies that ensure chemical plausibility and data quality. Together, these sections build a coherent argument for the current state of the art and motivate the methodological choices adopted in later chapters.

2.2 Reporting of Chemical Reaction Data

Chemical knowledge is disseminated through a wide range of sources, including patents, peer-reviewed articles, and academic theses, each with distinct conventions for reporting experimental procedures and outcomes. Reaction information in these documents is distributed across unstructured text, molecular diagrams, tabulated data, spectral records, and three-dimensional representations. This heterogeneity leads to fragmented and non-standardised data structures, which complicate systematic extraction, normalization, and cross-document comparison. As a result, establishing consistent relationships among experimental parameters, chemical entities, and reported outcomes requires dedicated computational approaches [1-4].

2.2.1 Scientific Literature

The scientific literature constitutes the largest and most diverse source of experimentally reported chemical reactions. A typical synthetic procedure in the organic chemistry literature contains several recurring linguistic and structural patterns, as illustrated in the example in Figure 2-1 [5]. Titles of synthetic procedures generally indicate the target compound, such as “1,1-dibromo-2,2-bis(chloromethyl)-3-methylcyclopropane” in this case. The procedural text employs stylized and formulaic constructs that fall into characteristic categories. These include chemical terms such as “DCM”, “triposgene”, and “silica gel”; action phrases such as “X was cooled to” or “Y was washed with”; and data phrases that encode experimental quantities, for example, “5.5 mL, 68.8 mmol, 4.0 equiv.” or “4 hours”. Because systematic chemical names can be lengthy, authors frequently use numerical labels to refer to compounds within the same publication. The final line in Figure 2-1 illustrates this practice through the reference “22”.

Synthesis of 1,1-dibromo-2,2-bis(chloromethyl)-3 methylcyclopropane 22:

Under an anhydrous atmosphere, triphosgene (10.00 g, 34.3 mmol, 2.0 equiv.) was dissolved in anhydrous DCM (50 mL). The solution was cooled to 0 °C. Anhydrous pyridine (5.5 mL, 68.8 mmol, 4.0 equiv.) was added dropwise over 20 min. After 4 hours at reflux, the reaction was cooled to r.t. and diluted with DCM (150 mL). The phases were separated, and the organic layer was washed with NaHCO₃ (sat. aq. sol., 150 mL). The crude mixture was filtered over silica gel (n-pentane 100%) and concentrated to afford 22 (4.98 g, 16.0 mmol, 93%) as white crystals.

Figure 2-1: Simplified excerpt from an organic synthesis procedure by Verschaeve *et al.* [5] indicating chemical terms (green), action phrases (yellow), and data phrases (blue).

The lack of a standardised naming convention to represent the chemical structural information has been a difficulty in mining chemical information [6, 7]. Although International Union of Pure and Applied Chemistry (IUPAC) offers a standardized framework, authors frequently employ abbreviations and common names for clarity and brevity [8-11]. Table 2-1 illustrates several different expressing methods and naming groups.

Table 2-1: Description and examples of the methods for expressing chemical structural information [6].

Expressing method	Description	Example
Systematic names	International Union of Pure and Applied Chemistry (IUPAC) names.	3-(3,4-dihydroxyphenyl)prop-2-enoic acid “Caffeic acid” used for 3-
Trivial names	Do not reflect the structure of the chemical substance.	(3,4-dihydroxyphenyl)prop-2-enoic acid

Semi systematic names	IUPAC-like, non-IUPAC names.	In “N-benzoylglycine” the part “benzoyl” is systematic, whereas “glycine” is the trivial name for α -aminoacetic acid.
Common or generic names	Names applied to a class of compounds.	Camphor, water, alcohol
Registered trademark/brand names	Identify the brand owner as the commercial source of products.	Aspirin
Company codes	A company code is used to identify the compound within the company.	ZD5077 = ICI204636 = ZM204636
Acronyms and abbreviations	Used to obtain short names.	DMS for dimethyl sulphate
Index and reference numbers	Numbers from Chemical Abstracts Service (CAS) registry, Beilstein registry numbers, etc.	CAS number of water: 7732-18-5
Anaphors	Compounds are named earlier in the text but co-referenced to a shorter name (the anaphor) later in the text.	“...bioactivity is found in compounds [1–7, 9–11] listed in Additional file 1...”
Sum formula	Consists of the elements contributing to a compound and the number of their occurrences.	C ₉ H ₈ O ₄
Chemical structures	Explicit and implicit structural representations.	Markush structures, e.g. R ₁ = CH ₃ , COOH, etc.

Next to textual description of chemical reactions, the most important information about chemical reactions is often hidden in images of structural formulas and reaction schemes [12]. A structural formula is a graphic representation of the molecular structure, showing how atoms are connected to

one another, even the chemical bonding is shown, either explicitly or implicitly. Figure 2-2 illustrates how a simple molecule like butane can be represented using different drawing styles: Lewis structures, Newman projections, condensed formulas, and skeletal formulas.

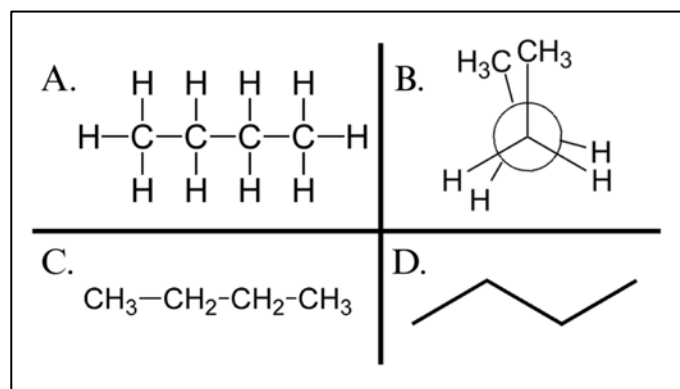


Figure 2-2: Example of different molecule representation styles: (a) Lewis's structure, (b) Newman projection, (c) condensed formula, (d) skeletal formula.

Chemical reactions are typically depicted with structural formulas, which help chemists visualize the molecules involved and the structural changes that occur during a transformation [12]. Figure 2-3 illustrates two such reactions. At first glance, these schemes appear straightforward: reactants are shown on the left of the arrow, products on the right, and reaction conditions and solvents above or below the arrow. On closer inspection, however, important nuances emerge. In reaction A, for example, several species are only referred to by boldface numbers (9 and 10), while in reaction B Markush structures, generic placeholders such as *R* and *Me*, denote unspecified substituents and methyl groups representing a group of related chemical compounds. Understanding precisely what these symbols represent and thus how to reproduce the reactions, requires familiarity with organic chemistry jargon and careful consultation of the surrounding text, figures, and experimental section. This reliance on implicit conventions and cross-referencing highlights how demanding the human-readable representation can be, and motivates the need for more explicit, machine-readable formats.

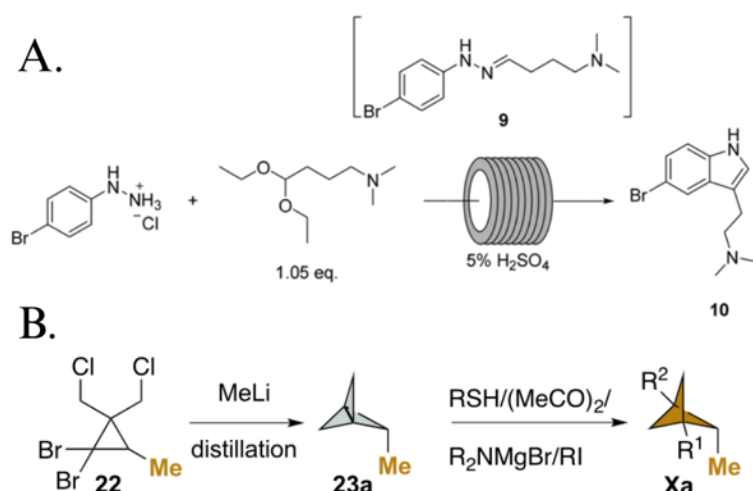


Figure 2-3: Two different chemical reactions examples. Figure A is an excerpt from Simoens et al. [13] illustrating a Fischer Indole reaction in flow. Figure B is an excerpt from Verschaeve et al. [5] showing a step in the synthesis of bridge-methylated Bicyclo[1.1.1]Pentanes.

2.2.2 Patents

Patents constitute a large repository of chemical knowledge and thus a key target for text mining [14, 15]. Compared to scientific articles, patents describing organic syntheses adopt a markedly different writing style and structure [16]. Whereas journal papers aim to communicate and justify concrete experimental results, patents are legal documents designed to secure broad intellectual property protection. This leads to repetitive, legalistic, and highly stylised language phrasing (“in one embodiment...”, “optionally...”) and extensive use of Markush structures. These are structural formulae that not only consist of chemical elements, but also of generic placeholders covering large classes of familiar compounds rather than a few well-defined molecules (Figure 2-4) [17, 18].

A crucial concept in patents is that of prophetic reactions (or prophetic examples): procedures that are projected or proposed but have not actually been carried out in the laboratory [18]. These are often recognisable by their verb tense: using the future or conditional (“will be stirred”, “may be heated”) rather than the past (“was stirred”, “was heated”), typical of factual experimental report [19]. Experimental sections are presented as “examples” rather than as a cohesive narrative, may mix real and prophetic examples, and often omit mechanistic discussion that are common in the scientific literature. Conditions, reagents, and yields can be dispersed across claims, description, and examples, frequently with wide ranges and numerous alternatives. This legalistic, forward-looking style

contrasts sharply with the concise, data-driven prose of research articles. It poses specific challenges for both human readers and automated extraction systems in organic synthesis [19, 20].

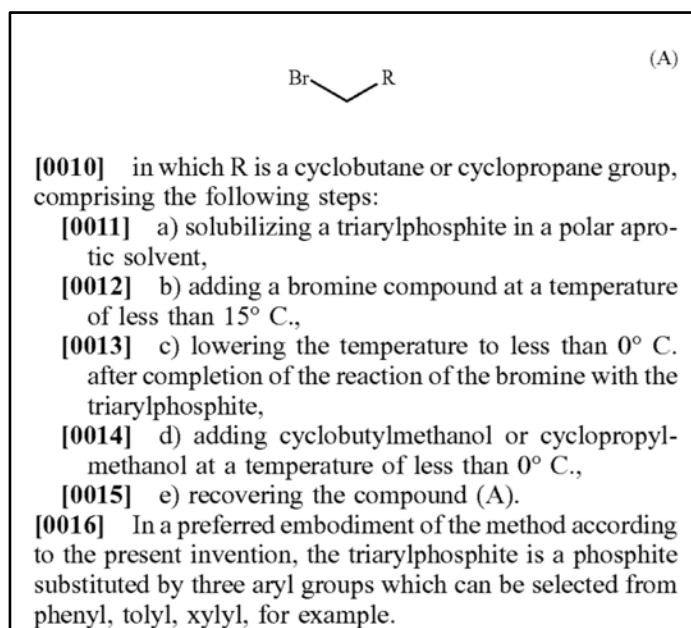


Figure 2-4: Experimental section of an organic chemistry patent describing how bromomethyl cyclopropane and bromomethyl cyclobutene can be synthesised. The example illustrates highly stylised writing and a Markush structure at the top (R) representing a cyclopropane or cyclobutene group. The example is adapted from Guerret [21].

2.3 Machine-Readable Reaction Data Formats

The commonly used chemical reaction formats prioritize human interpretability such as the IUPAC format and structural formula; however, they are hardly machine-intelligible for automated analysis [12, 22]. Machine-readable representations like SMILES [23] and InChI [24] use standardized, syntax-driven approaches in which each token corresponds unambiguously to a chemical feature. This minimizes information loss and facilitates computational processing of chemical data [25].

2.3.1 Strings

With the introduction of the computer, line notation became widely used in chemical nomenclature because linear strings are compatible with computer processing. Line notation is a typographical

notation system using ASCII characters (American Standard Code for Information Interchange) [26]. The most common machine-readable methods are SMILES [23], SMARTS [27] and InChI [24], which have been extended to represent reactions through the development of Reaction SMILES [28], Reaction SMARTS and RInChI [29]. Although these methods are based on rigid algorithms, they too are prone to inconsistencies in structural representations within and between databases [30].

Reaction SMILES

A Simplified Molecular-Input Line-Entry System (SMILES) is a method developed in the 1980s to represent chemical molecules in one string [23]. Figure 2-5 illustrates a SMILES string of p-chlorotoluene. A single molecule may be represented by multiple valid SMILES strings; to address this, algorithms have been developed to generate a unique, canonical SMILES representation for each molecule. For example CC(C)O, C(O)(C)C, or OC(C)C are all equivalent SMILES strings of isopropanol, but its canonical SMILES string is CC(C)O [31].

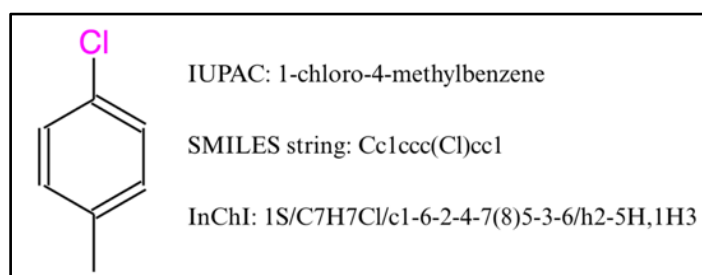
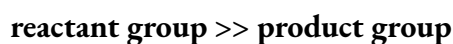


Figure 2-5: SMILES string and InChI from 1-chloro-4-methylbenzene created with the software from InChI Key[32] and ChemInfo[33].

The SMILES system has generalised from individual molecules to entire reactions through the development of Reaction SMILES [28]. In the Reaction SMILES notation, a reaction is divided into three segments: reactants, solvent(s), and products. These segments are concatenated with ‘>’ symbols in the form “reactant group > solvent group > product group”. Each segment is itself a collection of standard SMILES strings. When multiple species belong to the same group, for example in a bimolecular reaction, their SMILES strings are separated by a dot (‘.’). If no solvent is used, the solvent segment is left empty, yielding a double-arrow form:



The Reaction SMILES for the classic Fischer esterification reaction of acetic acid and ethanol into Ethyl acetate with sulphuric acid (Figure 2-6) is shown below:

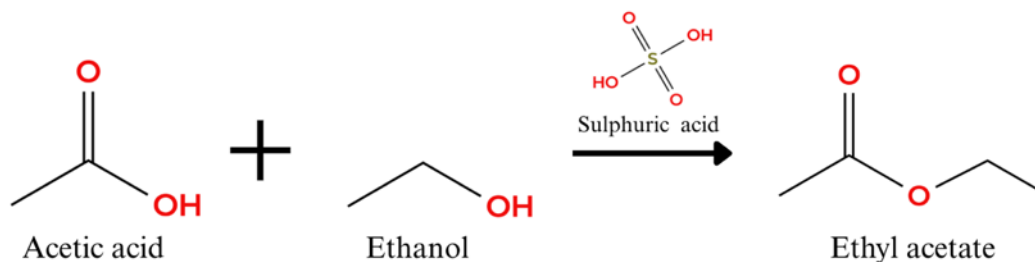
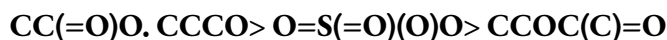


Figure 2-6: Fischer esterification reaction of acetic acid and ethanol into Ethyl acetate with sulphuric acid.

Reaction SMARTS

SMILES arbitrary target specification (SMARTS), is a representation of substructural patterns in molecules. SMARTS use rules that are straightforward extensions of SMILES. For example, to search a database for phenol-containing structures, one would use the SMARTS string [OH]c1ccccc1, which should be familiar to those acquainted with SMILES. In fact, almost all SMILES specifications are valid SMARTS targets. Using SMARTS, flexible and efficient substructure-search specifications can be made in terms that are meaningful to chemists[28].

SMARTS can also be extended to Reaction SMARTS, similar as Reaction SMILES. As with SMARTS for individual molecules, reaction SMARTS do not specify a single concrete reaction but rather encodes one or more substructures that must be present in the reactants and/or products. Syntactically, reaction SMARTS strings mirror Reaction SMILES: SMARTS patterns for reactants, solvents, and products are written separately and then combined as reactant substructure > solvent substructure > product substructure. For instance, the query:



describes an aromatic nitration pattern, where an aromatic carbon bearing a hydrogen in the reactant is transformed into an aromatic carbon substituted with a nitro group in the product. Any reaction

in which an aromatic C–H is converted into an aromatic C–NO₂ motif will match this reaction SMARTS.

RInChI

InChI is a structure-based chemical identifier, developed from 2000 to 2005. In one string it describes chemical substances in terms of layers of information – the atoms and their bond connectivity, tautomeric information, isotope information, stereochemistry and electronic charge. The InChIKey is a fixed length (27 character) condensed digital representation of the InChI that is not designed to be human-understandable [34]. In July 2006, IUPAC introduced InChI as a universal standard for formula representation.

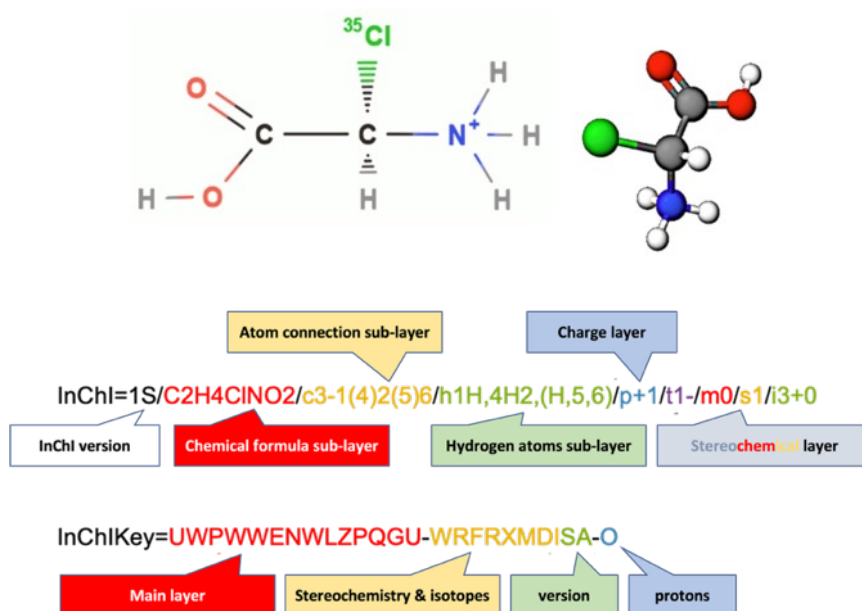


Figure 2-7: Representation of InChI and InChIKey with their components for the molecule 3-Chloroalanine, adapted from InChI Trust [35].

RInChI [29] is a line-based reaction identifier derived from InChI and organised into six layers. The first layer encodes the RInChI version. The second and third layers list the reactants and products, respectively, each species being represented by its InChI string; within a layer, individual molecules are sorted alphabetically and separated by an exclamation mark ('!'). The fourth layer describes species that are present on both sides of the reaction, such as catalysts or solvents. The fifth layer indicates which of the second or third layers should be interpreted as reactants, since these were

originally ordered alphabetically. The sixth layer reports the number of unidentified components in the second, third, and fourth layers. In a full RInChI string, layers are normally separated by a slash ('/'), except for the second, third, and fourth layers, which are delimited by the symbol '<>'. For example, the acid-catalysed esterification of acetic acid with ethanol to give ethyl acetate and water in sulphuric acid can be written as (Figure 2-6):

RInChI=1.00.1S/C2H4O2/c1-2(3)4/h1H3,(H,3,4)!C2H6O/c1-2-3/h3H,2H2,1H3<>C4H8O2/c1-3-6-4(2)5/h3H2,1-2H3!H2O/h1H2<>H2O4S/c1-5(2,3)4/h(H2,1,2,3,4)/d=

2.3.2 Other Reaction Data Exchange Formats

Chemical reaction data exchange formats are critical for capturing, sharing, and analysing detailed chemical information. Three commonly used formats are chemical table files, Chemical Markup Language and CDXML files. They enable comprehensive representation of reaction schemes, molecular structures, and metadata [36]. These structured formats support interoperability between research databases, drawing software, and laboratory informatics systems, ensuring the preservation of chemical complexity and context in digital workflows.

Chemical Table Files

The Molfile format is a line-based representation for individual molecular structures that encodes the atoms, bonds, connectivity and Cartesian coordinates of a single molecule. Figure 2-8 illustrates a Molfile for p-chlorotoluene. The file is structured as follows: line 1 contains the title, line 2 a timestamp, line 3 specifies the number of atoms and bonds as well as the Molfile version, lines 5–12 list the atomic coordinates and associated atom information, lines 13–20 define the bonds, and the file terminates with an end line.


```

1  Cc1ccc(Cl)cc1
2  JME 2017-11-16 Thu Nov 27 18:46:09 GMT+100 2025
3
4      8  8  0  0  0  0  0  0  0  0  0999 V2000
5      2.4249  2.1000  0.0000 C  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
6      2.4249  3.5000  0.0000 C  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
7      1.2124  4.2000  0.0000 C  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
8      0.0000  3.5000  0.0000 C  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
9      0.0000  2.1000  0.0000 C  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
10     1.2124  1.4000  0.0000 C  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
11     1.2124  5.6000  0.0000 Cl 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
12     1.2124  0.0000  0.0000 C  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
13     1  2  1  0  0  0  0
14     2  3  2  0  0  0  0
15     3  4  1  0  0  0  0
16     4  5  2  0  0  0  0
17     5  6  1  0  0  0  0
18     6  1  2  0  0  0  0
19     3  7  1  0  0  0  0
20     6  8  1  0  0  0  0
21 M  END

```

Figure 2-8: p-chlorotoluene stored in a Molfile created with ChemInfo [33]. The molfile consists of the following lines: line 1 is the title line, line 2 is a timestamp, line 3 shows the number of atoms and which Molfile type is used, the lines from 5 to 12 show the coordinates of the atoms, the lines from 13 to 20 describe the bonds and finally an end line.

Building on the Molfile specification, the RXN format is widely used to store information on a single chemical reaction, including its reactants, products and agents. Conceptually, an RXN file can be

regarded as an extension of the Molfile format that links multiple molfiles within a single reaction record [36] (Figure 2-9).

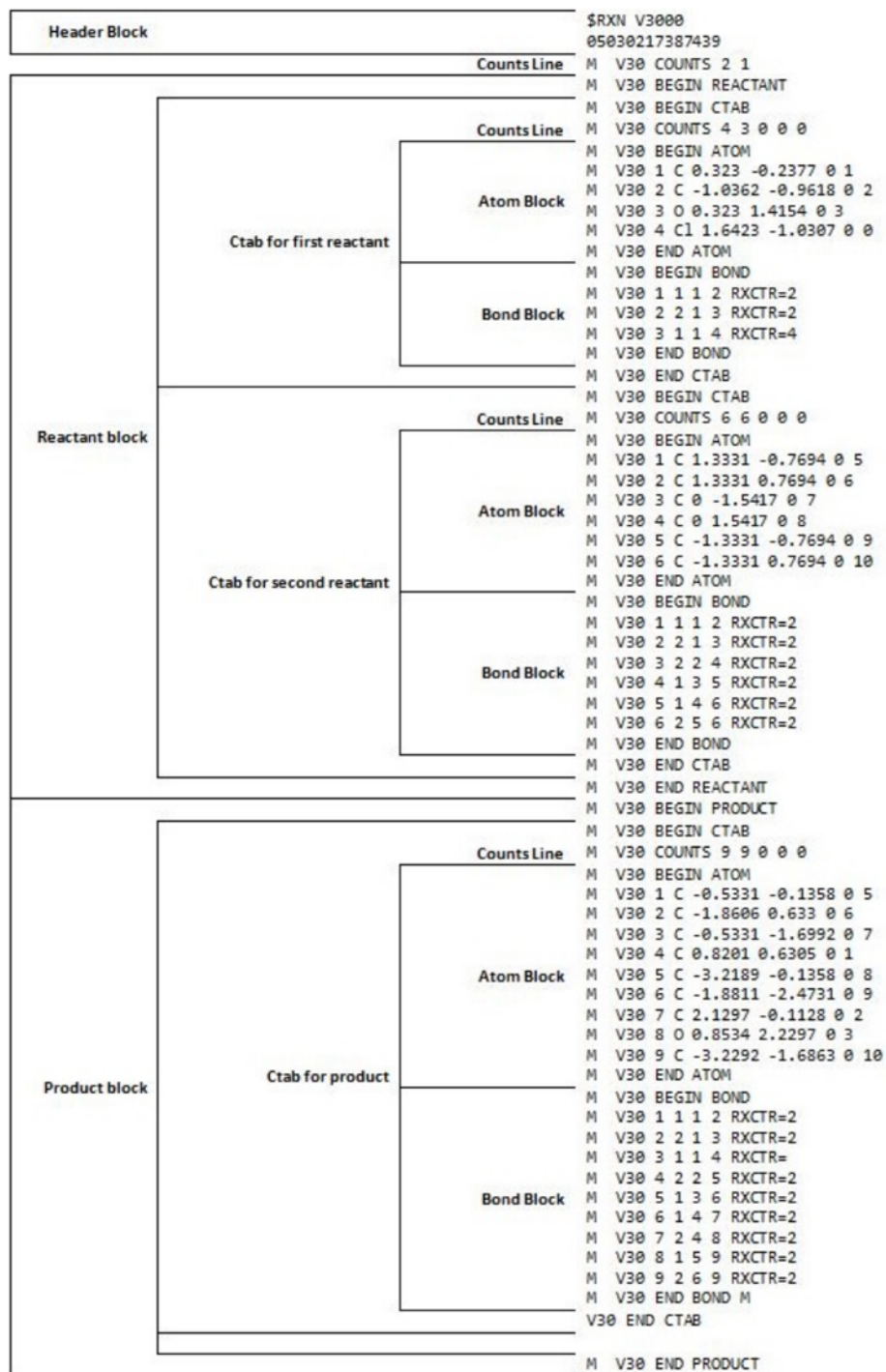


Figure 2-9: Example of a V3000 RXN file for the acylation of benzene from the Biovia report [37].

For representing reaction schemes that comprise multiple elementary steps, the Reaction Data File (RDF) format is commonly employed. RDF files can encapsulate several reactions together with

additional tagged fields and annotations for reaction conditions, making them well suited for database export, large-scale curation and batch processing workflows (Figure 2-10).

RD Header		\$RDFILE 1
		\$DATH 10/17/91 10:41
		\$RFMT \$RIREG 7439
		\$RXN
Rxn file header		1017911041 7439
First Rxn Record	#Reactants/ #products	2 1
	\$MOL	
	Molfile for 1 st reactant	10179110412D 1 0.00380 0.00000 315
		4 3 0 0 0 0 0 0 0 0 0
		1 4 1 0 0 0 4
	\$MOL	
	Molfile for 2nd reactant	10179110412D 1 0.00371 0.00000 8
		6 6 0 0 0 0 0 0 0 0 0
		5 6 2 0 0 0 2
	\$MOL	
Data Block for record	Molfile for product	10179110412D 1 0.00374 0.00000 255
		9 9 0 0 0 0 0 0 0 0 0
		6 9 2 0 0 0 2
	\$DTYPE rxn:VARIATION(1):rxnTEXT(1)	
	\$DATH CrCl3	
	\$DTYPE rxn:VARIATION(1):LITTEXT(1)	
	\$DATH A G Repin, Y Y Makarov-Zemlyanskii, Zur Russ Fiz-Chim, 44, p.2360, 1974	
	\$DTYPE rxn:VARIATION(1):CATALYST(1):REGNO	
	\$DATH \$RFMT \$RIREG 688	
		10179110412D 1 0.00371 0.00000 0
Start of next rxn record		4 3 0 0 0 0 0 0 0 0 0
		1 4 1 0 0 0 0
	\$DTYPE rxn:VARIATION(1):PRODUCT(1):YIELD	
	\$DATH 70.0	
	\$RFMT \$RIREG 8410	
	\$RXN	
		1017911041 8410
		2 1
	\$MOL	
		.

Figure 2-10: Example of an RDF containing information about a multi-step reaction[37].

Chemical Markup Language

Chemical Markup Language (CML) is an open, XML-based metadata format for representing chemical entities, including molecules, reactions, and spectra, while maintaining associated information and structure. It is compatible with visualization and database tools like JChemPaint, Jmol, MarvinView, and others [38-40]. Figure 2-11 illustrates an CML example of an NMR spectrum with peak assignments to the corresponding molecular structure.

```

<cml>
  <molecule id="mol1">
    <atomArray>
      <atom id="a1" elementType="C" x2="4.231" y2="1.1423"/>
      <atom id="a2" elementType="C" x2="4.231" y2="1.9673"/>
      <atom id="a3" elementType="C" x2="3.5167" y2="2.3797"/>
      <atom id="a4" elementType="C" x2="2.8022" y2="1.9673"/>
    </atomArray>
    <bondArray>
      <bond id="b1" atomRefs2="a1 a2" order="D" />
      <bond id="b2" atomRefs2="a2 a3" order="S" />
      <bond id="b3" atomRefs2="a3 a4" order="D" />
    </bondArray>
  </molecule>
  <spectrum type="NMR" moleculeRef="mol1">
    <parameterList>
      <parameter dictRef="jcamp:averages" title="averages">32</parameter>
      <parameter dictRef="jcamp:NMR.OBSERVEFREQUENCY">
        <scalar units="units:mhz">400</scalar>
      </parameter>
    </parameterList>
    <conditionList>
      <scalar dataType="xsd:string" dictRef="cml:field" units="siUnits:hertz">500</scalar>
      <scalar dataType="xsd:string" dictRef="cml:temp" units="siUnits:k">Unreported</scalar>
    </conditionList>
    <peakList>
      <peak xValue="3.36" xUnits="units:ppm" peakShape="sharp" id="p0" atomRefs="a1"/>
      <peak xValue="3.60" xUnits="units:ppm" peakShape="sharp" id="p1" atomRefs="a2" />
      <peak xValue="3.77" xUnits="units:ppm" peakShape="sharp" id="p2" atomRefs="a3"/>
    </peakList>
  </spectrum>
</cml>

```

Figure 2-11: CML example of an NMR spectrum with peak assignments to the corresponding molecular structure [41].

CDXML

CDXML is the XML-based format native to ChemDraw, a popular program for drawing chemical structures, used for encoding chemical structure and reaction diagrams in a human-readable and machine-readable form [42, 43]. ChemDraw also offers a binary CDX format, but CDXML is preferred for data exchange due to its XML structure.

To conclude, these new machine-readable formats enable the creation of large efficient databases of chemical data, which will improve organic synthesis research. However, most of the scientific literature and patents do not use machine-readable representation formats. To solve this problem, methods were developed to extract this non-machine-readable data and convert it to SMILES, RInChI and CDXML files.

2.4 The Evolution of Chemical Reaction Extraction Tools

2.4.1 The Roots

Chemists were not the first to automate data extraction from literature. Biomedical text mining and information retrieval are two research domains that have provided important foundations for chemical data extraction.

Bioinformatics

The biomedical community pioneered many of the first large-scale text-mining systems, such as ProMiner, GNAT and GeneTUKit [44-48]. Early work focused primarily on gene and protein name recognition [6]. Because biomedical and chemical texts share similar stylistic and structural characteristics (technical terminology, dense noun phrases, highly formulaic abstracts), a few chemical text-mining tools were directly inspired by biomedical systems. For instance, Hawizy et al. [49] explicitly aimed to develop a chemical counterpart to part-of-speech taggers (*e.g.* GeniaTagger [50]) and syntactic parsers (*e.g.* Enju[51]) when designing ChemicalTagger [49].

Information Retrieval

Secondly, textual information retrieval also has a rich history resulting in a valuable soil for chemical data extraction. It is the science of searching for relevant textual information in a document, for documents themselves, and for metadata that describes data [52]. The first approaches tried to retrieve textual information depending on a lexical match between words in users' Boolean requests and those in the database. Because of the tremendous diversity in the words people use to describe the same object (synonyms: *e.g.* different names for the same chemical) and words having different meanings (polysemes: *e.g.* financial bank and a park bank), lexical matching methods are necessarily incomplete and imprecise [53].

Building upon the foundational advances in biomedical text-mining and the strengths of information retrieval, chemical data extraction is positioned to overcome similar teething troubles and rapidly evolve. By leveraging lessons from these domains, chemical data extraction can achieve more accurate, efficient, and meaningful extraction of chemical knowledge from diverse textual sources.

2.4.2 Chronological Overview of Chemical Data Extraction

The origins of cheminformatics can be traced back to the late 1950s, when researchers first began to use computers to store, search and manipulate chemical structures and associated properties [54].

In the early 1980s, the first chemical reaction extraction tools were developed [55]. In parallel, Named Entity Recognition (NER) emerged as the standard in Natural Language Processing (NLP) and chemistry adopts by coining the term Chemical-NER (CNER) [56, 57]. CNER refers to the task of automatically identifying and extracting chemical entities such as compound names, formulas, and identifiers, from textual data, especially scientific literature [58]. CNER algorithms and tools are designed to recognize various types of chemical names, which can include systematic names, trivial names, abbreviations, family names, formulas, and database identifiers [57]. Methodologies applied to the problem of CNER include dictionary- and rule-based methods, as well as statistically based systems making use of different machine learning techniques and hybrid approaches [59]. A prominent example is Open-Source Chemistry Analysis Routines (OSCAR)[60] developed to automatically recognize chemical names in text and annotate them, using rules, dictionaries and ML [60]. Figure 2-12 summarizes the chemical data extraction landscape.

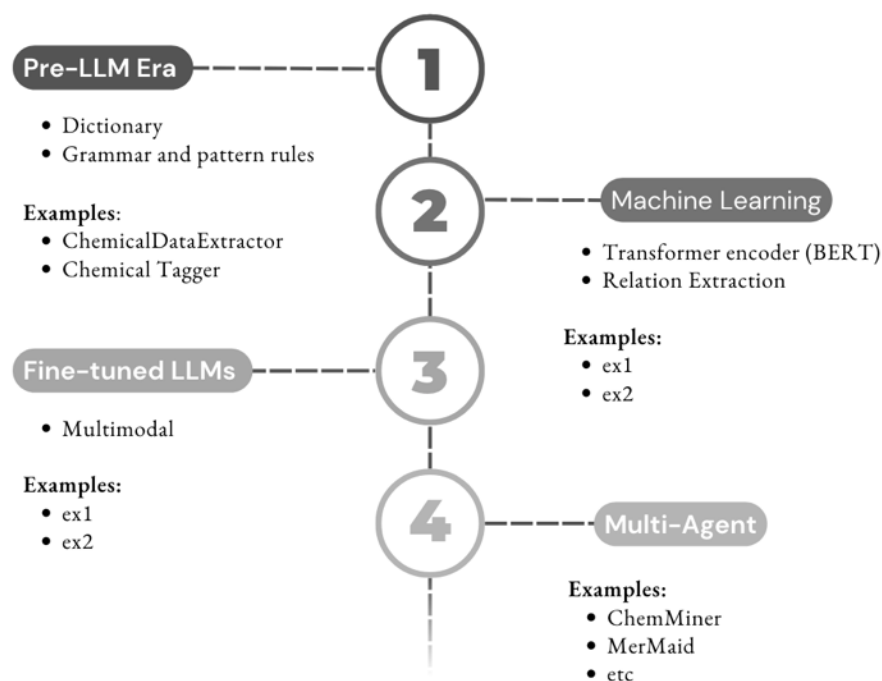


Figure 2-12: Chemical data extraction landscape.

Around 2012, ChemicalTagger [49] goes beyond CNER by grammar-based parsing experimental procedures into semantic phrases (actions, conditions, materials), which captures chemical meaning and relationships between entities: a first step toward full reaction/condition extraction [49]. Semantic parsing is the process in (NLP) of converting a natural language sentence (ex. synthesis instructions) into a structured, machine-understandable representation of its meaning, often known as a logical form. Building upon, Open Parser for Systematic IUPAC Nomenclature (OPSIN [61]) took a mixture of outputs from ChemicalTagger and employed a set of rules to determine four essential chemical roles, including product, reactant, solvent, and catalyst [61]. Thereby they extracted around 424,621 reactions from 65,034 organic chemistry patents from the United States Patent and Trademark Office (USPTO)[62]. These rule-based systems represent a significant step forward, but they are heavily dependent on manually designed rules and are sensitive to the noise introduced by either language use or preprocessing steps, which limits their scalability to nonpatent data such as journal articles [16].

Between 2013 and 2019, the community consolidates and creates the first benchmark. The CHEMDNER task (2015) creates large gold-standard corpora (~10k PubMed abstracts) and shared tasks for chemical and drug name recognition and document indexing [63]. Simultaneously,

ChemDataExtractor (2016) introduces a full chemistry-aware NLP pipeline: tokenization, POS-tagging, NER, phrase parsing, and rule-based relation extraction to build property and materials databases from the literature [1].

In the late 2010s and early 2020s, deep learning and large-scale mining entered the scene. Neural language models like BERT [64] from Google replace classic Conditional Random Fields (CRFs) [65] architecture for CNER and relation extraction, improving robustness in patents, abstracts, and full text [66]. With these new improvements, large-scale projects start mining tens of millions of articles and patents for compounds, reactions, and properties, combining text and image/structure recognition [67].

Recently, LLMs with multimodal character enhanced few-shot prompting improving chemical data extraction on precision, recall and F1-scores [68]. New architectures adapt CNER models to new chemical entity types with very few labelled examples (few-shot CNER). At the same time, general-purpose large language models (GPT-style[69], ChemBERTa-style domain models[70], etc.) start being used directly for chemical entity, property and reaction extraction, often combined with smaller specialist models and tools like OPSIN[61], ChemicalTagger[49] and ChemDataExtractor[1].

2.5 General Pipeline for Extracting Chemical Data

Most chemical reactions are not published as machine-readable formats, like SMILES, in scientific literature and patents [71]. Therefore, tools have been designed to extract chemical reactions automatically from text and store them in databases. All these tools have an efficient pipeline in common consisting of the following steps: preprocessing, tokenisation, feature processing, Chemical Named Entity Recognition (CNER)/tagging, normalisation, and relation extraction. Figure 2-13 shows several examples of pipelines.

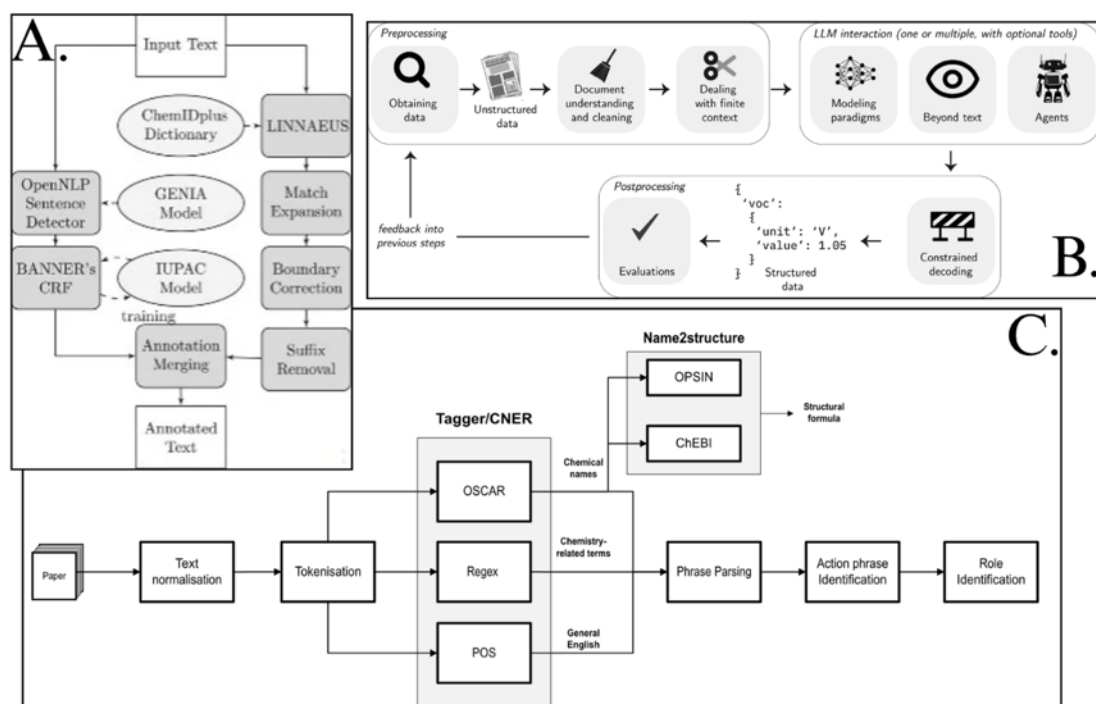


Figure 2-13: Illustration of established chemical data extraction pipelines. (A) adapted from Rocktäschel et al. [3], (B) from Jablonka et al. [4] and (C) from MC Swain et al.[1].

2.5.1 Preprocessing

Preprocessing starts on a document level where PDFs/HTML/XML files are converted to structured text and images, including page segmentation, OCR (Figure 2-14), and layout analysis to identify paragraphs, tables, and reaction schemes [72]. Next, paragraphs are split into the sentences and words. For example, ChemDataExtractor [1] uses a sentence splitter which detects sentence boundaries through unsupervised learning of common abbreviations and sentence starters. [1, 73].

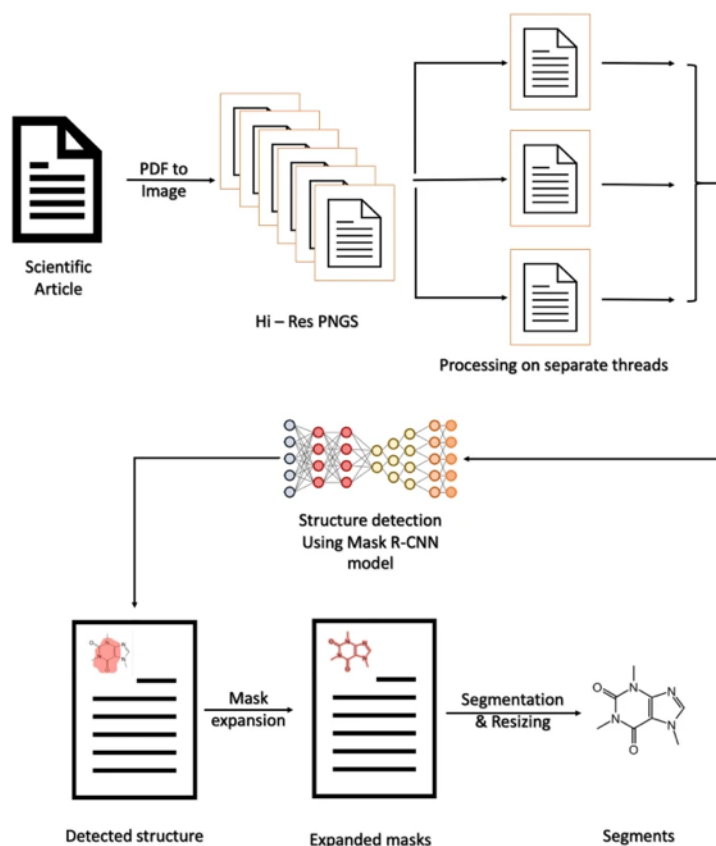


Figure 2-14: Excerpt of the DECIMER automatic segmentation pipeline for 2D chemical structures, adapted from K. Rajan et al. [74]

2.5.2 Tokenisation

Tokenisation is the process of splitting a phrase into smaller meaningful units called tokens, which are the basic elements for computer analysis in natural language processing (NLP). The token represents a string of characters that mostly correspond to words but also to punctuation or other discrete units of text *e.g.* "C₂H₆O" or "42". Many different splitting patterns are conceivable in NLP and hence many different tokenizers exist. For example, an adapted whitespace tokenizer which is developed for chemical names by ChemicalTagger [49]. Running the tokenizer on the normalised sentence above produces the following tokens in Figure 2-15.

Potassium carbonate (0.63 g , 4.56 mmol) and thiophenol (0.19 g , 1.69 mmol) were added to the 2-nitrobenzene sulfonamide

Figure 2-15: Adapted whitespace tokenizer used by ChemicalTagger [49]. All the tokens are denoted with a box.

2.5.3 Feature Processing

Feature processing step. Because natural language is highly context-dependent, the meaning and function of a single token cannot usually be inferred in isolation. Even in a chemical text, the simplest contextual representation already includes the previous, current and following word. In this setting, the probability that a token w_i belongs to a class _{i} (*e.g.*, “chemical”, “unit”, “temperature”) is modelled as depending on its immediate neighbours. i.e. a single token is enriched with features:

$$P(class_i | w_{i-1}, w_i, w_{i+1})$$

However, such a minimal context window is rarely sufficient for robust performance, and practical systems therefore exploit a richer set of features.

Typical feature sets for chemical information extraction include orthographic properties (presence of digits, hyphens, brackets, capitalization patterns), morphological cues (suffixes such as “-ol”, “-one”, “-ate”, “-amide”), lexical and dictionary information (membership in curated lists of solvents, reagents, catalysts, units or measurement terms) and local contextual features (neighbouring tokens, part-of-speech tags, n-grams) [75]. Constructing patterns that capture the different linguistic realisations of a given fact (*e.g.*, “was heated to 80 °C”, “the mixture was warmed at 80 °C”) typically presupposes a structural preprocessing step, in which tokens are assigned parts of speech and additional attributes, and multi-word expressions or idiomatic phrases are identified.

Table 2-2: Feature set from ChemDataExtractor [1].

Feature	Context	Description
Word	$w_{i-2}, w_{i-1}, w_i, w_{i+1}, w_{i+2}$	normalized lowercase token text
Bigrams	$w_{i-2}, w_{i-1}, w_i, w_{i+1}, w_{i+2}$	combinations of consecutive tokens
Word	shape	$w_{i-2}, w_{i-1}, w_i, w_{i+1}, w_{i+2}$
Brown	clusters	$w_{i-2}, w_{i-1}, w_i, w_{i+1}, w_{i+2}$
Length	w_i	number of characters in token
Prefixes	w_i	1–5 character prefixes
Suffixes	w_i	1–5 character suffixes

Hyphenated	w_i	contains a hyphen character
Alphabetical	w_i	contains only alphabetical characters
Case	w_i	upper, lower, or title cased
Number	w_i	number in digit or word form
Punctuation	w_i	contains only punctuation characters

2.5.4 Chemical Named Entity Recognition

At this stage, chemical named entity recognition (CNER) or tagging is performed. Depending on the system, this may rely on dictionary look-up, hand-crafted rules, classical machine-learning models, or large language models. The (feature-enriched) tokens are passed to the extraction component, which assigns a label to each token, such as B-CHEM, I-CHEM, UNIT, TEMP or OTHER. An example of tagging output produced by ChemicalTagger is shown in Figure 2-16

Figure 2-16 displays the tagging results from ChemicalTagger. The text is: "Potassium carbonate (0.63 g , 4.56 mmol) and thiophenol (0.19 g , 1.69 mmol) were added to 2-nitrobenzene sulfonamide 50 (0.50 g , 1.302 mmol)". In the original image, chemical entities like "Potassium carbonate", "thiophenol", and "2-nitrobenzene sulfonamide" are enclosed in double boxes. Regular expressions such as "(0.63 g , 4.56 mmol)", "(0.19 g , 1.69 mmol)", and "(0.50 g , 1.302 mmol)" are underlined.

Figure 2-16: Tagging results from ChemicalTagger [49]. Chemical tokens have been identified with double boxes, while regular expressions tokens are underlined.

A chemical reaction can be described as a process of chemical transformation from one set of chemical substances to another. A desired reaction schema is thus supposed to be informative enough to reproduce such a reaction by a chemist, so it should primarily include reactants, the products, and the reaction conditions (catalyst/reagents, workup reagents, solvent, temperature, pressure, time, reaction name and yield) [16]. Table 2-3 contains the detailed explanations for each of the predefined roles in the schema.

Table 2-3: Typical reaction record with detailed explanations of each specific role.

Reaction Role	Description
Product	Chemical substance that is the outcome (major product) of the reaction

Reactants	Chemical substances that contribute heavy atoms to the product
Catalyst/Reagents	Chemical substances that participate in the reaction but do not contribute heavy atoms (<i>e.g.</i> , acid, base, metal complexes)
Workup reagents	Chemical substances that are used after the reactions to terminate the reactions or obtain the products (<i>e.g.</i> , quenching reagents, extraction solvent, neutralizing acids/bases)
Solvent	Chemical substances that are used to dissolve/mix other chemicals, typically quantified by volume and used in super stoichiometric amounts (<i>e.g.</i> , water, toluene, THF)
Temperature	Temperature at which the reaction occurs
Pressure	Pressure at which the reaction occurs
Time	Duration of the reaction performed
Reaction type	Descriptions about the type of chemical reaction
Yield	Yield of the product

2.5.5 Parsing

In many pipelines, sentences are further simplified by parsing them into syntactic or semantic trees, which provide an explicit representation of dependencies between tokens and thereby facilitate downstream tasks such as relation extraction, where entities (*e.g.*, reactant, product) must be linked to conditions (*e.g.*, temperature, time, yield) [75]. In ChemicalTagger, phrase parsing aims to construct a chemical analogue of a Chomsky tree [49] using the Penn Tree bank part of speech tags[76], a hierarchical representation that organises tokens, i.e. words or chemical terms, into constituents according to grammatical principles. This tree structure clarifies the functional roles of the components in experimental descriptions and supports systematic analysis of chemical procedures.

Figure 2-17 illustrates a syntax tree for the sentence “Triphosgene was dissolved in DCM.” The root node Sentence is divided into a Noun phrase, containing the subject Triphosgene, and a Verb phrase, which encodes the predication about this subject. Within the verb phrase, the auxiliary past-tense verb (VBD) was and the main verb (VBD-add) dissolved form the core action, while the Prepositional phrase specifies experimental context via the preposition in and its embedded Noun phrase DCM, the solvent. Such syntactic trees make explicit the grammatical relations between tokens and thereby support downstream extraction of roles such as material, action and solvent in chemical procedure text [49].

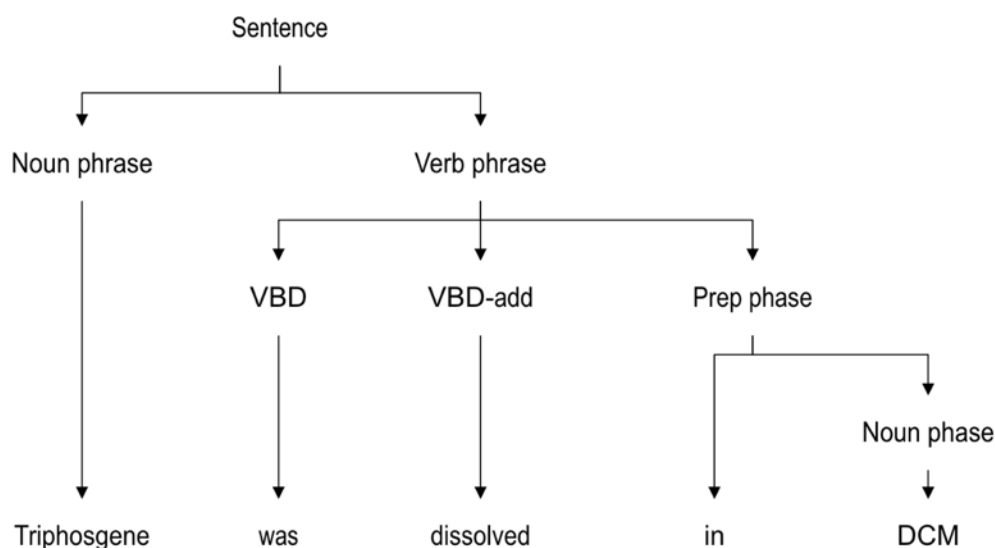


Figure 2-17: Syntax tree for the sentence “Triphosgene was dissolved in DCM.”

2.5.6 Normalization Step and Relation Extraction

This is sometimes addressed as a separate task from the NER. The entity normalization process involves mapping entities’ names to their canonical names and associating them with unique representations to help resolve issues arising from synonym variations and ambiguous abbreviations.

After normalisation, the system infers relations between the recognised entities. In the context of reaction extraction, this involves assigning roles such as REACTANT, PRODUCT, CATALYST or SOLVENT and linking these entities to experimental conditions (temperature, time, yield, atmosphere, etc.). Depending on the system, relations are derived using hand-crafted patterns,

dependency-based rules, or machine learning neural models operating on the tagged and parsed sentences. The resulting structured tuples or reaction graphs can then be stored in reaction databases or used for downstream tasks such as retrosynthetic analysis and yield prediction [77, 78]. Table 2-4 lists several chemical data extraction tools, established by Krallinger et al. [75].

Table 2-4: List of chemical data extraction tools made by Krallinger et al. [75].

Tool	Category	Reference
ChemicalTagger	Rule-/grammar-based chemistry NLP	[49]
Banner- Chemdner	Statistical / ML-based (CRF)	[79]
BC4-CHEMDNER	Statistical / ML-based (CRF)	[79]
becas-chemicals	Statistical / ML-based (CRF, online)	[80]
chemxseer-tagger	Statistical / ML-based (CRF)	[81]
CheNER	Statistical / ML-based (CRF)	[58]
tmChem	Statistical / ML-based (CRF)	[82]
Oscar3	Statistical / ML-based (Naïve Bayes)	[60]
Oscar4	Statistical / ML-based (MEMM/CRF-style)	[60]
ChemSpot	Hybrid (ML + dictionary)	[7]
ChemEx	Hybrid (ChemicalTagger-based)	[83]
LeadMine	Hybrid (rules + dictionary, commercial)	[84]
OntoGene	Hybrid / lexical lookup	[85]
Whatizit	Hybrid / ontology-based	[86]
MetaMap	Ontology-/concept-based	[87]
NCBO Annotator	Ontology-/concept-based	[88]

2.6 Overview Chemical Reaction Extraction Techniques

2.6.1 Dictionary-Based Systems

Dictionary-based systems refer to a family of techniques that find mentions in text by comparing it with a dictionary: an ordered collection of known names in a specific domain. The process of identifying the dictionary names in text is called matching or lookup. Figure 2-18 illustrates a simplified dictionary-based CNER.

Implementing a dictionary matching technique consists in producing a complete and comprehensive dictionary and a good quality matching method [10, 84]. Matching methods can either be lexicon, i.e. exact, or approximate. Hettne et al.[59] and Rebholz-Schuhmann et al. [89] are examples of dictionary-based systems used to extract small molecules via string matching methods .

Generally, the dictionary-based method offers high precision but is prone to low recall when spelling errors occur in the text. Additionally, this technique faces limitations as dictionaries become outdated, since maintaining and updating them demands considerable time and resources [6]. Furthermore, the complexities and the variability in the morphological structure of systematic IUPAC chemical names make it impossible to create a finite dictionary of such names [10, 58].

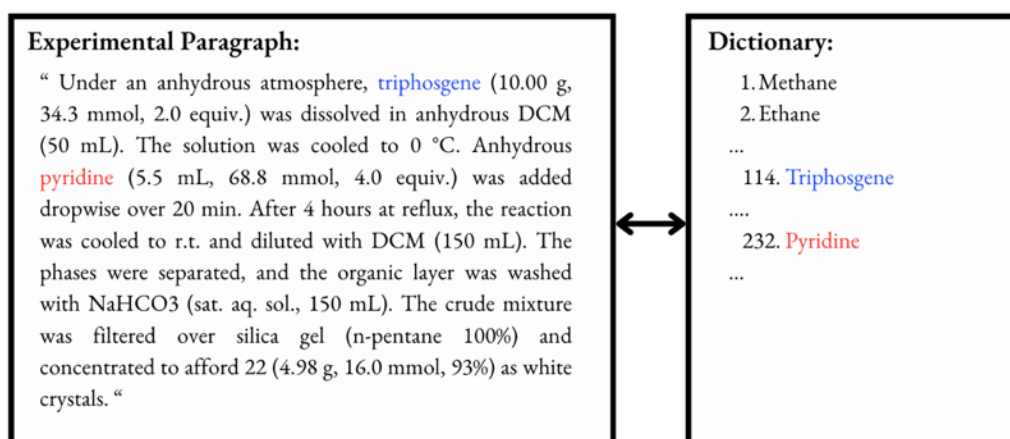


Figure 2-18: Example of dictionary-based CNER. The dictionary does not include the molecule DCM, so DCM is not recognised as a chemical entity in the experimental paragraph.

2.6.2 Rule-Based Systems

Rule-based systems identify and extract entity names (*e.g.*, chemical names) using collections of manually designed rules. These rules typically exploit grammatical information (*e.g.*, part-of-speech tags), syntactic structure (*e.g.*, word order and dependency relations), and often domain-specific dictionaries. Within this paradigm, two broad categories of rules are commonly distinguished [6]: pattern and context-based.

Pattern-based/regex rules capture orthographic or morphological regularities in the text, such as capitalization patterns, affixes, or characteristic character sequences of chemical classes. For example: “Aromatic compounds, such as nitrobenzene and chlorobenzene, were added dropwise to the reaction mixture”. In this case, “nitrobenzene” and “chlorobenzene” are identified as chemical entities because they match a morphological pattern: a functional-group prefix (*e.g.*, nitro-, chloro-, ...) followed by the aromatic core “benzene”. Another example of pattern rules comes from OSCAR4. It can recognise multi-token named entities like “ethyl acetate” with regular expression (regex) matching of chemical tokens to a set of pre-specified pattern rule such as “*yl *ate” [60].

ChemicalTagger provides another example: it uses a regex-based tagger to mark-up chemistry-related terms that OSCAR does not recognise [49]. These include domain-specific nouns such as “*solution*” and “*mixture*” and procedural verbs such as “*quench*” and “*evaporate*”. The regex-tagger relies on

regular expressions stored in a rules file, together with customised tags (a small domain-specific dictionary). This approach is well-suited for identifying fixed textual patterns such as SMILES strings or CAS numbers. Still, it is fragile: small variations in wording, formatting, or typos can cause matches to fail because the method does not account for context. Chemistry-related terms can include the following:

- Boldface numbers: labels that usually refer to a specific chemical in the experiment (*e.g.* "50" in the example paragraph).
- Action verbs: verbs describing experimental operations, such as adding, removing, dissolving, quenching, or evaporating.
- Physical states: descriptors of aggregation state, such as liquid (including oils), solid (including crystals), or gas.
- Units: standardised quantities such as mmol, g, and mL.

In practice, dictionaries of chemical name segments are used in pattern-based rules. A dictionary can be built automatically by segmenting chemical names with manually crafted heuristic rules [90]. There are also official dictionaries of name segments such as the Registry File Basic Name Segment Dictionary [91], which can be used to divide a chemical name into basic name segments. Figure 2-19 shows on how many different dictionaries and rules LeadMine relies on.

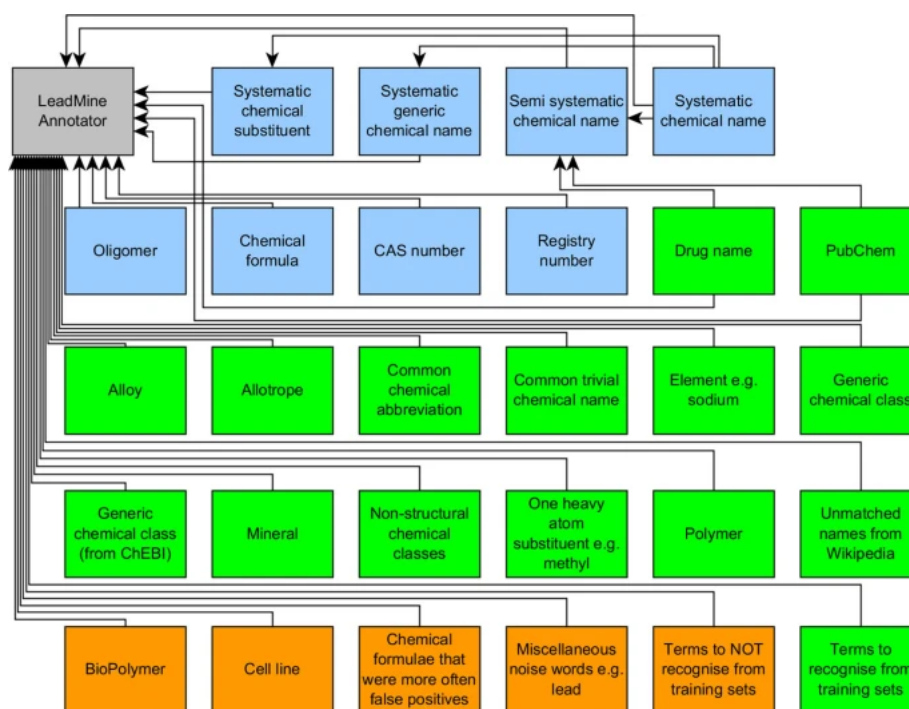


Figure 2-19: Dictionaries and grammar rules used by D. Lowe et al. in LeadMine [84]. (blue: grammars, green: traditional dictionaries, orange: blocking dictionaries)

Context-based rules infer entity types from the surrounding linguistic context, leveraging neighbouring words, phrases, and sentence structure. For instance, a simple context-based rule might state: “If a noun phrase follows the cue phrase *‘the organic solvent’*, then that noun phrase is a solvent name.”

Rule-based systems have notable strengths and weaknesses. Their advantages include high transparency and interpretability, as the decision logic is clearly encoded in explicit rules. Such systems provide consistent, accurate results within well-defined domains, often requiring less training data and offering rapid, reliable performance for tasks governed by expert knowledge [84]. However, rule-based systems struggle with scalability, as maintaining large, complex rule sets quickly becomes unwieldy. They cannot learn or adapt independently to new data or scenarios, and their rigidity makes them ineffective when confronted with ambiguity and incomplete information. Handling complex or nuanced problems with intricate relationships is also challenging, leading to reduced applicability to tasks outside their designed rule coverage. [6, 16, 92]

2.6.3 Machine-Learning Systems

Since around 2011, a new approach has been adopted for extracting chemical entities and reactions from scientific literature: machine learning (ML) [7]. In contrast to dictionary- and rule-based systems, which struggle with new chemical names, spelling variants, unseen jargon, or new molecules, ML models can generalise to unseen entities and variants. They have proved particularly effective at classifying morphologically complex IUPAC names, which are harder to handle than simple brand names, and at identifying synthesis steps in experimental procedures [7, 93].

Machine-Learning Fundamentals for Chemical Text

Basic Concepts

Machine learning is an artificial intelligence (AI) method which uses an algorithm that can “learn” patterns of training data and subsequently make prediction about new data. Any classical machine learning algorithm consists of the following elements: data, representation, model. All three elements are linked together and influence the overall performance as shown in Figure 2-20.

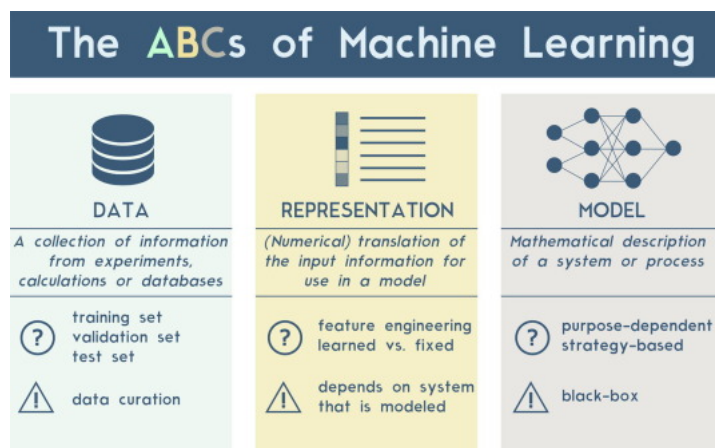


Figure 2-20: Illustration of the ABC of machine learning in Chemical Reaction Extraction, adapted from Dobbelaere *et al.*[94]

In the case of chemical reaction extraction, the **data** is a large volume of scientific literature and patents. Such annotated corpora may be created in-house for specialised use cases, but this is time-consuming and resource intensive. Alternatively, externally curated datasets can be used for training and evaluation, although publicly available labelled chemical text remains limited. A notable example

is the CHEMDNER corpus, comprising 10,000 PubMed abstracts with 84,355 chemical entity mentions manually annotated by expert curators. [63]. Other examples of open-source databases are USPTO [62], NextMove Patent Reaction Dataset [95] and the Open Reaction Dataset (ORD) [96].

In any machine-learning approach, the raw input must be converted into a numerical **representation**, commonly referred to as features. For chemical text, the simplest features are the previous, current and following word. In that case, the probability that word w_i belongs to $class_i$ also depends on w_{i-1} and w_{i+1} : $P(class_i | w_{i-1}, w_i, w_{i+1})$. However this simple system will not be very robust, thereby more features are typically used like orthographic properties (*e.g.* presence of digits, hyphens, brackets), morphological cues (suffixes such as “-ol”, “-one”, “-ate”, “-amide”), dictionary information (membership in lists of known solvents, reagents or units) and local context (neighbouring tokens or part-of-speech tags) [75]. Classical models such as CRFs, SVMs or random forests operate directly on such hand-crafted feature vectors. In contrast, modern neural models learn their own feature representations in the form of word or subword embeddings, which are dense numerical vectors induced automatically from large text corpora [97]. Figure 2-21 shows a potential feature vector for the token “DCM” in the sentence. The feature vector acts as a list of answers to certain questions.

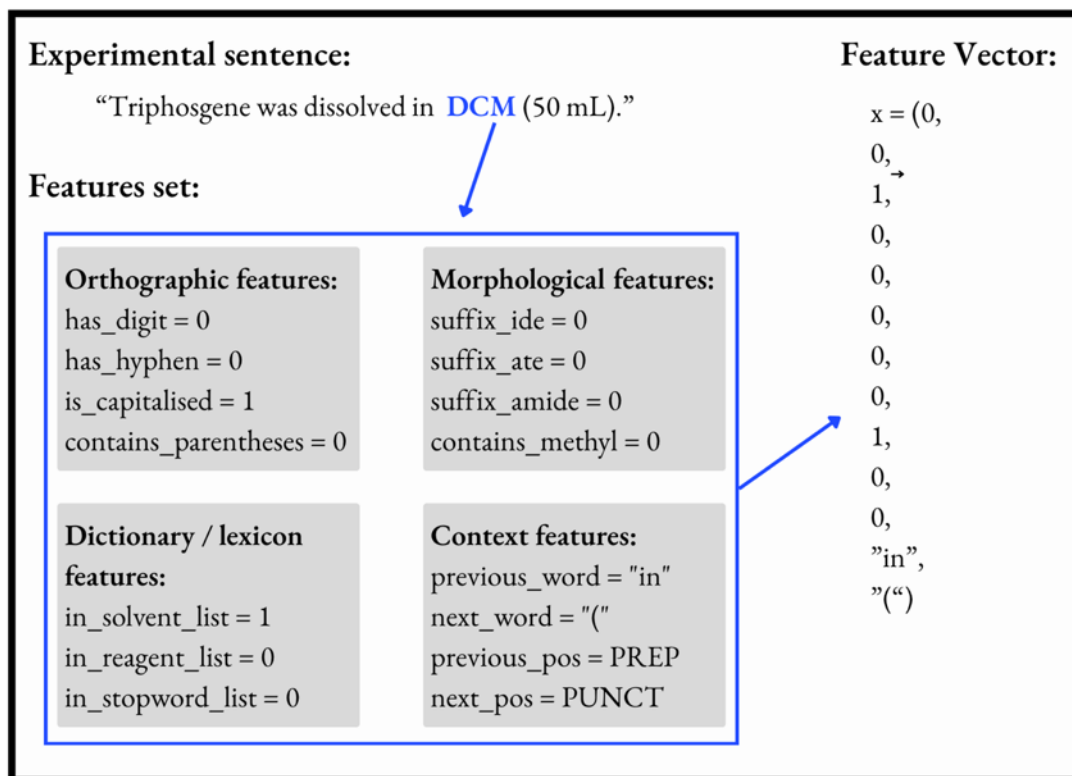


Figure 2-21: Example of a feature set and feature vector of a chemical entity “DCM” in an experimental synthesis section.

A **model** is the mathematical form of how the program is learning. Possible models are Conditional Random Field (CRF) [2], random forest regression [98] and a neural network [92]. It contains parameters, *i.e.*, weights, that will be adjusted during the learning process to improve the performance. Inside the model there is a loss function quantifying how far the model’s predictions deviate from the correct labels and thus provides the objective that training seeks to minimise. For multi-class classification tasks, such as predicting whether a token is part of a chemical name, the loss is typically the cross-entropy between the predicted class probabilities (q_i) and the true class (often represented by a one-hot vector p_i), penalising confident but wrong predictions more strongly than uncertain ones (Eq.(2-1)) [99]. The total number of classes is C . In sequence labelling models like CRFs, this idea is extended to entire label sequences by minimising the negative log-likelihood of the correct tag sequence given the input sentence. For regression problems, for example, predicting reaction yields or boiling points, the loss is often the mean squared error between predicted and true values (Eq. (2-2)).

$$\text{Cross - entropy loss function} = - \sum_{i=1}^c p_i \log (q_i) \quad (2-1)$$

$$\text{Squared error loss function} = (\hat{y}_i - y_i)^2 \quad (2-2)$$

Regardless of the specific form, training a model corresponds to optimising its parameters to minimise this loss over the training data. Several algorithms are used for this like gradient descent[100], stochastic gradient descent [101], etc. Each algorithm firstly computes the predictions, followed by comparing these with the actual value to calculate the loss. The gradient is then calculated to update the parameters. These steps are iterated until the loss is minimal [102].

Two major categories of machine learning are employed in chemical reaction extraction: supervised and unsupervised learning [103-107]. Supervised learning relies on annotated documents in which reaction mentions (*e.g.*, “methane is formed”) are labelled with their roles, such as product, reactant, or condition, allowing a model to learn how to assign these labels in new texts. [99, 108]. In contrast, unsupervised learning works on unlabelled corpora and seeks to uncover latent structure, such as clusters of similar reactions, typical condition patterns, or relationships between reagents and products. This is done purely from co-occurrence and similarity in the data, without predefined labels [99].

Sequence Labelling and Relation Extraction in Chemistry

Extracting chemical reactions consists of two steps: sequence labelling and relation extraction. In the first step, each token (word) gets a label (*e.g.*, B-CHEM, I-CHEM, B-TEMP, B-YIELD, REAGENT, SOLVENT, etc.). An example is given in Table 2-5. CRFs, transformers, and similar models are the workhorses here. In the second step, the relations between the labelled tokens are derived (*e.g.*, which temperature belongs to which step, which solvent is used for the product, etc.). The relation of reactants, products, and conditions can be formulated as a sequence-to-sequence task, in which machine learning outperforms other techniques. Reactions SMILES and other reaction strings are the perfect format for this task, as they present the reaction in one string. Other relation extraction

tools go much further by extracting complete reaction procedures using actions like “add”, “yield”, “extract”, etc[49]. To conclude, machine learning is an ideal tool for identifying these relations as they are more flexible than dictionary- and rule-based models [92].

Table 2-5: Labelling of words in Chemical Named Entity Recognition (CNER).

Word	Label
Sodium	B-Chem
chloride	I-Chem
was	O
dissolved	O
in	O
ethanol	B-Chem

Supervised Machine Learning in Chemical Data Extraction

Supervised learning refers to a class of machine-learning methods that are trained on labelled datasets, in which each input (feature vector) is paired with a target output. The objective is to learn a mapping from inputs to outputs that generalises to unseen data. In chemical data extraction, supervised classification algorithms such as logistic regression, support vector machines (SVMs), decision trees, and neural networks are commonly used to assign categorical labels (i, chemical, unit, temperature) to tokens, spans, or sentences. [99]

Classical Feature-Based Models: CRFs, SVMs, MEMMs

Feature-based models for supervised chemical information extraction are typically built on Conditional Random Fields (CRFs), Support Vector Machines (SVMs), and Maximum Entropy Markov Models (MEMMs). MEMMs perform sequence labelling by predicting the label of each token from a rich feature vector (character n-grams, orthographic patterns, dictionary features, local context) and the previous label; in OSCAR4[60], such MEMM-based predictions are combined with ontology lookup and finite automata for chemical prefixes to identify chemical, reaction,

enzyme and related entities [60] (Figure 2-22). SVMs, in contrast, treat each candidate entity or entity pair as a feature vector and learn a maximum-margin decision boundary between classes, making them effective for role classification and relation extraction (*e.g.*, distinguishing yields from temperatures or reactants from products), while structured SVMs extend this principle to structured outputs such as sequences or trees [109-111] (Figure 2-23). Similar to SVM, Naïve Bayes is also a classifier [112]. CRFs model the conditional probability of an entire label sequence given the input sequence, explicitly capturing dependencies between neighbouring labels without assumptions on the observation distribution, and have therefore become the standard choice for token-level tagging in systems such as ChemSpot (Figure 2-24) and ChemDataExtractor [1, 7, 11, 113, 114].

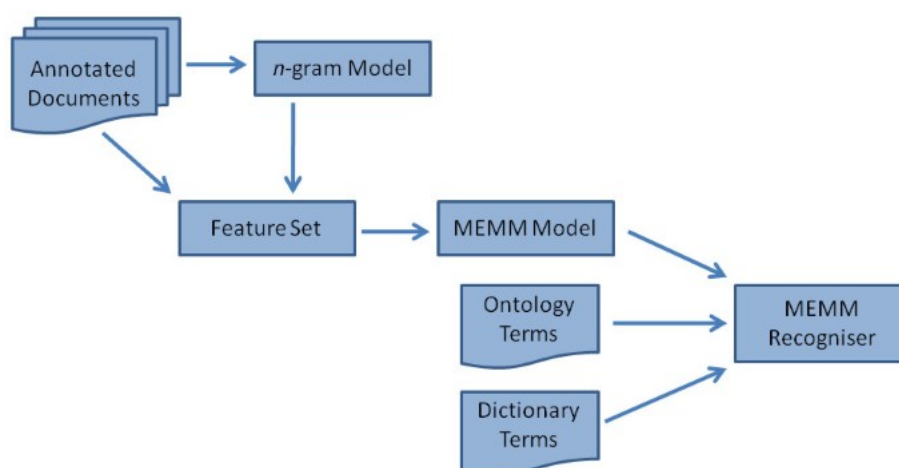


Figure 2-22: MEMMRecogniser architecture of OSCAR4 [60].

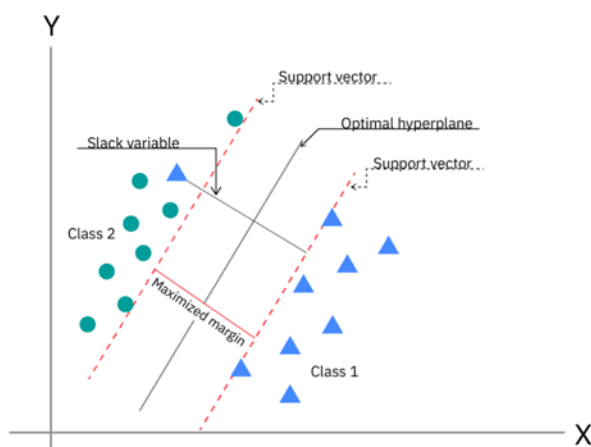


Figure 2-23: Example of a classification of two classes using a Support Vector Machine model from IBM [115].

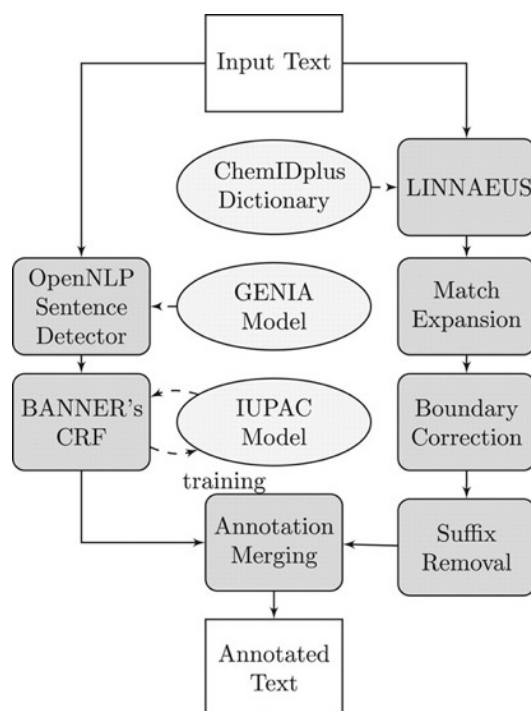


Figure 2-24: Overview of ChemSpot's architecture. The left branch corresponds to ChemSpot's CRF, and the right branch to its dictionary component. Both are independently used to annotate text. Subsequently, annotations of both components are merged, adapted from Rocktäschel et al. [7].

Deep Neural Networks for CNER and Relation Extraction

Deep neural networks represent the next stage beyond feature-based CRFs and SVMs for chemical text processing. Early work employed convolutional neural networks [99] and, more prominently, bidirectional LSTM-CRF (BiLSTM-CRF) architectures to vectorise and label sentences describing chemical reactions [116]. In these models, the BiLSTM yields rich contextualised representations for each token, while the CRF layer enforces globally coherent label sequences [117] (Figure 2-25). Such architectures have been used for core tasks including chemical named entity recognition (CNER), reaction role labelling (reactant, product, solvent, catalyst, etc.), and relation extraction, where sentences or entity pairs are classified to determine whether a particular relation holds (*e.g.*, whether a number is a yield or a temperature associated with a given step).

Compared with the shallow models discussed above, the main difference lies in the way features are constructed. Instead of relying on manually engineered orthographic, morphological, and dictionary features, deep networks operate on word or subword embeddings and learn additional layers of representation automatically during training. This substantially reduces the need for feature

engineering and enables the exploitation of longer-range context, for example, dependencies across clauses in complex experimental descriptions. However, training purely supervised deep models for reaction extraction remains difficult because high-quality annotations (entities, roles, relations) are expensive and require domain expertise [118].

To mitigate this data bottleneck, recent work adopts a pretrain–then–fine-tune paradigm with transformer-based encoders [119, 120]. Models such as ChemBERT [70] are pretrained in a self-supervised manner, meaning labels are automatically generated from unlabelled data, on large collections of unlabelled chemical literature, then coupled with task-specific output layers and fine-tuned on comparatively small, labelled datasets for reaction role labelling (Figure 2-26). ChemRxnBERT further performs adaptive pretraining on a reaction-focused sub corpus (texts explicitly describing at least one reaction and its main product), yielding a task-specialised encoder with improved performance [16]. Beyond token-level labelling, deep sequence-to-sequence models have been used to translate entire synthesis procedures into structured action sequences: for example, Vaškevičius *et al.* [116] combine a neural classifier to select true procedure paragraphs with a seq2seq model (LSTM, BiLSTM, or Transformer) that maps each sentence to a sequence of formal actions (ADD, QUENCH, FILTER, PURIFY) and parameters, illustrating how representation learning underpins end-to-end extraction of reaction workflows.

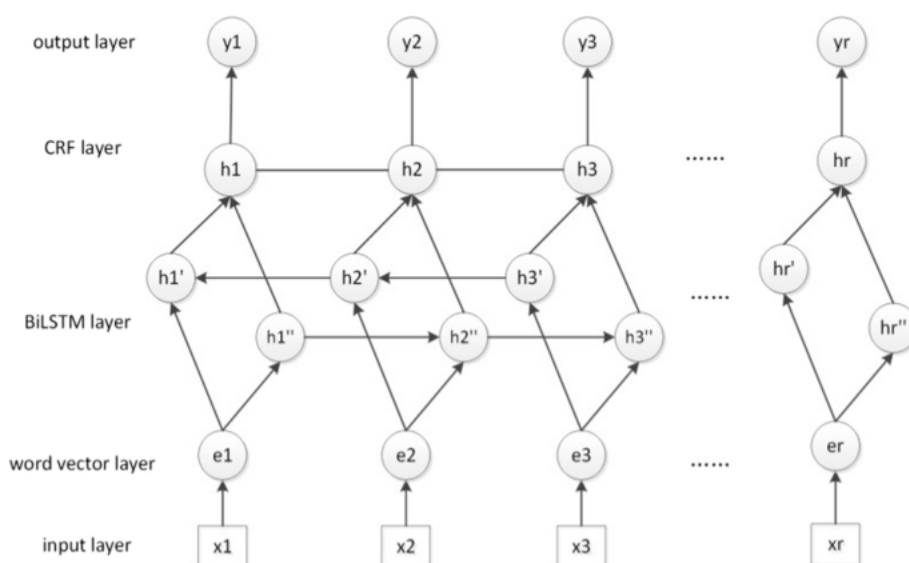


Figure 2-25: The graph model representation of BiLSTM-CRF model from [117].

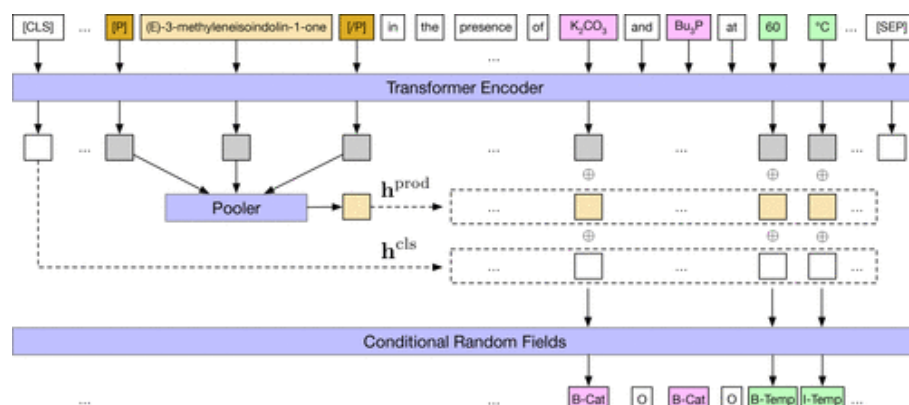


Figure 2-26: Example of transformer-CRF architecture of ChemBert [16].

Unsupervised Machine Learning

Unsupervised machine learning operates on unlabelled data and aims to uncover latent structure, regularities, and intrinsic relationships. In practice, unsupervised word representations (embeddings) are often learned from large corpora and then incorporated as additional features in downstream models [121]; under the distributional hypothesis, words with similar meanings tend to occur in similar contexts and therefore obtain similar embeddings [122]. Because these methods do not rely on manually annotated corpora, they scale readily to large collections of chemical text and reaction data, enabling the induction of previously unseen “reaction grammar” and the discovery of novel relationships between molecules, conditions, and transformations, making them more flexible than supervised models [99, 123, 124].

Word and Token Embeddings in Chemistry

Unsupervised models typically rely on word embeddings as features. Word embedding methods (*e.g.*, Word2Vec [125], GloVe [126]) learn dense vectors from word co-occurrence statistics such that words appearing in similar contexts are mapped to nearby points in embedding space. These representations support both intrinsic tasks, which refer to latent semantic relations (*e.g.*, analogies, synonym detection, acronym disambiguation), and extrinsic tasks, which use embeddings as input to downstream models for named entity recognition, part-of-speech tagging, or semantic role labelling [127-130]. For instance, when trained on synthesis procedures, “esterification” is expected to lie closer (by cosine distance) to “transesterification” or “acylation” than to “distillation”.

Zhang *et al.* [124] proposed a label-free data mining approach based on unsupervised syntactic distance analysis (SDA), which combines word-vector similarity with syntax-tree parsing to extract reaction information. Their pipeline consists of two stages: extraction and correlation. In the extraction stage, the method (i) identifies chemical entities using existing CNER tools (*e.g.* ChemDataExtractor [1, 2], ChemicalTagger [49], OSCAR4 [60], OPSIN [61]), (ii) detects domain-relevant keywords via word embeddings, (iii) derives physicochemical properties from characteristic phrase patterns and distributions, and (iv) recognises operational terms by comparing unknown verb stems to known experimental operations. In the correlation stage, sentences are parsed into English syntactic trees, and relations such as reactant–product and property–chemical are inferred by analysing relative positions and syntactic distances between nodes in these trees.

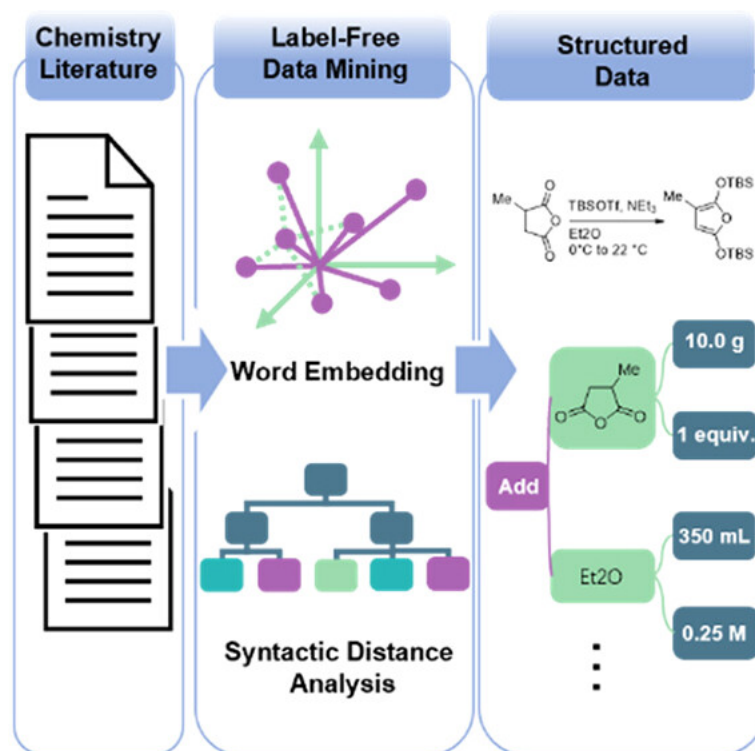


Figure 2-27: Example of the pipeline of a unsupervised extraction model [124].

In summary, unsupervised methods provide a complementary route to chemical reaction extraction by leveraging large volumes of unlabelled text to induce distributed representations and latent “reaction grammars” without the need for costly manual annotation [123]. They are particularly useful for discovering new operations, relations and terminology that lie beyond the coverage of existing dictionaries and rule sets, and for adapting to new subdomains or corpora with minimal

human intervention. However, purely unsupervised approaches typically lack the task-specific precision required for high-quality reaction databases and are therefore most effective when combined with supervised models and rule-based components. Figure 2-28 shows how various components of our natural language processing pipeline incorporate both unsupervised and supervised learning [1], illustrating that state-of-the-art systems increasingly rely on such hybrid architectures to balance scalability with accuracy.

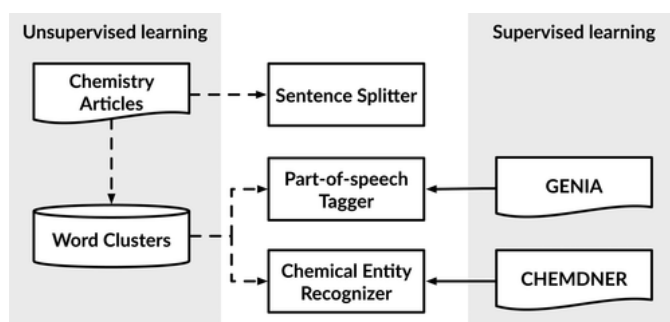


Figure 2-28: Supervised (solid lines) and unsupervised (dashed lines) training methods for machine learning-based NLP components. The sentence tokenizer relies entirely on unsupervised training using the raw text of chemistry articles, whereas the part-of-speech tagger and chemical entity recognizer combine unsupervised features from word clusters with supervised training from labelled corpora, such as GENIA (2000 MEDLINE abstracts with manually annotated part-of-speech tags) and CHEMDNER (10 000 PubMed abstracts with manually annotated chemical entity mentions). [1]

2.6.4 Optical Chemical Structure Recognition (OCSR)

Definition, Motivation and Challenges

Optical Chemical Structure Recognition (OCSR) is concerned with converting graphical depictions of chemical information, ranging from single molecules to full reaction schemes, into machine-readable representations such as connection tables or standard identifiers (SMILES, InChI). Conceptually, OCSR integrates methods from image processing, pattern recognition and sequence modelling to recover the underlying molecular graph and reaction topology from visual input [131]. The task is challenging because the same molecule or reaction can be drawn in many ways, with varying layouts, bond styles, arrow conventions, and annotations. At the same time, performance is strongly affected by image quality, document modality (scanned versus born-digital), and noise [131]. Research on OCSR dates back to the early 1990s, with systems such as Kekulé establishing the basic pipeline for recognising and reconstructing chemical structures from images [12]. Since

then, the field has evolved from rule-based and heuristic approaches towards machine-learning-based methods, mirroring broader developments in computer vision and cheminformatics [132].

OCSR Methodologies

Methodologies for OCSR have progressed from early rule-based systems that emulate human reading strategies to modern machine-learning approaches that treat structure and reaction recognition as statistical pattern-recognition or sequence-generation problems. Rule-based methods rely on explicitly encoded chemical and graphical heuristics to interpret strokes, bonds, atoms, arrows, and labels. In contrast, machine-learning systems learn mappings from images to structured outputs (*e.g.*, Reaction SMILES [28], RInChI [29] or reaction graphs) from data, often within encoder–decoder architectures inspired by image captioning. Together, these approaches define a spectrum from symbolic, highly interpretable systems to data-driven models that can generalise to more diverse drawing styles and complex reaction diagrams.

Rule-Based Systems

Early OCSR systems were predominantly rule-based and sought to mimic the visual interpretation process of a human chemist [12, 22, 133, 134]. These approaches typically decomposed a structure and reaction diagrams into low-level graphical primitives (*e.g.*, strokes, lines, polygons, and textual labels) and applied expert-designed heuristics to reconstruct atoms, bonds, ring systems, and, in simple cases, reaction arrows and roles [135]. A typical pipeline builds an intermediate vector “meta descriptor” of lines, junctions, and symbols, complemented by optical character recognition (OCR) for element symbols and annotations, and then uses deterministic rules to infer the corresponding connection table or reaction topology (Figure 2-29).

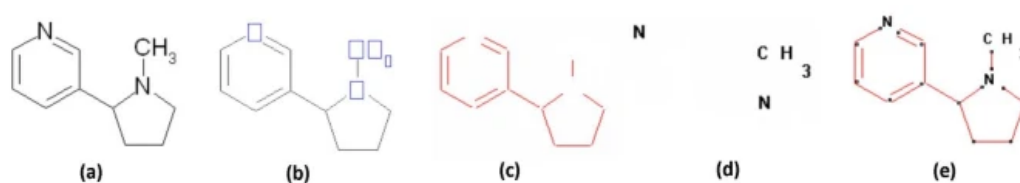


Figure 2-29: Recognition of chemical diagrams 2D images in ChemReader. (a) input image, (b) character-line separation, (c) bond recognised, (d) character recognised, and (e) topology reconstructed and data output. An excerpt from Park *et al.* from their OCSR tool ChemReader[136].

Reaction diagram parsing (reaction OCR) has traditionally been addressed with rule-based, pipelined systems such as early versions of ReactionDataExtractor [137] and OSRA [138]. Images were pre-processed (grayscale conversion, denoising, line detection, polygonal segmentation), arrows, molecular structures, and text labels were segmented using geometric heuristics, and reactants, products, and conditions were assigned based on their relative position to the reaction arrow. Reaction steps were then assembled into a graph by linking products of one arrow to reactants of the next. More recent hybrid pipelines, such as ReactionDataExtractor v2.0 [139] augment this scheme with deep-learning-based object detection followed by symbolic post-processing, but performance remains sensitive to layout variability, graph-style schemes, and noise, and the overall system still relies heavily on manually encoded domain knowledge [140].

Despite their pioneering role, rule-based approaches suffer from intrinsic limitations. Their accuracy depends on the completeness and correctness of the rule set, which is difficult to maintain given the diversity of drawing styles, historical conventions, and edge cases in the literature [136]. They are also susceptible to image defects (broken lines, specks, merged symbols), which can invalidate rule preconditions, and they struggle with reaction formats not anticipated by the rule designer, limiting generalisation across document types and reaction diagram styles [141].

Machine-Learning Systems

With advances in hardware, large-scale datasets, and deep learning, OCSR has increasingly shifted from rule-based heuristics to machine-learning-based systems. Initially, the problem was formulated as a multi-class image classification task, in which each structural image was mapped to a single string label. However, the virtually unbounded number of possible molecules rendered this approach impractical. Modern systems instead cast OCSR as an image captioning problem: given an input image, the model generates a variable-length sequence (*e.g.*, Reaction SMILES or RInChI), thereby combining techniques from computer vision and natural language processing [141].

Most contemporary OCSR architectures adopt an encoder–decoder design. A convolutional neural network (CNN), often pretrained via transfer learning (*e.g.*, Vision Transformers), serves as the encoder and extracts a dense feature representation from the input image [142, 143]. Figure 2-30 illustrates the encoder-decoder pipeline of Img2Mol. The decoder, implemented as a recurrent

neural network (RNN), an LSTM, a Transformer, then autoregressively generates the output string token by token. Attention mechanisms are commonly employed to allow the decoder to focus selectively on different spatial regions of the image during generation, mitigating information loss from compressing the entire image into a single fixed-length vector [144].

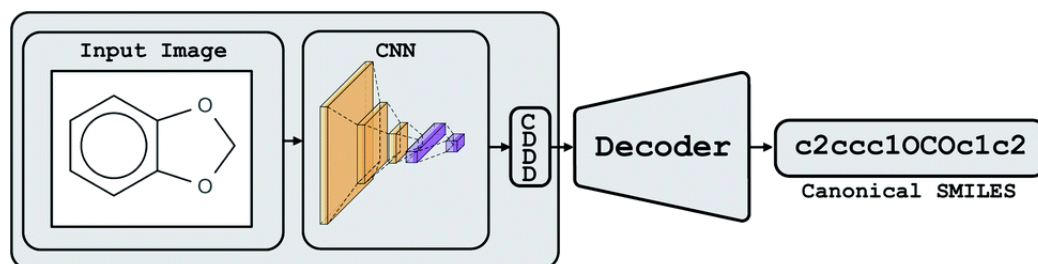


Figure 2-30: Excerpt of the Img2Mol pipeline illustrating its encoder–decoder architecture, adapted from Clevert *et al.* [143].

State-of-the-art reaction-specific systems, such as RxnScribe [121], formulate reaction diagram parsing as an end-to-end sequence-generation problem. A convolutional encoder extracts visual features, and a Transformer decoder generates a token sequence that jointly encodes bounding boxes, entity types ([Mol], [Txt], [Idt]) and their organisation into reactants, conditions and products under a fixed grammar, terminating each reaction with [Rxn] and the complete diagram with [EOS]. Off-the-shelf OCSR models (*e.g.* MolScribe [145]) and OCR tools are then used to convert molecular regions to SMILES and text regions to strings. This sequence-to-sequence formulation reduces error propagation across pipeline stages and substantially outperforms rule-based systems across diverse diagram styles, though residual chemically implausible outputs still require post-validation against chemical constraints. Transformer-based systems such as DECIMER [146, 147], which combine powerful vision encoders with chemically robust string formalisms and beam-search decoding, further improve accuracy. Still, their performance remains dependent on training data coverage and image modality, and hybrid strategies that route different image types to the most suitable OCSR or reaction OCR model remain an active research direction (Figure 2-31).

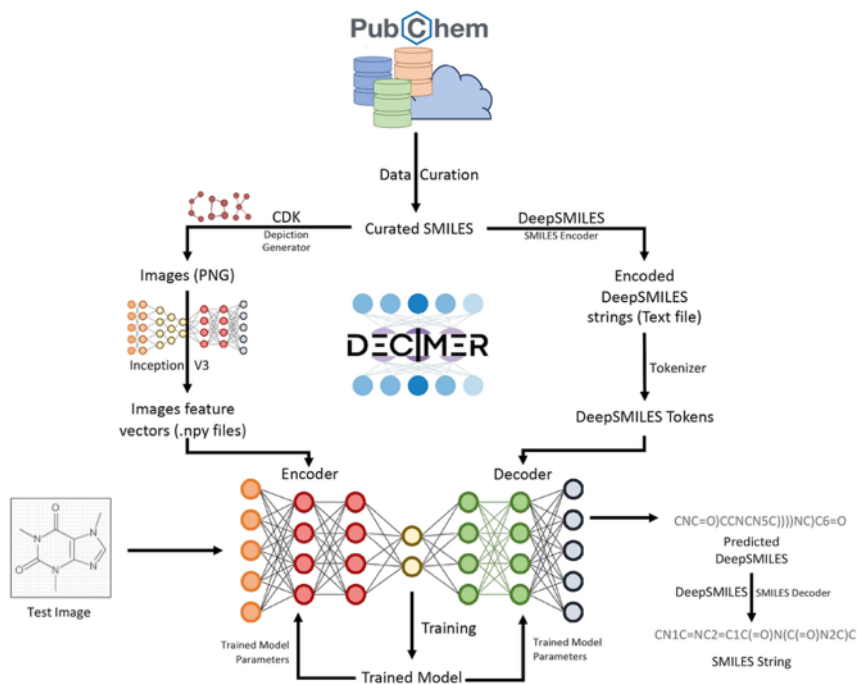


Figure 2-31: Excerpt of the DECIMER pipeline illustrating its encoder–decoder architecture, adapted from Rajan *et al.* [146].

Within this encoder–decoder paradigm, two architectural families have emerged that differ in what the decoder is asked to predict. *Image-to-sequence* approaches treat OCSR as a literal captioning task: the decoder autoregressively emits a SMILES string token by token, with chemical validity enforced only at decode time through constrained beam search or post-hoc validation. *Img2Mol* [143] and DECIMER [146] are canonical examples. *Image-to-graph* approaches instead have the decoder predict the molecular graph itself, including atoms, bonds, and (in some variants) their two-dimensional coordinates, which is only afterwards serialised to a SMILES or InChI string. Because the predicted output is structural rather than lexical, valence, connectivity, and stereochemistry constraints can be enforced during graph assembly rather than retroactively, which has consistently translated into stronger exact-match performance on perturbed and out-of-distribution images. MolScribe [145] established this paradigm; MolNexTR [148] extends it with a hybrid ConvNeXt–Transformer encoder and explicit reasoning about chirality and abbreviated groups, and is the per-molecule recogniser adopted in the pipeline described in Chapter 3.

More recently, Tang *et al.* [149] examined whether general-purpose vision-language models can be adapted to OCSR via supervised fine-tuning. Their model, MolSeek-OCR, fine-tunes DeepSeek-

OCR-2 [158] on a combined corpus of synthetic PubChem renderings and realistic USPTO-MOL patent images using a two-stage progressive strategy: parameter-efficient LoRA [163] followed by selective full-parameter fine-tuning with split learning rates for the visual and language components. The resulting model achieves exact-match accuracies broadly comparable to the strongest image-to-sequence baseline (DECIMER [146]), but it remains substantially behind state-of-the-art image-to-graph systems such as MolScribe [145] and MolNexTR [148] across every benchmark evaluated (e.g. 61.0 versus 92.6 % exact-match on USPTO-MOL and 29.9 versus 71.9 % on the ACS test set). The authors further report that reinforcement-style post-training and additional data-curation refinement fail to close this gap. Together, these results reinforce the architectural argument made above: explicit prediction of atom-and-bond geometry currently retains a clear advantage over autoregressive SMILES token generation for OCSR and motivates the choice of MolNexTR as the per-molecule recognition backbone of the pipeline described in Chapter 3.

2.6.5 Hybrid and Multi-Modal Systems

In practice, high-performance chemical reaction extraction systems are rarely purely dictionary-, rule-, machine-learning-, or OCSR-based; instead, they typically adopt hybrid architectures that integrate multiple components. The main motivation is to combine the complementary strengths of symbolic approaches (high precision, interpretability, explicit chemical constraints) with statistical models (robustness to noise and improved generalisation). A common design pattern applies machine learning to perception-heavy subtasks, such as token tagging, entity recognition, or diagram segmentation, followed by rule-based post-processing, curated lexicons, and cheminformatics toolkits to enforce domain constraints and normalise the extracted outputs [150].

Several established pipelines already follow this paradigm. For example, ChemSpot [7] and ChemDataExtractor [1], combine CRF-based chemical named entity recognition with lexicons, regular expressions, and rule-based post-processing for abbreviation resolution and template-driven extraction. Similarly, ReactionDataExtractor v2.0 [139] combines deep object detectors for arrows, structures, and textual regions with symbolic graph-building rules that infer reactant, product, and condition roles based on spatial arrangement around arrows. In OCSR, tools such as OSRA[138] or modern DECIMER-based [151] workflows are frequently integrated with name-to-structure

resolvers (*e.g.* OPSIN[61]) and RDKit-based [152] valence checks to ensure chemically implausible structures are corrected or discarded.

Beyond hybridisation across algorithmic paradigms, reaction information is frequently distributed across modalities (narrative text, tables, and figures), for example when conditions are tabulated or structures contain abbreviated fragments (*e.g.*, R-groups) [153]. Consequently, more complete extraction increasingly requires multi-modal systems that jointly aggregate and reconcile evidence across document regions and media [154]. Recent toolkits such as OpenChemIE [155] exemplify this direction by integrating extraction from text, images, and tables to automate the capture of experimental reaction records from synthesis papers. A further evolution is the growing use of LLMs as central extraction modules, where language(-vision) models can support end-to-end parsing and schema-grounded structuring while still benefiting from downstream chemical validation and normalisation steps within established cheminformatics pipelines [4].

2.7 LLMs and VLMs as New Tools for Chemical Reaction Data Extraction

Hybrid and multimodal designs are proven to be high-performance reaction extractors typically adopting learning-based components handle noisy perception, while symbolic rules and cheminformatics toolkits enforce chemical plausibility and normalise outputs [155]. Large Language Models (LLMs) and Visual Language Models (VLMs) extend this paradigm by enabling schema-grounded extraction from heterogeneous document content (text, layout, tables, figures) through a single instruction-driven interface [156, 157]. In practice, they shift the pipeline focus from many narrowly trained classifiers to a more flexible information-to-schema mapping, while preserving the need for downstream chemical validation [158]. Thus, the main advantage of LLMs and VLMs is their ability to solve tasks they have not been specifically trained on, compared to classical machine learning models which are only specialised in one task [4].

2.7.1 Conceptual Positioning Relative to Classical ML and Hybrid Systems

LLM/VLM-based extraction differs from classical ML and hybrid pipelines in three conceptually important ways.

First, task specification moves from model design to prompting. Instead of training separate models for named entity recognition, relation extraction, and template filling, an LLM can be instructed to produce a reaction record directly. Here, the prompt acts as a contract that defines (i) the target schema (fields and nesting), (ii) allowable units and value types, and (iii) what constitutes acceptable evidence [158].

Second, LLMs enable schema-grounded reasoning over context, which is often required in experimental text. Reaction procedures contain co-reference (“the mixture”, “the residue”), implicit operations, abbreviations, and context-dependent definitions that tend to break rigid rules. LLMs are not “more chemical” by themselves, but they are often more robust in mapping varied phrasing into a stable schema, especially when instructed to justify extractions using local evidence [156].

Third, VLMs provide a practical route to multimodal alignment. In many papers, critical conditions are written above reaction arrows, yields are embedded in schemes, and key parameters appear only in tables or captions. VLMs can interpret these visual elements and align them with nearby text, allowing extraction to operate over the document record rather than over isolated text blocks [132]. Figure 2-32 illustrates the parsing of a reaction diagram and how cross-modality interdependencies are resolved.

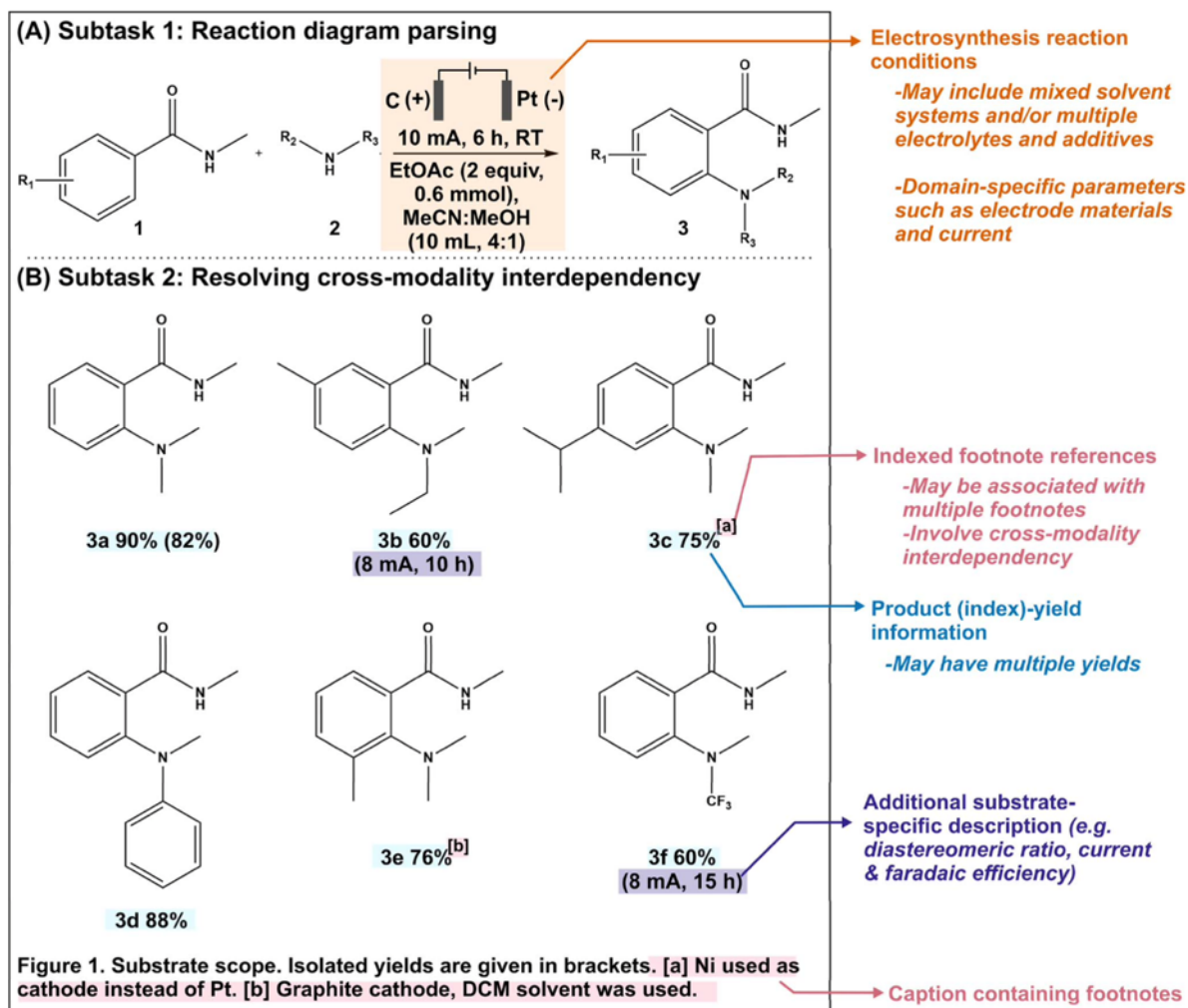


Figure 2-32: An example of a typical figure depicting the overall reaction diagram and substrate scope, with the corresponding caption. The figure is adapted from Leon *et al.* [157]. It highlights how the reaction diagram is parsed and how the cross-modality data interdependency is resolved.

Because LLMs/VLMs may still generate plausible but incorrect content, they should be framed as an interpretation layer, strong at reading and structuring, while correctness remains enforced by tool-based constraints [159].

2.7.2 LLM/VLM-based Extraction Methodology

A useful way to present modern LLM/VLM-based extraction without excessive model detail is to structure the methodology into four phases: preprocessing, LLM interaction, multimodal integration, and postprocessing/validation.

Preprocessing

The purpose of preprocessing remains the same as in the previous extraction techniques. Documents are converted into machine-readable representations through visual document understanding (VDU), typically including OCR for scanned pages or embedded text, followed by cleaning steps such as removing headers, author metadata, and reference lists[71]. For instance, the VLM model VisualHeist [156] and the new DeepSeek OCR V3 [160] model are capable of segmenting text, images and tables as shown in Figure 2-32. In particular for chemical reaction diagrams, segmenting the molecules from each other, as shown in Figure 2-33 is an important factor contributing to the overall performance [161].

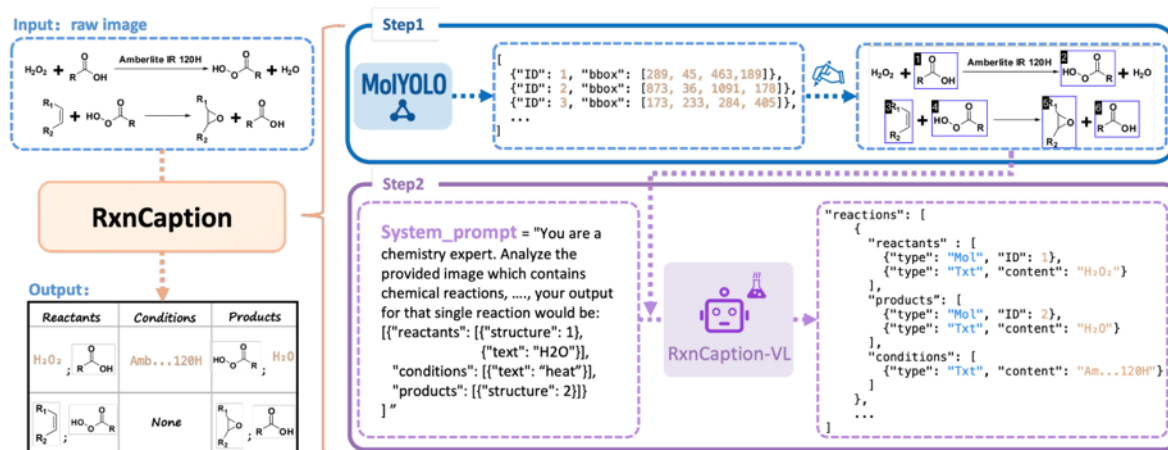


Figure 2-33: Workflow of parsing chemical reaction diagrams and creating a structured reaction record with a Large Vision-Language Model (LVLMs), adapted from Song *et al.* [161].

Since LLM context windows are finite, long documents are processed via chunking (fixed-length or section-aware semantic segmentation) or retrieval-augmented generation (RAG), where only the most relevant chunks (e.g., experimental procedures, captions, condition tables) are retrieved and provided as context. Overall, preprocessing directly constrains the evidence accessible to the model and thus the attainable extraction completeness [4].

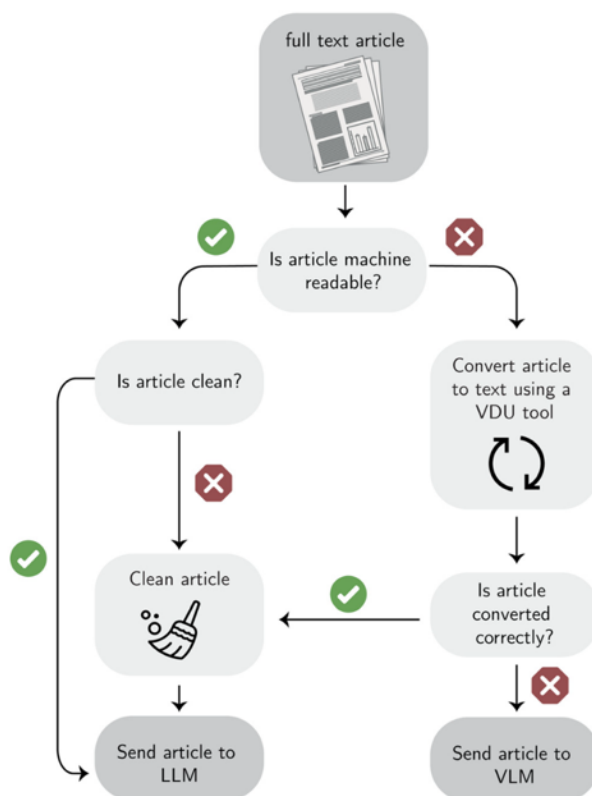


Figure 2-34: Example of a data preprocessing workflow adapted from Schilling-Wilhelmi *et al.* [4].

LLM Interaction

At the core of the pipeline, an LLM recognises chemically named entities (CNER) and relations between them. In practice, two modelling paradigms are used: prompt-based in-context learning and fine-tuning.

Prompt Engineering

Prompt engineering (in-context learning) is the most rapid deployment approach because it requires no additional training. In zero-shot settings, the model relies on its pretrained knowledge [162]; few-shot prompts provide a small number of exemplars to improve format stability and field completeness [163]. Chain-of-Thought (CoT) [164] prompting may support multi-step reasoning, but for extraction it is best treated as an internal aid while enforcing a strict final output schema. Crucially, prompting serves as task specification: the prompt should explicitly define the target data model like mandatory fields, unit conventions, missing-value handling, and, where possible, an evidence requirement for non-trivial entries [4].

Fine-tuning of LLMs

Fine-tuning (model adaptation) is warranted when prompting does not achieve the required fidelity, particularly for strict and nested outputs such as ORD-style procedures [4]. Supervised fine-tuning (SFT) on curated examples improves task alignment and output consistency, while parameter-efficient fine-tuning (PEFT) methods (e.g., LoRA[165]) reduce computational cost by updating only a small subset of parameters. Overall, fine-tuning should be presented as an engineering trade-off: prompting enables fast iteration and flexibility, whereas fine-tuning supports high-fidelity, schema-stable extraction [166].

Multimodal Integration (VLM/MLLM)

Vision–language models (VLMs), also referred to as multimodal large language models (MLLMs), process textual and visual inputs jointly by combining an image encoder with a language model to generate structured outputs conditioned on visual evidence, shown in Figure 2-35. In contrast to earlier multimodal pipelines that merged the outputs of separate text and image extractors, VLMs enable direct alignment of visual and textual information within a single reaction record, thereby resolving cross-modality dependencies more coherently [157].

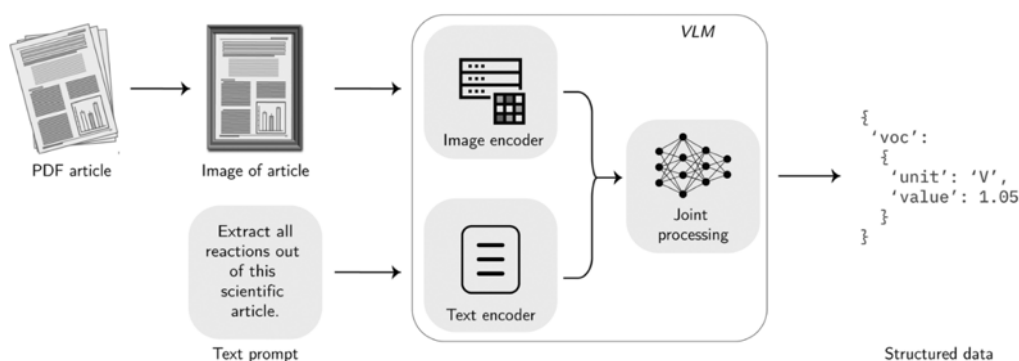


Figure 2-35: VLM workflow chemical data extraction workflow, adapted from Schilling-Wilhelmi *et al.* [4]. The VLM processes the text prompt and article’s figure simultaneously creating a structured reaction record.

Recent studies show that this integrated approach leads to substantial gains in extraction accuracy. VLM-driven pipelines such as MERMaId [156] achieve an end-to-end accuracy of approximately 87% when converting information from PDF documents into structured knowledge graphs, while its DataRaider module, based on a GPT-4o VLM [167], reports accuracies exceeding 92% for

reaction-parameter extraction. Similarly, RxnIM [71], a specialised multimodal large language model, attains an F1-score of 88% for reaction component identification, outperforming previous state-of-the-art methods. By contrast, general-purpose multimodal models (e.g., GPT-o3) consistently underperform on strictly defined chemical extraction tasks, highlighting the critical role of domain-specific training and task-oriented model optimisation in achieving high-fidelity multimodal chemical data extraction [71].

Agentic Systems

Agentic systems constitute an advanced methodological layer in automated chemical reaction extraction, extending large language models (LLMs) with structured mechanisms for tool use, planning, and reflection [168]. Rather than treating the LLM as a single-pass parser of experimental text, agentic approaches frame it as a controller that incrementally constructs, verifies, and refines reaction records. Agentic systems can be simple as in Figure 2-36 consisting of one agent with several tools or very complex as in Figure 2-37, which illustrates a complex network of agents who communicate with each other to execute tasks [4, 156, 169].

A defining characteristic of agentic systems is tool use, through which the LLM augments its reasoning with specialised chemical and document-analysis components, shown in Figure 2-36. In the context of reaction extraction, this includes OCSR models for recovering molecular structures from reaction schemes, OCR and table extractors for parsing conditions and yields, chemical databases and APIs for name normalisation and identifier resolution (e.g. via retrieval-augmented generation), and cheminformatics libraries for structural and stoichiometric validation. Frameworks such as ReAct [170] formalise this interaction by interleaving reasoning steps with tool invocations, enabling the LLM to dynamically assemble task-specific workflows (e.g. structure extraction → condition parsing → schema assembly). Although not explicitly designed as an autonomous agent, the MERMaid [156] pipeline exhibits agent-like behaviour by coordinating modular tools such as VisualHeist (for joint segmentation of reaction schemes, tables, and captions) and KGWizard (for schema-consistent database construction and entity disambiguation).

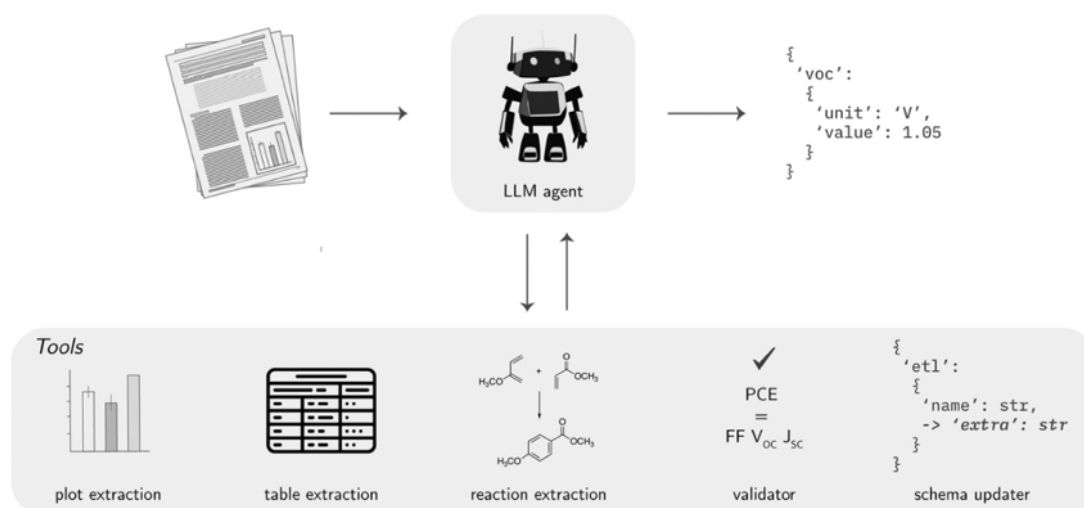


Figure 2-36: General workflow of a zero-level agent in chemical reaction data extraction. The agent receives scientific articles as text, figures and tables. The reasoning capabilities of the agent can decide which tool is best to use in extracting the chemical reaction data and converting it into a structured reaction record. The figure is adapted from Schilling-Wilhelmi *et al.* [4].

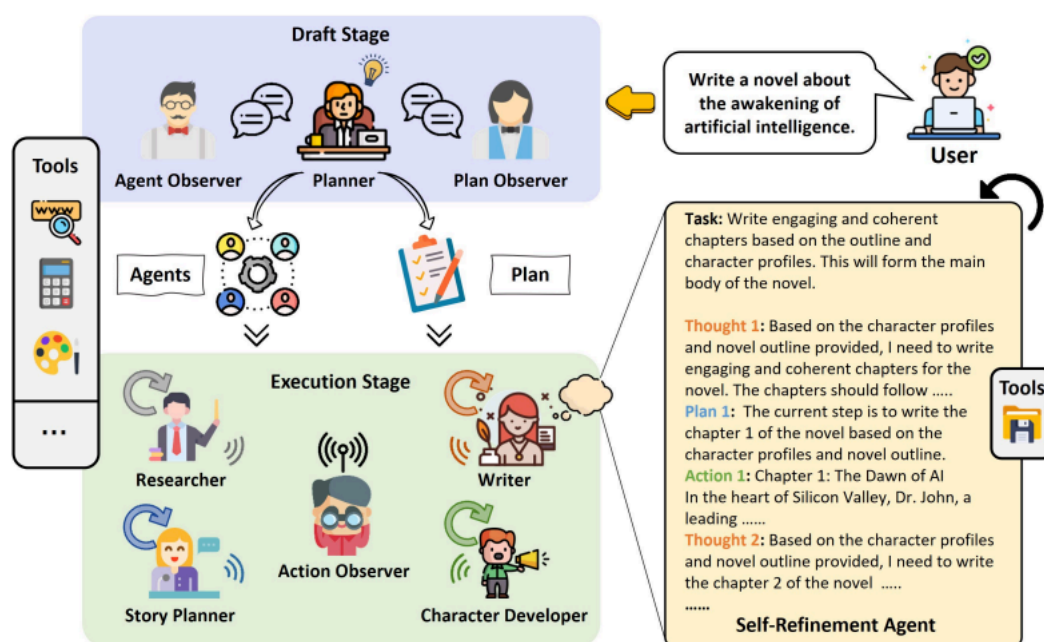


Figure 2-37: Workflow of multi-agent collaboration adapted from Chen *et al.* [169]. The workflow starts with a human query initiating the draft stage: different agents start planning and creating new agents for the execution stage. In the execution stage, new agents with different roles execute the plan using a variety of tools. The digital workflow is similar to one in human company.

A second core component is reflection, which enables iterative assessment and correction of extracted reaction data. Reflection mechanisms, such as self-critique [171], follow-up prompts, or Chain-of-Thought with reflection [172], are particularly valuable in reaction extraction, where hallucinated

reagents, misplaced conditions, or incorrect role assignments can otherwise remain undetected[173]. Within MERmaid [156], the DataRaider module illustrates agent-like reflection by internally validating extracted visual inputs, resolving cross-modal coreferences between reaction labels in figures and their textual definitions, and performing limited context-driven inference to complete conventional reaction details when explicitly absent. Its two-stage prompting framework further acts as a safety layer, ensuring that entry-specific modifications described in footnotes are correctly propagated to individual reaction records.

Despite their advantages, agentic systems introduce challenges that are especially relevant for reaction extraction. Finite context windows constrain the amount of procedural and graphical context that can be retained across reasoning steps; errors introduced during early tool calls (e.g. incorrect structure recognition) may propagate and amplify through subsequent reasoning; and poorly constrained objectives may lead to unstable or repetitive reasoning loops [158, 174, 175]. In addition, evaluation becomes more complex, as correctness must be assessed not only at the level of the final reaction schema but also with respect to the intermediate reasoning and validation steps. Consequently, agentic systems in chemical reaction extraction should be viewed as controlled orchestration frameworks that improve robustness, consistency, and scalability, rather than as fully autonomous extractors that replace chemical domain constraints[156].

Postprocessing and Validation

Postprocessing constitutes a critical and often iterative stage in automated chemical data extraction pipelines, following the initial output of LLMs and VLMs. Its primary objective is to transform raw model outputs into syntactically valid, chemically consistent, and interoperable records suitable for downstream analysis and database construction[4].

First, output control via constrained decoding is applied to enforce compliance with a predefined schema (e.g. JSON or ORD-like formats [96]). By restricting token generation to grammar-consistent structures and using deterministic decoding (typically at zero temperature), syntactic errors and formatting ambiguity are substantially reduced, ensuring reliable parsing and evaluation [176, 177].

Second, normalisation and coreference resolution are performed to guarantee consistency across extracted records [156]. Chemical entities are standardised by resolving names and aliases to canonical identifiers such as SMILES [23] or InChI [34] using tools like OPSIN [61] and database queries (e.g. PubChem [178]), while Retrieval-Augmented Generation (RAG) [179] is employed to detect and merge duplicate entities against existing records. Numerical values are converted to standard units to enable quantitative comparison.

Finally, chemical validation and quality control leverage domain-specific constraints that are unique to chemistry. Cheminformatics toolkits such as RDKit [180] and Rxn-INSIGHT [77] are used for valence checking, molecule sanitisation, and canonicalization, while reaction-level validation (e.g. atom mapping with tools such as RXNMapper [123] as shown in Figure 2-38) verifies mass and atom conservation between reactants and products. Complementary LLM-based self-verification strategies, including follow-up prompts or chain-of-verification schemes [181], can further reduce hallucinations by encouraging consistency checks. Together, these postprocessing steps ensure that extracted reactions are not only structurally correct, but also chemically plausible and database-ready [182].

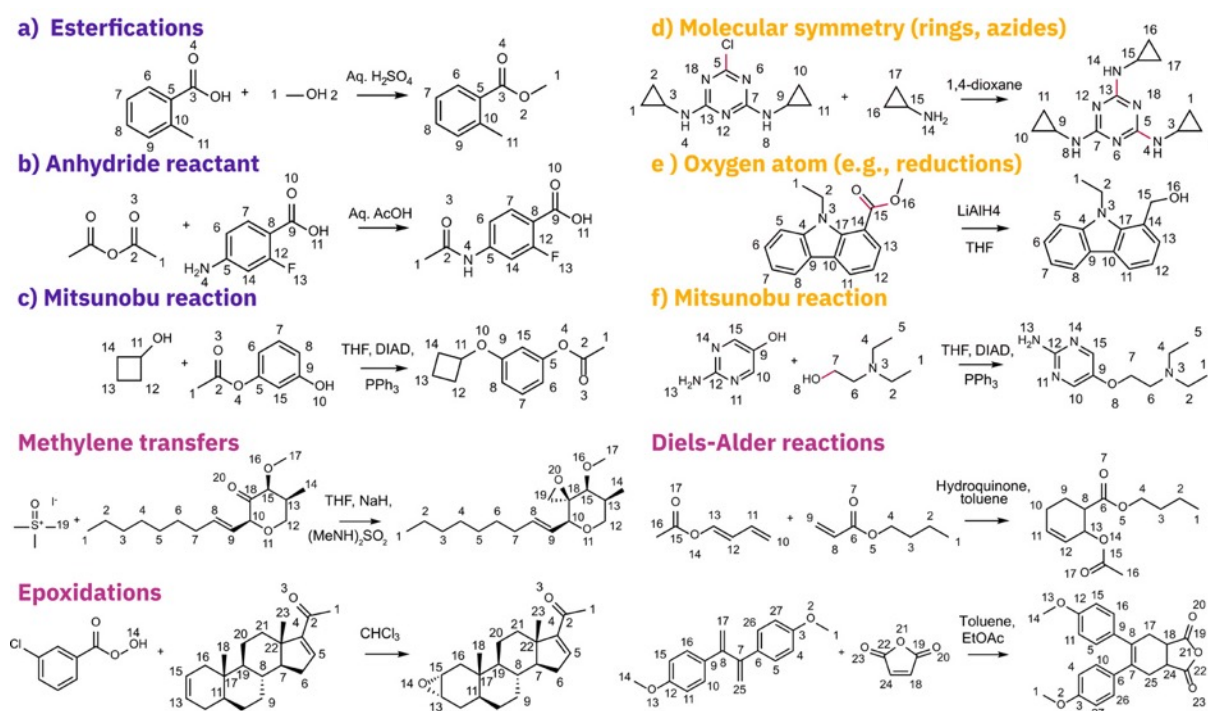


Figure 2-38: Several examples of atom mappings in reactions generated by RXNMapper, adapted from Schwaller *et al.* [123].

2.7.3 Strengths and Limitations

Automated chemical data extraction using LLMs and VLMs offers several distinct strengths compared with earlier rule-based and classical machine-learning approaches. These models exhibit increased robustness to linguistic variability, enabling them to handle paraphrasing, implicit procedural steps, and heterogeneous reporting styles commonly found in experimental chemistry. Their strong zero- and few-shot capabilities reduce reliance on large, fully annotated datasets, which is particularly advantageous given the cost and legal constraints of chemical data curation. In addition, VLMs provide a unified framework for multimodal integration, allowing reaction information distributed across text, tables, and figures to be aligned within a single structured representation [4, 183, 184].

Despite these advances, significant model-level limitations remain. A central concern is hallucination, whereby LLMs generate chemically incorrect or unsupported information, especially under vague or underspecified instructions. In agentic or multi-step reasoning settings, such errors can propagate and amplify across successive steps, leading to substantial deviations in the final extracted reaction record [185]. Furthermore, biases in the scientific literature, most notably the underreporting of negative or failed experiments, may be implicitly learned by models, limiting their robustness when applied to real-world or incomplete data scenarios.

Finally, operational and evaluation challenges constrain large-scale deployment. Multimodal and multi-agent systems incur high computational and financial costs, with inference time and token usage increasing rapidly as workflows become more complex. Evaluation is inherently difficult, as performance depends not only on the correctness of the final output but also on intermediate reasoning and tool-use decisions, for which ground truth is often unavailable [186]. Additional concerns include the need for extensive postprocessing and expert validation, as well as ethical and security risks associated with autonomous tool interaction. Consequently, current systems are best deployed within controlled, constraint-driven pipelines, where LLMs enhance extraction flexibility but remain bounded by chemical validation, human oversight, and safety mechanisms [187].

2.8 Evaluation Criteria and Metrics

To compare systems fairly, their fidelity must be assessed with consistent criteria. Standard metrics include precision, recall, and F1-score [16]. In extraction settings this is non-trivial: outputs often contain multiple, related fields in nested schemas, and many entities can be expressed synonymously. Careful manual audits are therefore valuable to identify recurrent errors. Sayeed et al. showed that targeted manual inspection can inform an automated evaluation pipeline for systematic, cross-system assessment [188].

The simplest setting can be thought of as extracting a single value. For example, if the task is collecting all chemicals in a document, the performance of a model could be scored by measuring how many chemicals were found (retrieved entries) out of all the ones reported in the documents (relevant entries) and how many of the extracted chemicals are correct. This first metric is called recall, Equation (2-3). The $\cap$ symbol denotes a set intersection between the set of entries, and $|\text{set}|$ indicates size of a set.

$$\text{Recall} = \frac{|(\text{relevant entries}) \cap (\text{retrieved entries})|}{|(\text{relevant entries})|} \quad (2-3)$$

The second measure is known as precision and is defined in almost the same way, except for the denominator being the retrieved entries, Equation (2-4).

$$\text{Precision} = \frac{|(\text{relevant entries}) \cap (\text{retrieved entries})|}{|(\text{retrieved entries})|} \quad (2-4)$$

Intuitively, recall can be thought of as how many of the values we expected are extracted correctly and precision as out of just the extracted values how many are correct. Based on those two definitions, additional metrics can be defined, such as the F1-score Equation (2-5) (which is the harmonic mean between precision and recall and known as the Dice score).

$$F1 - score = \frac{2 \text{ Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2-5)$$

Due to the infancy of evaluations of structured extractions, there is much confusion about how to use metrics meant for classification models for this task. We propose to define true positive (TP) as a

correct value extracted. False positive (FP) is when there is an extraction, but the value does not match what we expect, this is called an hallucination. False negative (FN) can be taken as a value that is in our ground truth but has not been extracted, they were missed by the extraction tool. True negative (TN) is not applicable and cannot be realistically defined for our task. True negative can be thought of as a value that we did not ask the extraction tool and it did not provide it. Therefore, it is every concept or word available in our document or vocabulary that hasn't been reported. Table 2-6 provides examples of these different error types one might encounter in structured data extraction. Using these definitions, we can calculate many metrics like the F1-score as it does not use true negative. The recall and precision based on true positive (TP), false positive (FP), and false negative (FN) looks like this.

$$\text{Recall} = \frac{|TP|}{|TP| \cap |FN|}, \text{ Precision} = \frac{|TP|}{|TP| \cap |FP|}, \text{ F1} = \frac{2 |TP|}{2 |TP| + |FP| + |FN|}$$

Table 2-6: Examples of True Positive, False Positive, False Negative and True Negative with from the content “Nitrogen (N₂) and hydrogen (H₂) react to form ammonia (NH₃) creating -92.28 kJ per mole of N₂”.

Outcome Type	Extracted data	Expected Data
True Positive (TP)	Reactants: N ₂ and H ₂	Reactants: N ₂ and H ₂
	Products: NH ₃	Products: NH ₃
	Enthalpy: -92.28 kJ	Enthalpy: -92.28 kJ
False Positive (FP)	Reactants: N ₂ and He	Reactants: N ₂ and H ₂
	Products: NH ₃	Products: NH ₃
	Enthalpy: -92.28 kJ	Enthalpy: -92.28 kJ
False Negative (FN)	Reactants: N ₂ and H ₂	Reactants: N ₂ and H ₂
	Products: NH ₃	Products: NH ₃
		Enthalpy: -92.28 kJ
True Negative (TN)	None	None

The performance of an extraction tool will be examined on an annotated gold standard dataset, the benchmark dataset that is considered the most reliable and accurate in a specific context. It is used to evaluate the quality of work done by individuals by comparing their results to this trusted dataset. In the best-case scenario, this dataset already exists so it is just plug-in and play. At worst you must create this dataset by your own, which is crucial but time consuming. Therefore, using an automation tool is recommended. For doing so, TeamTat can be a convenient choice, but there are also other tools available such as Label Studio, Doccano or Prodigy [189-192].

2.9 Conclusion and Current Challenges

This chapter has positioned chemical reaction extraction as an engineering response to a structural mismatch: reaction knowledge is reported at scale in articles and patents, yet remains dispersed across narrative prose, tables, and reaction schemes that are not directly reusable in machine-readable form. Against this backdrop, the chapter connected (i) the reaction representations that extraction systems aim to produce, (ii) the recurring pipeline stages that transform documents into structured reaction records, and (iii) the historical progression from symbolic approaches toward data-driven and multimodal systems. The overarching message is that reaction extraction is best understood as an end-to-end information-to-schema problem, where document heterogeneity, reporting conventions, and output requirements jointly determine which methods are viable in practice.

Several elements work well today under realistic constraints. Hybrid pipelines that combine statistical components for perception-heavy steps (e.g., entity recognition or diagram segmentation) with symbolic postprocessing remain a pragmatic route to high-fidelity records, because they leverage complementary strengths: robustness to linguistic variability on the one hand, and explicit chemical constraints and normalisation on the other. Modern OCSR and reaction-diagram parsers have also matured to the point that schemes can be converted into structured representations more reliably than earlier heuristic approaches, especially when integrated with downstream validation. In parallel, LLMs and VLMs extend this toolbox by improving schema-grounded mapping from diverse phrasing and mixed modalities, if outputs are tightly controlled (e.g., deterministic, schema-

constrained decoding) and followed by normalisation and chemical plausibility checks (e.g., identifier resolution, unit harmonisation, valence and mass-balance validation).

The main limitations are therefore less about the existence of capable components and more about robustness, traceability, and measurement across the full workflow. Upstream, the source material itself remains difficult: patents introduce legalistic phrasing, Markush variability, and prophetic examples; articles rely on abbreviations, co-reference, and implicit conventions; and key values may only appear in figures or tables, each of which increases sensitivity to preprocessing and segmentation quality. At the model level, LLM/VLM-based extraction remains exposed to hallucination and error propagation, particularly when prompts are underspecified or when multi-step/agentic workflows amplify early mistakes; computational cost further constrains large-scale deployment. Finally, evaluation is intrinsically hard for nested reaction schemas with synonymy and partial matches: standard precision/recall/F1 require careful operational definitions (including how to treat hallucinated fields), and reliable assessment typically demands gold-standard datasets plus targeted manual audits to understand systematic failure modes.

2.10 References

1. Swain, M. and J. Cole, *ChemDataExtractor: A Toolkit for Automated Extraction of Chemical Information from the Scientific Literature*. Journal of Chemical Information and Modeling, 2016. **56**(10): p. 1894–1904.
2. Mavračić, J., et al., *ChemDataExtractor 2.0: Autopopulated Ontologies for Materials Science*. Journal of Chemical Information and Modeling, 2021. **61**(9): p. 4280–4289.
3. Jablonka, K., et al., *14 examples of how LLMs can transform materials science and chemistry: a reflection on a large language model hackathon*. Digital Discovery, 2023. **2**(5): p. 1233–1250.
4. Schilling-Wilhelmi, M., et al., *From text to insight: large language models for chemical data extraction*. Chemical Society Reviews, 2025. **54**(3): p. 1125–1150.
5. Verschaeve, S.R., et al., *Methyl on the Bridge: 2-Methyl Propellane as Precursor for 1, 3-Substituted 2-Methylated Bicyclo [1.1. 1] Pentanes*. Chemistry–A European Journal, 2025: p. e02396.
6. Eltyeb, S. and N. Salim, *Chemical named entities recognition: a review on approaches and applications*. Journal of Cheminformatics, 2014. **6**(1): p. 17.
7. Rocktäschel, T., M. Weidlich, and U. Leser, *ChemSpot: a hybrid system for chemical named entity recognition*. Bioinformatics, 2012. **28**(12): p. 1633–40.
8. Banville, D.L., *Mining chemical structural information from the drug literature*. Drug discovery today, 2006. **11**(1-2): p. 35–42.
9. Brecher, J., *Name= struct: A practical approach to the sorry state of real-life chemical nomenclature*. Journal of Chemical Information and Computer Sciences, 1999. **39**(6): p. 943–950.
10. Vazquez, M., et al., *Text mining for drugs and chemical compounds: methods, tools and applications*. Molecular Informatics, 2011. **30**(6-7): p. 506–519.
11. Kolárik, C., et al. *Chemical names: terminological resources and corpora annotation*. in *Workshop on Building and evaluating resources for biomedical text mining (6th edition of the Language Resources and Evaluation Conference)*. 2008.
12. McDaniel, J.R. and J.R. Balmuth, *Kekule: OCR-optical chemical (structure) recognition*. Journal of Chemical Information and Computer Sciences, 1992. **32**(4): p. 373–378.
13. Simoens, A., et al., *Continuous flow synthesis of N, N-dimethyltryptamine (DMT) analogues with therapeutic potential*. RSC Medicinal Chemistry, 2025. **16**(1): p. 367–372.
14. Bregonje, M., *Patents: A unique source for scientific technical information in chemistry related industry?* World Patent Information, 2005. **27**(4): p. 309–315.

15. Southan, C., et al., *Tracking 20 years of compound-to-target output from literature and patents*. PLoS One, 2013. **8**(10): p. e77142.
16. Guo, J., et al., *Automated Chemical Reaction Extraction from Scientific Literature*. Journal of Chemical Information and Modeling, 2022. **62**(9): p. 2035–2045.
17. Dethlefsen, W., et al., *Computer storage and retrieval of generic chemical structures in patents. 11. Theoretical aspects of the use of structure languages in a retrieval system*. Journal of Chemical Information and Computer Sciences, 1991. **31**(2): p. 233–253.
18. Wagner, S., C. Sternitzke, and S. Walter, *Mapping Markush*. Research Policy, 2022. **51**(10): p. 104597.
19. Lowe, D. *Patents and Prophecy*. 2019 [cited 2025 18 nov]; Available from: <https://www.science.org/content/blog-post/patents-and-prophecy>.
20. CAS. *CAS coverage of prophetic substances*. [cited 2025 26 nov]; Available from: <https://www.cas.org/training/documentation/chemical-substances/prophetics>.
21. [FR], G.O. and G. Olivier, *METHOD FOR PRODUCING (BROMOMETHYL)CYCLOPROPANE AND (BROMOMETHYL)CYCLOBUTANE*, in *Espacenet*, M.M.L.S.F. [FR] and M.M.A.L.S. FRANCE, Editors. 2016.
22. Valko, A.T. and A.P. Johnson, *CLiDE Pro: The Latest Generation of CLiDE, a Tool for Optical Chemical Structure Recognition*. Journal of Chemical Information and Modeling, 2009. **49**(4): p. 780–787.
23. Weininger, D., *SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules*. Journal of Chemical Information and Computer Sciences, 1988. **28**(1): p. 31–36.
24. Heller, S.R., et al., *InChI, the IUPAC International Chemical Identifier*. Journal of Cheminformatics, 2015. **7**(1): p. 23.
25. Wigh, D.S., J.M. Goodman, and A.A. Lapkin, *A review of molecular representation in the age of machine learning*. Wiley Interdisciplinary Reviews: Computational Molecular Science, 2022. **12**(5): p. e1603.
26. Abrahams, P.W., *Typographical extensions for programming languages: breaking out of the ASCII straitjacket*. SIGPLAN Not., 1993. **28**(2): p. 61–68.
27. Daylight Chemical Information Systems, I. *SMARTS - A Language for Describing Molecular Patterns*. [cited 26 nov 2025]; Available from: <https://www.daylight.com/dayhtml/doc/theory/theory.smarts.html>.
28. School, D.S. *Reaction SMILES and SMIRKS*.

. 2001 [cited 18 nov 2025].
29. Grethe, G., et al., *International chemical identifier for reactions (RInChI)*. Journal of Cheminformatics, 2018. **10**(1): p. 22.

30. Akhondi, S.A., J.A. Kors, and S. Muresan, Consistency of systematic chemical identifiers within and between small-molecule databases. *Journal of Cheminformatics*, 2012. **4**(1): p. 35.
31. Neglur, G., R.L. Grossman, and B. Liu. *Assigning Unique Keys to Chemical Compounds for Data Integration: Some Interesting Counter Examples*. 2005. Berlin, Heidelberg: Springer Berlin Heidelberg.
32. International Chemical Identifier Generator (InChI key). 2022 2022/12/17 [cited 2026 25 nov]; Available from: <https://www.inchikey.info/international-chemical-identifier-generator-inchi-key/>.
33. Info, C. SMILES generator / checker. [cited 2025 26 nov]; Available from: https://www.cheminfo.org/flavor/malaria/Utilities/SMILES_generator_checker/index.html
https://www.cheminfo.org/flavor/malaria/Utilities/SMILES_generator_checker/index.html#.
34. Heller, S., et al., InChI - the worldwide chemical structure identifier standard. *Journal of Cheminformatics*, 2013. **5**(1): p. 7.
35. Trust, I. About the InChI Standard. [cited 2025 3 dec]; Available from: <https://www.inchi-trust.org/about-the-inchi-standard/>.
36. Dalby, A., et al., Description of several chemical structure file formats used by computer programs developed at Molecular Design Limited. *Journal of chemical information and computer sciences*, 1992. **32**(3): p. 244-255.
37. Biovia. CT File formats. 2021 [cited 2025 20 nov]; Available from: https://discover.3ds.com/sites/default/files/2020-08/biovia_ctfileformats_2020.pdf.
38. Murray-Rust, P. and H.S. Rzepa, CML: Evolution and design. *Journal of Cheminformatics*, 2011. **3**(1): p. 44.
39. Murray-Rust, P. and H.S. Rzepa, Chemical Markup, XML, and the Worldwide Web. 1. Basic Principles. *Journal of Chemical Information and Computer Sciences*, 1999. **39**(6): p. 928-942.
40. Murray-Rust, P. and H.S. Rzepa, Chemical Markup, XML, and the World Wide Web. 4. CML Schema. *Journal of Chemical Information and Computer Sciences*, 2003. **43**(3): p. 757-772.
41. Kuhn, S., et al., Chemical Markup, XML, and the World Wide Web. 7. CMLspect, an XML Vocabulary for Spectral Data. *Journal of Chemical Information and Modeling*, 2007. **47**(6): p. 2015-2034.
42. Nguyen, A., et al., ChemScanner: extraction and re-use(ability) of chemical information from common scientific documents containing ChemDraw files. *Journal of Cheminformatics*, 2019. **11**(1): p. 77.
43. Li, Z., et al., Personal Experience with Four Kinds of Chemical Structure Drawing Software: Review on ChemDraw, ChemWindow, ISIS/Draw, and ChemSketch. *Journal of Chemical Information and Computer Sciences*, 2004. **44**(5): p. 1886-1890.

44. Zweigenbaum, P., et al., *Frontiers of biomedical text mining: current progress*. Brief Bioinform, 2007. **8**(5): p. 358–75.
45. Altman, R.B., et al., *Pacific Symposium On Biocomputing 2015*. 2014: World Scientific.
46. Sætre, R., et al., *Proceedings of the Second BioCreative Challenge Workshop*. 2007.
47. Hakenberg, J., et al., *The GNAT library for local and remote gene mention normalization*. Bioinformatics, 2011. **27**(19): p. 2769–2771.
48. Huang, M., J. Liu, and X. Zhu, *GeneTUKit: a software for document-level gene normalization*. Bioinformatics, 2011. **27**(7): p. 1032–1033.
49. Hawizy, L., et al., *ChemicalTagger: A tool for semantic text-mining in chemistry*. Journal of Cheminformatics, 2011. **3**(1): p. 17.
50. Tsuruoka, Y., et al. *Developing a robust part-of-speech tagger for biomedical text*. in *Panhellenic conference on informatics*. 2005. Springer.
51. Hara, T., Y. Miyao, and J.i. Tsujii. *Evaluating impact of re-training a lexical disambiguation model on domain adaptation of an HPSG parser*. in *Proceedings of the Tenth International Conference on Parsing Technologies*. 2007.
52. Luk, R.W.P., *Why is Information Retrieval a Scientific Discipline?* Foundations of Science, 2022. **27**(2): p. 427–453.
53. Furnas, G.W., et al., *Human factors and behavioral science: Statistical semantics: Analysis of the potential performance of key-word information systems*. The Bell System Technical Journal, 1983. **62**(6): p. 1753–1806.
54. Bajorath, J., *Milestones in chemoinformatics: global view of the field*. J Cheminform, 2024. **16**(1): p. 124.
55. Reeker, L.H., E.M. Zamora, and P.E. Blower, *Specialized information extraction: automatic chemical reaction coding from English descriptions*, in *Proceedings of the first conference on Applied natural language processing*. 1983, Association for Computational Linguistics: Santa Monica, California. p. 109–116.
56. Jehangir, B., S. Radhakrishnan, and R. Agarwal, *A survey on Named Entity Recognition – datasets, tools, and methodologies*. Natural Language Processing Journal, 2023. **3**: p. 100017.
57. Tarasova, O.A., et al., *Chemical named entity recognition in the texts of scientific publications using the naïve Bayes classifier approach*. J Cheminform, 2022. **14**(1): p. 55.
58. Usié, A., et al., *CheNER: chemical named entity recognizer*. Bioinformatics, 2013. **30**(7): p. 1039–1040.
59. Hettne, K.M., et al., *A dictionary to identify small molecules and drugs in free text*. Bioinformatics, 2009. **25**(22): p. 2983–2991.
60. Jessop, D.M., et al., *OSCAR4: a flexible architecture for chemical text-mining*. Journal of Cheminformatics, 2011. **3**(1): p. 41.

61. Lowe, D.M., et al., *Chemical Name to Structure: OPSIN, an Open Source Solution*. Journal of Chemical Information and Modeling, 2011. **51**(3): p. 739–753.
62. Affairs, U.O.o.P. *United states patent and trademark office*. [Text]; Available from: <https://www.uspto.gov/>.
63. Krallinger, M., et al., *The CHEMDNER corpus of chemicals and drugs and its annotation principles*. Journal of Cheminformatics, 2015. **7**(1): p. S2.
64. Devlin, J., *Bert: Pre-training of deep bidirectional transformers for language understanding/arXiv preprint*. arXiv preprint arXiv:1810.04805, 2018.
65. Malarkodi, C., P.R. Rao, and S.L. Devi. *CLRG ChemNER: A Chemical Named Entity Recognizer@ ChEMU CLEF 2020*. in CLEF (Working Notes). 2020.
66. Islamaj, R., et al., *NLM-Chem, a new resource for chemical entity recognition in PubMed full text literature*. Scientific Data, 2021. **8**(1): p. 91.
67. Papadatos, G., et al., *SureChEMBL: a large-scale, chemically annotated patent document database*. Nucleic Acids Research, 2015. **44**(D1): p. D1220–D1228.
68. Rajan, K., et al., *MARCUS: molecular annotation and recognition for curating unravelled structures*. Digital Discovery, 2025. **4**(11): p. 3137–3148.
69. Radford, A., et al., *Improving language understanding by generative pre-training*. 2018.
70. Chithrananda, S., G. Grand, and B. Ramsundar, *ChemBERTa: large-scale self-supervised pretraining for molecular property prediction*. arXiv preprint arXiv:2010.09885, 2020.
71. Chen, Y., et al., *Towards large-scale chemical reaction image parsing via a multimodal large language model*. Chemical Science, 2025. **16**(45): p. 21464–21474.
72. Beard, E.J. and J.M. Cole, *ChemSchematicResolver: A Toolkit to Decode 2D Chemical Diagrams with Labels and R-Groups into Annotated Chemical Named Entities*. Journal of Chemical Information and Modeling, 2020. **60**(4): p. 2059–2072.
73. Read, J., et al. *Sentence boundary detection: A long solved problem? in Proceedings of COLING 2012: Posters*. 2012.
74. Rajan, K., et al., *DECIMER-Segmentation: Automated extraction of chemical structure depictions from scientific literature*. Journal of Cheminformatics, 2021. **13**(1): p. 20.
75. Krallinger, M., et al., *Information Retrieval and Text Mining Technologies for Chemistry*. Chemical Reviews, 2017. **117**(12): p. 7673–7761.
76. Marcus, M., B. Santorini, and M.A. Marcinkiewicz, *Building a large annotated corpus of English: The Penn Treebank*. Computational linguistics, 1993. **19**(2): p. 313–330.
77. Dobbelaere, M.R., et al., *Rxn-INSIGHT: fast chemical reaction analysis using bond-electron matrices*. Journal of Cheminformatics, 2024. **16**(1): p. 37.

78. Gao, H., et al., *Using machine learning to predict suitable conditions for organic reactions*. ACS central science, 2018. 4(11): p. 1465–1476.
79. Lu, Y., et al., *CHEMDNER system with mixed conditional random fields and multi-scale word clustering*. Journal of cheminformatics, 2015. 7(Suppl 1): p. S4.
80. Campos, D., S. Matos, and J.L. Oliveira, *A document processing pipeline for annotating chemical entities in scientific documents*. Journal of cheminformatics, 2015. 7(Suppl 1): p. S7.
81. Khabsa, M. and C.L. Giles, *Chemical entity extraction using CRF and an ensemble of extractors*. J Cheminform, 2015. 7(Suppl 1 Text mining for chemistry and the CHEMDNER track): p. S12.
82. Leaman, R., C.H. Wei, and Z. Lu, *tmChem: a high performance approach for chemical named entity recognition and normalization*. J Cheminform, 2015. 7(Suppl 1 Text mining for chemistry and the CHEMDNER track): p. S3.
83. Tharatipyakul, A., et al., *ChemEx: information extraction system for chemical data curation*. BMC bioinformatics, 2012. 13(Suppl 17): p. S9.
84. Lowe, D.M. and R.A. Sayle, *LeadMine: a grammar and dictionary driven approach to entity recognition*. Journal of Cheminformatics, 2015. 7(1): p. S5.
85. Rinaldi, F., et al., *OntoGene web services for biomedical text mining*. BMC bioinformatics, 2014. 15(Suppl 14): p. S6.
86. Rebholz-Schuhmann, D., et al., *Text processing through Web services: calling Whatizit*. Bioinformatics, 2007. 24(2): p. 296–298.
87. Aronson, A.R., *Effective mapping of biomedical text to the UMLS Metathesaurus: the MetaMap program*. Proc AMIA Symp, 2001: p. 17–21.
88. Funk, C., et al., *Large-scale biomedical concept recognition: an evaluation of current automatic annotators and their parameters*. BMC Bioinformatics, 2014. 15: p. 59.
89. Rebholz-Schuhmann, D., et al., *Text processing through Web services: calling Whatizit*. Bioinformatics, 2008. 24(2): p. 296–298.
90. Kemp, N. and M. Lynch, *Extraction of Information from the Text of Chemical Patents. 1. Identification of Specific Chemical Names*. Journal of Chemical Information and Computer Sciences, 1998. 38(4): p. 544–551.
91. *Registry file basic name segment dictionary*.
92. Vaucher, A.C., et al., *Automated extraction of chemical synthesis actions from experimental procedures*. Nature Communications, 2020. 11(1): p. 3601.
93. Kim, E., et al., *Materials Synthesis Insights from Scientific Literature via Text Extraction and Machine Learning*. Chemistry of Materials, 2017. 29(21): p. 9436–9444.
94. Dobbelaere, M.R., et al., *Machine Learning in Chemical Engineering: Strengths, Weaknesses, Opportunities, and Threats*. Engineering, 2021. 7(9): p. 1201–1211.
95. Apodaca, R.L. *The nextmove patent reaction dataset*. 2019 2019/01/28/; Available from: <http://depth-first.com/articles/2019/01/28/the-nextmove-patent-reaction-dataset/>.

96. Kearnes, S.M., et al., *The Open Reaction Database*. Journal of the American Chemical Society, 2021. **143**(45): p. 18820–18826.
97. Barnard, J. What Are Word Embeddings? | IBM. 2023 2023/12/29/T00:00:00.000; Available from: <https://www.ibm.com/think/topics/word-embeddings>.
98. Breiman, L., *Random forests*. Machine learning, 2001. **45**(1): p. 5–32.
99. Yang, C., *Review of the latest progress of AI and machine learning methods in the free energy kinetics estimation and synthesis analysis for organic chemistry applications*. Intelligent Pharmacy, 2025.
100. Boyd, S. and L. Vandenberghe, *Convex optimization*. 2004: Cambridge university press.
101. Kingma, D.P., *Adam: A method for stochastic optimization*. arXiv preprint arXiv:1412.6980, 2014.
102. Jung, A., *Machine learning: the basics*. 2022: Springer Nature.
103. Court, C.J. and J.M. Cole, *Auto-generated materials database of Curie and Néel temperatures via semi-supervised relationship extraction*. Scientific Data, 2018. **5**(1): p. 180111.
104. Dong, Q. and J.M. Cole, *Auto-generated database of semiconductor band gaps using ChemDataExtractor*. Scientific Data, 2022. **9**(1): p. 193.
105. Sierepeklis, O. and J.M. Cole, *A thermoelectric materials database auto-generated from the scientific literature using ChemDataExtractor*. Scientific Data, 2022. **9**(1): p. 648.
106. Kumar, P., S. Kabra, and J.M. Cole, *A Database of Stress-Strain Properties Auto-generated from the Scientific Literature using ChemDataExtractor*. Scientific Data, 2024. **11**(1): p. 1273.
107. Weston, L., et al., *Named Entity Recognition and Normalization Applied to Large-Scale Information Extraction from the Materials Science Literature*. Journal of Chemical Information and Modeling, 2019. **59**(9): p. 3692–3702.
108. Mohri, M., A. Rostamizadeh, and A. Talwalkar, *Foundations of machine learning*. 2018: MIT press.
109. Cortes, C. and V. Vapnik, *Support-vector networks*. Machine Learning, 1995. **20**(3): p. 273–297.
110. Koutroumbas, K. and S. Theodoridis, *Pattern recognition*. 2008: Academic Press.
111. Tang, B., et al., *A comparison of conditional random fields and structured support vector machines for chemical entity recognition in biomedical literature*. Journal of Cheminformatics, 2015. **7**(1): p. S8.
112. Townsend, J., et al. *Language technology for processing chemistry publications*. in *Proceedings of the fourth UK e-Science All Hands Meeting*. 2005.
113. McCallum, A., D. Freitag, and F.C. Pereira. *Maximum entropy Markov models for information extraction and segmentation*. in *Icml*. 2000.
114. Klinger, R. and K. Tomanek, *Classical probabilistic models and conditional random fields*. 2007: TU, Algorithm Engineering.

115. Kavlakoglu, E., What Is Support Vector Machine? | IBM. 2023.
116. Vaškevičius, M., et al., Deep learning-based automatic action extraction from structured chemical synthesis procedures. *PeerJ Comput Sci*, 2023. **9**: p. e1511.
117. Chen, L., et al., A deep learning based method for extracting semantic information from patent documents. *Scientometrics*, 2020. **125**(1): p. 289–312.
118. Jacobsson, T.J., et al., An open-access database and analysis tool for perovskite solar cells based on the FAIR data principles. *Nature Energy*, 2022. **7**(1): p. 107–115.
119. Devlin, J., et al. Bert: Pre-training of deep bidirectional transformers for language understanding. in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019.
120. Vaswani, A., et al., Attention is all you need. *Advances in neural information processing systems*, 2017. **30**.
121. Turian, J., L. Ratinov, and Y. Bengio. Word representations: a simple and general method for semi-supervised learning. in *Proceedings of the 48th annual meeting of the association for computational linguistics*. 2010.
122. Tshitoyan, V., et al., Unsupervised word embeddings capture latent knowledge from materials science literature. *Nature*, 2019. **571**(7763): p. 95–98.
123. Schwaller, P., et al., Extraction of organic chemistry grammar from unsupervised learning of chemical reactions. *Science Advances*, 2021. **7**(15): p. eabe4166.
124. Zhang, B., et al., Label-Free Data Mining of Scientific Literature by Unsupervised Syntactic Distance Analysis. *The Journal of Physical Chemistry Letters*, 2024. **15**(1): p. 212–219.
125. Mikolov, T., et al., Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
126. Pennington, J., R. Socher, and C.D. Manning. Glove: Global vectors for word representation. in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 2014.
127. Bakarov, A., A survey of word embeddings evaluation methods. *arXiv preprint arXiv:1801.09536*, 2018.
128. Schnabel, T., et al. Evaluation methods for unsupervised word embeddings. in *Proceedings of the 2015 conference on empirical methods in natural language processing*. 2015.
129. Liu, Y., et al. Exploiting task-oriented resources to learn word embeddings for clinical abbreviation expansion. in *Proceedings of BioNLP 15*. 2015.
130. Elton, D.C., et al., Using natural language processing techniques to extract information on the properties and functionalities of energetic materials from large text corpora. *arXiv preprint arXiv:1903.00415*, 2019.

131. Rajan, K., et al., *A review of optical chemical structure recognition tools*. Journal of Cheminformatics, 2020. **12**(1): p. 60.
132. Krasnov, A., et al., *Comparing software tools for optical chemical structure recognition*. Digital Discovery, 2024. **3**(4): p. 681–693.
133. Contreras, M.L., et al., *Computational perception and recognition of digitized molecular structures*. Journal of chemical information and computer sciences, 1990. **30**(3): p. 302–307.
134. Ibison, P., et al., *Chemical literature data extraction: the CLiDE Project*. Journal of Chemical Information and Computer Sciences, 1993. **33**(3): p. 338–344.
135. Casey, R., et al. *Optical recognition of chemical graphics*. in *Proceedings of 2nd International Conference on Document Analysis and Recognition (ICDAR'93)*. 1993. IEEE.
136. Park, J., et al., *Automated extraction of chemical structure information from digital raster images*. Chemistry Central Journal, 2009. **3**(1): p. 4.
137. Wilary, D.M. and J.M. Cole, *ReactionDataExtractor: A Tool for Automated Extraction of Information from Chemical Reaction Schemes*. Journal of Chemical Information and Modeling, 2021. **61**(10): p. 4962–4974.
138. Filippov, I.V. and M.C. Nicklaus, *Optical structure recognition software to recover chemical information: OSRA, an open source solution*. 2009, ACS Publications.
139. Wilary, D.M. and J.M. Cole, *ReactionDataExtractor 2.0: A Deep Learning Approach for Data Extraction from Chemical Reaction Schemes*. Journal of Chemical Information and Modeling, 2023. **63**(19): p. 6053–6067.
140. Qian, Y., et al., *RxnScribe: A Sequence Generation Model for Reaction Diagram Parsing*. Journal of Chemical Information and Modeling, 2023. **63**(13): p. 4030–4041.
141. Musazade, F., N. Jamalova, and J. Hasanov, *Review of techniques and models used in optical chemical structure recognition in images and scanned documents*. Journal of cheminformatics, 2022. **14**(1): p. 61.
142. Zheng, L., T. Zhang, and X. Yu. *Recognition of Handwritten Chemical Organic Ring Structure Symbols Using Convolutional Neural Networks*. in *2019 International Conference on Document Analysis and Recognition Workshops (ICDARW)*. 2019.
143. Clevert, D.-A., et al., *Img2Mol—accurate SMILES recognition from molecular graphical depictions*. Chemical science, 2021. **12**(42): p. 14174–14181.
144. Staker, J., et al., *Molecular Structure Extraction from Documents Using Deep Learning*. Journal of Chemical Information and Modeling, 2019. **59**(3): p. 1017–1029.
145. Qian, Y., et al., *MolScribe: Robust Molecular Structure Recognition with Image-to-Graph Generation*. Journal of Chemical Information and Modeling, 2023. **63**(7): p. 1925–1934.

146. Rajan, K., A. Zielesny, and C. Steinbeck, DECIMER: towards deep learning for chemical image recognition. *Journal of Cheminformatics*, 2020. **12**(1): p. 65.
147. Khokhlov, I., et al., Image2SMILES: Transformer-Based Molecular Optical Recognition Engine. *Chemistry-Methods*, 2022. **2**(1): p. e202100069.
148. Chen, Y., et al., MolNexTR: A generalized deep learning model for molecular image recognition. *Research Square*, 2024.
149. Tang, H., X. Dang, and J. Wang, Fine-tuning DeepSeek-OCR-2 for molecular structure recognition. *arXiv [cs.CV]*, 2026.
150. Wang, J., et al., From Tokenization to Self-Supervision: Building a High-Performance Information Extraction System for Chemical Reactions in Patents. *Front Res Metr Anal*, 2021. **6**: p. 691105.
151. Rajan, K., et al., DECIMER.ai: an open platform for automated optical chemical structure identification, segmentation and recognition in scientific publications. *Nature Communications*, 2023. **14**(1): p. 5045.
152. Landrum, G.O., *Rdkit/rdkit*. 2025, RDKit.
153. Chen, L.-Y. and Y.-P. Li, Machine learning-guided strategies for reaction conditions design and optimization. *Beilstein Journal of Organic Chemistry*, 2024. **20**: p. 2476–2492.
154. Wang, X., et al., Large-scale Multi-modal Pre-trained Models: A Comprehensive Survey. *Machine Intelligence Research*, 2023. **20**(4): p. 447–482.
155. Fan, V., et al., OpenChemIE: An Information Extraction Toolkit for Chemistry Literature. *Journal of Chemical Information and Modeling*, 2024. **64**(14): p. 5521–5534.
156. Leong, S.X., et al., MERMaid: Universal multimodal mining of chemical reactions from PDFs using vision-language models. 2025.
157. Leong, S.X., et al., Automated electrosynthesis reaction mining with multimodal large language models (MLLMs). *Chemical Science*, 2024. **15**(43): p. 17881–17891.
158. Zhang, W., et al., Fine-tuning large language models for chemical text mining. *Chemical science*, 2024. **15**(27): p. 10600–10611.
159. Ramos, M.C., C.J. Collison, and A.D. White, A review of large language models and autonomous agents in chemistry. *Chem Sci*, 2025. **16**(6): p. 2514–2572.
160. Wei, H., Y. Sun, and Y. Li, DeepSeek-OCR: Contexts Optical Compression. *arXiv preprint arXiv:2510.18234*, 2025.
161. Song, J., et al., RxnCaption: Reformulating Reaction Diagram Parsing as Visual Prompt Guided Captioning. *arXiv preprint arXiv:2511.02384*, 2025.
162. Choi, J. and B. Lee, Accelerating materials language processing with large language models. *Communications Materials*, 2024. **5**(1): p. 13.
163. Brown, T., et al., Language models are few-shot learners. *Advances in neural information processing systems*, 2020. **33**: p. 1877–1901.

164. Wei, J., et al., *Chain-of-thought prompting elicits reasoning in large language models*. *Advances in neural information processing systems*, 2022. **35**: p. 24824–24837.
165. Hu, E.J., et al., *Lora: Low-rank adaptation of large language models*. *ICLR*, 2022. **1**(2): p. 3.
166. Dagdelen, J., et al., *Structured information extraction from scientific text with large language models*. *Nature communications*, 2024. **15**(1): p. 1418.
167. Hurst, A., et al., *Gpt-4o system card*. *arXiv preprint arXiv:2410.21276*, 2024.
168. Masterman, T., et al., *The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey*. *arXiv preprint arXiv:2404.11584*, 2024.
169. Chen, G., et al., *Autoagents: A framework for automatic agent generation*. *arXiv preprint arXiv:2309.17288*, 2023.
170. Yao, S., et al. *React: Synergizing reasoning and acting in language models*. in *The eleventh international conference on learning representations*. 2022.
171. Shinn, N., et al., *Reflexion: Language agents with verbal reinforcement learning*. *Advances in Neural Information Processing Systems*, 2023. **36**: p. 8634–8652.
172. Wang, X., et al., *Self-consistency improves chain of thought reasoning in language models*. *arXiv preprint arXiv:2203.11171*, 2022.
173. Madaan, A., et al., *Self-refine: Iterative refinement with self-feedback*. *Advances in Neural Information Processing Systems*, 2023. **36**: p. 46534–46594.
174. Labate, A.B., et al., *Solving Context Window Overflow in AI Agents*. *arXiv preprint arXiv:2511.22729*, 2025.
175. Lu, R., Y. Li, and Y. Huo, *Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks*. *arXiv preprint arXiv:2508.13143*, 2025.
176. Geng, S., et al., *Generating structured outputs from language models: Benchmark and studies*. *arXiv e-prints*, 2025: p. arXiv: 2501.10868.
177. Michelle Pokrass, C.C., Melody Guan, Michelle Pokrass, Ted Sanders, Brian Zhang. *Introducing structured outputs in the api*. Available from: <https://openai.com/index/introducing-structured-outputs-in-the-api/>.
178. Kim, S., et al., *PubChem substance and compound databases*. *Nucleic acids research*, 2016. **44**(D1): p. D1202–D1213.
179. Lewis, P., et al., *Retrieval-augmented generation for knowledge-intensive nlp tasks*. *Advances in neural information processing systems*, 2020. **33**: p. 9459–9474.
180. RDKit, *Open-Source Cheminformatics Software*, RDKit, Editor. 2024, RDKit.
181. Dhuliawala, S., et al. *Chain-of-verification reduces hallucination in large language models*. in *Findings of the association for computational linguistics: ACL 2024*. 2024.
182. Patiny, L. and G. Godin, *Automatic extraction of FAIR data from publications using LLM*. 2023.

183. Kumari, M., R. Chauhan, and P. Garg, *Can LLMs revolutionize text mining in chemistry? A comparative study with domain-specific tools*. *Computer Standards & Interfaces*, 2025. **94**: p. 103997.
184. Huang, X., et al. *Application of large language models in chemistry reaction data extraction and cleaning*. in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 2024.
185. M. Bran, A., et al., *Augmenting large language models with chemistry tools*. *Nature Machine Intelligence*, 2024. **6**(5): p. 525–535.
186. Mohammadi, M., et al. *Evaluation and benchmarking of llm agents: A survey*. in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 2025.
187. Askill, A., M. Brundage, and G. Hadfield, *The role of cooperation in responsible AI development*. arXiv preprint arXiv:1907.04534, 2019.
188. Sayeed, H.M., T. Mohanty, and T.D. Sparks, *Annotating Materials Science Text: A Semi-automated Approach for Crafting Outputs with Gemini Pro*. *Integrating Materials and Manufacturing Innovation*, 2024. **13**(2): p. 445–452.
189. Islamaj, R., et al., *TeamTat: a collaborative text annotation tool*. *Nucleic Acids Research*, 2020. **48**(W1): p. W5–W11.
190. Sambetbayeva, M., et al. *A Multi-Level Annotation Model for Fake News Detection: Implementing Kazakh-Russian Corpus via Label Studio*. *Big Data and Cognitive Computing*, 2025. **9**, DOI: 10.3390/bdcc9080215.
191. Nakayama, H., et al., *doccano: Text Annotation Tool for Human*. 2018.
192. *Prodigy · An annotation tool for AI, Machine Learning & NLP*.

Chapter 3.

Data and Methods

3.1	INTRODUCTION	87
3.2	DATA	87
3.3	SOFTWARE	89
3.3.1	CHEMEAGLE	89
3.3.2	MULTI-AGENT FRAMEWORK	95
3.3.3	CHEMEAGLE HYBRID	103
3.3.4	SINGLE-AGENT PIPELINE	104
3.3.5	LEMATH-SYNTH	113
3.3.6	RXN-INSIGHT	116
3.4	CLOSED VERSUS OPEN-SOURCE MODELS	119
3.4.1	CLOSED-SOURCE MODELS	120
3.4.2	OPEN-SOURCE MODELS	123
3.4.3	HARDWARE	125
3.5	COST CALCULATIONS	125
3.6	BENCHMARK EVALUATION METHODOLOGY	126
3.6.1	EVALUATION METHODOLOGY	126
3.7	REFERENCES	130

3.1 Introduction

The methodological landscape for automated chemical reaction extraction surveyed, from rule-based and machine-learning systems through to LLM and VLM-based pipelines, and identified hybrid multimodal approaches with agentic orchestration as the current state of the art. The present chapter moves from that survey to the concrete data, software, and infrastructure underpinning the experiments reported in Chapters 4 and 5. Its purpose is to document, in sufficient detail to allow replication, every component the thesis evaluates.

The chapter is organised around four main concerns. First, the benchmark data set on which all subsequent results are computed. Second, the software stack used for extraction, comprising five extraction pipelines and one post-processing toolkits: the published ChemEAGLE system reinstalled locally, an original multi-agent framework built on AutoGen, the ChemEAGLE Hybrid variant, a single-agent vision-LLM pipeline instantiated behind several multimodal models, the Rxn-INSIGHT classification toolkit applied to every extracted reaction, and the LeMath-synth procedural-text track adapted for organic chemistry. Third, the model-hosting infrastructure, covering closed-source models accessed through Microsoft Azure AI Foundry and OpenRouter, and open-source models served on the HPC cluster. Finally, the evaluation methodology consisting of different criteria is discussed.

3.2 Data

All extraction systems documented in Section 3.3 are evaluated against a single shared benchmark, the 16-image set assembled from the open and subscription chemical literature. The set is a curated subset of the larger GT3 ground-truth corpus published alongside ChemEAGLE, chosen to cover the figure styles encountered most often in synthesis papers: reaction-scope tables, picture-style schemes, and reaction-condition tables. Each image has been cropped from the corresponding journal page so that no surrounding running text was retained, which isolates the extraction task from any out-of-figure context and ensures that every system reads the same visual evidence.

The 16 images are drawn from five journals: two from ACS Catalysis, three from Chemistry, a European Journal, one from Chemical Science, eight from Green Chemistry, and two from Nature Communications. Green Chemistry is the most strongly represented source and contributes a particularly diverse set of figure styles, ranging from scope tables with up to ten entries to condition-survey panels with six grouped catalysts and dense picture-style schemes containing as many as thirteen individually numbered products. All the images are included in the code base.

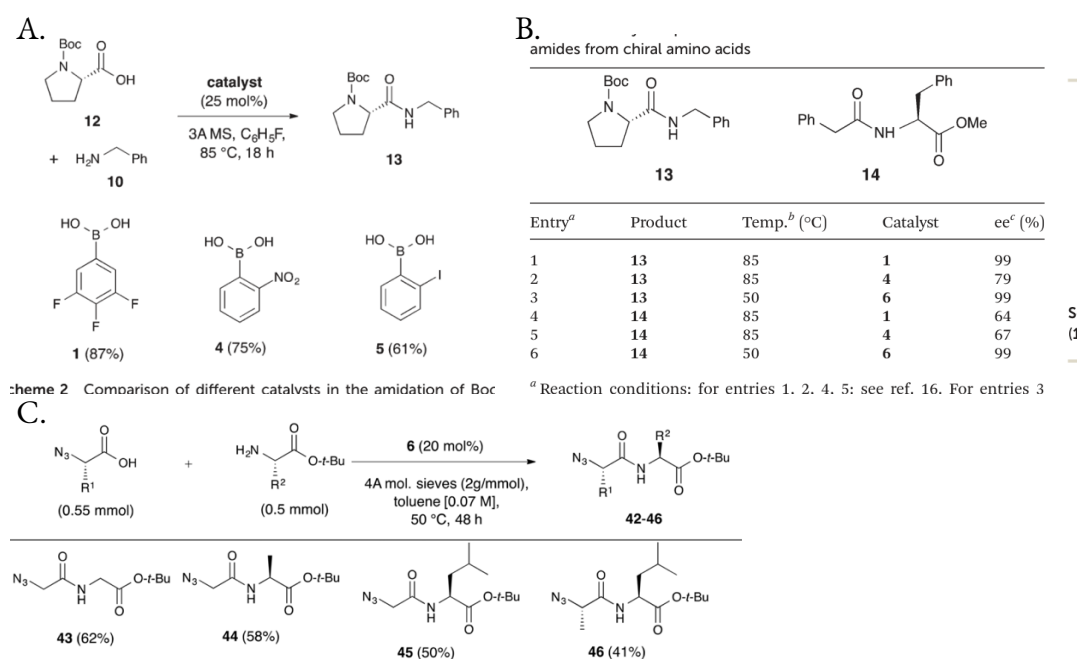


Figure 3-1: Some examples from the 16-image benchmark dataset. A. condition-survey panels, B. reaction-condition table and C. reaction-scope tables.

The ground-truth annotations for each image are taken from the GT3 corpus published alongside ChemEAGLE, derived from the upstream Benchmark on HuggingFace . The ground-truth schema is a flat list of reactions, each with reactants, products, and a flat conditions list. Every condition entry has the form (role, text, optional SMILES, optional label), where role is drawn from a closed enumeration covering reagent, solvent, catalyst, temperature, time, yield, atmosphere, loading, and pressure. Several annotation conventions affect the comparison. R-group placeholders are encoded as atom-mapped SMILES wildcards ([1*], [2*]) when the figure draws distinct R groups. Yields are recorded verbatim, including textual qualifiers such as “trace” and parenthesised scale notes. Each generic-header reaction is included as a separate entry with wildcard-encoded reactant and product SMILES, distinct from the per-variant entries that follow. Catalyst-class species are classified under

the role “reagent” in the ground truth, with the finer-grained taxonomy of catalysts, additives, solvents, and reductants used by the extraction systems collapsed into that single matching bucket at evaluation time.

The dataset comprises only 16 images, which is a recognised limitation on the statistical reliability of the results. However, as it serves solely as a verification set rather than a training set, it is sufficient for qualitative comparison between different pipelines. The principal advantage of such a small dataset is that it enables rapid iteration and testing of ideas at low computational cost.

3.3 Software

3.3.1 ChemEAGLE

Purpose and Claims

ChemEAGLE [2] is a multi-agent system designed to extract structured chemical reaction data from scientific literature. Rather than relying on a single model or algorithm, it coordinates a collection of specialised agents, each responsible for a distinct part of the extraction process. Each agent receives a certain context which contains the image, a specific prompt which tells the agent its character and its goals. To achieve these goals an agent can use specific tools, i.e. pieces of software. Figure 3-2 illustrate the complete workflow of ChemEAGLE.

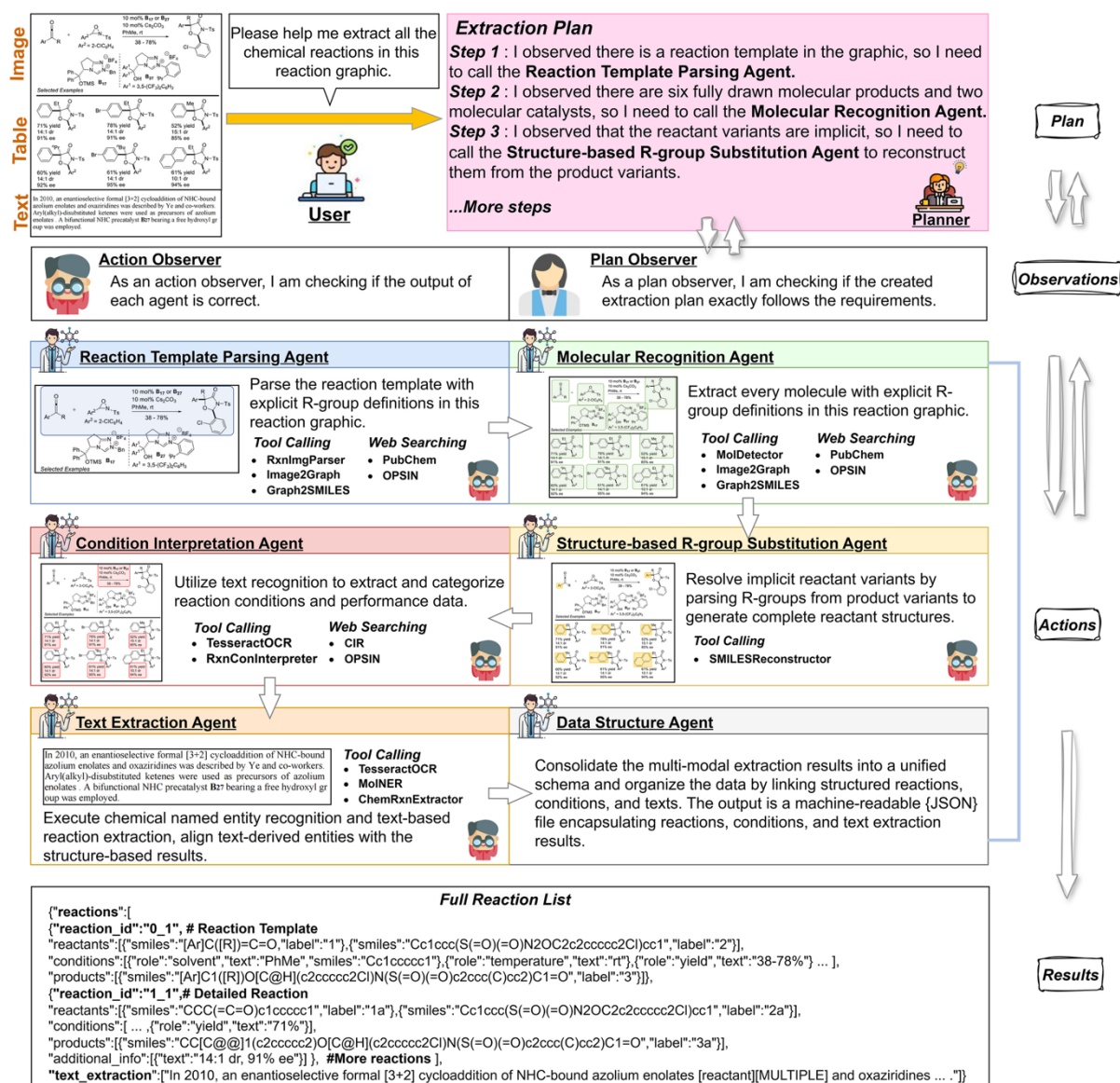


Figure 3-2: Architecture of the multi-agent reaction extraction pipeline from ChemEAGLE by Chen *et al.* [2]. A central Planner generates an extraction plan from the input graphic and delegates subtasks to six specialized agents responsible for reaction template parsing, molecular recognition, condition interpretation, R-group substitution, text extraction, and data structuring. Two observer modules supervise plan validity and per-agent output quality. The pipeline integrates OCR, chemical structure recognition, and named-entity recognition tools, and returns a structured JSON representation of the reactions, conditions, and associated text.

The reason for working with ChemEAGLE is because of its high reported numbers compared to other methods, listed in Table 3-1.

Table 3-1: ChemEAGLE's reported numbers. A soft match requires all reactant and product SMILES to exactly match the ground truth. A hard match further requires the SMILES in the reaction conditions to match as well.

Method	Soft match score	Hard match score
OpenChemIE (rule-based) [3]	0.46	0.39
MERmaid (gpt-4o single-agent) [1]	0.13	0.07
GPT-5 (general MLLM, single-agent) [4]	0.24	0.20
Claude 4.5 Sonnet (general MLLM, single agent) [5]	0.14	0.11
ChemEAGLE	0.78	0.76

Input

ChemEAGLE is capable of processing PDF and image formats. In the first step of processing a pdf a VLM is used for precise object detection of figure-caption and table-heading pairs. This visual extraction pipeline has been adapted from the MERmaid system [1]. The module relies on a Microsoft Florence-2-large VLM which has been fine-tuned on a dataset of 3,518 PDF pages for precise detection of figure-caption and table-heading pairs.

Pipeline

The workflow begins with a **Planner agent**, which analyses the input document and determines which subsequent agents to be invoked and in what order. This makes the system adaptive: the exact sequence of steps can differ depending on the structure of the paper being processed.

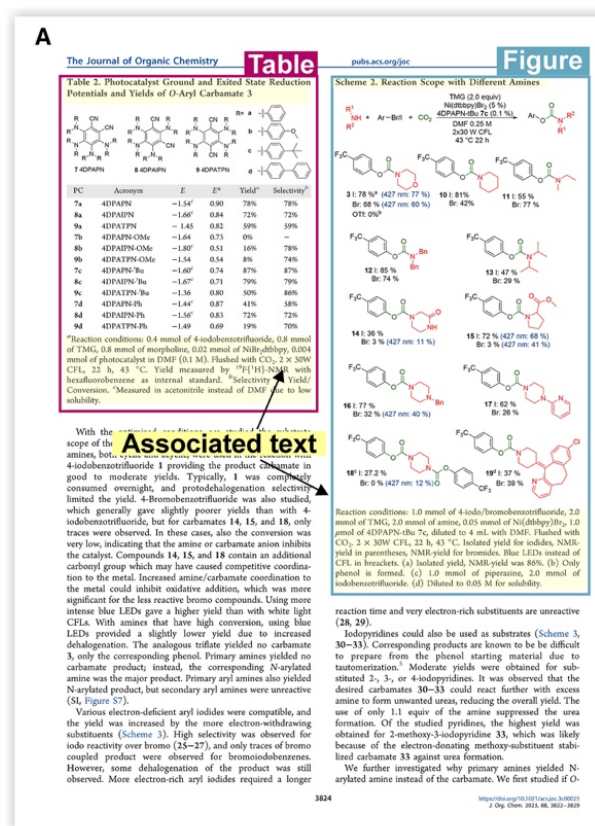


Figure 3-3: Detection of figures and tables with the fine-tuned vision-language model from MERmaid by Leon *et al* [1].

For papers containing reaction graphics with structure-based tables, the following agents are typically called in sequence. The **Reaction Template Parsing Agent** identifies the overall reaction scheme and converts it into a machine-readable format called SMILES, while also correcting any errors introduced during text recognition.

The **Molecular Recognition Agent** then locates and identifies individual molecules within the image and converts their graphical representations into SMILES strings. Where reactions contain variable substituents (so-called R-groups), the **Structure-based R-group Substitution Agent** extracts the specific variants from the table and reconstructs the corresponding SMILES for each. Most often detailed R-group fragments from the product variant structure table are extracted and utilised to reconstruct reactant SMILES.

Meanwhile, the **Condition Interpretation Agent** extract's reaction conditions from the text (temperature, solvent, catalyst, etc) and links them to the correct reaction instances. A separate **Text Extraction Agent** captures any additional relevant information from the surrounding text. Finally, the **Data Structure Agent** combines all extracted information into a single structured output record in JSON format, ensuring that every reaction variant is fully specified.

For papers where R-group variations are described in plain text rather than structural tables, the Planner skips the Molecular Recognition Agent and instead calls the **Text-based R-group Substitution Agent**, which reads the R-group definitions directly from the text and incorporates them into the reaction template.

This modular, agent-based design allows ChemEAGLE to flexibly handle the heterogeneous formats found throughout the chemical literature, combining visual, tabular, and textual information into a unified, machine-readable output.

Output

Figure 3-4: Example of extraction results from a reaction image in JSON format ChemEAGLE. The output consists of the different reactions, indicated by "reaction_id". Each reaction has four elements: reactants, conditions, products and additional info. shows the output of ChemEAGLE. This is a JSON file consisting of several reaction segments indicated by "reaction_id": "0_1" and

“reaction_id”: “0_2”. Each reaction segment consists of four building blocks: reactants, conditions, products, and additional information. Chemical molecules are represented by SMILES. This output format is adopted in the other frameworks (multi-agent framework and single agent) to create uniform results.

Input:

Table 1. Optimization of the Reaction Conditions^{2f}

entry	substrate	catalyst	solvent	temperature (°C)	3b ^b (%)
1	1a	A	toluene	80	99
2	1b	A	toluene	80	0

Output:

```

"file_name": "ACScat_2020.pdf_page002_table_01_s0.88.png",
"reactions": [
  {
    "reaction_id": "1_1",
    "reactants": [
      {
        "smiles": "CC(C)(C)OC(=O)N[C@@](H)C(=O)O",
        "label": "1a"
      },
      {
        "smiles": "NC1CCCCC1",
        "label": "2"
      }
    ],
    "conditions": [
      {
        "role": "reagent",
        "text": "catalyst A (5 mol%)",
        "smiles": null
      },
      {
        "role": "reagent",
        "text": "5 Å MS",
        "smiles": null
      },
      {
        "role": "solvent",
        "text": "toluene",
        "smiles": "CC1=CC=CC=C1"
      },
      {
        "role": "time",
        "text": "12 h"
      },
      {
        "role": "temperature",
        "text": "80°C"
      },
      {
        "role": "yield",
        "text": "99 %"
      }
    ],
    "products": [
      {
        "smiles": "CC(C)(C)OC(=O)N[C@@](H)C(=O)NC1CCCCC1",
        "label": "3a"
      }
    ],
    "additional_info": [
      "entry 1 in Table 1: substrate 1a (R = H), catalyst A, toluene, 80 °C, 99% yield",
      "General reaction conditions from figure: 1 (0.20 mmol, 1.0 equiv), 2 (0.20 mmol, 1.0 equiv); catalyst (5 mol%), 5 Å MS (200 mg), solvent (200 µL), 12 h."
    ]
  }
]

```

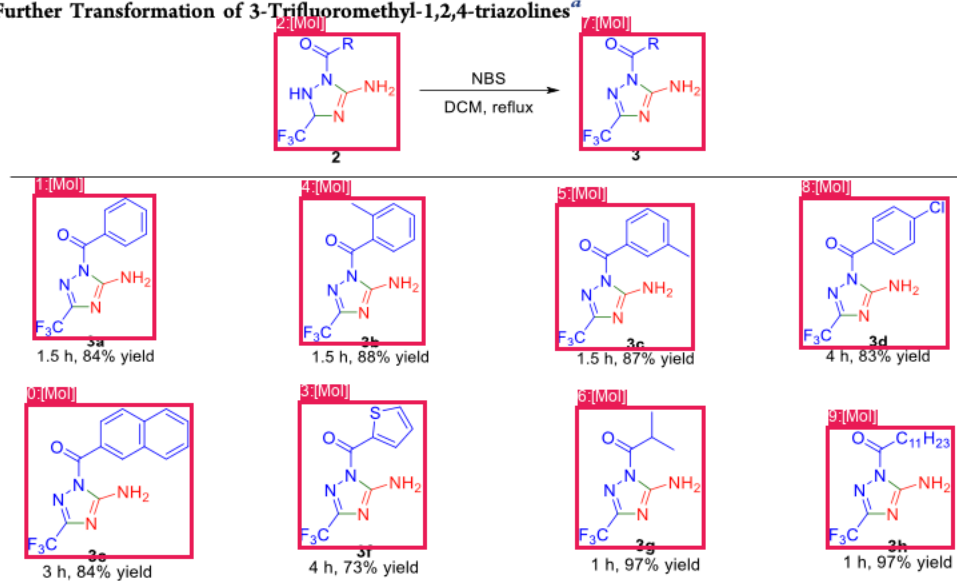
Figure 3-4: Example of extraction results from a reaction image in JSON format ChemEAGLE. The output consists of the different reactions, indicated by “reaction_id”. Each reaction has four elements: reactants, conditions, products and additional info.

Key Insights

Despite the pipelines' failure to reproduce similar matching scores as they claimed in their report (Table 3-1), two important components of the ChemEAGLE stack work in isolation and were retained as building blocks for the multi-agent framework described in Multi-Agent Framework.

MolDetect is a Pix2Seq-style molecule-bbox detector trained on chemistry-figure crops. On a substrate-scope figure, MolDetect reliably recovers one bbox per drawn molecule, illustrated in Figure 3-5.

Table 3. Further Transformation of 3-Trifluoromethyl-1,2,4-triazolines^a



^aReaction conditions: All reactions were carried out by using 0.2 mmol 2 and 1.5 equiv of NBS in 2 mL of CH₂Cl₂ at 40 °C. ^bIsolated yields.

Figure 3-5: MolDetect output on 'acs.joc.2c00176 example 2'. Ten bounding boxes are recovered: the two reaction-template molecules in the header strip and the eight concrete product variants below. Each bbox is then passed individually to MolNexTR, which on single-molecule crops returns canonical SMILES at ~0.79 confidence, versus ~0.004 confidence when called on the whole figure. This per-bbox routing is the key change that lifted product IoU on the suite.

MolNexTR is a Swin-Transformer image-to-graph-to-SMILES model. On single-molecule crops MolNexTR returns canonical SMILES with proper wildcard atoms (e.g. [*:1], [*:2]) for R-group placeholders, which is exactly the input format the downstream R-group decomposition consumes. The published model checkpoint from ChemEAGLE was used unchanged [6]. Without per-bbox routing, calling MolNexTR on an entire multi-molecule image returns degenerate dot-separated

SMILES at confidence ~ 0.004 ; with per-bbox routing, single-molecule crops return correct SMILES at confidence ~ 0.79 .

The **OPSIN-PubChem-CIR cascade** [7-9] is a name-resolution chain used to convert chemical names found in figure captions and footnotes into SMILES. ChemEAGLE's implementation (OPSIN for IUPAC names, PubChem for synonyms, CIR/Cactus as fallback, abbreviation lexicon for short labels like LDA) was adopted directly into the multi-agent framework as the `lookup_compound` tool.

The remaining ChemEAGLE components (figure classifier, R-group sub-agent, JSON aggregator, ChemNER, RxnExtractor, the table-extractor, and the corefdet model) were either not adopted or replaced by alternatives: the figure classifier and JSON aggregator were re-implemented as part of the multi-agent topology; the R-group sub-agent was replaced by a deterministic RDKit chain [10]; ChemNER[3], RxnExtractor[11], the table-extractor, and corefdet are listed as candidates for future integration but were not used in any of the experiments reported here.

The reproduction outcome reframed the project. If a six-agent pipeline cannot, in our hands, reach the published numbers on its own benchmark, then either the pipeline is fragile in ways the paper does not document, or the value the published agents add over single-agent baselines is smaller than reported, or both. Two complementary research directions follow naturally: (i) build a different multi-agent design over the same chemistry-DL toolchain and see whether a different orchestration recovers the published quality, and (ii) test whether a single, strong VLM (without any agent decomposition at all) already crosses the bar that the published multi-agent number sets. Firstly, the multi-agent framework will be pursued. Afterwards, the single agent system will be set up. A proper cross-system evaluation brings them together against the same ground truth.

3.3.2 Multi-Agent Framework

The multi-agent system is self-developed pipeline which integrates an LLM-driven agent team with chemistry-specific deep-learning models and a deterministic R-group decomposition layer. Across ten development phases, the system progressed from a baseline that produced 0% schema-valid output to a configuration that achieves 100% schema-valid extractions, mean product-set

intersection over union (IoU) of 0.55, and a perfect reactant- and product-IoU on a canonical R-group substrate-scope figure.

Agent Pipeline

Seven agents are wired into a group chat to communicate their findings to each other. The agents' roles are quite similar to those in ChemEAGLE. However, their functionality differs. ChemEAGLE only uses text-only LLM and dedicated Deep-Learning or OCR models to extract visual parts from the process (Table 3-2). In contrast, the multi-agent framework reads the image directly with a VLM and uses fewer additional tools. Figure 3-6 gives an overview of the pipeline. After the 5 specialised agents have emitted their findings on the image or explicitly opted out, the Data Structure agent turns their findings in a JSON results which is verified by the Verifier agent. If the Verifier agent does not agree with the JSON result, then the specialised agents can have a second attempt on the image. The run terminates when "TERMINATE" is mentioned by the Verifier agent or no proper results have been generated after two cycles.

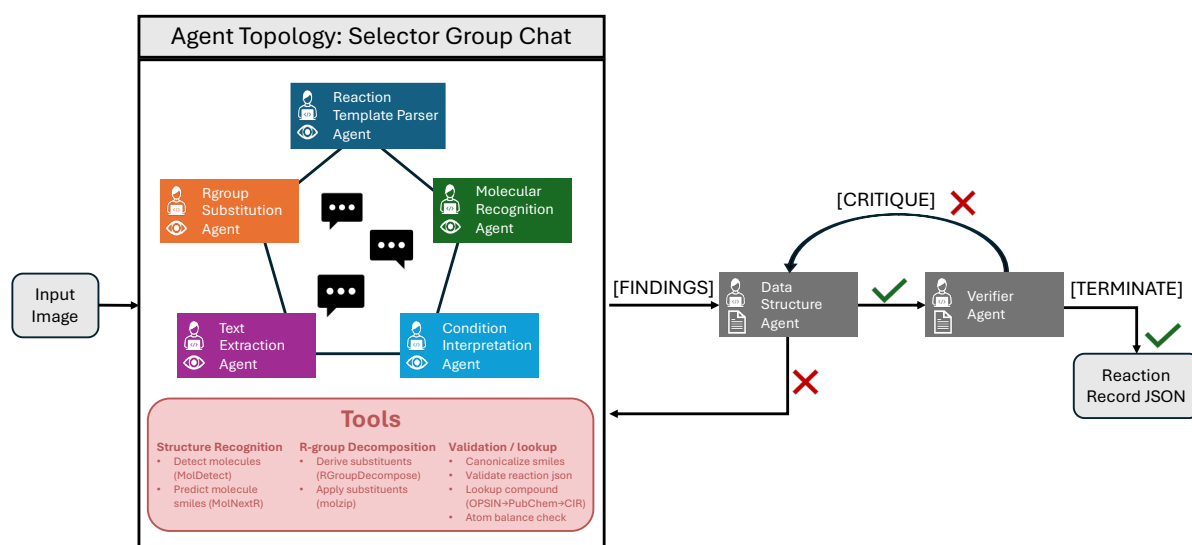


Figure 3-6: Agent topology. Five specialist agents (top row) each emit a [FINDINGS ...] card. The selector function then admits the Data Structure agent (the only JSON producer) and forces the verifier to speak immediately after; on [CRITIQUE] the cycle repeats up to twice before forced termination. An eye icon represents a VLM.

Table 3-2: Description of the agents used in the multi-agent framework.

Agent Name	Type	Description multi-agent framework
Reaction template parser	Vision	Identifies the overall reaction scheme(s); produces template SMILES with * wildcards for both reactant- and product-side templates.
Molecular recognition	Vision	Recognises molecules and extracts SMILES. Uses a deterministic ScriptedMolRecAgent that runs MolNexTR per molecule box and emits one entry per detected structure. Without MolNexTR, it falls back to a vision-LLM agent.
R-group substitution	Vision	For substrate-scope figures, derive R-groups via RDKit RGroupDecompose and substitute them into the reactant template via Chem.molzip to compute per-variant reactant SMILES.
Condition interpretation	Vision	Extracts conditions, classifies each by role.
Text extraction	Vision	Captures captions, footnotes and references that text the chemistry agents may have skipped.
Data structure	Text- Only	Azure gpt-5-mini, later promoted to gpt-5.4. The only agent allowed to emit JSON; aggregates the upstream [FINDINGS ...] cards into one ReactionRecord.
Verifier	Text- Only	Audits the JSON via and emits a [CRITIQUE ...] block on failure or TERMINATE on success. Bounded to two critique cycles before forced termination.

Tool Layer

A tools package provides RDKit, MolNexTR, and web-service-backed primitives that every agent can invoke. All tool invocations are traced during a run for audit. Tools that wrap external services

(lookup compound tool) are decorated with selective retries; transient HTTP 5xx errors (server-side) are retried with while permanent 404s (client-side) are not.

Table 3-3: The tool layer is exposed to the agents in the multi-agent framework.

Tool	Purpose	Used by
Canonicalize smiles	RDKit canonicalization + validity check	Most chemistry agents
Validate smiles	Light yes/no validity	All chemistry agents
Derive substituents	RDKit RGroupDecompose extracts R-groups from concrete molecules, given a wildcard template	R-group substitution
Apply substituents	RDKit Chem.molzip fuses R-fragments back into a template, preserving attachment positions	R-group substitution
Validate reaction JSON	Pydantic-backed schema validation, returns structured errors	Data structure, Verifier
Lookup compound	SMILES for multi-source names are resolved through a sequential cascade: OPSIN, then PubChem, then CIR.	Molecular recognition, Condition interpretation
Crop image region	Image cropping for "zoom in" requests.	Most vision agents
Atom balance check	Heavy-atom difference between reactant and product SMILES	Verifier (advisory)
Detect molecules	Rxnim MolDetect bbox detection of molecules in a chemistry figure	Reaction Template Parser, Molecular Recognition
Predict molecule smiles	Image-to-SMILES per bbox (MolNexTR); returns wildcard-bearing SMILES	Molecular Recognition, R-group Substitution

Development Phase

The project was developed in ten numbered phases. Each phase was independently shippable, and benchmark scores were captured at each stage to track regressions and gains. The phases are summarised below; for the full prompt files and per-phase tool-trace artefacts see the project repository.

Phase 1: Schema as Source of Truth and Retry Loop

The original prototype regex-extracted JSON from the agent's last message and silently saved the raw text. It accepted any string the agent produced, including condition role values like "equivalents". A pydantic v2 schema became the single source of truth. The condition-role was extended to include loading and pressure (categories that previously leaked into the additional info). A validation flow runs on the agent's draft; on failure, it constructs a list with validation errors: feedback message listing each pydantic error's location and re-prompts the Data Structure agent (up to 2 times).

Phase 2: Tool Layer

Tools package consists of several python files and deep-learning models: RDKit-backed SMILES canonicaliser, SMILES validator, r-groups enumerator, pydantic-backed reaction JSON validator, web-service compound lookup, image-cropper, and heavy-atom counter. All wrapped with a tracing decorator that logs each invocation per run. The enumerate r-groups tool was string-based at this stage: it replaced * tokens in a template with substituent SMILES and re-canonicalised. This was sufficient for simple substrate scopes but later proved insufficient when attachment-point information mattered (Phase 9).

Phase 3: Prompt and Role Redesign

The core architectural shift in this phase is that specialised agents stopped emitting their findings in JSON format. Each agent now emits a unique findings cards in a fixed plain-text format, and the Data Structure agent is the only JSON producer. A new Verifier agent was added with a hard rule that schema validity is its only blocking criterion, while chemistry sanity checks like atom-balance are advisory. Without this shift the pipeline produced three competing JSON drafts in different schemas (one each from R-group Substitution agent, Condition Interpretation agent, Text

Extraction agent) which Data Structure agent had to reconcile. Schema conformance was sub-50% under that regime; after this phase it climbed to 100% across the suite.

Phase 4: Coordination Redesign

A new dependency-gated coordination system of the agents has been implemented: SelectorGroupChat [12] from Microsoft. Instead of working in strict ordered pipeline, SelectorGroupChat implements a team where agents take turns broadcasting messages to all other agents. At the end of an agent's turn, the agent selects the next speaker based on the shared context, enabling dynamic, context-aware collaboration. This workflow is illustrated in Figure 3-7.

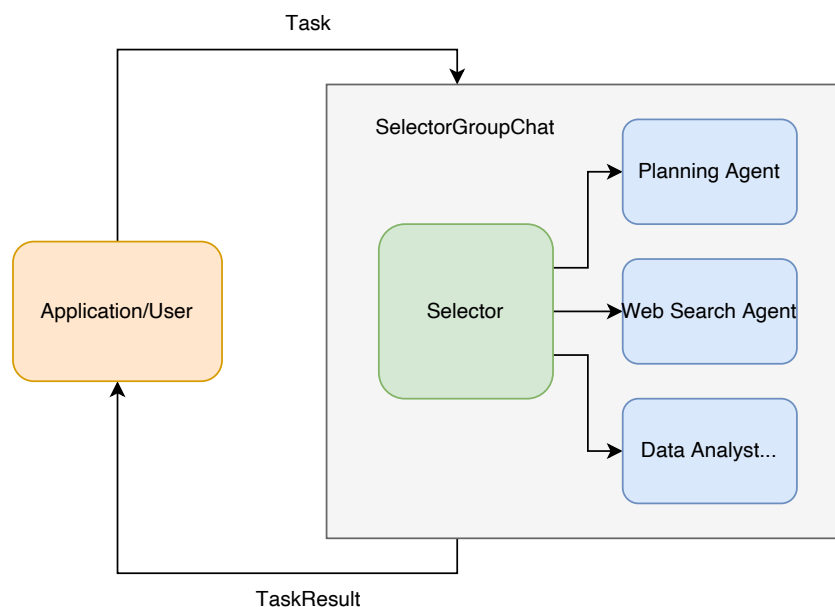


Figure 3-7: SelectorGroupChat is an agent coordination workflow [12] which is used in the multi-agent framework. SelectorGroupChat implements a team where agents take turns broadcasting messages to all other agents. At the end of an agent's turn, the agent selects the next speaker based on the shared context, enabling dynamic, context-aware collaboration.

Phase 5: Evaluation Harness

An evaluation method with two metric families has been implemented: reference-free (schema conformance check, SMILES validator, role enumeration compliance, reaction counter) and reference-based (intersection over union of reactants and products and a condition coverage checker). The evaluation runs the full benchmark with per-image timeouts, regression detection compared with a baseline, and a summary aggregating per-image and mean scores.

The original benchmark dataset was nine images sampled from three larger collections from ChemEAGLE[6]. The set spans three substrate-scope figures with R-group enumeration, three traditional reaction schemes, one single-reaction figure, and two OCR'd page crops. Reaction counts per image range from 1 to 13. Nine images is a small but deliberately varied evaluation set; it was later expanded to the 16-image benchmark.

Phase 7: MolNexTR Integration

ChemEAGLE's MolNexTR has been integrated as the tool to convert chemical images in to SMILES. MolNexTR returns canonical SMILES with `*/[N*]` wildcards, exactly what the downstream R-group decomposition consumes. The setup involved a separate Python 3.10 virtual environment with the dependency pin set. On a single-molecule crop, MolNexTR achieves 0.79 confidence with correct wildcard-bearing SMILES; on the full multi-molecule image of a substrate-scope figure, confidence drops to 0.004 because the model is trained for single-molecule inputs.

Phase 8: MolDetect Integration

The phase 7 limitation surfaced due to that agents asked MolNexTR to extract SMILES from full images which degrades performance. The repair was a separate molecule-bbox detector: MolDetect from ChemEAGLE, which is a deep-learning model. The workflow changed so that the agent calls MolDetect for a full image once to obtain N bounding boxes, then calls MolNexTR once per bounding box, yielding N high-confidence wildcard SMILES, as shown in Figure 3-5.

Initial experiments with the LLM-driven Molecular Recognition agent revealed it consistently under-counted: on a 13-variant substrate-scope figure it would call MolNexTR 18 times then write only 4 entries in its findings card, collapsing similar variants under labels like 1(a-m). Three rounds of prompt-tightening could not break this prior. The eventual fix was a deterministic Python script that bypasses the LLM entirely for this step. In turn, it runs the bbox detector and per-bbox MolNexTR inline and emits one molecule. The agent retains its slot in the team so the selector logic and the prompt contract for downstream agents are unchanged.

Phase 9: Deterministic R-group Derivation

By Phase 8 the pipeline reliably recognised the concrete products of substrate-scope figures with high product recall. The remaining bottleneck was per-variant reactant SMILES: most substrate-scope figures display only the products, and the corresponding reactants must be derived by structural reasoning ("subtract template from product to get R, substitute R into reactant template"). The Phase 8e prompt asked gpt-5.4 to do this reasoning visually. It worked partially: typically getting 4 to 5 out of 8 R-groups correct but failed systematically on small position errors (1-naphthyl versus 2-naphthyl, ortho versus para chlorine, isobutyl versus isopropyl). Figure 3-8 illustrates such an image.

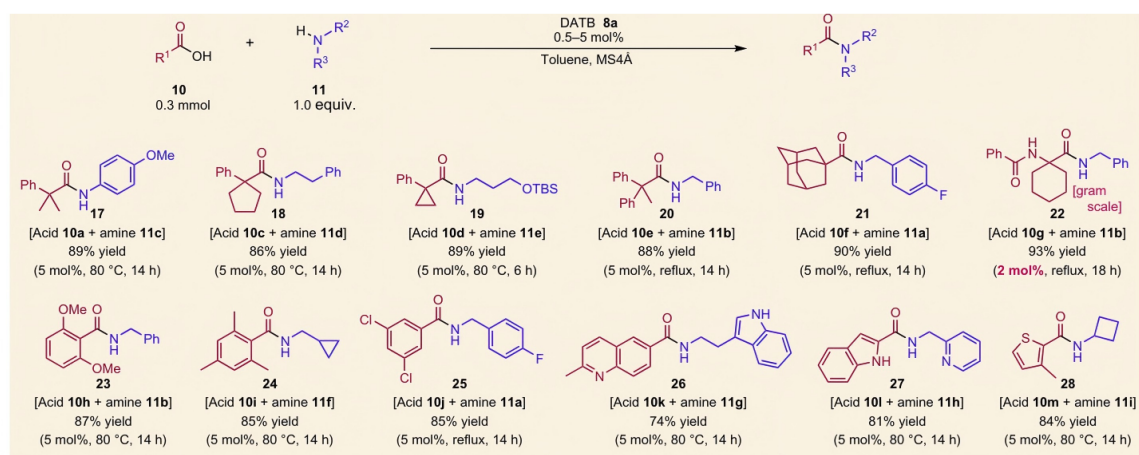


Figure 3-8: Substrate-scope figure (NC_2017.pdf_page004_picture_01) in which different reactions are specified by only showing a general reaction template at the top and their specific products at the bottom. The extraction pipeline must recognise the unique R-groups in the product and substitute them in the generic reactants SMILES to create each individual reaction template. The R-groups are identified by different colours.

Investigation of ChemEAGLE's SMILES reconstructor tool revealed that, despite the marketing in their README, ChemEAGLE's tool of that name is also LLM-driven (a label-and-classify step). They do not provide a deterministic R-group decomposition tool. The solution was to write our own with RDKit primitives.

The full pipeline call sequence on a substrate-scope figure is therefore: detect_molecules → per-bbox predict_molecule_smiles → one derive_substituents call against the product template → one apply_substituents call per variant against the reactant template → N specific reactant SMILES. Steps 1–2 are deep-learning (MolDetect + MolNexTR); steps 3–4 are pure RDKit graph operations and run in milliseconds with no LLM in the loop.

The full pipeline call sequence on a substrate-scope figure is therefore as follows: first molecules are detected by MolDetect, followed by MolNexTR predicting SMILES on each bounding box, then a single tool that derives the substituents against the product template, and finally one tool that applies substituents call per variant against the reactant template, yielding N specific reactant SMILES. Steps 1-2 rely on deep learning (MolDetect and MolNexTR), whereas steps 3-4 are pure RDKit graph operations that run in milliseconds with no LLM in the loop

Substituting one variant's R into a different template (e.g. the reactant template) gives the corresponding specific reactant. Two failure modes the chain cannot rescue: First, when MolNexTR returns the wrong concrete product SMILES, then the derive substituents tool faithfully extracts the wrong R-group and the reactant SMILES inherit the error (there is no internal consistency check); Second, when the product template and reactant template carry their R-groups on different ring positions but the figure does not draw the reactant template at all, the function has no way to verify the chosen reactant scaffold is correct, only that the substituent it injects is.

Phase 10: ChemEAGLE-comparable Evaluation Upgrade

To enable direct numerical comparison with ChemEAGLE's published results, the evaluation suite was extended beyond the IoU-based metrics: per-reaction binary matching (precision/recall/F1 by treating each reaction as either fully correct or fully wrong, rather than per-image set overlap).

3.3.3 ChemEAGLE Hybrid

ChemEAGLE Hybrid is a variant of the upstream ChemEAGLE pipeline designed independently. Its re-uses ChemEAGLE's chemistry deep-learning stack (MolDetect, MolNexTR, and the OPSIN, PubChem, CIR name-resolution cascade) but modifies the agent topology and the prompt set in different directions. The aim of the hybrid variant is to test whether modest changes to the orchestration layer above the existing chemistry tools can recover the published quality on the benchmark used here. The full design rationale and implementation are not the work of this thesis and are not documented in detail in this section. ChemEAGLE Hybrid is included in the experimental comparison because it was tested on the same 16-image benchmark and provides a third

reference point alongside the upstream ChemEAGLE reproduction and the original multi-agent framework. The numerical results for ChemEAGLE Hybrid are reported in Chapter 4.

3.3.4 Single-Agent Pipeline

The VLM single-agent pipeline is built around a uniform, model-agnostic agent loop in which a multimodal model is given an image of a chemical scheme together with two tools: an RDKit-backed SMILES validator tool, and a pydantic-validated extraction submission terminator. Several proprietary models were instantiated behind the same agent architecture: Anthropic Claude Opus 4.7, Opus 4.6, Microsoft Azure-hosted GPT-5.4, GPT-5.5, Google Gemini 3.1 Pro Preview, Google Gemini 3 Flash Preview, Gemini 3.5 Flash and xAI Grok 4.3 (accessed via OpenRouter). The post-processing step routes every successful extraction through Rxn-INSIGHT, producing a 29-column reaction-class /functional-group/fingerprint annotation per reaction. An eighth model, the open-weights Qwen3.6-27B, was added to the same architecture and served through OpenRouter for the benchmark runs reported in Chapter 4; the 27-billion-parameter checkpoint is also small enough to be self-hosted on the HPC cluster in float16 alongside the Qwen3-VL-8B-Instruct deployment described in Section 3.4.

Motivation

The single-agent pipeline was built for two reasons. First, an experiment: if the multi-agent decomposition is the lever that ChemEAGLE credits for its quality, then a simple single-agent baseline over a strong VLM should land well below the multi-agent and ChemEAGLE numbers; if it does not, the lever is somewhere other than the orchestration. Second, an engineering response: the multi-agent framework had reached a point at which incremental gains required prompt-following improvements that the VLM provided unreliably, and at the same time newer multimodal models (Claude Opus 4.7, Gemini 3 Flash) had become available with substantially stronger figure-reading performance than the GPT-5.4/GPT-5-mini class. A clean re-implementation around the strongest available models, without inheriting the multi-agent overhead, was a low-cost way to re-baseline the project. Figure 3-9 illustrates the visual reasoning capabilities of the seven VLMs compared to the image input cost. The figure is adopted from Artificial Intelligence [11], a platform

that compares AI models on a variety of benchmarks and criteria. The benchmark used in this comparison is the MMU Pro evaluation, which scores the performance of the model on 1000 images. The left upper quadrant is the most attractive one, as the costs are low and the performance is high making Gemini 3 Flash and Grok 4.3 are the most attractive models. However, if costs are not important, GPT-5.5 with high reasoning effort has the highest visual reasoning score.

Visual Reasoning vs. Image Input Price

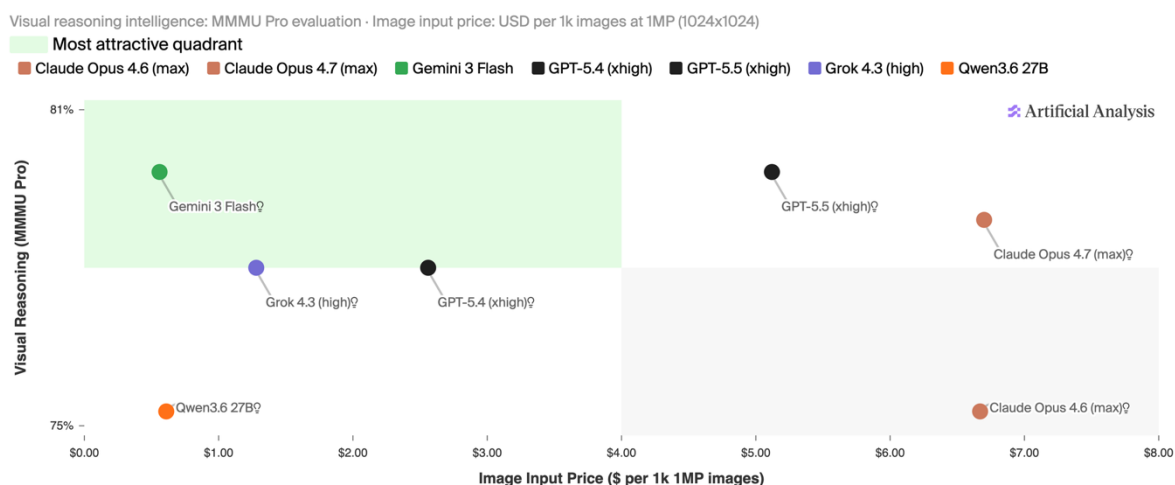


Figure 3-9: Comparison of the seven models on visual reasoning versus image input price. Figure is adopted from Artificial Intelligence [13], a platform that compares AI models on a variety of benchmarks and criteria. The benchmark used in this comparison is the MMU Pro evaluation, which scores the performance of the model on 1000 images.

A second comparison isolates the Opus 4.6 to Opus 4.7 generational jump on Anthropic's own visual-navigation evaluation (Figure 3-17) [14]. On ScreenSpot-Pro, Opus 4.7 at high resolution reaches 87.6% accuracy with tool use and 79.5% without; Opus 4.6 at low resolution reaches 83.1% with tool use and 57.7% without. This is consistent with the qualitative observation that newer multimodal models read complex chemical schemes more reliably than the GPT-5.4/GPT-5-mini class behind the original multi-agent framework, and it is part of the rationale for adopting Opus 4.7 as the strongest single-model baseline in the single-agent pipeline.

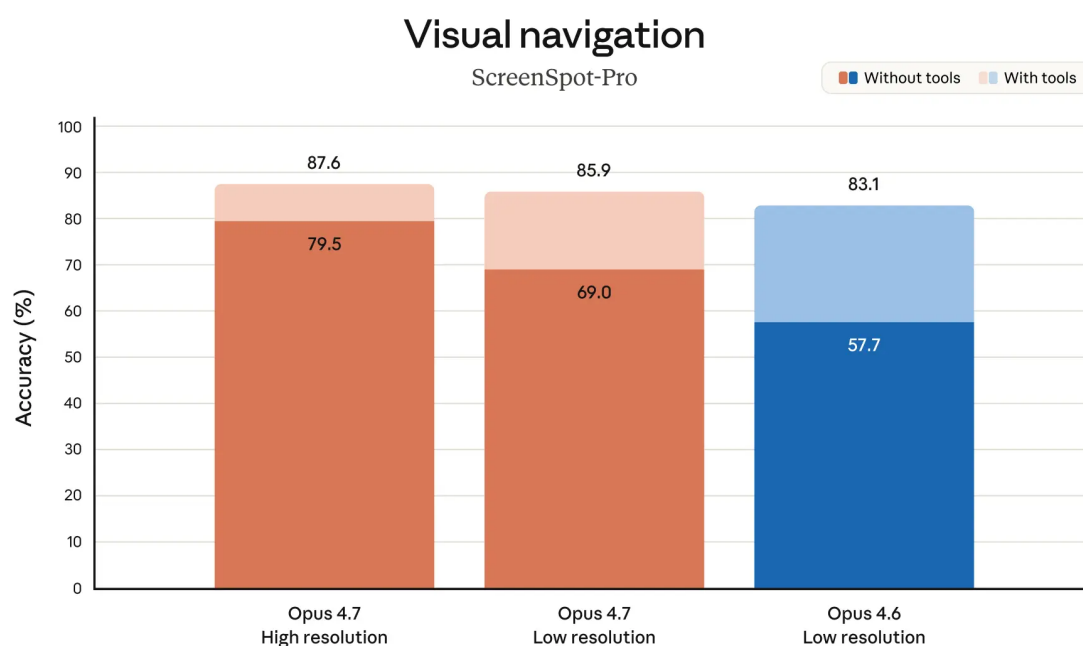


Figure 3-10: ScreenSpot-Pro visual-navigation accuracy for Claude Opus 4.7 (high and low resolution) and Claude Opus 4.6 (low resolution), with and without tool use [14].

Agent Loop

Extraction is performed by an agent loop in which a multimodal model is invoked iteratively until it calls the terminator tool. Each iteration of the loop consists of three steps: (i) the model is sent the conversation so far, including the image; (ii) the model's response is parsed for tool calls; (iii) each tool call is executed locally, and its result is appended to the conversation. The loop exits when the model invokes the submit extraction message, when an error event is emitted, or when a configurable step ceiling is reached.

A single **system prompt** is used across all multimodal models. It describes the extraction task, names each tool available to the agent, and lists a quality bar covering canonical SMILES, generic placeholders, yield qualifiers, reagent role disambiguation, and the explicit extraction termination requirement. Particular care is taken in two places: the prompt instructs the agent to call the SMILES validator on every drawn structure prior to submission, and it specifies that condition surveys (where one generic transformation is drawn against a gallery of alternative catalysts) are to be expanded into one reaction per labelled variant. The latter rule corrects a default behaviour in which several models would otherwise produce a single generic reaction and file the alternative catalysts as intermediates.

The system prompt was redesigned following Anthropic's published prompt-engineering recommendations for Claude Opus 4.7 [15], shown in Figure 3-11. The original version was a single markdown block consisting of a numbered "Workflow" followed by a flat "Quality bar" bullet list, with seven rules emphasised by uppercase MUST or NEVER directives and no worked examples. The revised prompt makes three structural changes that the documentation associates with measurable gains on long, instruction-heavy prompts. First, the content is partitioned into named XML sections (`<role>`, `<workflow>`, `<smiles_rules>`, `<reagent_rules>`, `<yield_rules>`, `<scope_and_condition_surveys>`, `<examples>`, `<self_check>`), so each rule can be located by tag rather than parsed from surrounding prose; the documentation explicitly recommends this pattern when a prompt mixes role, instructions, examples, and structured output. Second, two worked few-shot examples are introduced: one demonstrating atom-mapped wildcards (`[1*]`, `[2*]`) on a Suzuki-coupling scope header, and one demonstrating the expected reaction count for a four-variant catalyst gallery. Both correspond to the two extraction patterns that the original prompt had specified verbally but never illustrated. Third, the imperative register is softened: aggressive negations such as "Never repeat the headline percentage in `yield_note`" are replaced by positive specifications ("`yield_note` carries only information not already in `yield_pct`"), reducing the count of uppercase prohibitions from seven in the original to a single occurrence (ONCE, applied to the terminal `submit_extraction` call). This last change reflects guidance specific to Claude 4.6 and later, which are reported to overtrigger on prompts written for earlier models. A short `<self_check>` block is appended at the end of the prompt that enumerates four pre-submission invariants (non-empty canonical SMILES, populated reagent labels, numeric `yield_pct`, and consistent scope/variant counts) implementing the recommendation that the model verify its own output before terminating.

```

SYSTEM_PROMPT = """
<role>
You are an expert chemist extracting structured reaction data from a figure.
You have two tools: 'validate_smiles' and 'submit_extraction'.
</role>
<workflow>
1. Read the figure carefully. Identify every labelled compound, every reaction
equation, and any generic "scope" header drawn above a table of specific examples.
2. Draft SMILES for every drawn structure. Pay attention to stereochemistry
(wedges), protecting groups, and substituent labels (Me, Bn, Boc, TBS, ...).
3. Call 'validate_smiles' on every SMILES you wrote. If RDKit can't parse it,
fix the SMILES and re-validate.
4. Run the <self check> below. Then call 'submit_extraction' exactly ONCE with
the final payload and stop – no more text, no more tool calls.
</workflow>
<smiles_rules>
- Use the canonical SMILES returned by 'validate_smiles' in the final submission.
- For an unspecified R group, use '[*]'. When the figure labels distinct R groups
(R1, R2, ...), use atom-mapped wildcards: '[1*]', '[2*]', etc. Example: a generic
α-amino acid scaffold drawn with R becomes '[*][C@H](N)C(=O)OC'.
- Apply the wildcard rule to every drawn R group in every reaction, including
generic scope headers – not just the first reaction in the figure.
- Use 'smiles=""' only when no structure is drawn (pure text label). If the
figure draws a structure, give it SMILES.
- If a substituent is ambiguous (e.g. (CH2)n with unclear n), record your best
guess and flag it in 'extraction_notes'.
</smiles_rules>
<reagent_rules>
- Every reagent has 'label' or 'name' populated, preferably both. If no label is
shown, fall back to the common abbreviation (AcOH, TFA, DCM, HCONH2, ...) as 'name'.
- If the same compound serves as both reagent and reaction medium (e.g. formamide
as both acyl source and solvent), list it ONCE as a reagent with role='other'
and an appropriate loading; leave 'conditions.solvent' null. Listing it twice
would double-count it in downstream aggregation.
</reagent_rules>
<yield_rules>
- 'yield_pct' is the headline number (e.g. 96).
- 'yield_note' carries only information not already in 'yield_pct' – scale,
recovered SM, dr, ee, conditional qualifiers ("On 5 mmol scale", "after
recrystallization"). Don't echo the headline percentage.
- For yields shown as 'trace', 'NR', 'ND', leave 'yield_pct' null and put the
qualifier in 'yield_note'.
</yield_rules>
<scope_and_condition_surveys>
A figure often shows one generic transformation paired with multiple alternative
catalysts (or solvents, ligands, oxidants, ...) drawn as a labelled gallery –
typically 1, 2, 3, ... below or beside the scheme, sometimes grouped under banners
like "Catalysts active at >80 °C". Emit one Reaction per labelled variant, plus
the generic header itself as a separate Reaction.
Each variant:
- shares the same substrate and product as the header (wildcard SMILES, copied);
- differs only in its 'reagents' (the specific catalyst with its own SMILES +
label) and one banner condition (e.g. 'temperature: ">80 °C"');
- the variant catalysts go in 'reagents', not in 'intermediates'.
'entry_id' should match the figure's labelling (e.g. 'I', 'II', '1', '8b').
</scope_and_condition_surveys>
<examples>
<example>
Figure: a Suzuki coupling header drawn once at top – "ArBr + R-B(OH)2 → Ar-R" –
labelled "scope". Below it, three boxed examples labelled 1a (Ar = Ph, R = 4-MeOPh,
92%), 1b (Ar = 4-NO2-C6H4, R = vinyl, 78%), 1c (Ar = 2-thienyl, R = Cy, 65%).
Correct submission: 4 Reactions.
- entry_id "scope": substrate '[1*]Br', partner '[2*]B(O)O', product '[1*][2*]',
yield_pct null.
- entry_id "1a": substrate 'Brc1ccccc1', partner 'OB(O)c1ccc(OC)c1',
product 'COc1ccc(-c2ccccc2)c1', yield_pct 92.
- entry_id "1b", "1c": analogous, with concrete SMILES from the labels.
</example>
<example>
Figure: one C-H amination scheme at top. Below, four catalyst structures labelled
1-4 under a banner "Catalysts active at >80 °C". No per-variant yields printed.
Correct submission: 5 Reactions.
- entry_id "header": wildcard substrate/product copied from the scheme, no
specific catalyst, yield_pct null.
- entry_id "1"-4: each copies the wildcard substrate/product from the header,
has its specific catalyst as a reagent (with its own SMILES + label), and
'conditions.temperature = ">80 °C"'. yield_pct null.
</example>
</examples>
<self_check>
Before calling 'submit_extraction', verify:
- Every drawn structure has a non-empty canonical SMILES from 'validate_smiles'.
- Every reagent has 'label' or 'name' populated.
- Every 'yield_pct' is numeric or null – not a string.
- For scope/condition surveys: the number of Reactions equals
(1 generic header) + (number of labelled variants).
Fix any violations and re-submit.
</self_check>
Be thorough but don't fabricate. If you can't read part of the figure, say so in
'extraction_notes' rather than guessing.
"""

```

Figure 3-11: System prompt of agent consisting of several blocks: role, workflow, SMILES rules, reagent rules, yield rules, scope and condition surveys, two examples and a self-check. The prompt uses a XML layout and prompt-engineering recommendations for Claude Opus 4.7 [15].

The agent has access to two **tools**. The first tool parses a candidate SMILES string with RDKit and returns either a canonical SMILES, the molecular formula or an error explaining why the string could

not be parsed: `validate_smiles(smiles: str) → dict`. The tool is the agent's only mechanism for verifying that a drawn structure has been transcribed correctly: if RDKit cannot parse it, the agent is required to revise the SMILES. The second tool is the terminator: `submit_extraction(extraction: FigureExtraction) → dict`. Its input schema is the JSON-Schema of the pydantic `FigureExtraction` model. When the agent calls this tool, the supplied argument is validated against the pydantic model: on success, the agent loop exits and the validated object is returned; on failure, the validation errors are returned to the agent so that it can correct its submission and call again.

The agent loop is implemented as a generator that yields events as they occur (`step_start`, `step_done`, `complete`, `error`). The loop is bounded by a per-call step ceiling that defaults to 12. A "step" here counts one model round-trip, not one tool call: a single round-trip can contain several tools calls in parallel when the model elects to batch. For models that have been trained or instructed to batch tool calls (Anthropic Claude family, OpenAI GPT family), 12 roundtrips are sufficient for almost all benchmark images. For models that emit only a few tool calls at a time (Google Gemini family, xAI Grok), the same workload requires substantially more round-trips, and the ceiling was raised to 64 (Gemini) and 128 (Grok and Qwen3.6-27B) to provide adequate headroom; the higher ceiling on Qwen3.6-27B absorbs a non-converging `validate_smiles` retry loop observed on dense scope and optimisation tables, discussed in Chapter 4. The cause of these differences is examined later.

Multi-Provider Implementation

The agent loop is implemented once per model and shares the same extraction schema, the same tool set, and the same system prompt across all backends. The four extractor modules differ only in their wire-format translation layer: each adapts the native function-calling protocol of its Software Development Kit (SDK) to the common loop.

The **Anthropic SDK**, used for **Claude Opus 4.6 and 4.7**, exposes tool use through `messages.create`, where assistant responses contain interleaved text and `tool_use` content blocks. Multiple `tool_use` blocks can appear in a single assistant response; the implementation executes them in order and returns each result as a `tool_result` block in a subsequent user message. Tool-use blocks declare their schema as an `input_schema` JSON-Schema dictionary; the `submit_extraction` schema

is patched in at runtime from `FigureExtraction.model_json_schema()`. Image input is provided as a base64-encoded image content block.

Azure-hosted OpenAI GPT-5.4 and GPT-5.5 was accessed through Microsoft Azure AI Foundry, which exposes an OpenAI-compatible chat-completions endpoint. The OpenAI tool-use protocol declares each tool as (`"type": "function", "function": (name, description, parameters)`) and expects tool results to be returned as role: "tool" messages with a `tool_call_id` field. The schema-translation step rewrites the Anthropic-style schemas as OpenAI-style schemas; semantically the tools are identical. Image input is supplied as an `image_url` content part with a base64 data URL.

The **Google Generative AI SDK for Gemini 3.1 Pro Preview** and **Gemini 3 Flash Preview** use a different message shape: conversation history consists of Content objects with role "user" or "model", and tool results are passed back as user-role Part objects. Tool definitions are translated to Gemini's `function_declarations` schema, which is similar to but stricter than JSON-Schema (several JSON-Schema features such as `default`, `additionalProperties` are rejected and were removed during translation). The `tool_config` is set to `mode="ANY"` to encourage the model to call tools rather than emit free text.

OpenRouter [16] is a unified gateway that exposes many proprietary models through an OpenAI-compatible interface. Routing the agent through OpenRouter therefore requires no new schema-translation logic: the OpenAI extractor is reused with a custom `base_url`. The model identifier **x-ai/grok-4.3** was selected for evaluation. Grok's tool-call behaviour is closer to Gemini's than to Anthropic's: tool calls are rarely batched, and substantial reasoning text is emitted between calls.

The four extractor modules expose a single function each, named `extract_figure_stream_<provider>`, with a uniform signature: `(image_path, *, model, client, system_prompt) → Iterator[event]`. The downstream batch runner is provider-agnostic: it accepts the streaming output and writes the validated FigureExtraction to a per-image JSON file, alongside a metadata sidecar capturing input tokens, output tokens, tool calls, roundtrips, and wall time. This uniform output structure, but also the common loop, schema and prompt are what enables the model-by-model comparison.

Input-Image Resolution Ablation Setup

To quantify the effect of input-image down-sampling on extraction quality, a controlled ablation was conducted on the single-agent pipeline using Gemini 3 Flash Preview as the backbone. The 16-image benchmark was processed at five input resolutions: full (1/1) (Figure 3-12), one-half ($1/2 \times 1/2 = 1/4$ area) (Figure 3-13), one-quarter ($1/4 \times 1/4 = 1/16$ area) (Figure 3-14), one-eighth ($1/8 \times 1/8 = 1/64$ area) (Figure 3-15), and one-tenth ($1/10 \times 1/10 = 1/100$ area) (Figure 3-16). Each image was Lanczos-resampled to the target fraction of its original width and height; original image dimensions in the benchmark range from 663×393 pixels to 1487×840 pixels. The one-eighth condition was added a posteriori to localise the transition between robust extraction and collapse, which an earlier sweep using only the four extremes had placed somewhere between the one-quarter and one-tenth conditions.

Table 1. Optimization of the Reaction Conditions^a

Reaction scheme: BocHN-CH(R)-CO₂H (1a, R=H; 1b, R=Bn) + H₂N-CH₂-Ph (2, 1 equiv) $\xrightarrow[\text{temp., 12 h}]{\text{catalyst (5 mol\%), 5 \AA MS, solvent}}$ BocHN-CH(R)-C(=O)-NH-CH₂-Ph (3a or 3b)

entry	substrate	catalyst	solvent	temperature (°C)	3b ^b (%)
1	1a	A	toluene	80	99
2	1b	A	toluene	80	0
3	1b	B	toluene	80	0
4	1b	C	toluene	80	82
5	1b	C	toluene	65	31
6	1b	D	toluene	65	96 ^c
7	1b	D	PhF	65	78
8	1b	D	(CH ₂ Cl) ₂	65	68
9	1b	D	CPME	65	85
10	1b	D	DMF	65	0
11	1b	D	toluene	50	77
12	1b	none	toluene	65	0
13 ^d	1b	D	toluene	65	0

^aReaction conditions: **1** (0.20 mmol, 1.0 equiv), **2** (0.20 mmol, 1.0 equiv), catalyst (5 mol%), 5 Å MS, solvent, temperature, 12 h.

Figure 3-12: Full resolution (756 × 646 px) image used for the Resolution Ablation Setup (ACScat_2020 page 002, table 01).

Table 1. Optimization of the Reaction Conditions^a

entry	substrate	catalyst	solvent	temperature (°C)	3b ^b (%)
1	1a	A	toluene	80	99
2	1b	A	toluene	80	0
3	1b	B	toluene	80	0
4	1b	C	toluene	80	82
5	1b	C	toluene	65	31
6	1b	D	toluene	65	96 ^c
7	1b	D	PhF	65	78
8	1b	D	(CH ₂ Cl) ₂	65	68
9	1b	D	CPME	65	85
10	1b	D	DMF	65	0
11	1b	D	toluene	50	77
12	1b	none	toluene	65	0
13 ^d	1b	D	toluene	65	0

^aReaction conditions: 1 (0.20 mmol, 1.0 equiv), 2 (0.20 mmol, 1.0

Figure 3-13: Half resolution (378 × 323 px) image used for the Resolution Ablation Setup (ACScat_2020 page 002, table 01).

Table 1. Optimization of the Reaction Conditions^a

entry	substrate	catalyst	solvent	temperature (°C)	3b ^b (%)
1	1a	A	toluene	80	99
2	1b	A	toluene	80	0
3	1b	B	toluene	80	0
4	1b	C	toluene	80	82
5	1b	C	toluene	65	31
6	1b	D	toluene	65	96 ^c
7	1b	D	PhF	65	78
8	1b	D	(CH ₂ Cl) ₂	65	68
9	1b	D	CPME	65	85
10	1b	D	DMF	65	0
11	1b	D	toluene	50	77
12	1b	none	toluene	65	0
13 ^d	1b	D	toluene	65	0

^aReaction conditions: 1 (0.20 mmol, 1.0 equiv), 2 (0.20 mmol, 1.0

Figure 3-14: Quarter resolution (189 × 161 px) image used for the Resolution Ablation Setup (ACScat_2020 page 002, table 01).

Table 1. Optimization of the Reaction Conditions^a

entry	substrate	catalyst	solvent	temperature (°C)	3b ^b (%)
1	1a	A	toluene	80	99
2	1b	A	toluene	80	0
3	1b	B	toluene	80	0
4	1b	C	toluene	80	82
5	1b	C	toluene	65	31
6	1b	D	toluene	65	96 ^c
7	1b	D	PhF	65	78
8	1b	D	(CH ₂ Cl) ₂	65	68
9	1b	D	CPME	65	85
10	1b	D	DMF	65	0
11	1b	D	toluene	50	77
12	1b	none	toluene	65	0
13 ^d	1b	D	toluene	65	0

^aReaction conditions: 1 (0.20 mmol, 1.0 equiv), 2 (0.20 mmol, 1.0

Figure 3-15: Eighth resolution (94 × 80 px) image used for the Resolution Ablation Setup (ACScat_2020 page 002, table 01).

Figure 3-16: Tenth resolution (75×64 px) image used for the resolution ablation setup (ACScat_2020 page 002, table 01).

All five conditions reuse the same single-agent pipeline as described above. No context caching, no conversation history across images, and no fine-tuned variant of the model were used; each image was processed in an independent API call to gemini-3-flash-preview, which rules out cross-image information leakage. Ground-truth JSON files were copied unchanged into each per-condition input directory, and the batch driver was verified to enumerate only the *.png inputs so the ground-truth files were never transmitted to the model.

Per-condition outputs were scored with the same evaluation harness used for every other single-agent variant.

3.3.5 LeMath-synth

While the four pipelines documented in Sections 3.3.1 through 3.3.4 (ChemEAGLE, the multi-agent framework, ChemEAGLE Hybrid, and the single-agent pipeline) all extract structured reaction records from rendered figures, an organic-chemistry paper also carries a parallel body of synthesis information in its experimental procedure text. LeMath-synth is a procedural-text track that complements the figure-derived chemistry by recovering action sequences, stoichiometries, temperatures, and work-up details from the experimental prose. The two tracks are run independently and produce side-by-side records per paper; the integration into a single unified record is left as future work.

Motivation

ChemEAGLE only processes the visual content of a paper (reaction-scheme images and structure-bearing tables) and ignores the procedural prose where synthesis steps are described. Yet a substantial part of a synthetic paper's information sits in those text paragraphs: action sequences, exact

stoichiometries, atmosphere and temperature changes, work-up details. To complement ChemEAGLE on this dimension, the open-source LeMath-synth toolbox [17] was integrated into the pipeline as a second extraction track. Because LeMath-synth was originally designed for inorganic materials science, several modifications were made to make it usable on papers; both the original and the adapted version are kept in the project repository for comparison.

The Original Toolbox

LeMath-synth couples an LLM-driven text track with a VLM-driven figure track under a unified Pydantic ontology. The text track parses synthesis procedures using a structured prompt pipeline on top of Gemini 2.0 Flash; for each paper, it identifies the target material, extracts the corresponding procedure, and emits a JSON record. The figure track segments multi-panel figures with the DINO [18] zero-shot vision model, classifies them with a ResNet-152 fine-tuned on DocFig, and digitises quantitative plots with Claude 4 Sonnet. The built-in ontology covers 35 inorganic synthesis methods (sol-gel, hydrothermal, ball-milling, CVD, etc.) and 16 material classes; on a 2.5 k-procedure validation set Lederbauer *et al.* [17] report extraction quality correlating with expert annotations at Spearman $\rho \approx 0.72$.

Adaptation to Organic Chemistry

Three changes were made by the writer to apply LeMath-synth to organic synthesis papers. First, a new organic synthesis ontology was added alongside the original materials ontology, with 31 organic-relevant synthesis methods (cross-coupling, aldol reaction, Wittig, Diels–Alder, protection–deprotection, etc.), 14 target-compound classes (small molecule, natural product, pharmaceutical, heterocyclic compound, ...), and a 30-verb action vocabulary tailored to wet-chemistry work-ups (couple, cyclize, deprotect, recrystallize, chromatograph, ...).

Second, both extraction prompts were rewritten. The material-extraction prompt now instructs the LLM to extract not only final products but also key intermediates with their own preparation procedures in the Experimental section, using compound names or labels (e.g. "12: O-prenyl vinylogous ester") rather than chemical formulas. The synthesis-extraction prompt was expanded with six explicit instructions for multi-step organic synthesis: trace the full route from commercially

available starting materials, combine information from the Results/Schemes and Experimental sections, expand "General Procedure" references with the actual reagents and conditions for the compound at hand, avoid extracting only the final deprotection step, include intermediate preparation steps, and preserve all quantitative details (yields, enantiomeric excess (ee%), diastereomeric ratio (dr), selectivity).

Third, a set of helper utilities was added: an `expand_general_procedures()` Markdown post-processor that inlines "prepared following General Procedure I" references, a three-tier regex-based yield-recovery pass that retries from notes, last-step text, and per-step parentheticals when the LLM returns a null yield, consecutive-step deduplication, and a hard cap of 30 steps per synthesis. The figure track of the original toolbox was disabled, since organic-chemistry papers convey synthesis information through text and reaction schemes rather than the line plots LeMath-synth's figure pipeline targets.

The pipeline is exposed as a single command line interface (CLI) with a multi-provider preset (OpenAI / Gemini / Mistral) and a cost-tiered three-LLM split: a powerful model for synthesis extraction (default gpt-4.1), a cheap model for material extraction (gpt-4o-mini), and the same cheap model for the LLM-as-judge step (gpt-4o-mini). Per-run outputs are tagged with a timestamp and a user-supplied label so iterations on the prompts do not overwrite earlier results, and a `--no-eval` flag skips the judge for cheap structural debugging.

Integration in this Work

ChemEAGLE and the adapted LeMath-synth currently run as parallel extraction tracks: each produces its own JSON record per paper, and the two outputs sit side by side without automated reconciliation. The downstream feed into Rxn-INSIGHT uses the ChemEAGLE record only; the LeMath-synth record is retained as procedural context for human inspection and for cross-checking the reaction-scheme extractions. Bringing the two outputs into a single unified record per paper, for example by mapping LeMath-synth's per-step conditions onto ChemEAGLE's per-reaction conditions and resolving naming disagreements via a canonical SMILES key, is identified in Chapter 5 as an open piece of follow-up work.

3.3.6 Rxn-INSIGHT

Intro

Rxn-INSIGHT [19] is a chemical reaction analysis toolkit that classifies and analyses chemical reactions using Bond-Electron (BE) matrices, a representation that encodes changes in bond order during a transformation. Rather than relying on simple pattern matching, the tool examines the fundamental electronic rearrangements underlying each reaction, enabling mechanistic classification into categories such as oxidation, reduction, C-C coupling, and acylation. Beyond classification, Rxn-INSIGHT can identify named reactions (e.g., Suzuki coupling, Aldol addition), track participating functional groups, extract reaction templates in SMIRKS notation for retrosynthetic analysis, and perform stereochemistry tracking.

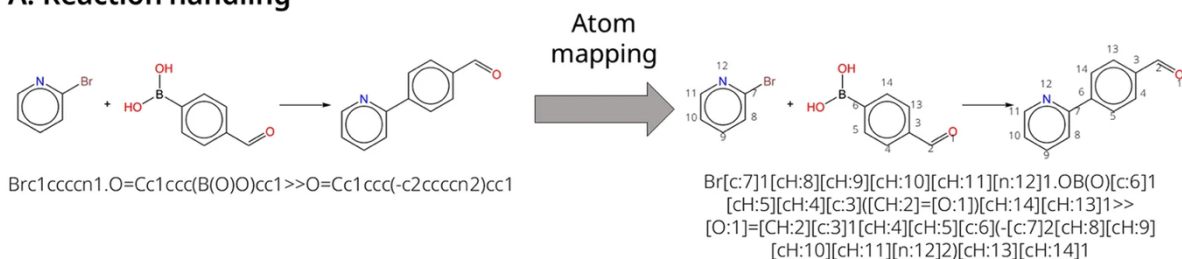
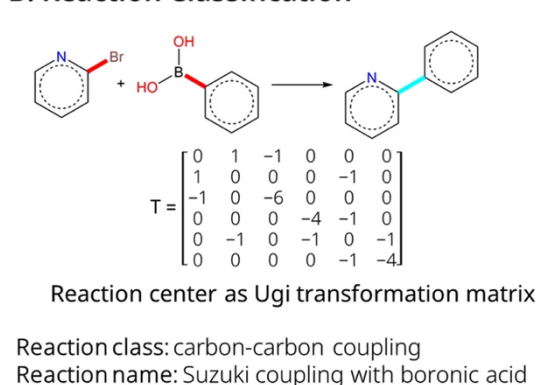
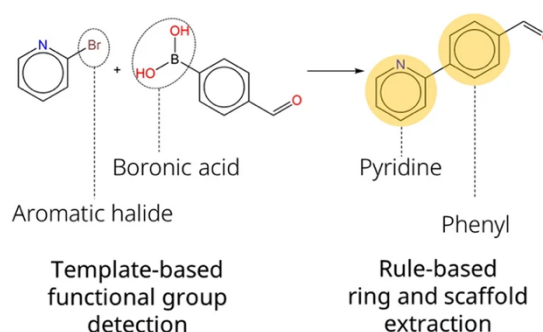
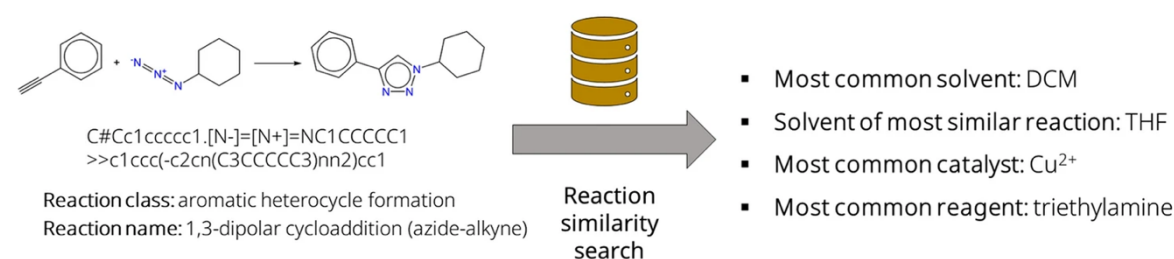
A. Reaction handling**B. Reaction Classification****C. Functional group and ring detection****D. Literature-based condition suggestion**

Figure 3-17: Overview of the Rxn-INSIGHT algorithm's workflow [19]. A. An example chemical reaction and its corresponding Reaction SMILES representation, with and without atom mapping. B. The Ugi transformation matrix for the example reaction, along with its assigned classification. C. Identification of functional groups, rings, and scaffolds within the reactants and products. D. Proposed reaction conditions inferred from the reaction class, scaffold, and functional groups.

Creating a Database

To build a reaction database, Rxn-INSIGHT's 'Database' class accepts either a pandas DataFrame or a CSV file containing reaction SMILES strings, alongside optional metadata columns for solvent, catalyst, reagent, yield, and literature reference as illustrated in Figure 3-18. Each reaction is passed through the full Rxn-INSIGHT analysis pipeline, covering atom mapping, BE-matrix construction, reaction classification, functional group detection, and template extraction, and the results are returned as an augmented DataFrame.

The resulting database contains a standardized set of output columns, including the classified reaction class and named reaction, functional groups present in both reactants and products, extracted SMIRKS templates, and two hash-based identifiers (TAG and TAG2) that allow grouping of reactions by exact or approximate transformation type. Additionally, four reaction fingerprint columns are generated by combining two fingerprint types (MACCS and Morgan) with two construction strategies: concatenated fingerprints, which join reactant and product fingerprints end-to-end, and difference fingerprints, which subtract the reactant fingerprint from the product fingerprint to emphasize what changes during the reaction. For large datasets, saving in Parquet format is recommended due to its superior compression and faster read speeds compared to CSV.

```
import rxn_insight as ri
import pandas as pd

data = {
    "rxn": ["OB(O)c1cccc1.Br1cccc1>>c1ccc(-c2cccc2)cc1"],
    "solvent": ["DMF"],
    "catalyst": ["[Pd]"],
    "yield": [85],
    "reference": ["Ref1"]}
df = pd.DataFrame(data)

db = ri.Database()
analyzed_df = db.create_database_from_df(
    df,
    reaction_column="rxn",
    solvent_column="solvent",
    catalyst_column="catalyst",
    yield_column="yield",
    ref_column="reference",
    classify=True,
    add_fp=True,
    n_jobs=4 )
```

Figure 3-18: Example Python code demonstrating the ingestion and processing of a single reaction entry using Rxn-INSIGHT's Database class [19].

Database Usage

Once the database is constructed, it supports a range of downstream analyses. Reaction class and named reaction distributions can be computed directly, providing an overview of the dataset composition. Solvent and catalyst usage can be broken down per reaction class or named reaction, and temporal trends can be tracked if publication year data is available. Similarity searching is also supported: given a query reaction, the `find_neighbors()` method first filters the database using the reaction's TAG hash to identify a relevant subset, then ranks candidates by Tanimoto similarity computed over the chosen fingerprint type, enabling practical similarity search across millions of reactions within minutes on standard hardware.

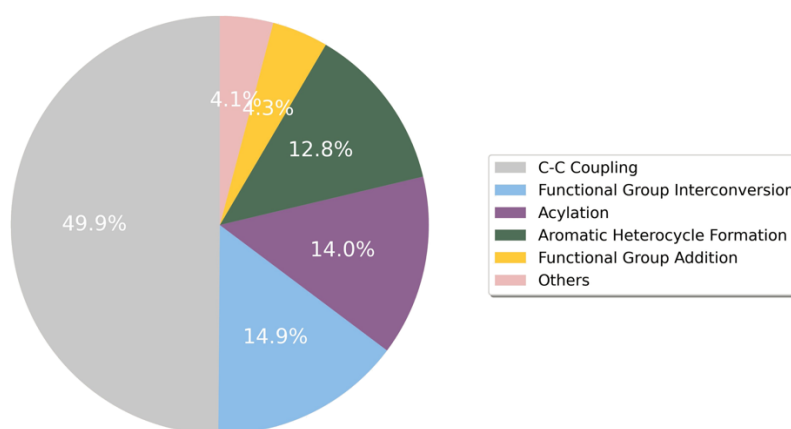


Figure 3-19: Reaction class distribution of 437 mechanochemistry reactions using Rxn-INSIGHT.

3.4 Closed versus Open-source Models

The different pipelines are all hosted around VLMs that can be either closed-source, accessed through a paid provider API, or open-source, served on local infrastructure. Each model used in this thesis has unique specification like the snapshot identifier, the context window, the knowledge cutoff, the modalities, and the host through which the model was reached. Pricing is summarised separately in Section 3.5.

Table 3-4 maps each model to the pipelines that use it and to the host through which it was reached.

Table 3-4: Models, pipelines, and hosting routes used in the different pipelines.

Model	Pipeline(s)	Host
GPT-5-mini	ChemEAGLE (local), Multi-Agent Framework	Azure AI Foundry
GPT-5.4	Multi-Agent Framework (primary), Single-Agent Pipeline	Azure AI Foundry
GPT-5.5	Single-Agent Pipeline	Azure AI Foundry
Claude Opus 4.6	Single-Agent Pipeline	Anthropic API
Claude Opus 4.7	Single-Agent Pipeline, Multi-Agent (Opus 4.7 backbone)	Anthropic API
Gemini 3 Flash Preview	Single-Agent Pipeline, ChemEAGLE-Gemini fork, ChemEAGLE Hybrid	Google AI Studio
Gemini 3.1 Pro Preview	Single-Agent Pipeline	Google AI Studio
Gemini 3.5 Flash	Single-Agent Pipeline	Google AI Studio
Grok 4.3	Single-Agent Pipeline	xAI API
Qwen3.6-27B	Single-Agent Pipeline	OpenRouter
Qwen3-VL-8B-Instruct	Open-source exploratory run only	HPC cluster (vLLM)

3.4.1 Closed-Source Models

The closed-source models documented below are reached through provider APIs. Microsoft Azure AI Foundry is the host for both OpenAI models, providing managed endpoints, regional deployments, and consolidated cost accounting under a single Azure subscription. Anthropic, Google, and xAI are reached through their native APIs.

GPT-5-mini

GPT-5-mini is a multimodal language model released by OpenAI on 7 August 2025 (snapshot gpt-5-mini-2025-08-07). It is a smaller and cheaper variant of GPT-5, with a 400,000-token context

window, a 128,000-token maximum output, and a knowledge cutoff of May 2024. The model accepts text and image inputs and emits text. It supports structured outputs, function calling, and the broader OpenAI tool surface including web search, file search, and code execution. GPT-5-mini is the inference backbone of the locally re-installed ChemEAGLE pipeline and of several helper agents in the original multi-agent framework [20].

GPT-5.4

GPT-5.4 is OpenAI's frontier multimodal model released on 5 March 2026 (snapshot gpt-5.4-2026-03-17). It supports a standard 272,000-token context window, with extended input up to 1,048,576 tokens through a higher-priced billing tier, and a 128,000-token maximum output. The knowledge cutoff is 31 August 2025. GPT-5.4 accepts text and image inputs and emits text, supports configurable reasoning tokens, and exposes a broad tool surface including web search, file search, code execution, computer use, and the Model Context Protocol. It is used both as the planner and primary worker model in the multi-agent framework and, separately, as a single-agent extractor [21, 22].

GPT-5.5

GPT-5.5 is OpenAI's next-generation multimodal model, released on 23 April 2026. It is a full pretraining run with new data and a reworked, agent-oriented architecture; pretraining completed in March 2026, and the API documentation reports a knowledge cutoff of December 2025. The model has a 1,048,576-token context window, supports text and image inputs with text output, and exposes the same tool surface and reasoning-token configurability as GPT-5.4. GPT-5.5 was used as a single-agent extractor [23, 24].

Claude Opus 4.6

Claude Opus 4.6 is Anthropic's frontier multimodal model released on 5 February 2026 (snapshot claude-opus-4.6). It has a 1,048,576-token context window in beta and a 128,000-token maximum output, with a knowledge cutoff of May 2025. The model accepts text and image inputs and emits text, supports extended thinking, and exposes tool use through the messages.create endpoint with

interleaved text and tool use content blocks as described in Section 3.3.4. Claude Opus 4.6 was used as a single-agent extractor [25].

Claude Opus 4.7

Claude Opus 4.7 is the successor to Opus 4.6, released by Anthropic on 16 April 2026 (snapshot claude-opus-4-7). It retains the 1,048,576-token context window and 128,000-token maximum output of its predecessor at standard pricing, with no long-context premium. The headline upgrade is vision: the maximum supported image resolution is increased to roughly 3.75 megapixels, approximately 3.3 times higher than Opus 4.6. This is the qualitative driver behind the figure-reading gains on dense scope tables and condition surveys documented in Chapter 4. Opus 4.7 was used as a single-agent extractor and as the backbone of the Multi-Agent with Claude Opus 4.7 re-baseline reported in Chapter 4 [14, 26].

Gemini 3 Flash Preview

Gemini 3 Flash Preview is Google DeepMind’s high-throughput reasoning model, released in preview in early 2026. It has a 1,048,576-token context window and a 65,536-token maximum output. The model accepts text, image, audio, video, and PDF inputs and emits text, and supports configurable reasoning levels (minimal, low, medium, high), structured output, function calling, and automatic context caching. The model was accessed through the Google Generative AI SDK as a single-agent extractor, as the model behind the ChemEAGLE-Gemini fork documented in Chapter 4, and as the backbone of the ChemEAGLE Hybrid variant designed [27].

Gemini 3.1 Pro Preview

Gemini 3.1 Pro Preview is Google’s reasoning-tier model, released in preview on 19 February 2026. It has a 1,048,576-token context window and a 65,536-token maximum output, with a knowledge cutoff of January 2025. The model accepts text, image, audio, video, and PDF inputs and emits text, and supports function calling, structured output, and search-as-a-tool. Gemini 3.1 Pro Preview was used as a single-agent extractor [28-30].

Gemini 3.5 Flash

Gemini 3.5 Flash is the generally-available successor to Gemini 3 Flash Preview, released on 19 May 2026. It has a 1,048,576-token context window and a 65,536-token maximum output, with a knowledge cutoff of January 2026. The modalities and tool support match Gemini 3 Flash Preview, with dynamic thinking enabled by default so that additional reasoning compute is allocated on harder inputs. The model was added to the single-agent suite late in the experimental campaign to test whether a generally-available Flash-tier model could match the preview models on figure extraction [31].

Grok 4.3

Grok 4.3 is xAI's reasoning model, released on 6 May 2026. It has a 1,000,000-token context window and a knowledge cutoff of December 2025 and operates with an always-on reasoning mode. The model accepts text and image inputs and emits text, and supports function calling, file input, implicit caching, and web search. Grok 4.3 was used as a single-agent extractor and reached through the xAI API [32, 33].

3.4.2 Open-Source Models

Open-source VLMs are a complement to closed-source providers because their weights can be hosted locally, which preserves data privacy, enables fine-tuning, and replaces per-token charges with one-time GPU-hour costs. The trade-off is operational: local serving requires GPU infrastructure, model loading from network storage can take more than ten minutes for multi-billion-parameter checkpoints, and disk quotas must be managed because vLLM caches and per-request logs grow quickly. To swap open-source backends without changing the agent loop, vLLM was used as the serving engine throughout. Two open-weight models were evaluated in this thesis: Qwen3.6-27B as a full benchmark participant, and Qwen3-VL-8B-Instruct as an exploratory single-image probe.

Qwen3.6-27B

Qwen3.6-27B is the dense open-weight flagship of Alibaba's Qwen3.6 family, released under the Apache 2.0 licence in April 2026. It has 27 billion parameters and a 262,144-token context window,

and is natively multimodal: it accepts text, image, and video inputs and emits text [34, 35]. For the benchmark reported in Chapter 4 the model was accessed through OpenRouter rather than the local HPC cluster, so that the agent loop and the wire-format adapter remained identical to the proprietary single-agent variants. An HPC-served evaluation of the same checkpoint using vLLM is straightforward to set up and is recorded in Chapter 5 as future work.

Qwen3-VL-8B-Instruct

Qwen3-VL-8B-Instruct, the previous-generation 8-billion-parameter vision-language Qwen, was used in a single-image probe to characterise local-serving throughput on the HPC cluster before any benchmark-scale run [36]. The model was served with vLLM in float16 on a single GPU node. On the canonical reaction figure shown as Figure 3-8, throughput averaged 28 tokens per second, and a complete extraction took roughly 17 minutes. The breakdown is summarised in Table 3-5. A newer Qwen3-VL checkpoint was also tested but rejected for benchmark use because its chain-of-thought reasoning produced prompts long enough to crash the pipeline and generated logs and caches that quickly filled the HPC disk quota. Relocating the cache to a higher-capacity disk resolved the storage problem but did not address the latency, which is why the benchmark Qwen3.6-27B run was kept on OpenRouter rather than HPC.

Table 3-5: Breakdown of open-source run with model Qwen3-VL-8B-Instruct on Figure 3-15.

Process step	Time (min)	Explanation
vLLM server startup	11	
R-group synthesis agent	2	
LLM call		
Final aggregation LLM call	2	
Deep Learning inference	1	Moldet, molnext, coref
Rest	1	(planner agent, observers' agent, encoding, parsing)

3.4.3 Hardware

The open-source experiments were run on a single compute node of the joltik partition of the VSC HPC cluster. Each node combines two 16-core Intel Xeon Gold 6242 processors (Cascade Lake architecture, 2.8 GHz base clock), four NVIDIA V100 GPUs with 32 GB of HBM2 memory each, 256 GiB of system RAM, and 800 GB of local SSD scratch storage. Nodes are interconnected with double EDR InfiniBand and run RHEL 9. The cluster comprises ten such nodes.

Qwen3-VL-8B-Instruct was served on this hardware with vLLM in float16 precision under CUDA 12.1.1. The exploratory single-image probe occupied one V100 GPU, on which the 8-billion-parameter checkpoint fit comfortably at this precision, so inference latency was bound by sequence length rather than memory bandwidth. The throughput numbers reported in Table 3-5 should be reproducible on any comparable Volta-class or newer accelerator with at least 32 GB of GPU memory running the same software stack. The closed-source pipelines were not served on this hardware; they ran against hosted endpoints from the developer workstation.

3.5 Cost Calculations

Per-system run costs are reported in Chapter 4 as both per-image and per-benchmark sums, allowing a direct cost-quality comparison across the twelve systems evaluated in this thesis (nine single-agent variants and three multi-step pipelines). The cost figures are derived from each system's metadata sidecars, which record input tokens, output tokens, and tool calls for every image processed. For each closed-source model and for Qwen3.6-27B accessed through OpenRouter, the USD cost of a single inference call is computed as the sum of input tokens multiplied by the per-million input-token list price and output tokens multiplied by the per-million output-token list price, with both prices taken from the provider's public pricing page at the time of the experiments and stored in a single JSON configuration file. Reasoning tokens, where supported (GPT-5.4, GPT-5.5, Claude Opus 4.6 and 4.7, the Gemini 3 family, Grok 4.3, and Qwen3.6-27B in its hybrid-thinking mode), are billed as output tokens by every provider and are aggregated under the output total. Tool calls carry no extra cost beyond the additional tokens consumed in the request and response payloads, which are already

counted in the input and output totals. Wall-clock time is recorded per image and summed per benchmark run as a secondary efficiency metric, but it is not converted into a financial cost because the closed-source endpoints are billed per token rather than per second. Qwen3-VL-8B-Instruct ran exclusively on the joltik HPC partition (Section 3.4.3) as an exploratory single-image probe and is not part of the benchmark cost accounting; its equivalent cost would be the GPU-hour utilisation reported by the cluster scheduler. Provider list prices were snapshotted on 6 May 2026 and are stored alongside the run metadata.

3.6 Benchmark Evaluation Methodology

To systematically and quantitatively assess the performance of the pipelines developed in the previous chapter (ChemEAGLE [2], multi-agent framework, ChemEAGLE Hybrid and the eight single-agent variants) on chemical reaction extraction, a dedicated evaluation pipeline was developed. The pipeline compares the structured JSON output produced by a pipeline against a ground truth dataset, computing a range of metrics that capture both binary correctness and continuous structural similarity. The evaluation operates entirely at the reaction level: each extracted reaction is treated as an independent unit of prediction, and correctness is determined by comparing the chemical structures it contains against those in the reference annotation. The evaluation uses a similar approach as in the ChemEAGLE paper.

3.6.1 Evaluation Methodology

SMILES Canonicalization

A fundamental challenge in comparing chemical structures represented as SMILES strings is that the same molecule can be written in many different but equivalent ways. For example, ethanol can be written as CCO, OCC, or C(O)C, all of which are valid but textually distinct. To ensure that chemically identical structures are recognised as equal regardless of their surface representation, all SMILES strings are passed through RDKit’s [10] canonicalization procedure before any comparison is made.

In addition to strict canonicalization, the script offers a loose matching mode designed to handle a specific representational ambiguity that arises in practice: the use of a bare wildcard atom (*) as a stand-in for an implicit hydrogen. Some tools and annotators write [2*]N([1*])[H] while others write *N([1*])[2*], referring to the same abstract molecule. Under strict canonical comparison these would be treated as different, introducing false mismatches. The loose canonicaliser resolves this by converting all unlabelled wildcard atoms (those with no isotope index) to explicit hydrogen atoms and then removing all explicit hydrogens to make them implicit, so that both notations collapse to the same canonical form. Crucially, isotope-labelled wildcards such as [1*] and [2*], which encode R-group identity in generic reaction templates, are left untouched, preserving their chemical meaning.

Reference-free Metrics

A subset of metrics evaluates the extraction independently of the ground-truth benchmark. These reference-free metrics assess internal consistency and structural well-formedness rather than correctness against a reference. The SMILES validity rate is the fraction of SMILES strings in the extraction that RDKit can parse to a non-null Mol object, and it captures whether the pipeline produces chemically well-formed structures. The role enumeration compliance is the fraction of condition entries whose role field belongs to the permitted enumeration, which verifies that the structured output respects the schema's controlled vocabulary. The reaction count is simply the number of reactions produced by the pipeline; interpreted jointly with the ground-truth count, it flags systematic over- or under-extraction.

Reference-based Metrics

Reactant intersection over union (IoU) and Product IoU are micro-averaged Jaccard indices [37] over the set of canonical SMILES on the corresponding side of each reaction, Eq. (3-1). Both are computed with RDKit canonicalization (with chirality stripped, since some ground-truth files use abbreviated stereo notations). In the ideal case IoU is 1, as shown in Figure 3-20.

$$\text{Jaccard Index} = J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (3-1)$$

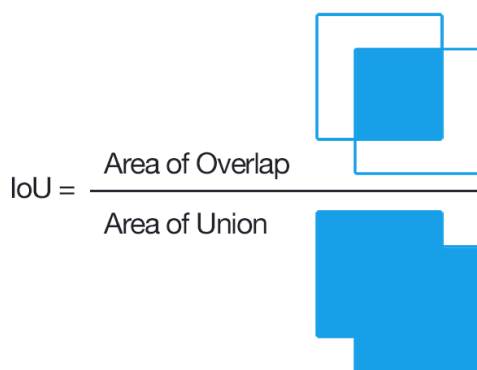


Figure 3-20: Visualisation of intersection over union (IoU) [38].

Two other metrics have been added to get more insight into the performance. Firstly, partial match F1 is a Hungarian-aligned per-reaction Jaccard on the union of reactant and product canonical SMILES. A pair counts when Jaccard index is more than 0.5. Secondly, constitution match F1 is a soft match with stereochemistry stripped. This identifies whether the model specifically struggles on stereochemistry or produces wrong skeletons in general.

Matching Criteria

Two binary matching criteria are defined, representing different levels of strictness: a soft and hard match. A soft match is awarded when the set of reactant SMILES and the set of product SMILES in a predicted reaction exactly equal the corresponding sets in a ground truth reaction, after canonicalization. This criterion tests whether the system correctly identified the core chemical transformation without requiring any information about the reaction conditions to be correct. A hard match is a stricter criterion that extends the soft match by additionally requiring that the set of condition also matches the ground truth.

Both criteria use set-based rather than list-based comparison, meaning that the order in which reactants, products, or conditions appear in the JSON does not matter. This reflects the fact that there is no intrinsic ordering among the molecules participating in a reaction.

Condition Matching

Condition coverage is computed as the recall of (role, text) pairs from the ground truth that are recovered by the prediction. Two variants are reported. Strict matching treats two condition entries as equal if and only if their role fields are equal and their text fields are equal after lowercasing and

whitespace collapsing. Lenient matching applies four deterministic normalisations before comparison: numeric–unit whitespace collapse ("82 %" and "82%" unified), parenthesis stripping ("acoh 20 mol%" and "acoh (20 mol%)" unified), role bucketing (catalyst and reagent collapse into a single reagent_bucket), and SMILES-based unification (entries with shared canonical SMILES match regardless of textual differences; for example, cf3co2h and trifluoroacetic acid match because they share OC(=O)C(F)(F)F).

The motivation for reporting both metrics is methodological. The strict number is what existing literature reports and is the more conservative measure. The lenient number is what one would obtain after manual disambiguation of cosmetic differences. On the present benchmark, applying the four normalisations to the strict numbers approximately doubles them, from a mean of 0.35 to a mean of 0.72 across the single-agent variants. This gap is interpreted as a measurement of formatting noise rather than chemistry correctness.

Bipartite Matching

A single image may contain multiple reactions, and the predicted set of reactions for an image will generally not be in correspondence with the ground truth set. Naively comparing reactions position by position would penalise the system for extracting reactions in a different order than the annotator, which is not meaningful. To address this, a maximum bipartite matching procedure is used to find the best possible alignment between predicted and ground truth reactions. The matching is formulated as a bipartite graph where one side contains the predicted reactions and the other contains the ground truth reactions. An edge is drawn between a predicted reaction and a ground truth reaction if and only if they satisfy the relevant matching criterion (soft or hard). The maximum matching in this graph, the largest set of edges such that no reaction appears more than once, gives the number of true positives (TP): reactions that were correctly predicted. This is computed using a depth-first augmenting path algorithm (Hopcroft–Karp style) [39], which finds the maximum cardinality matching efficiently. Separate maximum bipartite matchings are computed for soft and hard criteria, yielding soft true positives and hard true positives independently. The number of false positives is the number of predicted reactions not included in the matching, and the number of false negatives is the number of ground truth reactions not covered.

Precision, Recall, and F1

From the true positive counts obtained by bipartite matching, standard information retrieval metrics are computed for each image. Precision (P) measures the fraction of predicted reactions that are correct. A high precision indicates that pipeline avoids hallucinating reactions that are not present in the image. Recall (R) measures the fraction of ground truth reactions that were successfully extracted. A high recall indicates that the system does not miss reactions. F1 score is the harmonic mean of precision and recall, providing a single balanced summary: $2 \cdot P \cdot R / (P + R)$. It is zero if either precision or recall is zero and reaches one only when both are perfect.

These three metrics are computed separately for soft and hard matching, giving six per-image scores in total. Across the full benchmark, a micro-average is reported: the total soft (or hard) true positives across all images are divided by the total predicted and total ground truth reaction counts respectively, yielding micro-averaged precision, recall, and F1 that weight each individual reaction equally regardless of which image it came from.

3.7 References

1. Leong, S.X., et al., *MERmaid: Universal multimodal mining of chemical reactions from PDFs using vision-language models*. 2025.
2. Chen, Y., et al., *A multi-agent system enables versatile information extraction from the chemical literature*. arXiv [cs.AI], 2026.
3. Fan, V., et al., *OpenChemIE: An Information Extraction Toolkit for Chemistry Literature*. *Journal of Chemical Information and Modeling*, 2024. **64**(14): p. 5521–5534.
4. OpenAi, *GPT-5 System Card*. 2025, OpenAI.
5. *Introducing Sonnet 4.6*.
6. *CYF200127/ChemEAGLEModel at main*. 2026.
7. Lowe, D.M., et al., *Chemical Name to Structure: OPSIN, an Open Source Solution*. *Journal of Chemical Information and Modeling*, 2011. **51**(3): p. 739–753.
8. Kim, S., et al., *PubChem substance and compound databases*. *Nucleic acids research*, 2016. **44**(D1): p. D1202–D1213.
9. *International Chemical Identifier Generator (InChI key)*. 2022 2022/12/17 [cited 2026 25 nov]; Available from: <https://www.inchikey.info/international-chemical-identifier-generator-inchi-key/>.

10. RDKit, Open-Source Cheminformatics Software, RDKit, Editor. 2024, RDKit.
11. Guo, J., et al., Automated Chemical Reaction Extraction from Scientific Literature. *Journal of Chemical Information and Modeling*, 2022. **62**(9): p. 2035–2045.
12. Selector Group Chat – AutoGen.
13. Vision Models: LLMs with Image Input Capabilities | Artificial Analysis.
14. Introducing Claude Opus 4.7.
15. Prompting best practices. Claude API Docs.
16. OpenRouter. OpenRouter.
17. Lederbauer, M., et al., LeMat-Synth: a multi-modal toolbox to curate broad synthesis procedure databases from scientific literature. *arXiv [cs.DL]*, 2025.
18. Caron, M., et al. Emerging properties in self-supervised vision transformers. in *Proceedings of the IEEE/CVF international conference on computer vision*. 2021.
19. Dobbelaere, M.R., et al., Rxn-INSIGHT: fast chemical reaction analysis using bond-electron matrices. *Journal of Cheminformatics*, 2024. **16**(1): p. 37.
20. GPT-5 mini Model | OpenAI API. Available from: <https://developers.openai.com/api/docs/models/gpt-5-mini>.
21. Maak kennis met GPT-5.4. OpenAI, 2026.
22. GPT-5.4 Model | OpenAI API.
23. Maak kennis met GPT-5.5. OpenAI, 2026.
24. GPT-5.5 Model | OpenAI API.
25. Claude Opus 4.6.
26. What's new in Claude Opus 4.7. Claude API Docs.
27. Gemini 3 Flash | Gemini Enterprise Agent Platform. Google Cloud Documentation.
28. Gemini 3.1 Pro Preview | Gemini API. Google AI for Developers.
29. Gemini 3.1 Pro. Google DeepMind.
30. Gemini 3.1 Pro | Gemini Enterprise Agent Platform. Google Cloud Documentation.
31. Gemini 3.5 Flash. Google DeepMind.
32. Grok 4.3 | xAI Docs.
33. xAi, Grok 4 Model Card. 2025, xAI.
34. QwenLM/Qwen3.6. 2026, Qwen.
35. Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model. Alibaba Cloud Community.
36. qwen3-vl:8b-instruct.
37. Murphy, A.H., The Finley affair: A signal event in the history of forecast verification. *Weather Forecast.*, 1996. **11**(1): p. 3–20.
38. Rosebrock, A., Intersection over Union (IoU) for object detection, in *PyImageSearch*. 2016.
39. Hopcroft, J.E. and R.M. Karp, An $O(n^{5/2})$ Algorithm for Maximum Matchings in Bipartite Graphs. *SIAM J. Comput.*, 1973. **2**(4): p. 225–231.

Chapter 4.

Results and Discussion

4.1	INTRODUCTION	134
4.2	CHEMEAGLE	135
4.2.1	REPRODUCTION OUTCOME	135
4.2.2	VLM EVALUATION IN CHEMEAGLE	136
4.3	MULTI-AGENT FRAMEWORK	137
4.3.1	PHASE PROGRESSION	137
4.3.2	MULTI-AGENT VERSUS CHEMEAGLE ON THE SIXTEEN-IMAGE BENCHMARK	139
4.3.3	MULTI-AGENT WITH CLAUDE OPUS 4.7	140
4.3.4	FAILURE-MODE CASE STUDIES	142
4.3.5	MULTI-AGENT LIMITATIONS	144
4.4	SINGLE AGENT	144
4.4.1	TOOL-USE BEHAVIOUR ACROSS PROVIDERS	145
4.4.2	REDESIGNED PROMPT	146
4.4.3	OPUS 4.6 VERSUS OPUS 4.7 CASE STUDIES	146
4.4.4	INPUT-IMAGE RESOLUTION ABLATION	154
4.5	CROSS-SYSTEM EVALUATION	157
4.6	LE MATH-SYNTH	167
4.6.1	TEST CASE	167
4.7	DISCUSSION	168
4.7.1	MODEL CHOICE AS THE DOMINANT DETERMINANT OF EXTRACTION QUALITY	169
4.7.2	COMPARATIVE PERFORMANCE AND RESIDUAL ADVANTAGES OF THE MULTI-AGENT PIPELINE	170
4.8	REFERENCES	171

4.1 Introduction

Chapter 3 documented the four extraction pipelines and two post-processing toolkits that make up the experimental setup of this thesis, together with the benchmark images and ground-truth annotations against which every system is evaluated. The present chapter reports the results of running each pipeline against that benchmark and compares the systems on a single shared evaluation methodology. Following the Chapter 1 Outline, it asks how performance scales across the multimodal models behind the single-agent pipeline and how the original multi-agent framework compares with the ChemEAGLE Hybrid variant and the published ChemEAGLE baseline. It also probes the cost-quality trade-off, the per-provider tool-use discipline that shapes that trade-off, the specific perception events that drive the differences between Claude Opus 4.6 and Opus 4.7 on chemistry figures, and the feasibility of the same agentic methodology applied to procedure-text input.

The chapter is organised as follows. Section 4.2 reports the ChemEAGLE reproduction outcome and a frontier-model backbone variation. Section 4.3 presents the multi-agent framework’s phase-by-phase progression and head-to-head comparison against ChemEAGLE, including a model-swap experiment with Claude Opus 4.7 and a failure-mode analysis. Section 4.4 documents the single-agent pipeline observations on tool-use discipline across providers, a prompt redesign, an Opus 4.6 versus Opus 4.7 case-study set, and an open-weights Qwen3.6-27B variant. Section 4.5 brings every system together on a single ground truth and reports aggregate quality, cost, wall time, and per-system failure modes. Section 4.6 reports a feasibility test of the same agentic methodology adapted to procedure-text rather than figure input, using the LeMath-Synth LLM-as-judge framework on a single organic-chemistry paper. Section 4.7 summarises the chapter.

4.2 ChemEAGLE

4.2.1 Reproduction Outcome

ChemEAGLE completed 12 of 16 images on the 16-image benchmark. Four images failed due to either an `IndexError` or an `AttributeError` from their Python code.

The four failures are reproducible across cold restarts and clean checkout. They occur inside the upstream MolNexTR module on a step that converts the predicted atom-bond graph into a canonical SMILES string; ChemEAGLE’s wrapper does not catch the exception, so the entire image is dropped from the run. None of the failed images is structurally pathological: three are scope tables and one is a picture-style scheme. Three of them are extracted successfully by the multi-agent and single-agent pipelines reported later. Table 4-1 shows on the results of twelve images ChemEAGLE did process.

Table 4-1: Local ChemEAGLE metrics on the twelve images of the 16-image benchmark that the system did not crash on. The headline partial-F1 of 0.39 is approximately half the value reported in the paper (0.78 soft-F1).

Metric	Value
Mean product IoU	0.39
Mean reactant IoU	0.45
Mean soft F1	0.29
Mean partial F1	0.39
Mean constitution F1	0.29
Mean condition recall	0.55
Mean condition precision	0.85
N schema pass	12/16

The gap between the published 0.78 soft-F1 and the locally observed 0.39 partial-F1 is too large to attribute to evaluation-metric differences alone (soft-F1 is stricter than partial-F1 in most respects,

so the gap should run the other way if anything). It is also too large to attribute to benchmark composition. The remaining hypotheses are (i) a software-environment difference that affects orchestration but not core inference, (ii) a model-level difference (the paper’s "best" row is Claude 4.5 Sonnet, while the local run used GPT-5-mini per the upstream defaults), or (iii) a tuning-set / evaluation-set overlap not documented in the paper. Disentangling these is outside the scope of this thesis; the existence of the gap, on the other hand, is the motivating empirical observation for developing the multi-agent framework and single-agent pipeline.

4.2.2 VLM Evaluation in ChemEAGLE

To probe whether a stronger frontier VLM would lift ChemEAGLE’s extraction quality, the upstream pipeline was forked, every Azure-OpenAI call was rewired to Google’s gemini-3-flash-preview via the OpenAI-compatible shim, and the fork was run against the same 16-image benchmark used for the local GPT-backed reference. The expectation was that Gemini, marketed as a strong multimodal reasoning model [1], would either match or meaningfully exceed GPT on this task. In practice the two pipelines came out essentially indistinguishable: identical schema-pass count (12/16), Gemini marginally ahead on reactant IoU (0.51 versus 0.45), constitution F1 (0.34 versus 0.29), and condition recall (0.57 versus 0.55), while GPT held a clear lead on condition precision (0.85 versus 0.74); Gemini tends to over-emit plausible-looking conditions that are not in gold. The headline F1 metrics (soft 0.28 versus 0.29, partial 0.38 versus 0.39) are within rounding noise of each other. The takeaway is that swapping in a more capable LLM does not translate into a proportionally better extraction here; the bottleneck is no longer raw VLM quality but the surrounding scaffold (the OCR-derived molecular graph, the rigid tool-call orchestration, and the brittle JSON-merge step downstream), which constrain what any backbone model can contribute.

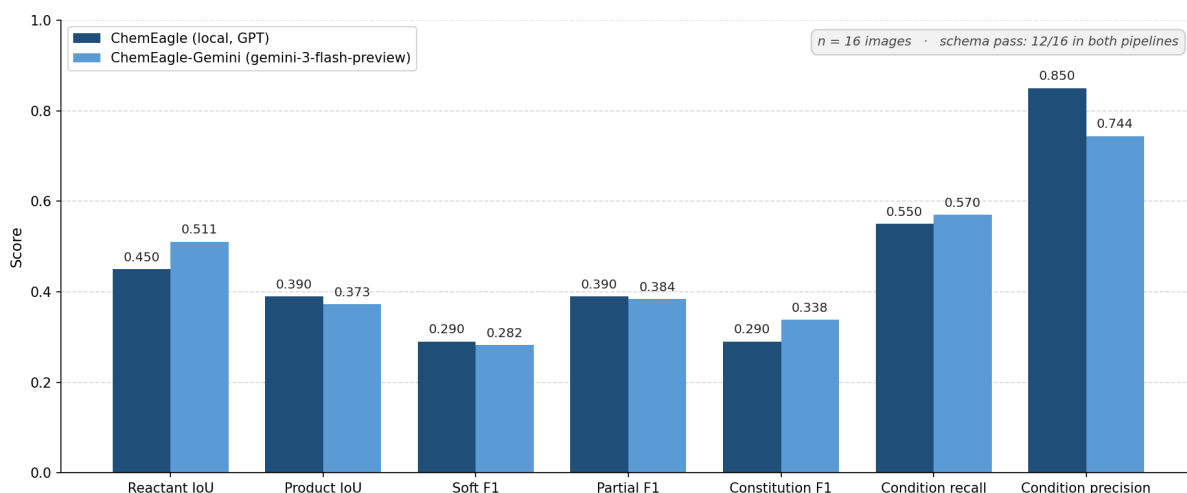


Figure 4-1: ChemEAGLE model change on the 16-image benchmark. Aggregate per-image means for ChemEAGLE's reference local pipeline (dark blue, GPT) and for the fork that routes every chat call to gemini-3-flash-preview via the OpenAI-compatible setup (light blue). Both pipelines pass schema validation on the same 12 of 16 images. Across the seven aggregate metrics the two backbones are within rounding noise of each other on the headline F1 scores (soft, partial, constitution); Gemini holds a small edge on reactant IoU, constitution F1, and condition recall, while GPT retains a clear advantage on condition precision (0.85 versus 0.74), reflecting Gemini's tendency to over-emit plausible but non-gold reaction conditions. Bar labels are mean values across the 16 images.

4.3 Multi-Agent Framework

4.3.1 Phase Progression

The cleanest demonstration of the full deterministic R-group pipeline is the canonical case `acs.joc.2c00176 example 2` (gold: 8 reactions) across four configurations illustrated in Figure 4-2.

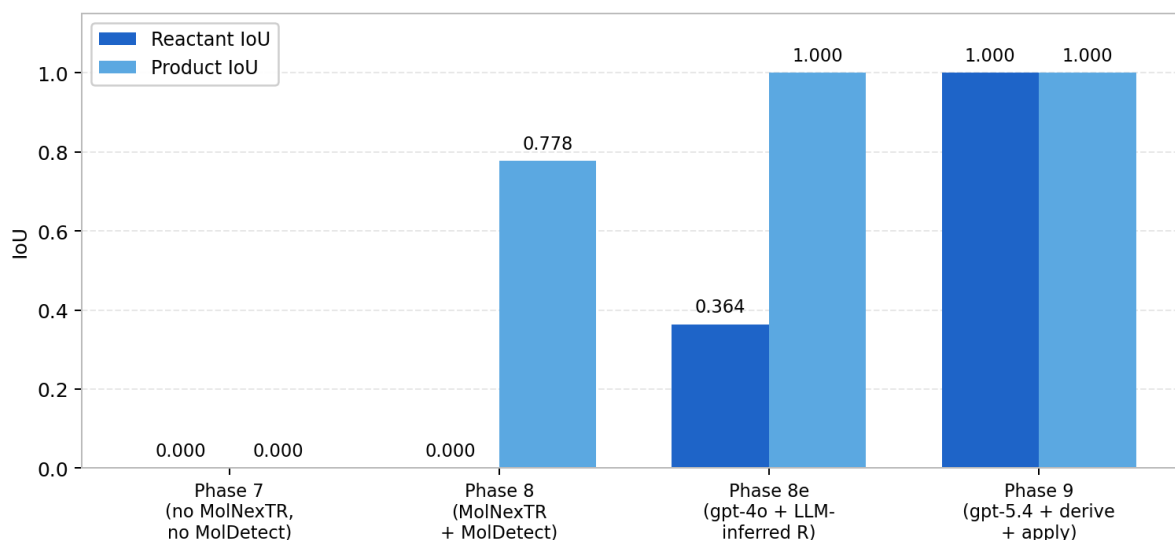


Figure 4-2: Canonical R-group case across phases. Product intersection over union (IoU) stabilises at 1.0 once MolNexTR is used per-bbox (Phase 8). Reactant IoU is the harder problem and requires the deterministic R-GroupDecompose and molzip chain (Phase 9).

Figure 4-3 shows the aggregate results across a nine-image benchmark.

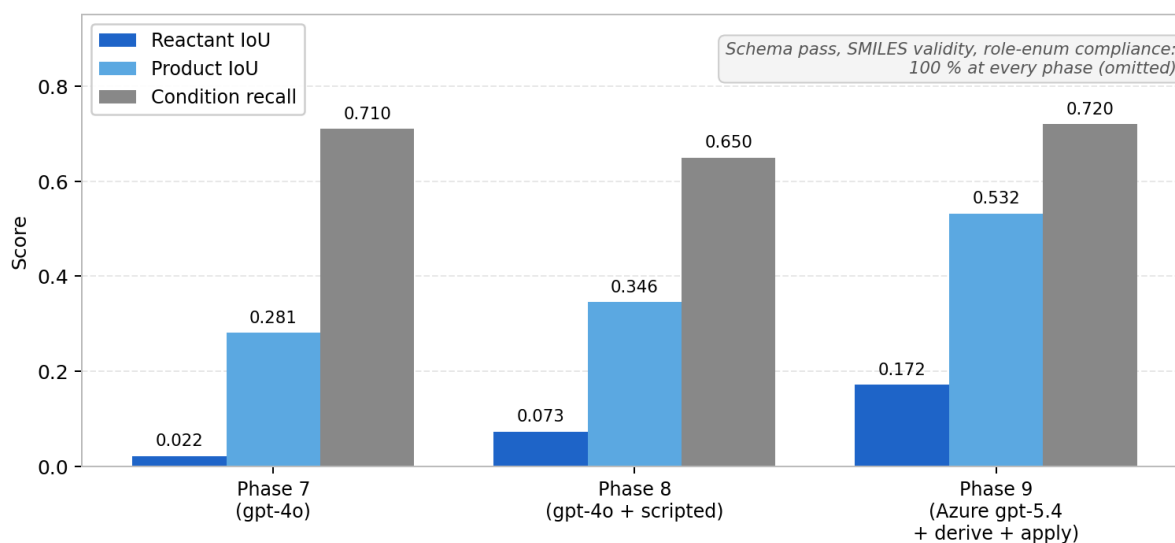


Figure 4-3: Aggregate metrics across the nine-image benchmark.

The pipeline's progression from baseline to final form produced three quantitatively distinct gains, each tied to a specific architectural intervention. Schema validity rose from 0% to 100% as a direct consequence of Phases 1, 3, and 4, where pydantic-backed validation, a retry loop, and verifier coordination jointly constrained the model's output space, reframing the VLM as a JSON-producer rather than a chat partner. The mean product IoU improved from 0.28 to 0.53 (90% relative gain)

once Phase 8 replaced VLM with a dedicated chemistry stack of MolDetect bbox detection, per-bbox MolNexTR, and a scripted molecular recognition routine, showing that domain-specific deep-learning models at the agent's tool layer outperform general-purpose VLM on chemistry-specific tasks where such models exist. The reactant IoU on the canonical R-group case rose from 0 to a perfect 1.00 with Phase 9's RDKit RGroupDecompose and molzip chain, confirming that rule-followable structural transformations are better handled by deterministic algorithms than by VLM reasoning, even with the strongest available model. Together, these results suggest a consistent architectural posture: constrain the output, delegate perception to specialised models, and reserve symbolic manipulation for deterministic code.

4.3.2 Multi-Agent versus ChemEAGLE on the Sixteen-Image Benchmark

A larger benchmark was built from the ChemEAGLE benchmark, partly to break the dependence on the original nine-image set and partly to test images flagged by the principal investigator as representative of the journal-paper distribution. The benchmark is referenced as the 16-image benchmark.

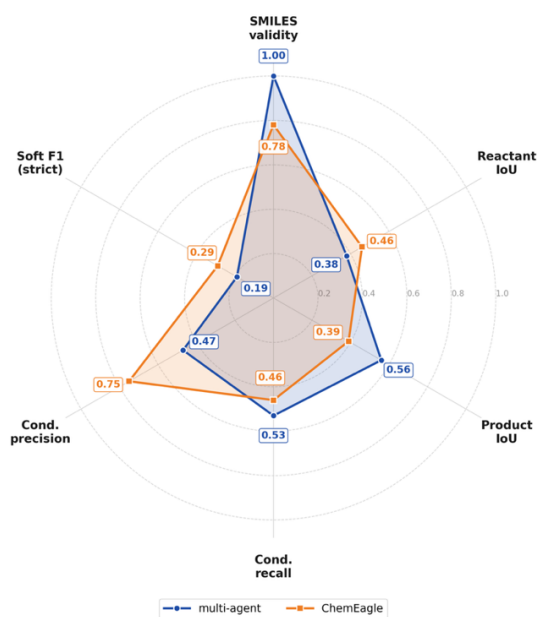


Figure 4-4: Multi-agent versus locally run ChemEAGLE on the 16-image benchmark.

The same sixteen images and gold files were run through the locally installed ChemEAGLE reference and the final multi-agent comparator. Both systems passed schema validation on all sixteen images.

ChemEAGLE outperforms on reactant IoU (+0.08), soft F1 (+0.10) and condition precision (+0.28). The multi-agent pipeline outperforms on product IoU (+0.17), condition recall (+0.07), and SMILES validity (+0.22). The product-side gap is driven by the deterministic ScriptedMolRecAgent which enumerates one MolNexTR call per detected bbox, so for substrate-scope tables every variant product is recovered; ChemEAGLE's loop tends to truncate at the first few variants. Every multi-agent SMILES roundtrips through RDKit canonicalization, while ChemEAGLE emits ~22 % of SMILES that do not parse.

4.3.3 Multi-Agent with Claude Opus 4.7

To test whether the multi-agent framework's partial-F1 gap to the strongest single-agent variants is dominated by the GPT-5.4 vision encoder rather than by the orchestration design, the framework was re-run on the 16-image benchmark with Claude Opus 4.7 substituted for GPT-5.4 across every agent. All other wiring is identical to the GPT-5.4 baseline reported above: MolNexTR is on, the seven-agent selector team is unchanged, and the verifier is capped at two critiques. The text-only agents (data_structure and verifier) also run on Opus 4.7.

The Opus 4.7 multi-agent passes schema validation on all sixteen images and emits 100% RDKit-valid SMILES with full role-enumeration compliance. Aggregate metrics are summarised in the table below alongside the GPT-5.4 multi-agent baseline and the single-agent Opus 4.7 comparator.

Table 4-2: Multi-agent model swap to Claude Opus 4.7. Aggregate metrics on the 16-image benchmark for the GPT-5.4 multi-agent baseline, the Opus 4.7 multi-agent re-run, and the single-agent Opus 4.7 comparator. All systems use the same gold annotations and the same scoring code

Metric	GPT-5.4 multi-agent	Opus 4.7 multi-agent	Opus 4.7 single-agent
Product IoU	0.55	0.80	0.76
Reactant IoU	0.39	0.67	0.68
Soft F1	0.22	0.64	0.60

Partial F1	0.37	0.78	0.85
Constituent F1	0.22	0.68	0.71
Cond. Recall	0.77	0.68	0.78
Cond. Precision	0.71	0.49	0.69
Schema valid (of 16)	15	16	16
Wall time (sum, s)	5,486	3,799	662
Tokens (in + out)	3.09 M	2.94 M	0.33 M
LLM calls	345	274	15
Cost (USD)	\$6.42	\$11.73	\$3.52

Three observations follow. First, the Opus 4.7 multi-agent improves on the GPT-5.4 multi-agent across every structural metric: Partial F1 rises from 0.37 to 0.78, Product IoU from 0.55 to 0.80, Soft F1 from 0.22 to 0.64, and Constituent F1 from 0.22 to 0.68 . These data indicate that the multi-agent framework’s published shortfall against the strongest single-agent variants is driven primarily by the GPT-5.4 vision encoder rather than by the orchestration design.

Second, the Opus 4.7 multi-agent does not close the gap with the single agent paired with the same model. The single-agent Opus 4.7 still leads on Partial F1 (0.85 versus 0.78), Constituent F1 (0.71 versus 0.68), and Reactant IoU (0.68 versus 0.67). The multi-agent outperforms on Product IoU (0.80 versus 0.76) by a small margin, but the margin does not propagate to the other structural metrics.

Third, the orchestration overhead remains substantial. The Opus 4.7 multi-agent consumes 2.94 million tokens against 0.33 million for the single-agent call (a roughly nine-fold token amplification), runs for 3,799 seconds against 662 seconds (approximately 5.7 times longer in wall time), and costs \$11.73 against \$3.52 at the same OpenRouter pricing (3.3 times more). Wall time per image is roughly 237 seconds for the multi-agent against 41 seconds for the single-agent call.

One regression accompanies the model upgrade: condition extraction degrades. Condition recall drops from 0.77 in the GPT-5.4 multi-agent to 0.68 in the Opus 4.7 variant, and condition precision drops from 0.71 to 0.49. Inspection of the per-image runs shows the multi-agent attaching separate role-loading entries to every catalyst row even when the gold bundles loading into the catalyst entry. This is an artefact of the Data Structure agent prompt’s enumeration encouragement interacting

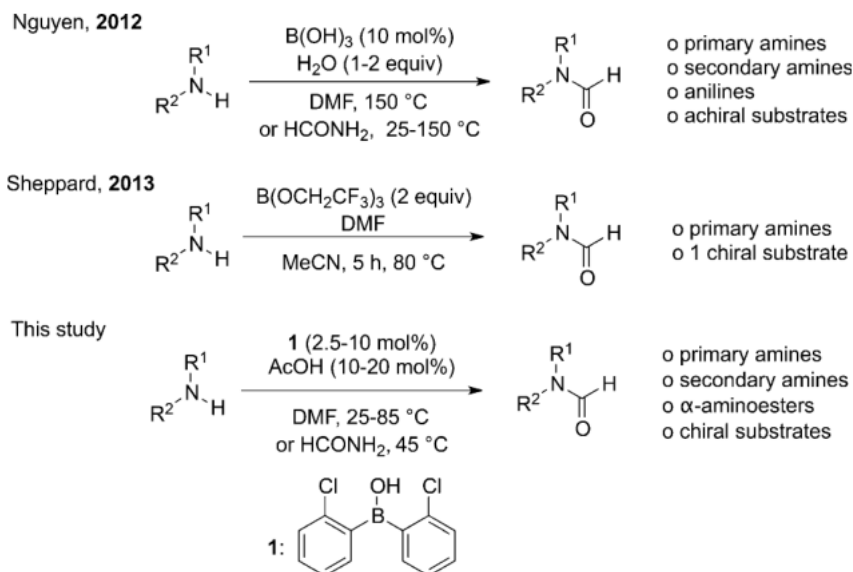
with Opus 4.7’s verbosity, not a property of the orchestration itself. The combined picture is that the multi-agent framework’s orchestration does not pay back its cost on this task even under the most favourable model substitution.

4.3.4 Failure-mode Case Studies

Aggregate scores hide what actually breaks. Four images, each representative of a distinct failure mode, illustrate two independent weaknesses that the current multi-agent pipeline cannot solve. The same architectural choices that produce the headline gains also produce these failures, and they are not orthogonal bugs but the cost side of those design decisions.

Case A: Page Crops with Text-Fragment Hallucinations

Page-cropped scans that contain a mix of structures, captions, equations, and reference text. Figure 4-5 illustrates an image where this failure occurs. Both images score reactant IoU and product IoU are 0.00 despite passing schema validation cleanly. MolDetect was trained on chemistry-figure crops and treats high-contrast text fragments (reference numbers, footnote markers, in-line abbreviations) as molecule bboxes; MolNexTR, called per bbox, then fabricates plausible-looking but completely wrong SMILES from the cropped pixels. Every fabricated SMILES is RDKit-valid, so the verifier loop never fires; the JSON looks reasonable end-to-end. This failure mode is the most difficult to overcome: a downstream consumer reading the output cannot tell that the chemistry deep-learning stack misrouted the figure. No prompt change addresses this; the failure is upstream of every agent.



Scheme 1. Formylating transamidations promoted by boron-based catalysts.

Figure 4-5: Image from the 16-image benchmark on which the multi-agent pipeline hallucinated due to wrong page crops with highly dense text-fragment (CEJ_20216 page 1).

Case B: Dense Substrate-Scope Grid with R-Fragment Leakage.

In Figure 4-6, product detection performs adequately, with a product IoU of 0.50 and full reaction recall, yet reactant IoU remains markedly low at 0.04. Inspection reveals that the agent invoked `derive_substituents` but deposited the resulting fragments (e.g. `[*:1]CC(C)C`) directly into the output JSON without subsequently calling `apply_substituents` to reconstitute them into the reactant template. The `[*:N]` Pydantic validator introduced after this run converts this omission into an explicit schema error but does not address the underlying cause: variance in the agent's adherence to multi-step tool-chaining instructions. As a consequence, repeated inference on the same input is non-deterministic.

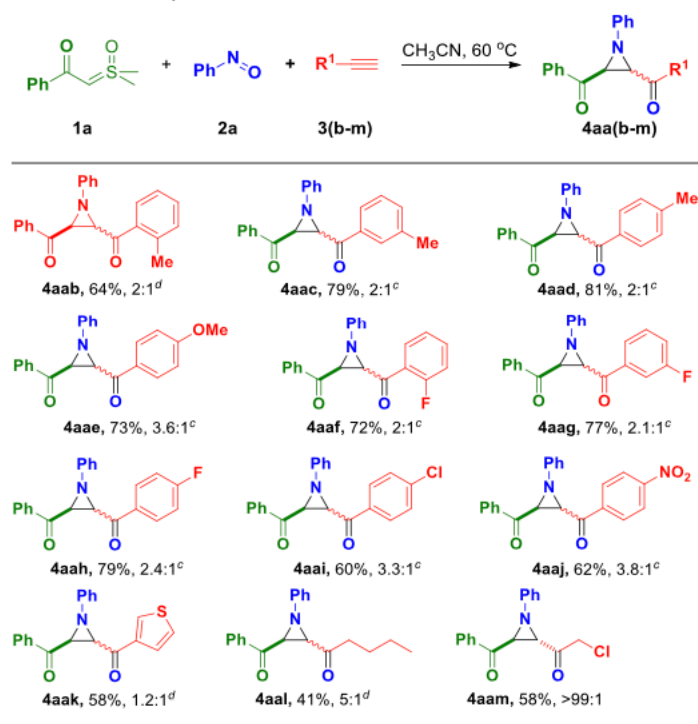
Scheme 4. Scope of Various Terminal Alkynes^{a,b}

Figure 4-6: Image on which the multi-agent pipeline fails due to a dense substrate-scope grid with R-fragment leakage.

4.3.5 Multi-Agent Limitations

To conclude, cases A and B illuminate complementary failure modes that are structural rather than incidental to the multi-agent framework's design. Case A demonstrates the cost of unconditional reliance on the chemistry deep-learning toolchain, while Case B exposes the limitations inherent in delegating deterministic tool-chaining to a language model. Neither failure admits remediation within the current architecture without introducing compensatory trade-offs elsewhere.

4.4 Single Agent

All the single-agent variants share the same agent loop, the same FigureExtraction schema, the same two RDKit-backed tools, and the same system prompt, differing only in the multimodal model behind the SDK adapter. Per-model aggregate quality and cost numbers are reported in the Cross-System Evaluation section below. This section documents four cross-cutting observations made

during those runs: how tool-use discipline differs across providers, the impact of a prompt redesign performed mid-evaluation on Claude Opus 4.7, the differences between two Claude Opus generations (4.6 and 4.7) traced through four image-level case studies, and the cost-quality profile of an open-weights variant accessed via OpenRouter.

4.4.1 Tool-Use Behaviour across Providers

Part of the apparent performance gap between models on identical agent infrastructure is a gap in tool-use discipline: how aggressively each model batches independent tool calls within a single round-trip, and how much non-actionable reasoning text it emits between them.

Claude Opus 4.6 and 4.7 average about nine calls per image over four to six round-trips. GPT-5.4 and GPT-5.5 batch comparably, with smoke tests recording up to twenty-two calls in a single round-trip. The three Gemini variants (3 Flash Preview, 3.1 Pro Preview, 3.5 Flash) share the same parallel-function-call profile and land in the same range, with representative runs dispatching 6.5 to 19 SMILES-validation calls per round-trip on the densest figures. Qwen3.6-27B batches at a similar rate on regular content (about 11.7 calls per image) but falls into a non-converging SMILES-validation retry loop on dense scope and optimisation tables, emitting chemist shorthand (Me, Et, Bn, OAc) that fails RDKit parsing and exhausting the step ceiling on five of the sixteen images. Grok 4.3 is the outlier in the other direction, issuing only one to three calls per round-trip and interleaving substantial reasoning text, which inflates both its round-trip count and its per-round wall time.

A per-provider step ceiling follows directly. A default of twelve round-trips suffices for the Anthropic, OpenAI, and Gemini families; Grok and Qwen3.6-27B require one hundred and twenty-eight, in Grok's case to absorb sparser parallelism and verbose intermediate reasoning and in Qwen's case to absorb the shorthand-SMILES retry tails. The elevated ceiling reflects differences in training-time emphasis on parallel tool dispatch, not a deficiency in the quality of individual tool calls.

Tool-use discipline and extraction quality are largely orthogonal on this benchmark: Gemini 3 Flash Preview attains the highest Product IoU while batching heavily, GPT-5.4 records the lowest Product IoU despite excellent batching and Grok 4.3 scores well despite sparse batching and high verbosity.

The dominant determinant of structural extraction quality is the multimodal model's vision encoder, and this is not simply correlated with general model capability.

4.4.2 Redesigned Prompt

The redesigned prompt was evaluated against the original on the sixteen-image benchmark using Claude Opus 4.7, with all other parameters held fixed. Under the IoU- and F1-based evaluator that drives the heatmap reported in the Cross-system Evaluation section below, the picture inverted (reactant IoU $0.68 \rightarrow 0.65$, soft-match F1 $0.60 \rightarrow 0.58$, partial F1 $0.85 \rightarrow 0.84$, while product IoU rose $0.76 \rightarrow 0.78$). The largest figure-level effect was on CEJ_2016 page001 (see Figure 4-5), where the original prompt failed to detect any of the three drawn reactions and the revised prompt recovered all three; this is consistent with the new prompt's explicit instruction that generic scope headers must be emitted as separate reactions. However, the aggregate per-metric deltas of one to three percentage points lie within the range that single-run language-model stochasticity can plausibly produce on a sample of sixteen figures, and repeat runs were not performed to estimate that run-to-run variance.

To conclude, the redesigned prompt cannot be shown to produce a statistically distinguishable change under the metrics used in this work, despite a clear improvement on at least one previously failing figure. Establishing whether the prompt-design choices documented here yield a real aggregate effect would require multiple independent runs of each prompt version to separate prompt-effect from sampling noise or a larger benchmark.

4.4.3 Opus 4.6 versus Opus 4.7 Case Studies

The aggregate cross-system evaluation reports a sizeable gap between the Claude Opus 4.7 and Claude Opus 4.6 single-agent variants on every structural metric. Because the two variants share identical agent infrastructure and differ only in the multimodal model behind the SDK adapter, the source of that gap is informative about where extraction quality is actually decided in this pipeline. This subsection walks through four images on which the two models diverge in qualitatively different ways: an optimization table where the gap is pure efficiency, a substrate-scope figure where it is a

fivefold quality difference, a catalyst gallery where the metric ties but the chemistry does not, and a conditions table where a single perception event drives the score from 3 of 3 down to 0 of 3.

Case Study 1: Optimization Table

The first image is Table 1 of an ACS Catalysis 2020 paper on Boc-amino-acid amidation, with a generic reaction at the top and a 13-row optimization grid varying substrate, catalyst, solvent, and temperature for a yield of product 3b. Both models return byte-identical extractions on this figure, so accuracy is not the variable: what differs is the route to the answer. Opus 4.7 emits a single 286-character thinking summary, fires five `validate_smiles` calls in parallel for the five distinct building blocks (Boc-Gly-OH, Boc-Phe-OH, benzylamine, Boc-Gly-NHBn, and Boc-Phe-NHBn) and submits on the next turn for six tool calls across two steps. Opus 4.6 produces over 3 000 characters of thinking re-examining a single ambiguity (whether reagent 2 is benzylamine, phenethylamine, or cinnamylamine), eventually settles on the correct interpretation, and then runs the loop through twelve tool calls across six steps. Because each step re-sends the prompt and image to the model, the behavioural difference compounds into roughly a 2x input-token gap (32 k versus 14 k), a 1.7x output-token gap, and a 2.4x wall-clock speed up for Opus 4.7 (47 s versus 114 s) for byte-identical output and the same score. This image is the cleanest available illustration of the same-quality-less-spend change documented in the Opus 4.7 release notes [2], where adaptive thinking is more concise by default and fewer tool calls are issued because the model reasons over the figure once before validating.

Table 1. Optimization of the Reaction Conditions^a

entry	substrate	catalyst	solvent	temperature (°C)	3b ^b (%)
1	1a	A	toluene	80	99
2	1b	A	toluene	80	0
3	1b	B	toluene	80	0
4	1b	C	toluene	80	82
5	1b	C	toluene	65	31
6	1b	D	toluene	65	96 ^c
7	1b	D	PhF	65	78
8	1b	D	(CH ₂ Cl) ₂	65	68
9	1b	D	CPME	65	85
10	1b	D	DMF	65	0
11	1b	D	toluene	50	77
12	1b	none	toluene	65	0
13 ^d	1b	D	toluene	65	0

^aReaction conditions: **1** (0.20 mmol, 1.0 equiv), **2** (0.20 mmol, 1.0 equiv), catalyst (5 mol%), 5 Å MS, solvent, temp., 12 h.

Figure 4-7: Table 1 of an ACS Catalysis 2020 paper on Boc-amino-acid amidation, with a generic reaction at the top and a 13-row optimization grid varying substrate, catalyst, solvent, and temperature for the yield of product 3b. Opus 4.6 and Opus 4.7 return byte-identical extractions on this figure; the gap reduces to a 2.4x wall-clock and 2x input-token efficiency advantage for Opus 4.7 (six tool calls in two steps versus twelve in six).

Case Study 2: Substrate-Scope Table

The second image is Figure 2 of a 2017 Nature Communications paper on direct amidation catalysed by the boronic-acid scaffold DATB 8a. It shows a generic reaction at the top and twelve worked examples 17 to 28 spanning a diverse scope of aryl, alkyl, alpha,alpha-disubstituted, adamantyl, spirocyclic, indolyl, quinolinyl, pyridyl, and thiophenyl substrates. The structures vary substantially row to row and are drawn at small scale within a 6-by-2 product grid. On this figure the gap between the two models is no longer one of cost alone: Opus 4.7 scores 10 of 13 against the ground truth while Opus 4.6 scores 2 of 13, a fivefold quality difference accompanied by a 4.3x wall-clock gap and a 6x input-token gap. Both models correctly read the two simplest products (18, 1-phenylcyclopentanecarboxamide with phenethylamine, and 19, 1-phenylcyclopropanecarboxamide with the TBS-protected aminopropanol). The remaining eleven Opus 4.6 outputs show a consistent pattern: the model returns a chemically valid SMILES of the correct class but reads the details wrong. Ring-system substitutions account for the largest cluster: the bicyclic disubstituted skeleton of 22 is

replaced by a 4-fluorobenzamide (shown in Figure 4-9), 2-methylquinoline collapses to benzene in 26, and the 3-methylthiophene of 28 is fused into a benzothiophene. Substituent count and position errors account for the rest: a 2,4,6-trimethylbenzamide loses two methyls in 24, a 3,5-dichlorobenzamide becomes 3,4-dichloro in 25, an alpha,alpha-diphenyl-alpha-methylacetamide drops its alpha-methyl in 20, and a 4-methoxyanilide gains a spurious CH₂ spacer in 17.

The Opus 4.6 thinking trace on this image runs to over 10 000 characters and documents its own uncertainty in real time, yet the model never returns to the image at higher fidelity and commits to its initial reading. Because the SMILES validation tool only checks that a SMILES is parseable, all thirty-nine SMILES from Opus 4.6 validate successfully despite eleven of thirteen being structurally wrong. The failure mode is consistent with the high-resolution image support change in the Opus 4.7 release notes: the maximum accepted image resolution rose from 1.15 MP on Opus 4.6 to 3.75 MP on Opus 4.7, and dense small-molecule grids are precisely the input class where down sampling to the lower cap obliterates the features (5- versus 6-membered rings, ortho versus meta versus para substituents, fused 5-6 versus 6-6 bicycles, isolated methyls) that distinguish the rows. Extra reasoning over a degraded image representation does not recover detail lost at the input stage, which points to a perception-stage rather than reasoning-stage bottleneck.

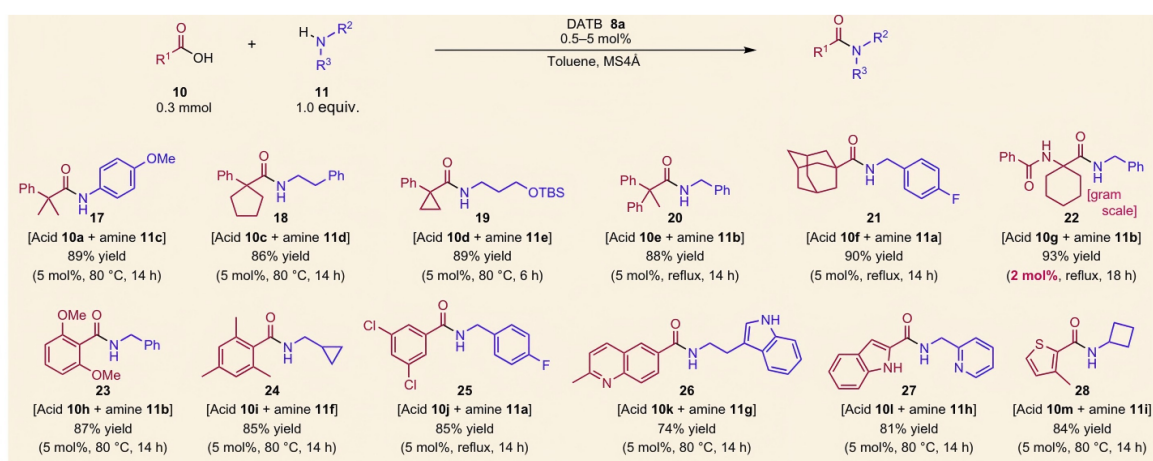


Figure 4-8: Case Study 2 input image. Figure 2 of a 2017 Nature Communications paper on direct amidation catalysed by the boronic-acid scaffold DATB **8a**, showing a generic reaction and twelve worked examples 17 to 28 across a diverse substrate scope drawn at small scale in a 6-by-2 product grid. Opus 4.7 scores 10 of 13 against the ground truth (effectively 12 to 13 of 13 once two ground-truth artefacts are corrected) while Opus 4.6 scores 2 of 13, isolating a perception-stage rather than reasoning-stage bottleneck.

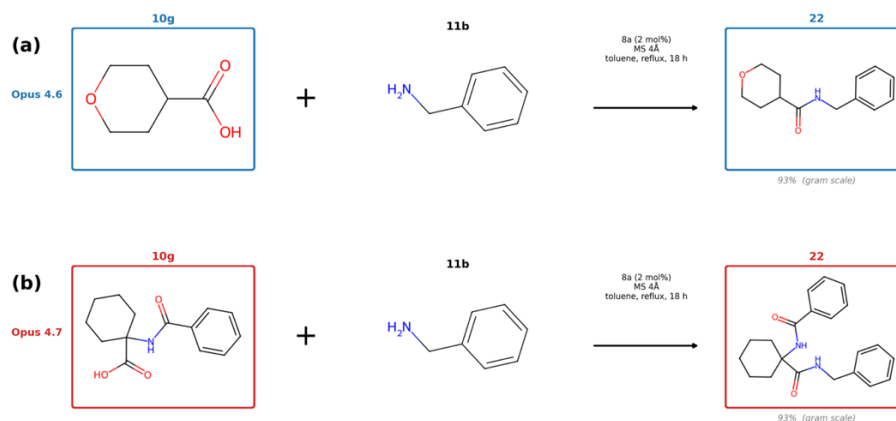


Figure 4-9: Extraction results of compound 22 by Opus 4.6 and Opus 4.7. Both models extract reactant 11B and the conditions correctly. Product 22 has been incorrectly extracted by Opus 4.6: the bicyclic disubstituted skeleton of 22 is replaced by a 4-fluorobenzamide. The extraction of reactant 10g is directly affected by this mistake.

Case Study 3: Boronic-Acid Catalyst Gallery

The third image is Figure 1 of a 2015 Green Chemistry review on boronic-acid-catalysed amidation. It is not a reaction figure: it contains one generic amidation scheme at the top and a gallery of six known catalysts (3,4,5-trifluorophenyl-B(OH)₂; the Hall MIBA-type ortho-(diisopropylaminomethyl)-meta-CF₃ phenylboronic acid; an N-methylpyridinium boronic acid; ortho-nitrophenyl-B(OH)₂; ortho-iodophenyl-B(OH)₂; and a 2-iodo-4-methoxyphenyl-B(OH)₂) grouped by temperature regime. On the metric defined in Section 4.2 the score is a perfect 7 of 7 tie. Inspection of the predictions reveals that the tie is an artefact of how the ground truth and the eval are constructed rather than evidence that the models extracted the same content. The ground truth encodes seven identical templated reactions with reactants [1*]C(=O)O and product [1*]C(=O)N[2*] and contains no chemical information about the catalysts. Both models produced richer extractions than the ground truth requires, populating the reagents field with a specific catalyst SMILES per row, but the eval scores only reactants and products, so the catalyst SMILES (which is the substantive content of the figure) does not affect the score. Direct comparison of the catalyst SMILES against what is drawn shows Opus 4.7 returning all six catalysts correctly, while Opus 4.6 makes two perception errors that are invisible to the metric: catalyst 2 is read as a benzamide rather than a benzylamine (the CH₂ linker between the aryl ring and the diisopropylamine is replaced by a C(=O) and one isopropyl group is dropped), and catalyst 3 has its boronic acid placed at the 2-

position rather than the 4-position of the N-methylpyridinium ring. If the eval scored reagents in addition to reactants and products, the result on this image would be 6 of 6 catalyst structures for Opus 4.7 against 4 of 6 for Opus 4.6, the same vision-quality gap that dominated Case Study 2, hidden here by metric coverage. The reasoning trace on this image also surfaces a behaviour not visible in the previous cases. On catalyst 2 the Opus 4.6 trace describes the structure correctly at first ("the boronic acid group is clearly at the top, but I need to carefully trace where the carbonyl-amide group and the trifluoromethyl are positioned") and then justifies the inserted carbonyl group by appeal to prior knowledge ("looking at the literature on Hall's MIBA catalyst, I'm confident the structure has a carbonyl group connecting the ring to the amide"). The model is not extracting what is drawn; it is replacing what is drawn with a chemically related catalyst it knows from training data. The Opus 4.7 trace on the same image describes catalysts feature by feature without invoking named compounds and produces the correct CH₂-linked SMILES. This contrast is consistent with the more-literal-instruction-following behaviour documented in the Opus 4.7 release notes, generalized from instructions to image content.

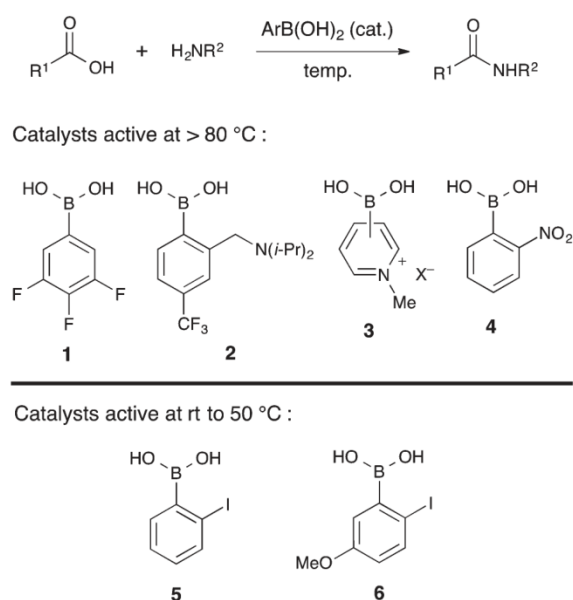


Fig. 1 Known boronic acid catalysts for direct amidation reactions.

Figure 4-10: Case Study 3 input image. Figure 1 of a 2015 Green Chemistry review on boronic-acid-catalysed amidation, comprising a generic amidation scheme above a gallery of six known catalysts grouped by temperature regime. The reactant/product metric ties at 7 of 7 because the ground truth

encodes only templated reactions; direct comparison of the catalyst SMILES shows Opus 4.7 returning all six correctly while Opus 4.6 makes two perception errors invisible to the metric.

Case Study 4: Comparative-Conditions Study

The fourth image is Table 2 of the same Nature Communications paper as Case Study 2. It is a conditions comparison for a single substrate pair (N-benzoyl-L-valine 10p with L-leucinamide 11r to give dipeptide 39) under three different conditions: DATB 8a at 5 mol% in toluene at 50 °C (75% yield, stereo chemically pure); EDCI at 1.0 equivalent in CH₂Cl₂ at room temperature (89% yield, diastereomeric mixture); and auxiliary catalysts 14 or 15 at 10 mol% (no conversion). The agent's task is to encode the three condition rows as three reactions sharing the same substrates and product but differing in conditions and notes. This is the largest quality gap in the case-study set: Opus 4.7 returns 3 of 3 in 33 s, Opus 4.6 returns 0 of 3 in 346 s. Both models read leucinamide 11r correctly. The single difference is in substrate 10p: where the figure draws the valine alpha-isopropyl group, Opus 4.6 reads only a single methyl and encodes the substrate as N-benzoyl-alanine. Because all three condition rows share the same product, the substrate error propagates to every row and a one-carbon misread cascades into 0 of 3 matched reactions, 0 of 3 matched products, and a product precision of zero. The inverse of Case Study 1 holds: one perception success on a shared structure earlier produced twelve correct rows; here one perception failure produces three wrong rows. Scope tables and conditions tables both amplify per-feature perception accuracy into per-row scores. The Opus 4.6 thinking trace on this image runs to 14 765 characters, the largest in the set, and is dominated by sequential never-resolved hypotheses about substrate 10p: hippuric acid, N-formylphenylalanine, an oxazolidinone, a Boc- or Cbz-protected carbamate, then a commitment to N-Bz-Ala. The crucial line in the trace ("I'm going in circles trying to parse this structure, let me just settle on the most chemically sensible interpretation and move forward with that") indicates that resolution comes not from acquiring more information from the image but from selecting the most chemically plausible candidate from prior. The candidate selected (alanine) is wrong but is common, easier to draw, and more frequent in textbook examples than valine in a peptide-coupling context. This is the same failure mode observed for catalyst 2 in Case Study 3, expressed even more transparently. The cost gap on this image is correspondingly the largest in the set: Opus 4.6 spends 10.5x the wall-clock time

(346 s versus 33 s), 3.2x the input tokens, 5.6x the output tokens, and 22x the thinking-character count of Opus 4.7 and produces three incorrect rows rather than three correct ones.

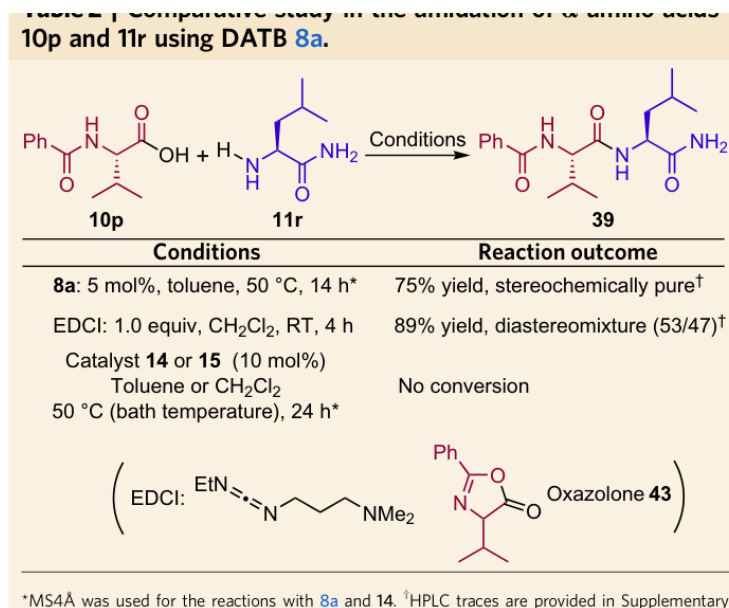


Figure 4-11: Case Study 4 input image. Table 2 of the same Nature Communications paper as Case Study 2, comparing three conditions (DATB 8a in toluene at 50 °C, EDCI in CH₂Cl₂ at room temperature, and auxiliary catalysts 14 or 15) for coupling N-benzoyl-L-valine 10p with L-leucinamide 11r to dipeptide 39. Opus 4.7 returns 3 of 3 in 33 s; Opus 4.6 misreads the valine α -isopropyl as a methyl, encodes the substrate as N-benzoyl-alanine, and the single perception error propagates to 0 of 3 matched reactions across all three rows.

Summary

Across the four cases, the gap between Opus 4.6 and Opus 4.7 reduces to two themes. First, vision quality dominates the score on chemistry figures: in every case where the metric distinguishes the two models, the gap can be traced to a perception event at the input stage (an isopropyl group, a CH₂ linker, a ring fusion, a methyl count, a ring-position index) that no amount of downstream reasoning, validation, or schema handling recovers. Both variants share the same agent loop, prompt, tool set, and validator, so the score difference reduces in each case to what is read off the page. Second, when vision fails, Opus 4.6 falls back to chemical priors rather than to low-confidence output: the thinking trace shows the model selecting the most chemically plausible candidate from training data when the image is ambiguous, rather than abstaining, flagging uncertainty, or requesting a different view. The release-note behaviour of more literal instruction following surfaces here as a tendency for Opus 4.7 to read figures more literally and to invoke named compounds less. The cost picture follows

the quality picture rather than trading against it: producing the wrong answer is not cheaper than producing the right answer in this agent, and on chemistry figures specifically the Opus 4.7 vision improvements yield both better extractions and lower compute spend per image [3].

4.4.4 Input-Image Resolution Ablation

The methodology for this ablation is given in Section 3.3.4. In summary, the 16-image benchmark was processed at five Lanczos-resampled input resolutions (full, 1/2, 1/4, 1/8, and 1/10 of original width and height) by the Gemini 3 Flash Preview single-agent pipeline under an otherwise identical configuration, and all five conditions were scored against the same ground truth with. The macro-averaged reference-based metrics for each condition are reported in Table 4-3.

Table 4-3: Reference-based metrics by input resolution. All values are macro-averages across the 16 images, scored against the ground truth of the 16-image benchmark. The best value in each row is shown in bold.

Metric	Full	Half	Quarter	Eighth	Tenth
Schema pass (of 16)	1.00	1.00	1.00	1.00	1.00
SMILES validity	0.99	1.00	1.00	0.97	0.99
Role-enumeration compliance	1.00	1.00	1.00	1.00	1.00
Reactant IoU	0.57	0.61	0.56	0.24	0.15
Product IoU	0.79	0.75	0.67	0.27	0.07
Cond. recall (strict)	0.42	0.45	0.44	0.22	0.15
Cond. recall (lenient)	0.81	0.82	0.83	0.38	0.24
Cond. precision (lenient)	0.70	0.70	0.71	0.31	0.16
Soft match F1	0.46	0.51	0.45	0.13	0.00
Hard match F1	0.04	0.04	0.04	0.00	0.00
Partial F1 (Jaccard ≥ 0.5)	0.79	0.78	0.78	0.25	0.06
Constituent F1	0.58	0.59	0.63	0.13	0.00

The headline trajectory is a quality plateau through the one-quarter condition followed by a rapid collapse: Partial F1 reads 0.79, 0.79, and 0.78 at full, one-half, and one-quarter resolution, drops to

0.25 at one-eighth, and reaches 0.06 at one-tenth. Product IoU, Soft F1, and Constituent F1 follow the same shape; on Soft F1 and Constituent F1 the one-half and one-quarter conditions slightly exceed the full-resolution baseline, well within the run-to-run noise expected on a 16-image benchmark scored at non-zero temperature. The plateau through one-quarter resolution is consistent with the low-information-per-pixel character of chemical schemes, in which large white margins, high-contrast atom labels, and bond lines that remain several pixels wide even after fourfold down-sampling preserve legibility well below the original 700 to 1500 pixel dimensions of the benchmark images.

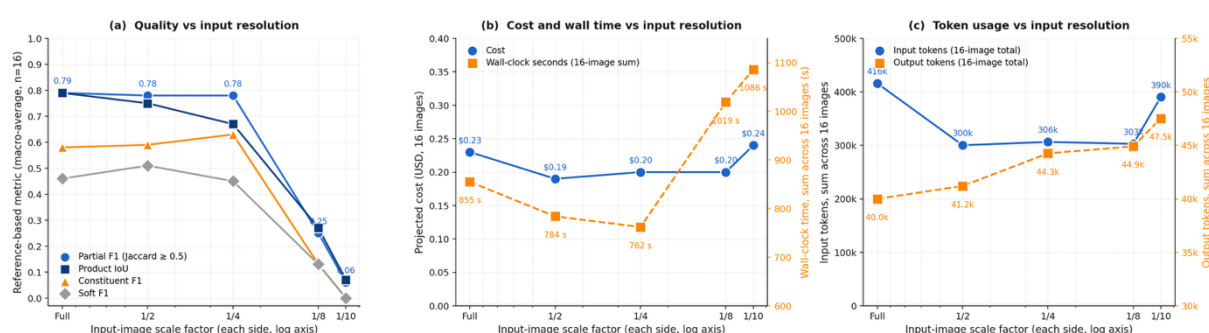


Figure 4-12: Resolution ablation of the Gemini 3 Flash Preview single-agent pipeline on the 16-image benchmark. (a) Reference-based quality metrics versus input-image scale factor; quality is essentially flat from full to one-quarter resolution, drops sharply at one-eighth, and collapses at one-tenth. (b) Projected USD cost (left axis) and summed wall-clock time across the 16 images (right axis) versus the same scale factor; neither cost nor time decreases monotonically with image size, and the lowest-resolution condition is the most expensive in both quantities. (c) Total input tokens (left axis) and total output tokens (right axis) summed across the 16 images; input tokens fall and then rebound, while output tokens, fixed in volume by the Pydantic JSON output schema, are monotonically non-decreasing.

Two patterns matter in the cost and token panels of Figure 4-12: Resolution ablation of the Gemini 3 Flash Preview single-agent pipeline on the 16-image benchmark. (a) Reference-based quality metrics versus input-image scale factor; quality is essentially flat from full to one-quarter resolution, drops sharply at one-eighth, and collapses at one-tenth. (b) Projected USD cost (left axis) and summed wall-clock time across the 16 images (right axis) versus the same scale factor; neither cost nor time decreases monotonically with image size, and the lowest-resolution condition is the most expensive in both quantities. (c) Total input tokens (left axis) and total output tokens (right axis) summed across the 16 images; input tokens fall and then rebound, while output tokens, fixed in volume by the Pydantic JSON output schema, are monotonically non-decreasing.. First, neither

projected USD cost nor summed wall-clock time decreases monotonically as resolution drops: the one-half and one-quarter conditions are weak local minima at approximately \$0.19 to \$0.20 across the 16 images, the one-eighth condition matches them on cost but is about a third slower in wall time, and the one-tenth condition is the most expensive in both quantities at \$0.24 and 1086 s. Second, the underlying mechanism is visible in panel (c): input tokens fall 28 % from full to one-half resolution and then plateau, but output tokens, which are pinned to the fixed Pydantic JSON output schema, are monotonically non-decreasing across the entire sweep (40 000 at full resolution, 47 515 at one-tenth). Because output tokens are priced approximately eight times higher than input tokens in the Gemini 3 family, the absence of any decrease in output volume neutralises the input-side savings. Image down-sampling is therefore not a useful cost-reduction lever for this pipeline.

The transition between robust extraction and collapse is concentrated in a narrow region of the resolution axis (a 2× linear, 4× area step between the one-quarter and one-eighth conditions) rather than being distributed across the sweep. The one-eighth condition itself exhibits bimodal per-image behaviour: five of the 16 images yield full or near-full recall and ten yield zero correct reactions. An earlier inspection of the one-tenth data had suggested a minimum-edge pixel-count threshold around 70 px, but the one-eighth data do not support that hypothesis. The largest minimum edge in the bucket (105 pixels) yielded zero correct reactions and the smallest (45 pixels) yielded perfect recall (5 of 5), with the per-image distributions of minimum edge overlapping substantially across the success and failure buckets. The pattern cannot be described as a deterministic pixel-count threshold; establishing whether a threshold exists, and what image features control it, would require repeat sampling at each condition on a larger benchmark.

A separate methodological observation follows from the failure regime. At one-tenth resolution, reference-based Partial F1 falls to 0.06 while schema validity and SMILES validity both remain at 99 to 100 %: the model continues to emit on average 7.9 reactions per image, every reaction passes the Pydantic schema validator, and every SMILES string is parseable by RDKit. Inspection of the predictions indicates that the model produces chemically plausible but image-unrelated reactions. The implication is that any quality-assurance scheme for this pipeline that relies solely on schema-pass or SMILES-validity checks would silently accept the one-tenth output as a successful extraction.

Reference-based evaluation against held-out ground truth is therefore not a sanity check but a necessary safeguard, a finding that reinforces the validity metrics reported in Section 4.5.

4.5 Cross-system Evaluation

Twelve extraction systems were evaluated against the same sixteen-image ground truth: nine single-agent variants (Claude Opus 4.7, Claude Opus 4.6, GPT-5.4, GPT-5.5, Gemini 3.1 Pro Preview, Gemini 3 Flash Preview, Gemini 3.5 Flash, xAI Grok 4.3, and the open-weights Qwen3.6-27B accessed via OpenRouter) and three multi-step baselines (multi-agent, ChemEAGLE, ChemEAGLE_Hybrid). As Figure 4-13 shows, the five single-agent variants paired with capable multimodal models (Claude Opus 4.7, GPT-5.5, Gemini 3.1 Pro Preview, Gemini 3 Flash Preview, and xAI Grok 4.3) outperform all three multi-step baselines on the structural metrics (Product IoU, Reactant IoU, Soft F1, Partial F1, Constitution F1). On Product IoU specifically, Gemini 3 Flash Preview attains 0.79 and Claude Opus 4.7 attains 0.76, both of which exceed the strongest baseline (multi-agent) by approximately 0.21–0.24 absolute. Within the single-agent family, however, the spread is dramatic: GPT-5.4 attains a Product IoU of 0.18, the lowest score in the entire table. This provides the most direct "the model dominates the framework" observation the present benchmark provides identical agent infrastructure produces results spanning 0.61 absolute on Product IoU when the underlying multimodal model is varied. Qwen3.6-27B sits below the proprietary single-agent frontier on Partial F1 (0.66) and below every proprietary variant except GPT-5.4 on Product IoU (0.40); it is comparable to Opus 4.6 on Partial F1 (0.64) while extracting only eleven of the sixteen images. Gemini 3.5 Flash, evaluated later in May 2026, lands at Partial F1 = 0.80 with full schema coverage (16 of 16) and sits between Gemini 3.1 Pro Preview (0.83) and Gemini 3 Flash Preview (0.79) on quality at roughly three times the per-image cost of Gemini 3 Flash Preview and a quarter of the per-image cost of Opus 4.7.

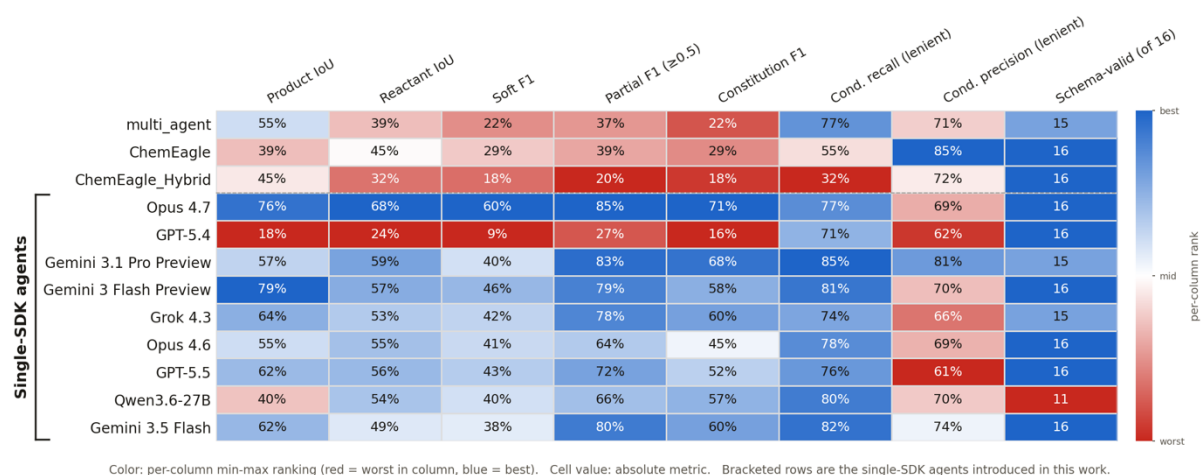


Figure 4-13: Heatmap of all 10 systems on the 16-image benchmark. Colour is per-column min-max ranking.

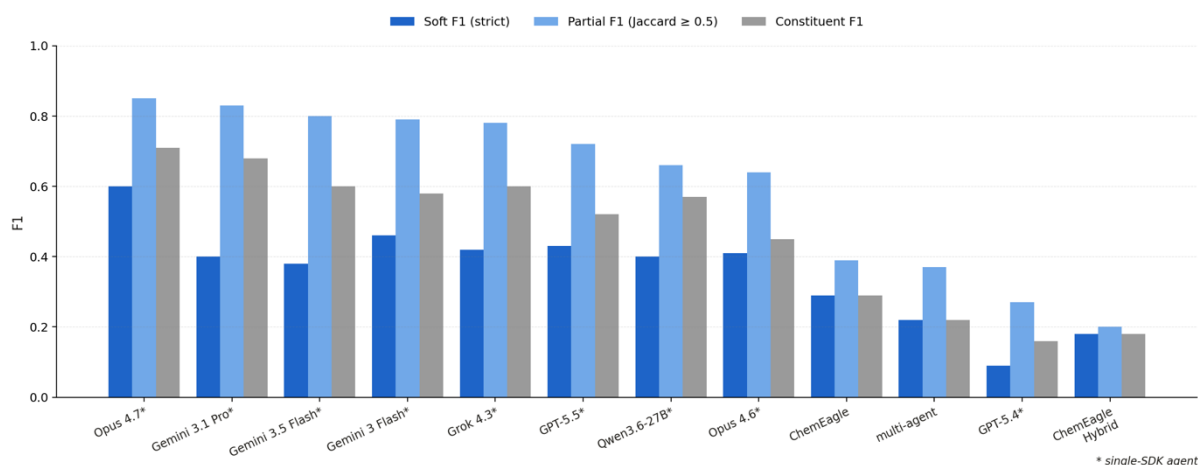


Figure 4-14: Per-metric ranked bar charts comparing the single-agent variants with the multi-step baseline systems. The single-agent Opus 4.7 shows the highest F1-scores.

Under the lenient matcher described in Section 4.2, all nine single-agent variants score in the 0.71 to 0.85 range on condition recall, with Gemini 3.1 Pro Preview leading at 0.85 and Gemini 3 Flash Preview at 0.81. This range overlaps with the multi-step baselines (multi-agent at 0.77, ChemEAGLE at 0.55, ChemEAGLE Hybrid at 0.32) and brackets all of them. The lenient figure reflects substantive agreement on what conditions are present rather than exact textual agreement with the annotator's style and is therefore the more meaningful indicator of chemistry coverage on this benchmark.

Cost versus Quality

Every system in the comparison was re-run on the 16-image benchmark with token usage captured per image. USD cost is computed from public list prices for each model (input + output tokens). Results sorted by partial-F1 shown in .

Table 4-4: Cost versus partial-F1 on the sixteen-image benchmark. ChemEAGLE could not process four of the sixteen images due to internal MolNexTR errors and is reported over $n = 12$ successful extractions. Gemini 3.1 Pro Preview and Grok 4.3 each timed out on one image and are reported over $n = 15$. USD costs are either reported from OpenRouter or Azure rates as of 2026-05-08; Opus 4.7 uses the empirical OpenRouter bill (\$3.52) rather than the sticker projection because of an undisclosed surcharge.

System	n / 16	Partial F1	USD cost	Tokens (in + out)	Wall (sum, s)
Single-agent Opus 4.7	16	0.854	\$3.52	334 k	662
Single-agent Gemini 3.1 Pro Preview	15	0.832	\$2.01	1,390 k	1,574
Single-agent Gemini 3.5 Flash	16	0.804	\$0.88	386 k	637
Single-agent Gemini 3 Flash Preview	16	0.790	\$0.28	464 k	855
Single-agent Grok 4.3	15	0.780	\$3.86	992 k	6,662
Single-agent GPT-5.5	16	0.723	\$3.62	259 k	1,588
Single-agent Qwen3.6-27B	11	0.66	\$0.93	350 k	3,797
Single-agent Opus 4.6	16	0.637	\$2.25	651 k	1,209
ChemEAGLE (gpt-5-mini)	12	0.394	\$0.61	817 k	6,854
multi_agent (gpt-5.4)	16	0.370	\$6.42	3,089 k	5,486
Single-agent GPT-5.4	16	0.273	\$0.85	185 k	879
ChemEAGLE_Hybrid (gpt-5-mini)	16	0.200	\$0.36	282 k	2,969

In Figure 4-15, the cost per image versus partial F1 is shown. Opus 4.7 single-agent is the best performing system with a partial F1 score of 0.85. Gemini 3 Flash Preview is the cheapest competitive single-agent. Qwen3.6-27B, the opensource model, sits close to the fitted line at roughly \$0.085 per

image and Partial F1 = 0.66. Gemini 3.5 Flash sits in the upper-middle of the Pareto at roughly \$0.055 per image and Partial F1 = 0.80, reproducing most of Opus 4.7's quality (0.85) at a quarter of its cost; this is now the second cheap-and-strong option after Gemini 3 Flash Preview. The multi-agent system clearly underperforms; it has the highest cost for a rather low partial F1 score. The most surprising result is the low partial F1 score of single-agent GPT-5.4.

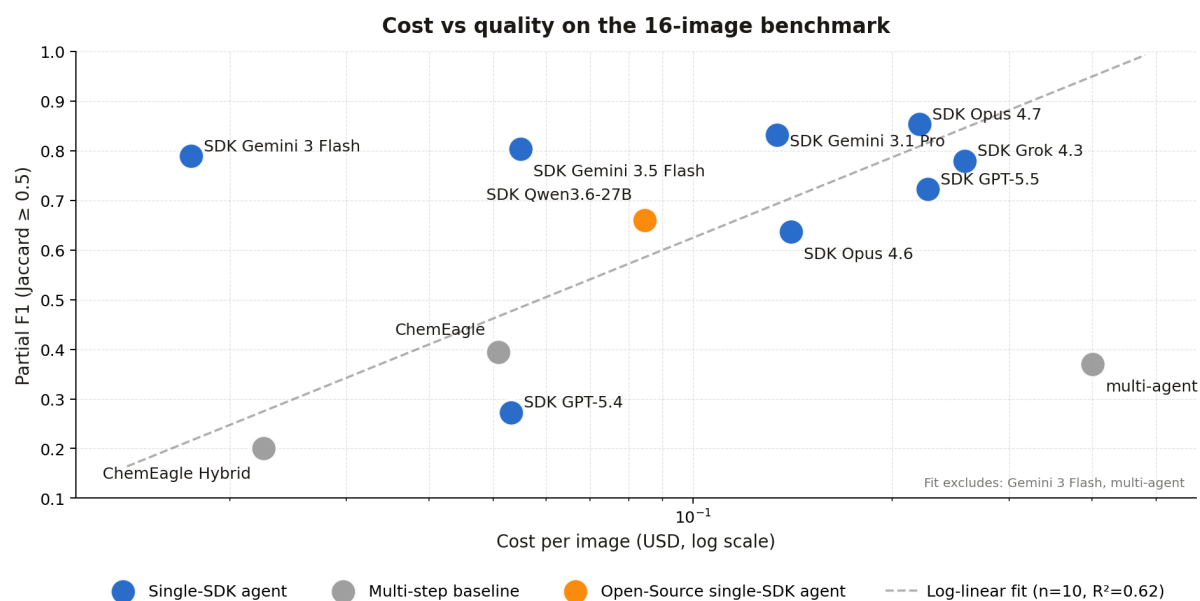


Figure 4-15: Cost-quality trade-off across all twelve systems on the sixteen-image GT3 benchmark. Each point is plotted by mean USD cost per image (logarithmic scale, horizontal axis) against Partial F1 with a Jaccard threshold of 0.5 (vertical axis). Blue markers denote the proprietary single-agent variants; the orange marker denotes the open-weights Qwen3.6-27B accessed via OpenRouter; grey markers denote the three multi-step baselines (ChemEAGLE, ChemEAGLE Hybrid, and the original multi-agent framework). The dashed line is a log-linear fit through the nine systems that lie close to the trend ($R^2 = 0.62$). Gemini 3 Flash Preview and the multi-agent framework are excluded as outliers.

Figure 4-16 shows that Opus 4.7 both the fastest (41 s/image) and the highest-scoring ($F1 = 0.85$) single-agent. Grok 4.3 takes 11 times longer (444 s/image) for a 0.07 lower F1. The two heaviest pipelines, namely multi-agent (343 s/image) and ChemEAGLE (571 s/image), sit in the $F1 = 0.37$ to 0.39 band, far below the single-agent frontier. This is due the fixed step sequence of the multi-agent pipelines which copies the history of VLM calls in the next call, increasing the process time each step. Wall time tracks per-provider throughput and pipeline orchestration cost, not extraction quality, so latency is a free parameter for picking systems on this task. Qwen3.6-27B sits between Opus 4.6 and Grok 4.3 at approximately 345 s per image for $F1 = 0.66$, the slowest competitive single-

agent option after Grok 4.3. Gemini 3.5 Flash sits at approximately 40 s per image for $F1 = 0.80$, matching Opus 4.7's wall time and confirming that the Gemini family's parallel-function-call discipline closes the latency gap with Anthropic on this benchmark.

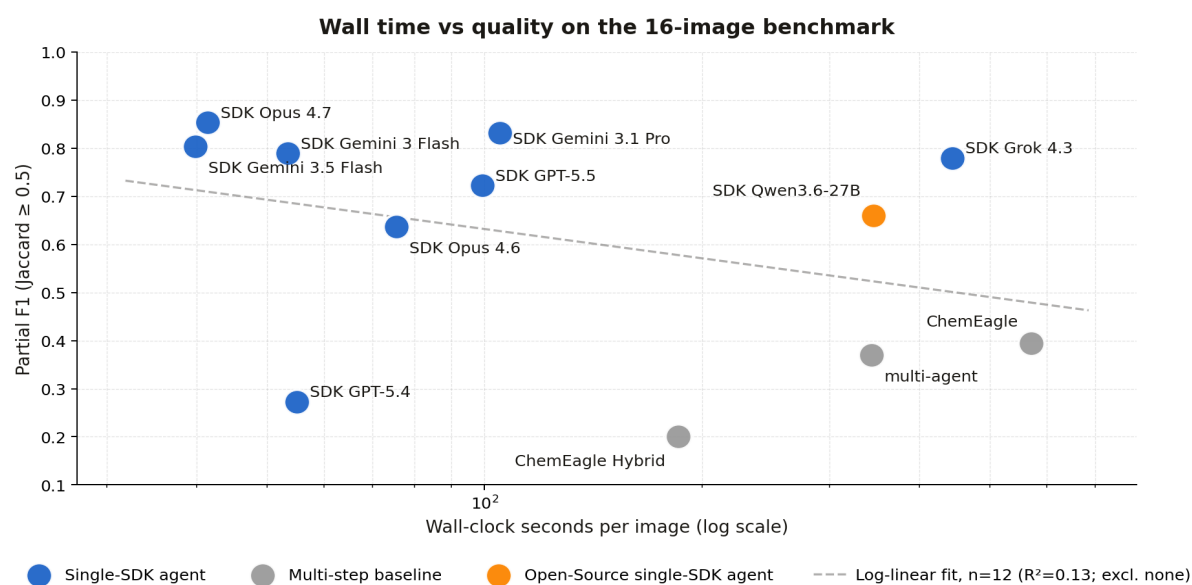


Figure 4-16: Partial F1 versus mean wall-clock seconds per image across all twelve systems on the sixteen-image benchmark (log-x). Blue markers denote the proprietary single-agent variants; the orange marker denotes the open-weights Qwen3.6-27B accessed via OpenRouter; grey markers denote the three multi-step baselines (ChemEAGLE, ChemEAGLE Hybrid, and the original multi-agent framework). The trend line is slightly negative ($R^2 = 0.13$).

Figure 4-17 illustrates that using more tokens does not on average buy more quality. Opus 4.7 attains the highest F1 (0.85) at only 21k tokens/image, while multi-agent consumes 193k tokens/image (roughly 9× more) for an F1 of 0.37. GPT-5.5 sits at the same token spend as ChemEAGLE Hybrid (16-18k tokens/image) yet is +0.52 F1 above it, and Gemini 3 Flash matches Grok 4.3's F1 (0.79 versus 0.78) at less than half the tokens. Tokens spend is therefore a poor predictor of extraction quality; the choice of model and pipeline shape dominate. Qwen3.6-27B uses about 32 k tokens per image, comparable to Opus 4.6 at roughly 41 k, with both models landing near Partial F1 of 0.65; this further confirms that tokens spend is not predictive of quality on this benchmark. Gemini 3.5 Flash uses about 24 k tokens per image (17 k input plus 8 k output), below Gemini 3 Flash Preview at roughly 29 k and well below Gemini 3.1 Pro Preview at 93 k for similar quality.

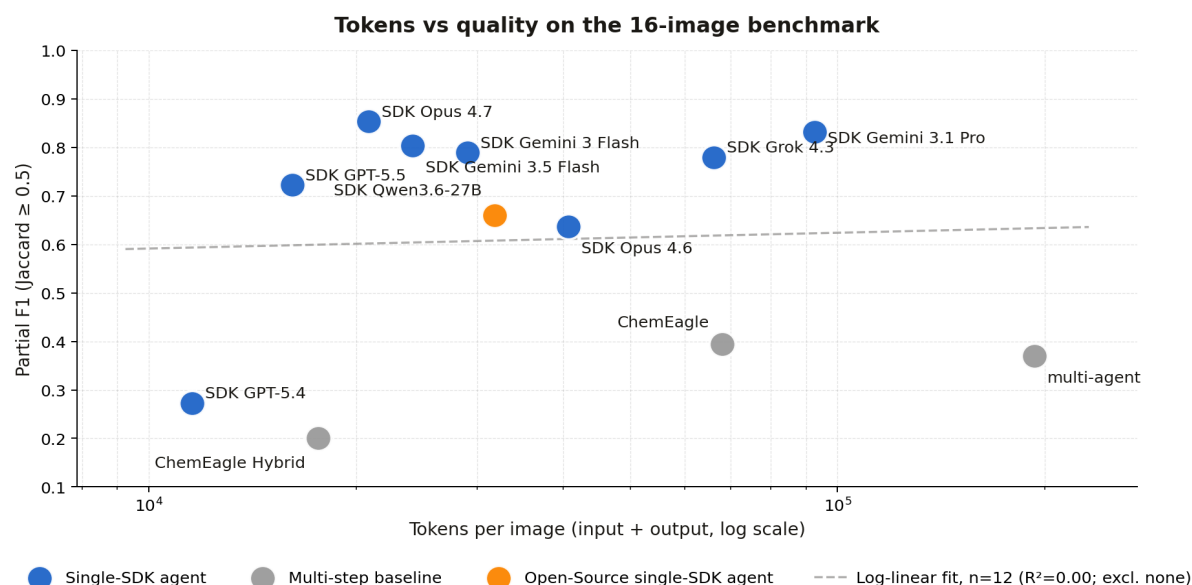


Figure 4-17: Partial F1 versus tokens-per-image (input + output, log-x). Blue markers denote the proprietary single-agent variants; the orange marker denotes the open-weights Qwen3.6-27B accessed via OpenRouter; grey markers denote the three multi-step baselines (ChemEAGLE, ChemEAGLE Hybrid, and the original multi-agent framework). The log-linear fit across all systems is essentially flat ($R^2 = 0.00$).

The principal conclusion is that, for the 16-image benchmark, replacing a six-agent pipeline with a single LLM call to a strong vision model is the dominant lever on every axis. Gemini 3 Flash gets +0.42 F1 above the multi-agent system at 1/23rd the cost and 1/6th the wall time. Opus 4.7 lifts F1 another +0.06 over Flash but at 13× the cost, only justified for last-mile quality where dollars do not matter. The multi-agent pipeline's value proposition therefore needs to be re-stated in terms other than F1 on this benchmark: it remains the only system that emits SMILES at 100 % validity through an enforced RDKit round-trip, and it remains the only system whose per-step trace is human-inspectable at the agent-by-agent level. These qualities matter for downstream auditing, regulatory documentation, and integration with structural-verifier loops, but that the partial-F1 scorer does not reward. A second cheap option emerges with Qwen3.6-27B at \$0.085 per image, although its Partial F1 of 0.66 is well below Gemini 3 Flash's 0.79, so its role on the Pareto front is as a low-cost mid-quality fallback rather than a frontier system. However, the cost could be much lower if the open-source model is installed on local hardware. Gemini 3.5 Flash extends the cheap-and-strong band of the Pareto front: at \$0.055 per image with Partial F1 = 0.80 and full schema coverage, it offers roughly Opus 4.7-class quality at a quarter of the cost, with the same 40-second wall time. A separate ablation

reported in Section 4.4.4, in which the same Gemini 3 Flash Preview pipeline was run on the 16-image benchmark at five input resolutions, shows that input-image down-sampling is not an additional cost-reduction lever: input-token savings are neutralised by the fixed-volume Pydantic output schema.

Failure Modes by System

Per-image inspection of zero or near-zero scores reveals systematic differences in failure modes between systems.

Claude Opus 4.7 scores 100% SMILES validity on every image. Its three lowest Product IoU images are dominated by stereo errors rather than connectivity errors, the only variant in the comparison where this is true. ACScat_2020 page002 table01 (Figure 4-7) shows a systematic mismatch across all thirteen predictions where Opus 4.7 writes the same benzyl-on- α -carbon structure that every other variant emits against a gold encoding a benzoyl group; ACScat_2020 page003 picture01 (Figure 4-18) shows a universal α -carbon stereo flip on the Boc-dipeptide coupling table that reduces Product IoU to 0.07 while Partial F1 remains at 0.88; CEJ_2016 page004 table02 (Figure 4-18) shows the same stereo-flip behaviour with one trailing connectivity error. The model produces consistently short output (around two thousand tokens per image) and almost never emits malformed structures.

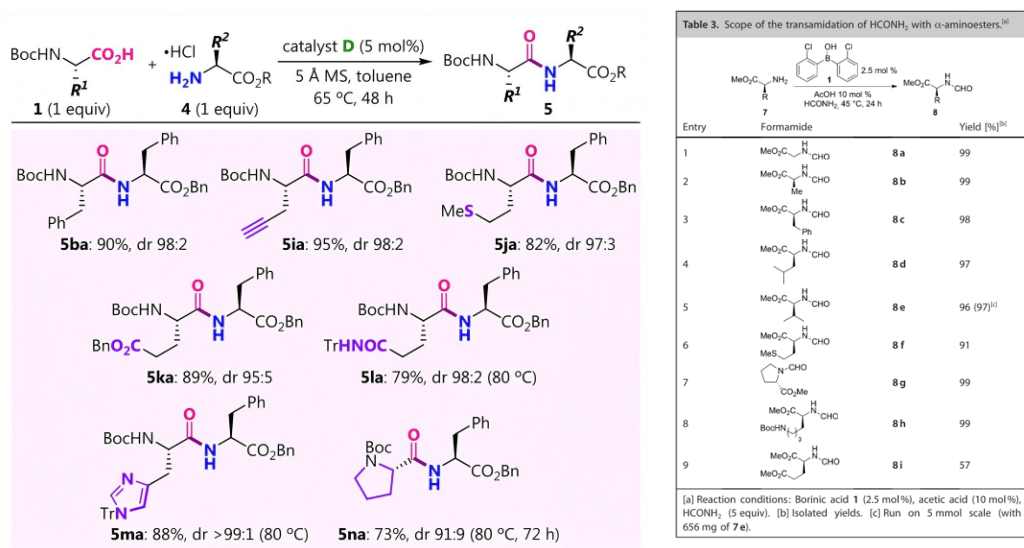
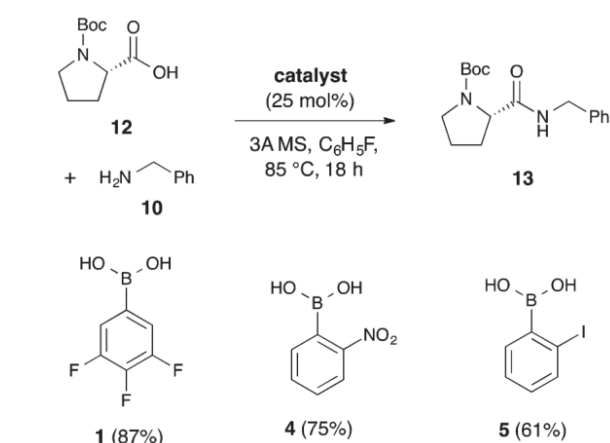


Figure 4-18: Two image from the 16-image benchmark. From left to right: ACScat_2020 page003 picture01 and CEJ_2016 page004 table02.

Claude Opus 4.6 also reaches passes all 16 images with 100% SMILES validity, but its per-image scores are markedly more uneven than Opus 4.7's. Its three lowest images replace Opus 4.7's stereo

failures with connectivity failures. On ACScat_2020 page002 table01 (Figure 4-7), Opus 4.6 omits the explicit α -carbon hydrogen present in Opus 4.7's output and produces twelve connectivity misreads plus one methylene shift across the fourteen R-group variants. NC_2017 page005 picture01 (Figure 4-11) is the most distinctive failure of any variant: three of the four Boc-dipeptide products are rewritten with a cinnamoyl-glycine chain in place of the Val-NH-Bz residue, and a fourth cinnamoyl-oxazoline reaction is hallucinated. Opus 4.6 is the only variant in the seven whose lowest-three set includes a fully hallucinated reaction. ACScat_2020 page003 picture01 (Figure 4-18) shows the same stereo-flip pattern as Opus 4.7 plus two real connectivity errors on the trityl-protected imidazole and glutamate sidechains.

GPT-5.4 scores 100% SMILES validity but its dominant failure mode across the three lowest Product-IoU images is a phenethyl-instead-of-benzyl methylene shift. On CEJ_2016 page002 table02 (Figure 4-19), all fourteen R-group variants emit O=CNCc1cccc1 against a gold [H]C(=O)NCc1cccc1, propagating the extra methylene uniformly. ACScat_2020 page002 table01 (Figure 4-7) inherits the shared benzoyl-vs-benzyl gold mismatch across all thirteen predictions, and on CS_2016 page002 table02 (Figure 4-19r) the model writes CC[C](C(=O)NCc1cccc1)c1cccc1, an RDKit-valid but chemically dubious tertiary carbon with no implicit hydrogen, against a gold encoding the same heavy-atom skeleton with a normal tertiary CH. These are not random hallucinations, but consistent vision-encoder misreads of specific atom-count and connectivity motifs.



cheme 2 Comparison of different catalysts in the amidation of Boc

Table 1. Optimisation of the transamidation of DMF with benzylamine.		
Entry	Deviation from standard conditions	Yield [%] ^[a]
1	none	73
2	2-Cl-4-F-(C ₆ H ₃) ₂ B(OH) ₂ 4 ^[b]	54
3	Ph ₂ B(OH) ₂ 5 ^[b]	6
4	2-Cl-C ₆ H ₄ B(OH) ₂ 6 ^[b]	22
5	reaction run for 72 h	82
6	reaction run at 45 °C	98
7	reaction run at 65 °C for 4 h	99
8	no 1, no AcOH	0
9	only 1	34
10	only AcOH	19
11	only BnCO ₂ H ^[c]	15
12	only HCO ₂ H ^[c]	11
13	only CCl ₃ CO ₂ H ^[c]	9
14	only CF ₃ CO ₂ H ^[c]	1

[a] Isolated yields. [b] Instead of 1. [c] Instead of AcOH.

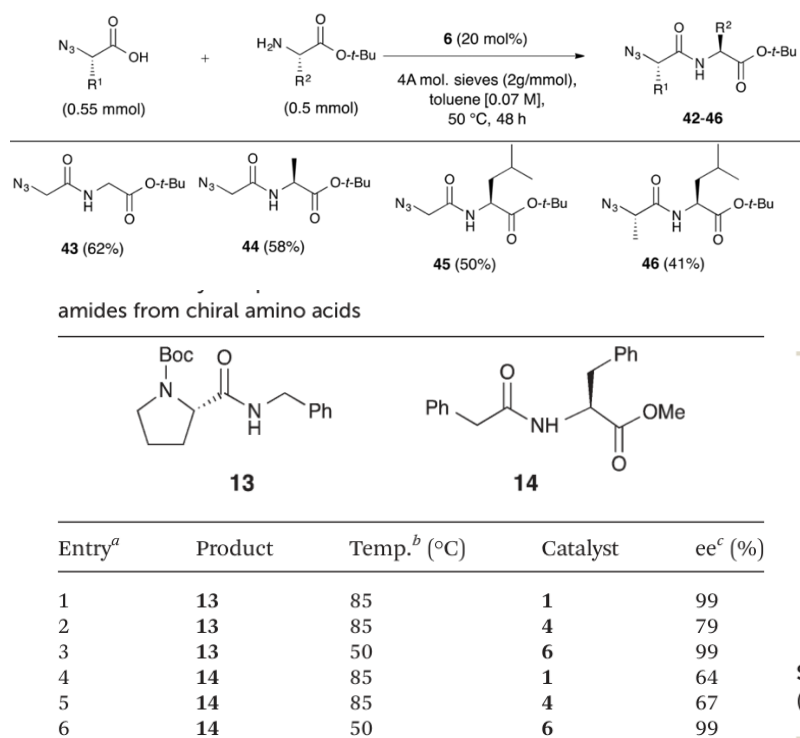
Figure 4-19: Two image from the 16-image benchmark. From left to right: GC_2015 page003 picture03 and CS_2016 page002 table02

GPT-5.5 is a strict improvement over GPT-5.4 (Partial F1 0.72 versus 0.27) but exhibits the same family of failure mode in concentrated form. Schema validity is 100% and passes all 16 images. Two

images in the lowest-three set carry the methylene shift: GC_2015 page003 picture03 (Figure 4-19) emits ...C(=O)NCCc1cccc1 against a gold ...C(=O)NCCc1cccc1 across four reactions, and GC_2015 page003 table02 (Figure 4-19) emits O=C(CCc1cccc1)N1CCCC1 against O=C(Cc1cccc1)N1CCCC1 across four more. ACScat_2020 page003 picture01 (Figure 4-18) is the same universal α -stereo flip seen on Opus 4.7 and Opus 4.6 with one additional methylene loss on the glutamate sidechain. The connectivity error is identical in shape to the misread documented for GPT-5.4 above. GPT-5.5 reads the rest of the benchmark essentially correctly, which is why its aggregate sits between Gemini 3 Flash and the multi-step baselines despite two near-zero outliers on Product IoU.

Gemini 3.1 Pro Preview scored on 15 of 16 images: one image (NC_2017 page004, Figure 4-8) timed out at the 64-step ceiling. The same image was the only failure for the multi-agent baseline at its 1200-second wall-clock cap. Among the 15 images Gemini 3.1 Pro did process, the three lowest Product-IoU cases are ACScat_2020 page002 table01 (Figure 4-7) (twelve of fourteen predictions inherit the shared benzoyl-vs-benzyl mismatch), NC_2017 page005 picture01 (Figure 4-11) (a universal α -stereo flip across the three Boc-dipeptide products with one duplicate reaction hallucinated), and GC_2015 page003 picture03 (Figure 4-19) (a uniform proline α -stereo flip that reduces Product IoU to 0 while Partial F1 remains at 1.00).

Gemini 3 Flash Preview scored on all 16 images, produced the highest Product IoU on the benchmark, and its three lowest Product-IoU images are dominated by stereo errors. ACScat_2020 page003 picture01 (Figure 4-18) shows eight α -stereo flips on the dipeptide coupling products, ACScat_2020 page002 (Figure 4-7) inherits the twelve-fold benzoyl-vs-benzyl mismatch, and GC_2015 page007 picture03 (Figure 4-20) shows two α -stereo flips on the Asp and Leu α -azide products. The most distinctive failure surfaces only on the Partial-F1 ranking on GC_2015 page004 table01 (Figure 4-20), a catalyst-screening table for a single Boc-Pro-NH-Bn product where the gold lists only the product per catalyst, Flash populates each reaction with two hallucinated reactant SMILES (the obvious retrosynthetic precursors Boc-Pro-OH and benzylamine). Product IoU is 1.00 but Reactant IoU collapses to 0 because gold leaves reactants empty, dragging Partial F1 below the 0.5 threshold for every reaction. This is a scope mismatch with the gold annotation convention rather than literal reaction hallucination, since the reaction count is correct.



^a Reaction conditions: for entries 1. 2. 4. 5: see ref. 16. For entries 3

Figure 4-20: Two images from the 16-image benchmark (Above: GC_2015 page007 picture03, below: GC_2015 page004 table01).

xAI Grok 4.3 scored on 15 of 16 images. The four initial failures were all step-ceiling exhaustions rather than completions with low scores; after the per-call step ceiling was raised from 64 to 128, three of the four were extracted successfully. One image (GC_2015 page007 picture_03 (Figure 4-20), a dense picture-style figure) continued to exhaust the 128-step ceiling and was excluded from the per-system aggregate. Among the 15 images Grok did process, the three lowest Product-IoU cases are all stereo-driven: ACScat_2020 page003 picture01 (Figure 4-18) (six α -stereo flips plus one connectivity drop and one substituent-position error on the trityl-imidazole regioisomer), GC_2015 page003 picture03 (Figure 4-19) (four proline α -stereo flips), and NC_2017 page005 picture01 (Figure 4-11) (three α -stereo flips on the Boc-dipeptide products). Grok also inherits the GC_2015 page004 table01 (Figure 4-20) reactant-hallucination scope mismatch flagged on Gemini Flash, surfaced only by the Partial-F1 ranking.

Multi-step baselines. The multi-agent framework's failure modes (page-crop hallucinations, R-fragment leakage, traditional schemes without an R-group structure, paper-specific shorthand) are documented in the case studies above and apply directly. ChemEAGLE's four hard failures (see Chapter 4, ChemEAGLE reproduction outcome) are upstream MolNexTR exceptions; the upstream wrapper does not catch them. ChemEAGLE Hybrid completed every image but scored below both other multi-step systems on partial-F1.

Cross-variant patterns. Across all seven single-agent variants, four images cluster as systematic rather than variant-specific failures. ACScat_2020 page002 table01 (Figure 4-7) fails for six of the seven variants with the same benzoyl-vs-benzyl mismatch on the α -carbon: all seven models read the same chemistry against a gold annotation, and the systematic agreement among independent multimodal models suggests the gold annotation for this image may itself be incorrect; a manual review of the gold file is warranted. ACScat_2020 page003 picture01 (Figure 4-18) fails for five of the seven variants with the same universal α -carbon stereo flip on a Boc-dipeptide coupling table; the image draws wedge-bond stereo descriptors in a form that all multimodal models in the benchmark mis-call, indicating a benchmark-level rather than model-level ambiguity. GC_2015 page003 picture03 (Figure 4-19) fails for four variants (GPT-5.5, Gemini 3.1 Pro Preview, Gemini 3 Flash Preview, Grok 4.3) with a uniform proline α -stereo error. GC_2015 page004 table01 (Figure 4-20) surfaces a different pattern: Gemini 3 Flash Preview and xAI Grok 4.3 both populate hallucinated reactants on a product-only catalyst-screening table, dragging Partial F1 to zero on each reaction while keeping Product IoU perfect. None of these patterns are addressable by prompt iteration; they reflect either annotation ambiguity or a structural mismatch between the agents' default reasoning and the benchmark's scoring conventions.

4.6 Le Math-synth

4.6.1 Test Case

The adapted pipeline was applied to a single organic-chemistry paper, Keerthana *et al.* [4], reporting a chemo- and regioselective route to O- and C-prenylated resorcinol natural products. The pipeline extracted 13 compounds: all 8 named natural-product targets (sofalcone, derricidin, derricin, isocordoin, 4-hydroxyderricin, crotaorixin, xanthoangelol, 4'-O-geranylisoliquirtigenin) and 5 of the labelled vinylogous-ester intermediates. Each compound has a synthesis route ranging from 3 to 11 steps (mean 6.4) and yields ranging from 32 % to 98 % (mean 76 %). Every compound received a label and a yield. The chemistry classification was internally consistent: 11 of the 13 compounds were tagged as aldol reaction (correct, since the chalcone-forming Claisen–Schmidt condensation is an aldol variant), and the 2 vinylogous-ester intermediates were tagged as nucleophilic substitution (correct, since O-prenyl/O-geranyl ethers are formed via SN2 alkylation).

To assess extraction quality, the LLM-as-judge framework supplied with LeMath-synth (DspyGeneralSynthesisJudge) was applied with gpt-4o-mini as the judge, scoring each extraction on seven Lederbauer-defined axes plus an overall score, on a 1–5 scale. The aggregate scores across the 13 compounds are summarised in Table 4-5.

Table 4-5: LeMath-synth (organic adaptation) judge scores on Keerthana et al. (2026), n = 13 compounds, scale 1–5, judge model gpt-4o-mini.

Criterion	Mean	Std. dev.	Min – Max
Overall	4.31	0.25	3.8 – 4.8
Structural completeness	4.19	0.33	4.0 – 5.0
Material extraction	4.46	0.25	4.0 – 5.0
Process steps	4.23	0.39	3.5 – 5.0
Equipment extraction	4.00	0.54	3.0 – 5.0
Conditions extraction	3.85	0.47	3.0 – 5.0
Semantic accuracy	4.50	0.29	4.0 – 5.0
Format compliance	4.88	0.30	4.0 – 5.0

Two patterns are visible in these numbers. First, format compliance is essentially saturated (4.88/5), confirming that the Pydantic schema and the constrained-vocabulary prompts reliably produce parseable, schema-conformant JSON (a prerequisite for any downstream use). Second, conditions extraction is the weakest dimension (3.85/5), reflecting the difficulty of pulling temperatures, durations, atmospheres, and concentrations out of dense Experimental-section prose. This is consistent with the limitations the LeMath-synth authors themselves report and points at conditions parsing as the most useful target for prompt-engineering follow-ups.

4.7 Discussion

The remaining sections of this chapter step back from the per-system numbers to interpret what they imply about the design choices underlying the comparison. Two discussion threads structure the

analysis. First, on this task and benchmark, the multimodal model itself emerges as the dominant determinant of extraction quality, with the choice of orchestration strategy accounting for a substantially smaller residual contribution. Second, we examine why the multi-agent pipeline scored below the single-agent pipeline on partial-F1 despite its more elaborate architecture, and which residual advantages of the multi-agent design persist once that performance gap is acknowledged.

4.7.1 Model Choice as the Dominant Determinant of Extraction Quality

A principal observation is that, on this task and this benchmark, the choice of multimodal model accounts for a 0.61-point absolute spread in Product IoU (0.18 for GPT-5.4 to 0.79 for Gemini 3 Flash Preview) on identical agent infrastructure, while the choice of orchestration (single agent versus six agents) accounts for a substantially smaller residual on the same data. The most controlled experimental comparison is the single-agent panel: every variant uses the same loop, the same two tools, and the same system prompt; only the model is swapped. The 0.61-point spread that remains is, by construction, attributable to the model alone. The same point is reinforced by the within-model resolution ablation in Section 4.4.4: across the Gemini 3 Flash Preview pipeline alone, varying input resolution from full to one-quarter shifts Partial F1 by less than 0.01, which is at least an order of magnitude smaller than the spread observed when the model itself is varied.

This claim should be qualified by a counterpoint visible in the same data. The multi-agent pipeline running on GPT-5.4 reaches 0.55 Product IoU on the sixteen-image benchmark, whereas the single-agent pipeline with the same GPT-5.4 model reaches only 0.18. That 0.37 product-IoU gap is a substantial framework effect at a fixed model, and it runs in the opposite direction of the headline claim. The two observations are best reconciled as follows: when the multimodal model is strong (Claude Opus 4.6 and 4.7, the Gemini 3 family, Grok 4.3), the single-agent pipeline already captures most of the available signal and the additional orchestration of the multi-agent framework contributes little; when the multimodal model is weak (GPT-5.4 as evaluated here), the multi-agent framework’s chemistry deep-learning components (MolDetect, MolNexTR) and deterministic R-group chain rescue what the LLM cannot read directly. The framework matters when the model is weak; the model matters more than the framework when the model is strong. This pattern is directly confirmed by the Opus 4.7 multi-agent run reported above: substituting Claude Opus 4.7 for GPT-5.4 across every agent of the multi-agent framework lifts Partial F1 from 0.37 to 0.78, but the

resulting Opus 4.7 multi-agent still does not beat the single-agent Opus 4.7 (0.78 versus 0.85). With a strong model, the framework no longer adds value.

Two alternative explanations should be considered before this is accepted as a strong claim. First, the system prompt was iterated against responses from the Anthropic family during development; it may be slightly tuned to Claude’s response patterns, and a prompt iterated against a different model could narrow the spread. Second, the per-provider step-ceiling differences reflect a real engineering accommodation that is not captured in the prompt itself. Neither alternative is sufficient to explain a Product-IoU difference of 0.61, but both should be acknowledged. A minimal-prompt ablation across the same seven models, designed to isolate the model contribution from any prompt-design effects, would help discriminate between these explanations. This ablation was not run in the present work and is identified as future work in Chapter 5.

4.7.2 Comparative Performance and Residual Advantages of the Multi-Agent Pipeline

The multi-agent pipeline scored 0.37 partial-F1 on the 16-image benchmark, against 0.85 for the single-agent pipeline paired with Claude Opus 4.7. The gap is large due to three contributing factors.

First, the chemistry-DL stack the multi-agent pipeline uses (MolDetect and MolNexTR) is not as accurate as the vision encoders inside the strongest current multimodal models. MolDetect was trained on chemistry-figure crops and works well on clean substrate-scope tables, but it fabricates bboxes on page crops that mix structures with text. Once MolDetect is wrong about where the molecules are, MolNexTR cannot rescue the situation. The multimodal models read the entire figure jointly and do not have this two-stage failure mode.

Second, multi-agent prompt-following variance is a real cost. On a given image, two consecutive runs of the multi-agent pipeline can produce different scores because the LLM occasionally skips the apply substituents tool after the derive substituents tool or collapses similar variants in a substrate-scope figure. The single-agent loop is not immune to variance, but the variance is bounded by the schema-validating terminator: if the agent emits a malformed FigureExtraction, the loop tells it so and it tries again. The multi-agent loop has more places where things can go wrong silently.

Third, the multi-agent framework’s GPT-5.4 / GPT-5-mini model split was chosen at a time when those were the strongest models available and was not re-baselined when Claude Opus 4.7 and the Gemini 3 family arrived. To remove this confounder, the multi-agent framework was re-run on the sixteen-image benchmark with Claude Opus 4.7 substituted for GPT-5.4 across every agent, with

the same wiring and the same MolNexTR backbone otherwise (Section 4.3.3, Multi-Agent with Claude Opus 4.7). The Opus 4.7 multi-agent reaches Product IoU 0.80, Partial F1 0.78, and Constituent F1 0.68, an improvement of roughly 0.41 absolute on Partial F1 over the GPT-5.4 multi-agent baseline (0.37). These data indicate that the multi-agent framework's shortfall against the single-agent pipeline is dominated by the GPT-5.4 vision encoder rather than by the orchestration design itself. The Opus 4.7 multi-agent does not, however, close the gap with the single agent paired with the same model: single-agent Opus 4.7 reaches Partial F1 0.85, Constituent F1 0.71, and Reactant IoU 0.68, all of which exceed the corresponding multi-agent figures, while costing \$3.52 compared with \$11.73 (approximately one third of the cost) and finishing in eleven minutes compared with sixty-three minutes (approximately 5.7 times shorter in wall time) on the sixteen-image benchmark. The multi-agent framework therefore does not pay back its orchestration overhead on this task even under the most favourable model substitution.

What the multi-agent pipeline retains qualitative advantages. It is the only system in the comparison that emits SMILES at 100 % validity through an enforced RDKit round-trip; ChemEAGLE emits roughly 22 % of SMILES that do not parse, and the single agents rely on the SMILES validation tool being called by the model rather than enforced. It is also the only system whose per-step trace is human-inspectable at the agent-by-agent level: a downstream auditor can see exactly which agent contributed which finding, which tool calls were made, and which validation rounds fired. These properties matter for regulatory documentation and for integration with structural-verifier loops, but neither is rewarded by partial-F1.

A reasonable production architecture would therefore not be the multi-agent pipeline as currently built, but a hybrid: route most images through a single-agent call to a strong vision model for cost and quality and reserve the multi-agent path for images that require a per-agent audit trail. The cost model and routing logic for such a hybrid are not implemented in this thesis but are an obvious direction for follow-up work.

4.8 References

1. OpenRouter. OpenRouter.
2. What's new in Claude Opus 4.7. Claude API Docs.
3. Building with extended thinking. Claude API Docs.
4. Keerthana, S., et al., *Chemo- and Regioselective Strategy for the Synthesis of O- and C-Prenylated Resorcinol-Based Natural Products from Vinylogous Esters*. The Journal of Organic Chemistry, 2026. **91**(10): p. 3717–3731.

Chapter 5.

Conclusion and Future Work

5.1	INTRODUCTION	173
5.2	LIMITATIONS	173
5.3	CONCLUSIONS	174
5.4	FUTURE WORK	177
5.4.1	BENCHMARK AND EVALUATION	177
5.4.2	REACTANT-SIDE IMPROVEMENTS	178
5.4.3	ARCHITECTURE EXPERIMENTS	178
5.4.4	KNOWLEDGE DISTILLATION	179
5.5	REFERENCES	180

5.1 Introduction

The preceding analysis reported the per-system and cross-system results of running the four extraction pipelines against the sixteen-image benchmark, including aggregate quality metrics, cost-quality trade-off curves, and per-system failure modes. This chapter steps back from those numbers to interpret what they imply about the design choices underlying the comparison and to identify the directions in which the work should be extended.

The interpretation of the principal findings, centred on the observation that the multimodal model itself dominates extraction quality and on the question of why the multi-agent pipeline scored below the single-agent pipeline on partial-F1 despite its more elaborate architecture, has already been folded into the discussion that accompanies the results. What remains is organised around three concerns. First, a candid statement of the limitations of the present evaluation, covering benchmark size and composition, the closed-source character of the evaluated models, the heuristic nature of the condition matcher, and the gaps in reproducibility metadata. Second, the conclusions of the thesis. Third, a structured programme of future work, grouped under benchmark and evaluation improvements, reactant-side improvements, and architecture experiments.

5.2 Limitations

Benchmark size. Sixteen images are sufficient to demonstrate the trends reported but is too small to support fine-grained model rankings. The standard error on each macro-averaged metric is large, and the ordering between similarly scoring systems (e.g. Claude Opus 4.7 at 0.76 Product IoU and Gemini 3 Flash Preview at 0.79) should not be over-interpreted. The same caveat applies in reverse to the tail: the two-place gap between Gemini 3.1 Pro Preview (0.832 partial-F1) and Gemini 3 Flash Preview (0.790 partial-F1) is well within run-to-run variance.

Benchmark composition. The sixteen images skew toward Green Chemistry (8 of 16). Within-journal stylistic conventions could systematically favour or disadvantage particular models. A larger, journal-balanced benchmark could address this.

Condition matcher heuristics. The lenient condition matcher applied throughout Chapter 4 collapses several formatting variants (numeric-unit whitespace, parenthesis style, role bucketing,

SMILES-based name unification) into a single canonical form before comparison. The four normalisations are sufficient to absorb most cosmetic differences observed in this benchmark, but they are hand-tuned and do not cover all formatting variants a chemist would consider equivalent. A more principled approach, for example token-level alignment against a controlled vocabulary or an entailment-based matcher built on a small judge model, would be more robust and would reduce reliance on hand-tuned regular expressions.

Closed-source models. Of the nine evaluated single-agent models, eight (Claude Opus 4.7 and 4.6, GPT-5.4 and 5.5, Gemini 3.1 Pro Preview, Gemini 3 Flash Preview, Gemini 3.5 Flash, and Grok 4.3) are proprietary; the eighth, Qwen3.6-27B, is open-weights but was accessed via OpenRouter rather than self-hosted on the HPC cluster for the benchmark runs reported here. The two GPT-5 deployments used by the multi-agent and ChemEAGLE systems remain proprietary. The agent infrastructure is provider-agnostic, but the absolute results obtained for the eight proprietary single-agent variants cannot be reproduced exactly without access to the respective APIs. A fully reproducible run of the Qwen3.6-27B configuration on a self-hosted vLLM endpoint, alongside additional open-weights multimodal checkpoints (Llama-Vision, larger Qwen-VL variants), is a natural follow-up.

Reproducibility polish. A run today is reproducible to the extent that the LLM is, but the artefact set lacks the metadata needed to fully reconstruct the configuration after the fact. Prompt-content hashes, image SHAs, deterministic seeds, and tenacity-wrapped retries on remaining tools are required for clean ablation.

5.3 Conclusions

This thesis began with an attempt to reproduce ChemEAGLE, a published multi-agent pipeline for structured reaction extraction from chemistry figures and ended with a single-agent vision-LLM pipeline that outperforms ChemEAGLE, ChemEAGLE Hybrid, and an original multi-agent framework on every aggregate metric on a sixteen-image benchmark. The path between those two endpoints documented (i) the local non-reproducibility of ChemEAGLE’s headline numbers, (ii) the construction of an alternative multi-agent framework that integrates a deterministic R-group decomposition layer not present in ChemEAGLE and reaches 0.55 mean Product-IoU on the same benchmark, (iii) the brief evaluation of a second variant, and (iv) the implementation and evaluation of the single-agent pipeline against seven proprietary multimodal models.

In parallel to these four pipelines, LeMath-synth was adapted from its original inorganic-materials focus to organic synthesis and applied to the procedural prose of the benchmark papers, providing a complementary text-derived extraction route alongside the figure-derived pipeline.

The principal numerical finding is that, on this task and this benchmark, the multimodal model itself is the dominant variable: identical agent infrastructure produces a 0.61-point spread in Product-IoU when the model is varied, substantially larger than the spread observed when the orchestration is varied at fixed model. This result encapsulates a broader principle: the framework matters when the model is weak, and the model matters more than the framework when the model is strong. Among the evaluated providers, Claude Opus 4.7 achieves the highest extraction quality at the highest cost, Gemini 3 Flash Preview achieves competitive quality at substantially lower cost, and Gemini 3.1 Pro Preview occupies an intermediate position in both dimensions. Every multi-step pipeline, including the multi-agent framework, ChemEAGLE, and ChemEAGLE Hybrid, is outperformed in partial-F1 by the best single-agent configurations at comparable or lower cost. A within-model resolution ablation on the same benchmark reinforces this finding: varying input image resolution from full to one-quarter shifts Partial F1 by less than 0.01 on the Gemini 3 Flash Preview pipeline, an order of magnitude smaller than the model-to-model spread, while at one-tenth resolution the same pipeline collapses to Partial F1 = 0.06 yet retains near-perfect schema and SMILES validity. Decreasing the resolution of the input image will not reduce any computational costs. The latter dissociation is a methodological caveat for any future deployment of the pipeline: schema-pass and SMILES-validity checks alone are not sufficient quality signals, and reference-based evaluation against held-out ground truth remains a necessary safeguard.

The qualitative findings are equally important. The multi-agent pipeline’s value proposition is no longer highest partial-F1 but agent-level auditability and guaranteed SMILES validity through an enforced RDKit round-trip. A production architecture that combines the cost-quality profile of a single-agent call with the auditability of a multi-agent decomposition is a natural follow-on direction but lies outside the scope of this thesis.

Beyond the headline numbers, the pipeline reported here is one building block in a broader trend toward AI co-scientists: language-model agents that read literature, run analyses, and propose experiments alongside human researchers [1]. The introduction motivated this work in those terms. The result is concrete. A single-agent figure-extraction step at roughly \$0.20 per image and forty seconds per image is fast enough to live inside an interactive workflow rather than a batch job. A reasonable integration path for a synthetic chemist or chemical engineer is to embed the extractor in the literature-triage step that already happens at the start of every project. The chemist drops a paper into the agent, the agent extracts every reaction in every figure (with conditions and roles), the

structured output is indexed alongside the procedural text extracted via the LeMath-synth adaptation, and the resulting database becomes searchable across substrate scope, catalyst family, and condition windows. The same primitives enable an inverse workflow: a chemist asks for every published amidation that ran below 50 °C with a Boc-protected substrate, and the agent answers from its own extraction database rather than from full-text search [2]. Neither workflow requires the chemist to leave their existing reference manager or electronic notebook; the agent is invoked through a slash command, a chat-bot, or a notebook cell. The cost-quality results in Chapter 4 are what make this practical: at a few cents per image and under a minute per image with Gemini 3 Flash Preview, the extractor can run interactively on every paper a chemist opens.

A second, more ambitious integration sits at the interface between this work and the emerging autonomous-lab industry. Companies such as Dunia Innovations [3] and Radical AI [4] operate closed-loop platforms in which AI proposes formulations, a robotic platform executes them, and the resulting measurements update the model for the next round. CuspAI [5] and Lila Sciences [6] frame the same architecture as “scientific superintelligence” applied to materials and therapeutics, in Lila’s case pairing the model with a proprietary AI Science Factory of laboratory instruments. FutureHouse [7] and its commercial spinout Edison Scientific [8] take the closely related AI-Scientist view: a loop of literature reading, hypothesis generation, and experiment design. What every one of these systems requires, and what no closed-loop platform can buy off the shelf, is a structured representation of the chemistry literature that the AI side of the loop can read. The figure-extraction pipeline reported here is precisely that representation layer: it converts the published reaction record into machine-readable triplets that a closed-loop planner can search over, retrieve from, and condition its proposals on. Within the four-tier mental model FutureHouse uses (tools, science assistants, AI scientist, human researcher), this thesis sits at the boundary between the tool layer (MolDetect, MolNexTR, RDKit) and the science-assistant layer (an agent that reads a chemistry figure end-to-end). The natural next step is to wire the extractor into one of these closed-loop platforms as the literature-conditioning input, and to measure whether the autonomous loop converges to viable formulations faster when its hypothesis generator is grounded in extracted, structured reaction history rather than full-text search.

A practical caveat applies to both integration paths. The proprietary multimodal models that currently dominate the cost-quality Pareto front are accessed through provider APIs and have a finite useful lifetime. They are deprecated, retrained, or repriced on the provider’s schedule rather than the users, and the chemistry-vision performance of any successor model is not guaranteed to be monotonically better than the current one. A model that happens to have ingested a chemistry-heavy

training mix today may be superseded by a successor that has not, with no announcement and no rollback.

5.4 Future Work

5.4.1 Benchmark and Evaluation

A larger and more diverse benchmark. Sixteen images are sufficient for the trends reported here but inadequate for fine-grained ranking. A benchmark of approximately 100 images, sampled to balance journal sources and figure styles, with multiple-annotator ground truth and inter-annotator agreement scores, would enable confident statements about relative model ranking. GT3.csv and GT4.csv from the upstream benchmark from ChemEAGLE cover ~95 additional images that could be re-annotated for this purpose.

Smarter condition matching. The lenient condition matcher applies four heuristic normalisations to condition entries before comparison. A more principled approach, for example an entailment-based matcher built on a small judge model, would be more robust and would reduce reliance on hand-tuned regular expressions.

Open-source model evaluation. The provider-agnostic agent infrastructure has now been exercised against an open-weights model (Qwen3.6-27B accessed via OpenRouter), extending the evaluation beyond the seven proprietary single-agent variants and the two GPT-5 deployments used by the multi-agent and ChemEAGLE systems. The remaining gap is self-hosted execution: running the same Qwen3.6-27B checkpoint on the HPC cluster via vLLM, in float16 alongside the existing Qwen3-VL-8B-Instruct deployment, would close the API-dependence loop and make the entire single-agent evaluation reproducible without paid API access. A second open task is sampling variance: the single-pass Qwen3.6-27B result is less stable in observed behaviour than the Anthropic and OpenAI single-pass results, with one image (ACScat 2020 optimisation table) extracted successfully in an isolated probe but hitting the step ceiling in the full benchmark run. An N at least 3 protocol across providers is required for defensible per-image scores. Extending the comparison to additional open-weights vision-language models (Llama-Vision, larger Qwen-VL variants) is a natural further step.

Minimal-prompt ablation across the seven single-agent variants. The current system prompt was iterated against Anthropic models during development. As discussed in Section 4.7.1 of Chapter 4, this introduces a confounder between the model contribution and any prompt-design

contribution to the observed 0.61-point Product-IoU spread. A minimal system prompt across the same seven models, paired with the same two RDKit-backed tools and the same termination contract, would isolate the model contribution. If the relative ranking is preserved under prompt minimisation, the model-dominance claim is strengthened.

5.4.2 Reactant-Side Improvements

A consistent pattern across every aggregate metric reported in this thesis is that product structures are recovered with substantially greater fidelity than reactant structures. The substrate-scope deterministic chain introduced in Phase 9 is sound in principle, but its output is contingent on the accuracy of MolNexTR’s prediction for the corresponding product; furthermore, it is inapplicable to figures that do not follow a substrate-scope layout. Three concrete directions for addressing this dominant residual error are outlined below.

Reaction-matrix R-group derivation via Rxn-INSIGHT [9]. The Rxn-INSIGHT framework exposes a reaction-matrix representation that aligns reactant and product molecular graphs at the atom level. The central hypothesis is that incorporating a reaction-matrix step could localise R-groups even in cases where the product template and reactant template place the wildcard substituent at different ring positions. The proposed implementation preserves the existing derive substituents and apply substituents functions, introduces a parallel reaction-matrix path behind a feature flag, and evaluates both paths on the benchmark before committing to a replacement. This staged approach ensures that any improvement in reactant IoU is empirically verified rather than assumed.

Direct consumption of MolNexTR’s atom-bond graph. The MolNexTR module already returns per-atom symbols and a per-bond adjacency list when invoked. In the current multi-agent pipeline, this graph is discarded in favour of parsing MolNexTR’s emitted SMILES string with RDKit, an operation that is lossy with respect to attachment-point information for wildcard atoms. ChemEAGLE addresses this by consuming the graph representation directly, thereby preserving the correspondence between graph nodes and figure-coordinate atoms. Adopting the same approach in the multi-agent pipeline would eliminate this information loss.

5.4.3 Architecture Experiments

Hybrid routing. The cost-quality Pareto front in Chapter 4 has three cheap-tier options rather than one: Gemini 3 Flash Preview combines near-frontier quality (Partial F1 = 0.79) with the lowest cost in the comparison (\$0.018 per image); Gemini 3.5 Flash sits one tier up at \$0.055 per image with

Partial F1 = 0.80 and full schema coverage on sixteen of sixteen images, effectively offering Opus 4.7 class quality at a quarter of the per-image cost; and Qwen3.6-27B sits below both on quality (Partial F1 = 0.66) but at a similar low-cost tier (\$0.085 per image). Gemini 3 Flash Preview and Gemini 3.5 Flash are the two strongest cheap-and-strong candidates, while Qwen3.6-27B is best positioned as a low-cost mid-quality fallback rather than a frontier system. The development of cost-quality optimal routing, in which the agent decides per image which model to query and may cascade from a fast model to a more expensive one only when the fast model's confidence is low, is a promising direction for production deployment, and the presence of an open-weights model on the Pareto opens an additional self-hosted fallback tier.

Manager-and-specialists “argue” topology for the multi-agent path. Restructure the team so that Data Structure agent becomes a manager that interrogates each specialist about whether their finding belongs in the final JSON, with explicit chat turns where the specialist defends their contribution and the manager either accepts or rejects it. The selector function would gate the conversation according to figure type, so that a single-reaction figure bypasses most of the back-and-forth, while an N-variant substrate scope should force the manager to confirm with `rgroup_substitution` that exactly N reactions are emitted.

Use colour cues for R-group identification. Some papers draw R1 / R2 / R3 in distinct colours (or colour-code attachment points). MolDetect bboxes are currently grayscale; running a per-bbox dominant-colour clustering before MolNexTR and exposing the colour as a side-channel to `rgroup_substitution` would let the agent disambiguate R1 versus R2 visually.

5.4.4 Knowledge Distillation

The proprietary multimodal models evaluated in this thesis are accessible only for a finite window. They get superseded by successor versions on the provider's schedule, sometimes with capability shifts that move chemistry-vision performance up or down without warning. A particular vintage of a frontier model may have ingested a chemistry-heavy training mix and become unusually strong on reaction figures; the next vintage may not, and the 0.61-point Product-IoU spread reported in Section 4.7.1 may well evaporate or invert the next time the family is updated. A way to make the dependence on a particular vintage of frontier model more durable is to distil the chemistry-extraction behaviour of the strongest current model into a smaller, deployable model whose weights stay under the user's control.

The proposed approach is a standard teacher-student setup adapted to the structured-output character of the task [10]. The teacher (Claude Opus 4.7 single-agent, the current Pareto leader on

Partial F1) is run on a few thousand chemistry figures sampled from a journal-balanced corpus; the resulting (image, extracted-JSON) pairs are filtered on schema validity, RDKit-validity of every emitted SMILES, and downstream agreement with the deterministic R-group toolchain wherever it applies. The filtered corpus then fine-tunes a smaller open-weights vision-language model. Qwen3.6-27B already serves as the open-weights single-agent baseline in this thesis (Partial F1 = 0.66 at \$0.085 per image) and is a natural first student; a smaller candidate (in the 7B parameter range) would test the parameter-budget threshold below which distillation no longer recovers the teacher's performance. Two questions need to be answered. First, how much of the Opus 4.7 partial-F1 transfers to the student? Second, where does the transfer fail? The latter is the more interesting question; failure is likely to concentrate on long-tail substrate-scope figures where reasoning rather than perception the bottleneck is, in which case the failure analysis itself diagnoses what the teacher is actually doing.

Distillation is unusually attractive for this task because the chemistry-extraction output is structured and machine-checkable. RDKit-valid SMILES, role enumeration, and Pydantic schema validation give a free, cheap filter on the teacher's outputs during corpus construction and on the student's outputs during training: bad teacher traces are rejected before they ever enter the training mix, and the student's validation loss can be computed against ground-truth structured fields rather than only against raw token cross-entropy. A successful distilled student would also remove the per-image proprietary-pricing dependence reported in Section 4.5, opening on-premises deployment for groups whose data cannot leave the lab.

5.5 References

1. Gottweis, J., et al., *Accelerating scientific discovery with Co-Scientist*. Nature, 2026.
2. Li, J., et al., *ReactionSeek: LLM-powered literature data mining and knowledge discovery in organic synthesis*. Nature Communications, 2026.
3. *Dunia*.
4. *Autonomous Lab | Radical AI*.
5. *culp.ai*.
6. *LILA | Scientific Superintelligence*.
7. *FutureHouse*.
8. *Edison Scientific | The AI Platform for scientific R&D*.
9. Dobbelaere, M.R., et al., *Rxn-INSIGHT: fast chemical reaction analysis using bond-electron matrices*. Journal of Cheminformatics, 2024. **16**(1): p. 37.

10. Wang, L. and K.-J. Yoon, *Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks*. IEEE transactions on pattern analysis and machine intelligence, 2021. **44**(6): p. 3048–3068.

Chapter 6.

Societal Reflection

6.1	INTRODUCTION	183
6.2	SUSTAINABILITY AND THE SUSTAINABLE DEVELOPMENT GOALS	183
6.3	ETHICAL IMPLICATIONS	184
6.4	SCIENTIFIC AND EPISTEMIC IMPACT	185

6.1 Introduction

This thesis investigates whether modern vision-language models can extract structured chemical-reaction data directly from journal figures, and how the resulting pipelines compare to ChemEAGLE, the current published baseline. The work sits at the intersection of synthetic chemistry, machine learning, and scientific publishing, and its outcomes carry implications that reach beyond the laboratory. This reflection considers three societal perspectives. First, sustainability and the United Nations Sustainable Development Goals (SDG). Second, the ethics of building extraction pipelines on top of copyrighted scientific figures and opaque commercial models. Third, the epistemic consequences for chemistry as a discipline when knowledge bases are populated by machine readers rather than human curators. The aim is not to defend the methodology, but to identify where it advances a societal good, where it introduces risk, and what would need to change for the next generation of similar work to do better

6.2 Sustainability and the Sustainable Development Goals

The most visible sustainability angle of this thesis maps onto SDG 9 (Industry, Innovation and Infrastructure) and SDG 12 (Responsible Consumption and Production). A reliable extractor turns the existing literature into a structured input for high-throughput experimentation, retrosynthesis planning, and machine-learned reaction optimisation. The chemical industry currently rediscovers procedures because the original conditions sit locked inside figure panels that no database indexes. Closing that gap reduces redundant solvent use, redundant catalyst screening, and the carbon footprint of repeating reactions that have already been run somewhere else. The seven-model single-agent pipeline benchmarked in this work is one step toward that infrastructure.

The same SDG 12 framing exposes a tension. Running Claude Opus 4.7, Claude Opus 4.6, GPT-5.5, GPT-5.4, Gemini 3.1 Pro Preview, Gemini 3 Flash Preview, and Grok 4.3 across a benchmark of any meaningful size consumes nontrivial compute. Frontier inference at the scale used here, with

multiple roundtrips per figure and parallel tool calling for SMILES validation, has a real energy cost. The thesis does not measure that cost, and that is itself a limitation worth naming. A more responsible follow-up would budget and report the energy consumed per extracted reaction, in the same way that life-cycle assessments are now expected for catalysts and solvents.

SDG 13 (Climate Action) sits behind both points. The downstream value of the extracted data is climate-relevant when it feeds greener synthesis-route planning, but only if the extraction itself does not erase those gains. The discipline of measuring and publishing the compute cost is the obvious place where the current methodology should improve.

6.3 Ethical Implications

The ethical surface of this work has three faces.

The first is data provenance. Every figure in the benchmark originates from a copyrighted journal article. The thesis treats those figures as research inputs under fair use, which is the prevailing academic norm, but the legal status of training or evaluating large models on copyrighted scientific figures is genuinely unsettled. Reproducing this methodology at industrial scale, with millions of figures rather than a curated benchmark, would force a clearer conversation with publishers about licensing. Acknowledging this openly is more useful than pretending the question is already resolved.

The second is hallucination. A vision-language model that returns a SMILES string and a temperature, with no internal uncertainty signal, is asking a downstream chemist to trust a black box. In synthetic chemistry the cost of a hallucinated condition is not abstract. It is wasted reagent, wasted solvent, and in some cases a safety incident. The pipeline mitigates this with SMILES validation as a tool call, and the canonical condition enumeration is deliberately narrow (reagent, solvent, catalyst, temperature, time, yield, atmosphere, loading, pressure) to limit the room for fabrication. These are improvements over a naive prompt, but they do not eliminate the risk. The honest framing is that the pipeline produces drafts that still need a chemist in the loop, and the report should be read with that caveat.

The third is accountability and reproducibility. Six of the seven models evaluated are closed-weight commercial endpoints. A future researcher cannot rerun the exact pipeline against the exact model that produced a published number, because the providers retire and silently update those endpoints. This is a structural problem for science, not a flaw unique to this thesis, but the methodology inherits it. A more robust scientific record would either pin to open-weight checkpoints where possible or publish the raw model outputs alongside the aggregated metrics so that the comparison can be inspected even after the model behind it is gone.

6.4 Scientific and Epistemic Impact

The third lens is what this kind of work does to chemistry as a knowledge system. If LLM-extracted reaction records become the dominant source of training data for the next generation of retrosynthesis and yield-prediction models, the biases of the extractors propagate forward. A model that systematically misreads stoichiometry in older figures, or that defaults to common solvents when the original is ambiguous, does not just produce wrong rows in a database. It shifts the prior of every downstream chemistry model trained on that database.

The ChemEAGLE comparison in this thesis is the right kind of pushback against that drift. By holding a new pipeline up against a published baseline on the same figures, the work makes the failure modes visible rather than hiding them inside an aggregate accuracy number. That is the methodological contribution most worth preserving in future work. The limitation is that the benchmark itself, like every benchmark in this area, is small relative to the literature it claims to represent, and its composition reflects the choices of whoever curated it. A reflective improvement would be to publish the benchmark with explicit provenance for each figure, and to invite the community to extend it with deliberately adversarial cases such as older typography, hand-drawn schemes, and non-English captions, so that the next round of models cannot quietly overfit to the easy parts of the distribution.